

Fundamentals of Database Systems

A Comprehensive Course: Relational Databases & NoSQL

Mr. Hussein Younis

2026

Table of contents

Preface	1
How to Use This Book	1
Prerequisites	2
Copyright	2
How to Cite This Work	2
Primary Reference & Acknowledgements	3
 Part I: Foundations	 4
Introduction to Database Systems	5
Motivation: Why Not Just Use a Text File?	5
The Six Problems That Appear Immediately	6
“OK, I Will Write a Program to Handle All That”	10
The Art of Solution: The DBMS	10
What Is a Database?	11
The Database Is Just Files — The DBMS Is the <i>Container</i>	11
Key properties of a database	13
Data Models	14
Classification of Data Models	14
The Relational Model (Briefly)	14
Database Schema vs. Instance	16
Worked Example	16
Database Languages	17
Database Users and Administrators	17
DBA Responsibilities	18
Where Does the DBMS Run?	18
Advantages of Using a DBMS	19
Summary	19
Review Questions	20
Conceptual	20
Notepad Scenario (Section recap)	21
Data Models	21
Three-Schema Architecture	22
Deployment Tiers	22
Data Languages	23
True / False (with explanation)	23
Multiple Choice	24
Design Exercise	25
SQL / Query Preview	25
 Database System Concepts & Architecture	 27
The Database Design Lifecycle	27
Phase Deliverables	29

Table of contents

Three-Schema Architecture (ANSI/SPARC)	29
The Three Levels	32
Data Independence — An AAUP Example	32
DBMS Deployment Architecture	33
1-Tier (Single-Tier) Architecture	34
2-Tier (Client-Server) Architecture	36
3-Tier Architecture	37
Tier Architecture at a Glance	40
Why Tier Count Matters	40
Connection Pooling — A Critical Optimisation for 2-Tier and 3-Tier	41
Beyond 3-Tier: Distributed and Cloud Deployments	42
Choosing the Right Architecture	42
DBMS Component Architecture (Detailed)	43
Key Sub-Components	45
A Single SELECT — The Full Journey	45
The System Catalog (Data Dictionary)	46
What the Catalog Stores	46
Why Catalog Statistics Matter	46
Types of Database Systems	47
By Deployment Location	47
By Number of Users	47
Requirements Collection and Analysis	48
What to Gather	48
Techniques	48
Worked Example: University Registration System	48
Summary	49
Review Questions	49
Conceptual	49
Short-Answer / Compare	49
True / False (with explanation)	50
Multiple Choice	50
Design Exercise	50
SQL / Query Exercise	50
Part II: Data Modeling	51
Entity-Relationship (ER) Modeling	52
What Is an ER Diagram?	52
Why ER First?	52
ER Notations Overview	53
Chen Notation — Symbol Reference	54
Entities	54
Strong Entity	55
Weak Entity	55
Attributes	56
Attribute Types	56
Keys in ER Diagrams	59
Key Attribute (Strong Entity)	60
Partial Key (Weak Entity)	60
Relationships	61
Degree of a Relationship	61

Table of contents

Cardinality Ratios	61
Participation Constraints	63
Relationship Attributes	64
Recursive (Unary) Relationships	64
Ternary Relationships	65
Worked Example: The Company Database	67
Requirements	67
Step 1 — EMPLOYEE Entity	67
Step 2 — DEPARTMENT Entity + MANAGES Relationship	68
Step 3 — WORKS_FOR Relationship	70
Step 4 — PROJECT Entity	70
Step 5 — CONTROLS Relationship	71
Step 6 — WORKS_ON Relationship	72
Step 7 — SUPERVISION Relationship	73
Step 8 — DEPENDENT Weak Entity + HAS (Full Diagram)	74
Summary	75
Min-Max Notation	77
What Do min and max Mean?	77
Placement Rule	77
Illustrative Example — EMPLOYEE and DEPARTMENT	78
Company Database in Min-Max Notation	78
Chen vs Min-Max: Side-by-Side Examples	80
Specific Value Constraints: (0, 3), (2, 5), and Beyond	90
Crow's Foot Notation	90
Symbol Reference	91
Reading Rule	92
Crow's Foot Examples	92
ER-to-Relational Mapping and Final Schema	97
Summary	97
Review Questions	98
Conceptual	98
Short-Answer / Compare	98
True / False (with explanation)	98
Multiple Choice	99
Design Exercise	99
Enhanced Entity-Relationship (EER) Modeling	101
Subclasses, Superclasses, and Inheritance	101
Type Inheritance	102
Specialization	102
Constraints on Specialization	102
Combining the Two Constraints	103
Generalization	104
Specialization Hierarchies and Lattices	104
Aggregation	105
Why Aggregation?	105
Category (UNION) Types	106
EER-to-Relational Mapping	107
Mapping Specialization — Four Options	107
Mapping Aggregation	108
Mapping Categories	108
EER vs. Object-Oriented Mapping	109

Table of contents

Summary	109
Review Questions	110
Conceptual	110
Short-Answer / Compare	110
True / False (with explanation)	110
Multiple Choice	111
Design Exercise	111
SQL / Query Exercise	111
 Part III: The Relational Model	 112
The Relational Data Model	113
A Brief History — Why the Relational Model?	113
The Problems Before 1970	113
What Codd Set Out to Fix	113
From ER to Relational — Vocabulary Bridge	114
Where Do Tables Come From?	116
Mathematical Foundation	116
Domain Specification Components	116
Key Terminology	118
Properties of Relations	119
Attribute Types	119
Classification Overview	119
1. Simple (Atomic) Attributes	120
2. Composite Attributes	120
3. Single-Valued Attributes	121
4. Multi-Valued Attributes	121
5. Stored Attributes	122
6. Derived Attributes	122
7. NULL Attributes	123
Summary: Attribute Types at a Glance	124
Formal Schema Notation	124
Style 1 — Formal Notation (Used in This Book)	124
Style 2 — Shorthand Notation	125
Style 3 — SQL CREATE TABLE (Full Constraint Notation)	126
Comparing the Three Styles	126
Company Database — Logical Schema	127
Complete Key Taxonomy	131
Definitions	132
Choosing a Primary Key	134
Key Analysis — Worked Examples	135
Example 1 — Students Table (Multiple Candidate Keys, Nullable Alternate Key)	136
Example 2 — Scoreboard (Composite Candidate Key)	137
Example 3 — Employee / Department / Project (Multi-Table)	138
Example 4 — Online Store Orders (Surrogate Key, Composite Trap)	139
Example 5 — University Students (Nullable Candidate Keys)	140
Example 6 — Self-Referencing FK (Employee Hierarchy)	140
Integrity Constraints	141
Domain Constraint	141
Entity Integrity Constraint	141
Referential Integrity Constraint	142

Table of contents

Key Uniqueness Constraint	142
Foreign Key Actions	142
Example 1 — What Each Action Does	143
Example 2 — Multiple FK References: One RESTRICT Blocks Everything	144
NULL Values and Three-Valued Logic	146
Three-Valued Logic	146
Business Rules: DB Constraints vs. Application Logic	147
Handling Constraint Violations in Applications	148
What Each Operation Can Violate	148
Referential Actions Recap	148
Application-Level Response Strategies	149
ER-to-Relational Mapping	151
Step 1 — Strong Entity Types	152
Step 2 — Weak Entity Types	154
Step 3 — Binary 1:1 Relationships	156
Step 4 — Binary 1:N Relationships	160
Step 5 — Binary M:N Relationships	162
Step 6 — Multivalued Attributes	163
Step 7 — N-ary Relationships ($n > 2$)	166
FK Referential Action Reference	167
NULL Summary	168
Solved Examples	168
Final Company Database Schema	187
Formal Schema (Complete)	187
Full SQL Script (dependency order)	187
Summary	190
Review Questions	191
Conceptual	191
Short-Answer / Compare	191
True / False (with explanation)	192
Multiple Choice	192
Design Exercise	192
SQL Exercise	193
Relational Algebra	194
Company Database Schema for RELAX	194
Entity-Relationship Overview	197
What Is Relational Algebra?	198
Result Schema Notation	198
How Results Are Presented in This Chapter	198
Fundamental Operations	199
1. Select — σ (Filter Rows)	199
2. Project — π (Filter Columns)	200
3. Union — $\cup$ (Combine Two Relations)	202
4. Set Difference — $-$ (In One But Not the Other)	203
5. Cartesian Product — $\times$ (All Combinations)	203
6. Rename — ρ (Rename Relation or Attributes)	205
Additional (Derived) Operations	207
7. Intersection — $\cap$	207
8. Natural Join — $\bowtie$	208
9. Theta Join — $\bowtie_{\theta}$ (Join on Any Condition)	211
10. Outer Joins	213

Table of contents

11. Division — $\div$ (Universal Quantification)	214
Composed Queries — Company Database Examples	215
12. Extended Relational Algebra — Aggregation (γ)	217
Algebra $\rightarrow$ Execution Plan and Query Optimization	221
Expression Trees	221
Equivalence Rules	221
Cost Estimation	222
Before-and-After Optimization Example	222
Relational Algebra vs. SQL — Correspondence	223
Summary	223
Review Questions	224
Conceptual	224
Short-Answer / Compare	224
True / False (with explanation)	224
Multiple Choice	225
Design / Think-Through	225
SQL / RELAX Exercise	225
 Part IV: Structured Query Language (SQL)	 226
SQL Basics	227
From Algebra to Executable Queries	227
SQL Overview	228
SQL Sub-languages	228
Data Types	228
DDL — Data Definition Language	229
CREATE TABLE	229
MySQL	229
PostgreSQL	230
Oracle	231
Constraint Summary	231
Auto-Increment / Identity Columns	232
MySQL	232
PostgreSQL	232
Oracle	233
ALTER TABLE	233
DROP and TRUNCATE	234
MySQL	234
PostgreSQL	234
Oracle	234
DML — Manipulating Data	235
INSERT	235
UPDATE	236
DELETE	236
SELECT — Querying Data	236
Logical Evaluation Order	236
Basic SELECT	237
WHERE — Filtering Rows	237
ORDER BY	238
Limiting Rows / Pagination	238
MySQL	238

Table of contents

PostgreSQL	239
Oracle	239
Aggregate Functions	240
GROUP BY and HAVING	240
Joins	241
CROSS JOIN (Cartesian Product)	241
INNER JOIN	242
OUTER JOINs	242
MySQL	243
PostgreSQL	243
Oracle	243
SELF JOIN	243
Join Type Visual Summary	244
Subqueries	245
Scalar Subquery	245
Row / Table Subquery in WHERE	245
Correlated Subquery	245
EXISTS and NOT EXISTS	246
Set Operators: UNION, INTERSECT, EXCEPT	246
MySQL	246
PostgreSQL	247
Oracle	247
Interactive: Join Type Explorer	248
Chapter Summary	248
Review Questions	248
Conceptual	248
Applied	249
Analysis	249
Dialect Awareness	250
Design	250
Synthesis	250
Advanced SQL	251
Building on the Basics	251
Common Table Expressions (CTEs)	251
Single CTE — Readable Subquery	252
Multiple CTEs — Chained Steps	252
Recursive CTE	252
MySQL	253
PostgreSQL	253
Oracle	254
Window Functions	254
Ranking Functions	255
Analytic / Offset Functions	255
Running Aggregates	255
Frame Specification (ROWS vs RANGE)	256
MySQL	256
PostgreSQL	256
Oracle	256
Window Functions vs GROUP BY	257
Views	257
Creating and Querying Views	257

Table of contents

MySQL	257
PostgreSQL	258
Oracle	258
Updatable vs Non-Updatable Views	258
Stored Procedures and Functions	259
Stored Procedures	259
MySQL	259
PostgreSQL	260
Oracle	261
Stored Functions	261
MySQL	261
PostgreSQL	262
Oracle	262
Procedures vs Functions vs Triggers	262
Triggers	263
Trigger Anatomy	263
Audit Trigger Example	263
MySQL	263
PostgreSQL	264
Oracle	265
Validation Trigger — Rejecting Bad Data	265
JSON Functions	266
JSON Storage	266
MySQL	266
PostgreSQL	266
Oracle	267
Reading JSON Values	267
MySQL	267
PostgreSQL	268
Oracle	268
Modifying JSON	269
MySQL	269
PostgreSQL	269
Oracle	270
Chapter Summary	270
Review Questions	271
Conceptual	271
Applied	271
Analysis	271
Dialect Awareness	272
Design	272
Synthesis	272
LEFT OUTER JOIN	272
RIGHT OUTER JOIN	273
FULL OUTER JOIN	273
Subqueries (Nested Queries)	273
Scalar Subquery	273
Subquery with IN / NOT IN	273
Subquery with EXISTS / NOT EXISTS	274
Correlated Subquery	274
ANY / ALL	275

Table of contents

Common Table Expressions (CTE)	275
Recursive CTE	275
Views	276
Creating a View	276
Querying a View	276
Modifying a View	276
Dropping a View	276
Updateable Views	277
Triggers	277
Trigger Syntax (PostgreSQL)	277
Trigger Timing and Events	277
OLD and NEW pseudo-records	278
Stored Procedures and Functions	278
Stored Procedure (PostgreSQL)	278
Function (returns a value)	279
Stored Procedures vs. Triggers vs. Functions	279
Window Functions	279
Common Window Functions	279
Summary	280
Review Questions	280
 Part V: Database Design Theory	 281
Normalization & Functional Dependencies	282
The Problem: Everything in One Table	282
Update Anomalies	283
Functional Dependencies (Recap)	283
FDs in the OrderLine Table	284
Step-by-Step Normalization of OrderLine	284
Step 1 — Is OrderLine in 1NF?	284
Step 2 — Check for 2NF (Eliminate Partial Dependencies)	285
Step 3 — Check for 3NF (Eliminate Transitive Dependencies)	285
Step 4 — Check for BCNF	286
Step 5 — Verify Lossless Decomposition	287
Final Normalized Schema	288
Normal Form Summary	289
BCNF Decomposition — A Case Where 3NF $\neq$ BCNF	289
Multivalued Dependencies and 4NF	290
Canonical Cover (Minimal Cover)	291
Dependency Preservation	291
Chapter Summary	291
Review Questions	292
Conceptual	292
Applied	292
Analysis	293
Diagram / Visualization	293
Design	293
Synthesis	294

Part VI: Storage, Indexing & Query Processing	295
Storage & Indexing	296
Why Storage Matters	296
Storage Hierarchy	297
Records and Page Layout	298
Record Formats	298
Slotted Page	299
File Organizations	300
Index Concepts	300
Taxonomy	300
Clustered vs Unclustered: I/O Impact	300
B+ Tree Index	301
Structure	301
Cost Analysis	303
Interactive: File Scan vs. Index Lookup	303
Interactive: B+ Tree Order Explorer	303
Hash Indexes	303
Extendible Hashing	304
Bitmap Indexes	304
Example	305
Bitwise Operations for Multi-Predicate Queries	305
When to Use Bitmap Indexes	306
Oracle Bitmap Index	306
Index Selection Guidelines	306
When to Create an Index	306
Composite (Multi-Column) Index	307
MySQL	307
PostgreSQL	307
Oracle	307
Chapter Summary	308
Review Questions	309
Conceptual	309
Applied	309
Analysis	310
Diagram	310
Design	310
Synthesis	310
Storage Hierarchy	310
Units of Disk Transfer	311
Records and Pages	311
Record Types	311
Page / Block Layout	311
File Organizations	312
Index Concepts	312
Index Terminology	312
Clustered vs. Unclustered	312
B+ Tree Index	313
Properties	313
B+ Tree Operations	313
When to Use a B+ Tree Index	313

Table of contents

Hash Index	314
Static Hashing	314
Extendible Hashing (Dynamic)	314
Index Selection Guidelines	314
When to Create an Index	314
Multi-Column (Composite) Indexes	314
Creating / Dropping Indexes in SQL	315
EXPLAIN — Verifying Index Usage	315
Summary	315
Review Questions	316
Query Processing & Optimization	317
The Journey of a SELECT	317
Query Processing Pipeline	319
Stage 1 — Parser	320
Stage 2 — Rewriter	320
Stage 3 — Optimizer	320
Stage 4 — Execution Engine	320
Translation to Relational Algebra	320
Logical Optimization: Equivalence Rules	321
Cost Model	322
Key Statistics	322
Operator Cost Estimates	322
Selection Algorithms	322
Sequential Scan	322
Binary Search (Sorted File)	323
B+ Tree Index Scan	323
Join Algorithms	323
Nested-Loop Join	323
Sort-Merge Join	324
Hash Join	324
Algorithm Selection Guide	324
Pipeline vs Materialization	326
EXPLAIN and EXPLAIN ANALYZE	326
MySQL	326
PostgreSQL	327
Reading an EXPLAIN Output	328
Putting It All Together: A Worked Plan	328
Chapter Summary	330
Review Questions	330
Conceptual	330
Applied	331
Analysis	331
Diagram	331
Design / Synthesis	332
Query Processing Pipeline	332
Parsing and Translation	332
Query Optimization Overview	333
Why Optimization Is Hard	333
Two Phases of Optimization	333
Cost Model	333
Statistics in the System Catalog	334

Table of contents

Selection Algorithms	334
Linear Scan (Sequential Scan)	334
Binary Search	334
Index-Based Selection	334
Join Algorithms	334
1. Nested-Loop Join (NLJ)	335
2. Sort-Merge Join (SMJ)	335
3. Hash Join	335
Algorithm Comparison	335
Join Ordering and Plan Enumeration	336
Left-Deep vs. Right-Deep vs. Bushy Trees	336
Dynamic Programming Optimization	336
Query Plan Reading (EXPLAIN)	336
Summary	337
Review Questions	337
 Part VII: Transaction Management	 338
Transaction Processing	339
Why Transactions?	339
What Is a Transaction?	339
ACID Properties	340
Worked Examples — What Goes Wrong Without Each Property	340
Without Atomicity	340
Without Consistency	340
Without Isolation	340
Without Durability	341
Transaction States	341
Concurrency Anomalies	341
Lost Update	341
Dirty Read (Reading Uncommitted Data)	342
Non-Repeatable Read	342
Phantom Read	342
Isolation Levels	342
MySQL	343
PostgreSQL	343
Oracle	344
Write-Ahead Logging (WAL)	344
Log Record Format	344
Commit Flow with WAL	345
Recovery: Undo and Redo	346
ARIES Recovery Algorithm (3 Phases)	346
Checkpointing	347
Savepoints	347
MySQL	347
PostgreSQL	347
Oracle	348
Chapter Summary	348
Review Questions	349
Conceptual	349
Applied	349

Table of contents

Analysis	349
Diagram	350
Design	350
Synthesis	350
What Is a Transaction?	350
ACID Properties	351
Transaction States	351
Concurrency Issues	352
1. Lost Update	352
2. Dirty Read (Temporary Update)	352
3. Unrepeatable Read	352
4. Phantom Read	352
Transaction Isolation Levels	352
Concurrency Control: Two-Phase Locking (2PL)	353
Lock Types	353
Lock Compatibility Matrix	353
2PL Protocol	353
Strict 2PL	354
Deadlocks	354
Deadlock Detection	354
Deadlock Prevention	355
Recovery: Logging and ARIES	355
Why Recovery Is Needed	355
Write-Ahead Logging (WAL)	355
Commit Protocol	355
Summary	355
Review Questions	356
Concurrency Control & Scheduling	357
From Problem to Protocol	357
Schedules	357
Notation	357
Serial vs Concurrent	358
Conflict Serializability	358
Conflicting Operations	358
Conflict-Equivalent Schedules	358
Precedence (Serialization) Graph	358
Worked Example	359
View Serializability	359
Recoverability	359
Recoverable Schedules	359
Cascading Rollback	360
Cascadeless Schedules	360
Two-Phase Locking (2PL) — Full Protocol	360
Lock Types	360
The Two Phases	361
2PL Variants	361
Deadlocks	362
Wait-For Graph	362
Deadlock Resolution	362
Deadlock Prevention Strategies	363

Table of contents

Timestamp-Based Concurrency Control	363
Rules	363
Multiversion Concurrency Control (MVCC)	364
Core Idea	364
MVCC and Isolation Levels	364
Optimistic Concurrency Control (OCC)	365
Three Phases	365
Interactive: Isolation Level Anomaly Explorer	365
Chapter Summary	365
Review Questions	366
Conceptual	366
Applied	366
Analysis	367
Diagram	367
Design	367
Synthesis	367
Serializability	368
Conflict Serializability	368
Precedence Graph (Serialization Graph)	368
View Serializability	369
Recoverability	369
Cascading Rollback	369
Cascadeless Schedules	370
Concurrency Control Protocols	370
2PL Variants (Review and Extension)	370
Timestamp-Based Concurrency Control	370
Multiversion Concurrency Control (MVCC)	371
Optimistic Concurrency Control (OCC)	371
Summary: Isolation Implementations	371
Timestamp Protocol Example	372
Summary	372
Review Questions	372

Part VIII: NoSQL & Modern Databases 373

Introduction to NoSQL Databases 374	374
Beyond the Relational Model	374
Motivation: Challenges at Internet Scale	374
The CAP Theorem	375
BASE vs. ACID	376
The Four NoSQL Data Models	376
Key-Value Stores	376
Redis (Core Commands)	377
Use Cases	377
Document Stores	377
Column-Family Stores (Wide-Column)	378
Cassandra CQL	378
Design Principles	379
Graph Databases	379
Neo4j Cypher	380
Use Cases	380

Table of contents

NoSQL vs. RDBMS: Full Comparison	381
When to Choose NoSQL	384
Chapter Summary	385
Review Questions	385
Conceptual	385
Applied	386
Analysis	386
Diagram	387
Design	387
Synthesis	387
MongoDB — Document Databases	388
From Concepts to Queries	388
MongoDB Architecture	388
Core Concepts	388
BSON — Binary JSON	389
Getting Started with MongoDB Shell	389
CRUD Operations	389
Create — Inserting Documents	389
Read — Querying Documents	390
Comparison Query Operators	390
Logical Query Operators	391
Querying Arrays and Nested Documents	391
Update — Modifying Documents	392
Delete — Removing Documents	393
Aggregation Pipeline	393
Common Pipeline Stages	393
Aggregation Examples	394
\$lookup — Join Between Collections	394
\$unwind — Deconstructing Arrays	395
Viewing and Removing Indexes	396
EXPLAIN in MongoDB	396
Schema Design Patterns	396
Embedding (Denormalization)	396
Referencing (Normalization)	397
Common Patterns	397
MongoDB vs. SQL — Side-by-Side	397
Transactions in MongoDB	398
Chapter Summary	399
Review Questions	399
Conceptual	399
Applied	400
Analysis	400
Diagram	401
Design	401
Synthesis	401
Appendices	403
Database APIs and Connectivity	403

Database APIs and Connectivity	404
§A.0 Connecting Applications to Databases	404
A.1 Database Connectivity Standards	405
The Connectivity Stack	405
A.2 JDBC — Java Database Connectivity	406
Connection Lifecycle	406
PreparedStatement — Preventing SQL Injection	406
Transaction Control in JDBC	407
A.3 Python Database Connectivity	407
psycopg2 — PostgreSQL from Python	407
pymongo — MongoDB from Python	408
A.4 REST APIs and Databases	409
HTTP Verbs to CRUD Mapping	409
Minimal Flask REST API (Python)	409
REST API — JSON Response Examples	410
A.5 Connection Pooling	411
Java (HikariCP)	411
Python (SQLAlchemy)	412
psycopg2 (ThreadedConnectionPool)	412
A.6 ORM — Object Relational Mapping	413
SQLAlchemy — Python ORM	413
ORM CRUD Operations	413
ORM vs Raw SQL	414
Summary	414
Review Questions	415
Conceptual	415
Applied	415
Analysis	416
Design	416
 A Guided Tour of Database Families	 418
 A Guided Tour of Database Families	 419
70 Years of Database Models	420
1. Flat File Model	420
2. Hierarchical Model	421
3. Network Model (CODASYL)	422
4. Relational Model	423
5. Object-Oriented (OODB)	424
6. Object-Relational (ORDBMS)	425
7. NoSQL Models	425
7a. Key-Value Stores	425
7b. Document Stores	426
7c. Column-Family (Wide-Column) Stores	427
7d. Graph Databases	428
8. NewSQL & Cloud-Native Distributed SQL	428
Side-by-Side Comparison	429
 Advanced Constraint Enforcement	 431
 Advanced Constraint Enforcement	 432
C.1 Parking-Permit Reassignment — ON DELETE SET NULL with a Status Trigger	432

Table of contents

C.2 OFFERS Timing — Enforcing “Within the First Semester” with a Trigger	434
C.3 Specific Value Constraints $(0, 3)$, $(2, 5)$, ... — Enforcement Strategies	435
RELAX — Interactive Relational Algebra Calculator	439
How to Use	439
Quick Reference — Symbol Bar	439
Example Queries	439

Preface

This book serves as the primary reference for the **Database Systems** course in the Computer Engineering curriculum. It bridges classical relational database theory with modern NoSQL technologies, equipping students with both the conceptual foundations and practical skills demanded by today's software industry.

How to Use This Book

The book is organized into **eight parts** spanning 15 chapters plus one appendix, designed for a **14–16 week semester**:

Part	Chapters	Weeks	Theme
I — Foundations	1–2	1–2	What databases are and how they work
II — Data Modeling	3–4	3–4	Designing with ER and EER diagrams
III — Relational Model	5–6	5–6	Formal relational theory and algebra
IV — SQL	7–8	7–8	The universal database language
V — Database Design	9	9–10	Normalization and functional dependencies
VI — Storage & Query	10–11	11–12	How databases store data and execute queries
VII — Transactions	12–13	13–14	Reliability, ACID, and concurrency
VIII — NoSQL	14–15	15–16	NoSQL landscape and MongoDB

Prerequisites

Students are expected to have:

- Proficiency in at least one programming language (C, Python, or Java)
 - Basic data structures knowledge (lists, trees, hash tables)
 - Familiarity with set theory and propositional logic
-

Copyright

Fundamentals of Database Systems: A Comprehensive Course — Relational Databases & NoSQL

© 2026 Hussein Younis. All rights reserved. Computer Engineering Department.

This work is an original educational resource authored by Hussein Younis. All text, figures, diagrams, interactive widgets, and source code are the intellectual property of the author unless otherwise attributed. Third-party works cited throughout the text (including Elmasri & Navathe, *Fundamentals of Database Systems*, and the official MongoDB documentation) remain the property of their respective authors and publishers and are used here under fair-use / academic quotation principles.

The book is distributed for **educational use** within the Database Systems course. Readers may read, study, and quote short excerpts in academic work with proper citation. Redistribution, derivative works, translations, and commercial use require prior written permission of the author.

How to Cite This Work

If you reference this book in a paper, thesis, report, or presentation, please use one of the citation formats below.

IEEE

H. Younis, *Fundamentals of Database Systems: A Comprehensive Course — Relational Databases & NoSQL*. Computer Engineering Department, 2026.

APA (7th ed.)

Younis, H. (2026). *Fundamentals of database systems: A comprehensive course — Relational databases & NoSQL*. Computer Engineering Department.

BibTeX


```
@book{younis2026fundamentals,  
  title      = {Fundamentals of Database Systems: A Comprehensive Course --  
                Relational Databases \& NoSQL},  
  author     = {Younis, Hussein},  
  year       = {2026},  
  publisher  = {Computer Engineering Department},  
  edition    = {1st}  
}
```

For permission requests, corrections, or academic collaboration, please contact the author through the Computer Engineering Department.

Disclaimer: The material in this book is provided “as is” for educational purposes. While every effort has been made to ensure accuracy, the author makes no warranty regarding completeness or fitness for a particular purpose.

Primary Reference & Acknowledgements

The theoretical foundations of this book follow **Elmasri & Navathe**, *Fundamentals of Database Systems*, **7th Edition** [1]. MongoDB content follows the official MongoDB documentation [2] and [3].

All rights to cited works belong to their respective authors and publishers. The author gratefully acknowledges these sources, whose scholarship made this teaching resource possible.

Part I: Foundations

Introduction to Database Systems

Chapter 1

Without data, you're just another person with an opinion.

—W. Edwards Deming

Motivation: Why Not Just Use a Text File?

You have data. Lots of it. Where do you put it?

Before databases existed, every programmer answered that question the same way: a text file. It seems reasonable — open Notepad, type your records, save. Let us follow that decision to its natural conclusion.

The Notepad Scenario

Imagine you are building a simple student-grade system for your department. You create three text files:

students.txt

```
001, Ahmed Khalidi, Computer_Eng
002, Sara Hasan,      Computer_Eng
003, Omar Zayed,      Information_Sys
```

grades.txt

```
001, Database, 92
001, Networks, 85
002, Database, 78
003, Database, 90
```

departments.txt

```
Computer_Eng,    Dr_Mansour, Building_A
Information_Sys, Dr_Younis,  Building_B
```

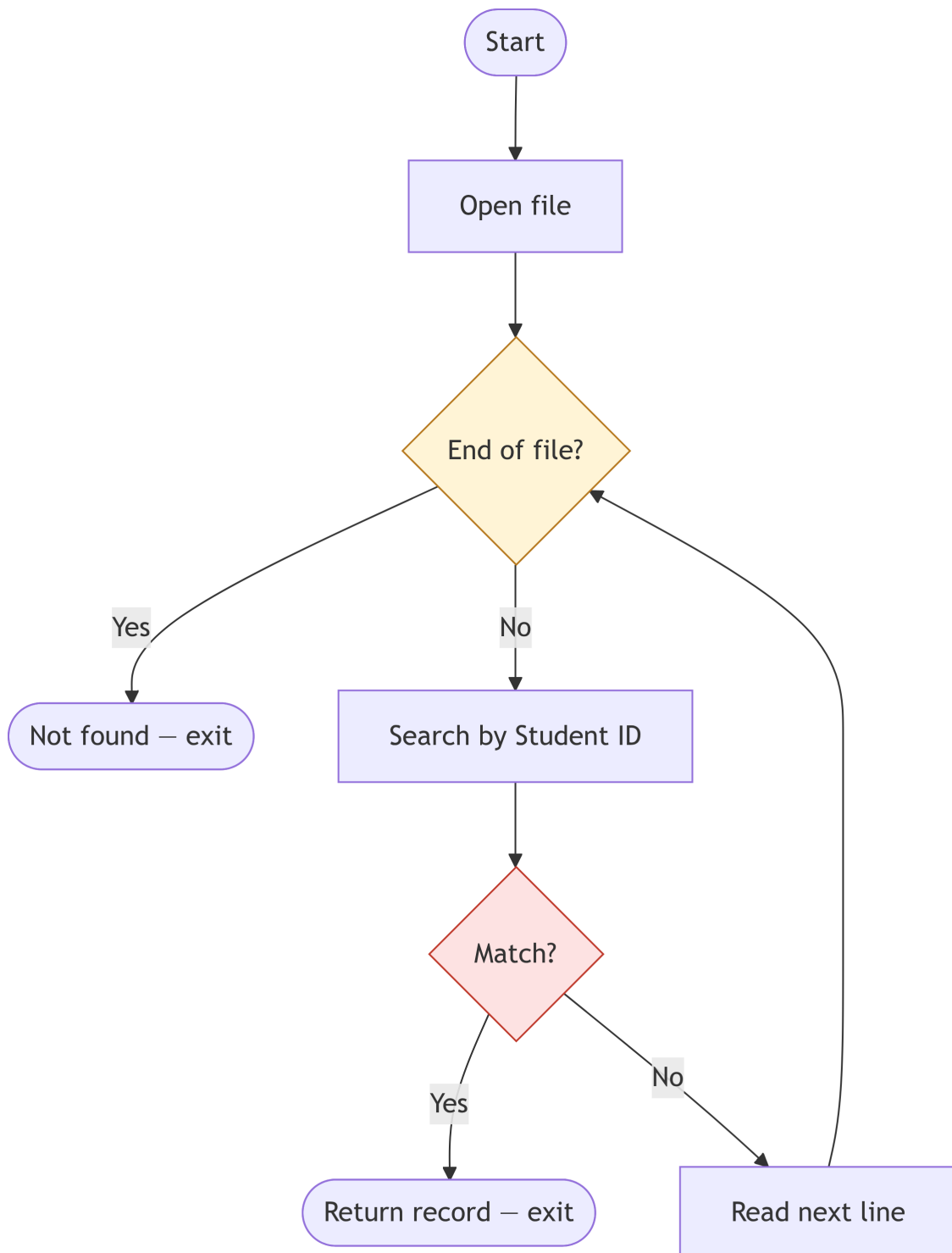
Simple. Clean. Until it is not.

The Six Problems That Appear Immediately

! Problem 1 — Search is $O(n)$

To find Ahmed's grades, your program reads every line of `grades.txt` from top to bottom until it finds 001. With 1,000 students that is manageable. With 1,000,000 students — a national exam registry — that scan takes several *seconds* per query, every query.

Search flow — how the scan actually works:



Pseudocode — linear search:

```

Algorithm LINEAR_SEARCH(file, key):
    open file
    for each line in file:                // worst case: N iterations
        fields ← split(line, ",")
        if fields[0] == key:
            output line                    // match found
    close file
    // Cost: best = O(1), average = O(N/2), worst = O(N)
    // 1 M rows × ~1 μs/row ≈ 1–3 seconds PER QUERY

```

! Problem 2 — Updates break consistency

Ahmed transfers departments. You update `students.txt`. Two weeks later someone notices a derived report still shows the old department. You forgot to update another file that duplicated the value.

Example — the real issue:

`students.txt` (*updated — new department*)

001, Ahmed Khalidi, Information_Sys

`enrollment_report.txt` (*stale — still says old department*)

001, Ahmed Khalidi, Computer_Eng, Database, 92

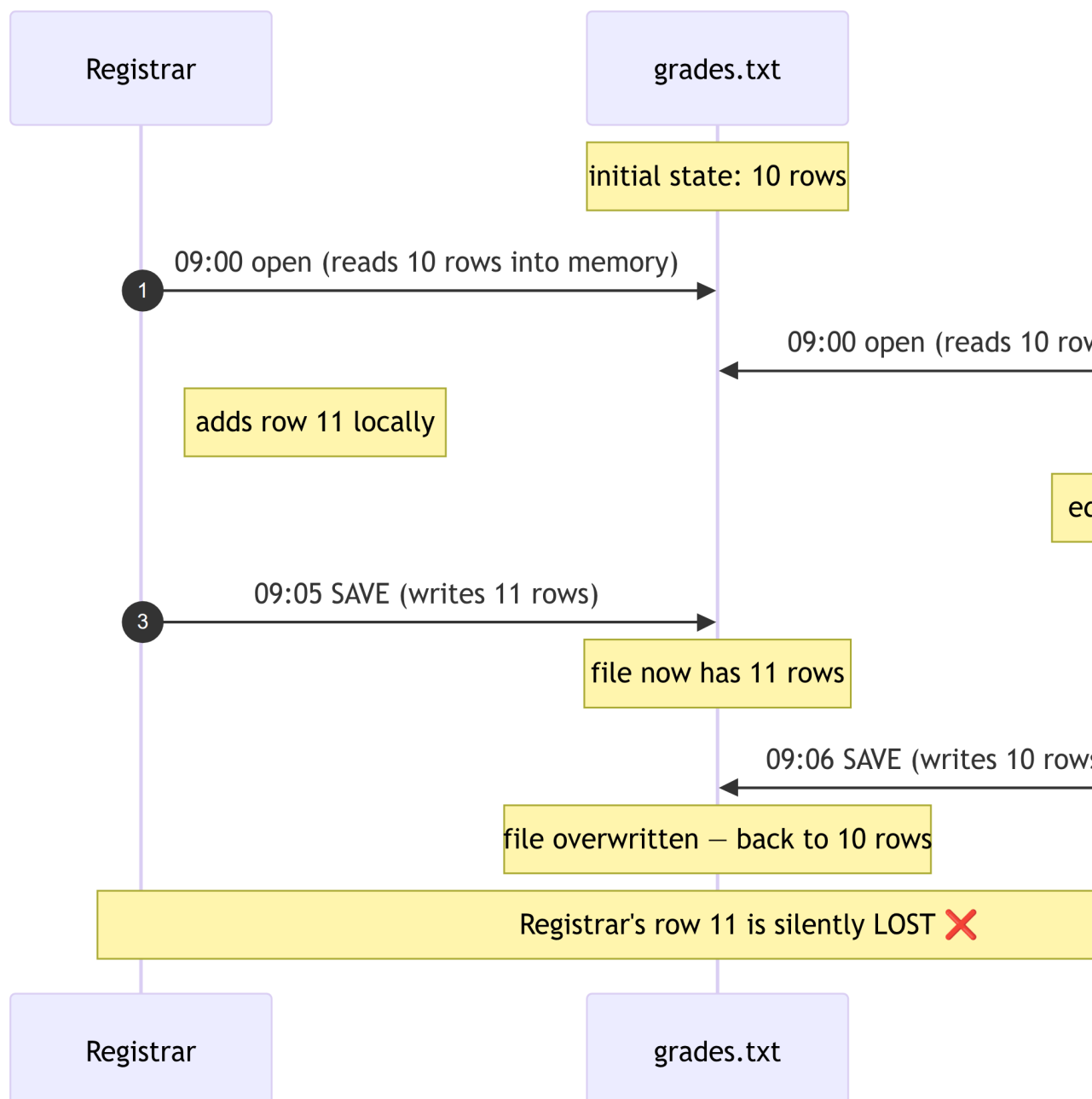
Two files, two truths, no way for the system to detect the conflict.

! Problem 3 — Concurrent access destroys data

Two users edit `grades.txt` at the same time. The last save wins — the earlier save is silently overwritten.

Example — the real issue:

Time	Registrar	Secretary
09:00	opens <code>grades.txt</code> (10 rows)	opens <code>grades.txt</code> (10 rows)
09:05	adds row 11, saves	editing row 4...
09:06	—	saves (only 10 rows)
09:07	Registrar's row 11 is gone . No error. No warning.	

Timeline of the Lost Update:

This is the classic Lost Update anomaly — one transaction's work vanishes because another transaction overwrote the same file.

! Problem 4 — Relationships require 40+ lines of code

"List all students in Building A with a grade above 80."

Example — the real issue:

```
# 1. Find departments in Building A
depts = []
for line in open("departments.txt"):
    name, head, bld = [x.strip() for x in line.split(",")]
```

```
if bld == "Building_A":
    depts.append(name)

# 2. Find students in those departments
students = {}
for line in open("students.txt"):
    sid, name, dept = [x.strip() for x in line.split(",")]
    if dept in depts:
        students[sid] = name

# 3. Cross-reference grades
for line in open("grades.txt"):
    sid, course, grade = [x.strip() for x in line.split(",")]
    if sid in students and int(grade) > 80:
        print(students[sid], course, grade)
```

Three nested loops, three file opens, manual joins. In SQL this is **one line**.

! Problem 5 — No types, no validation

Nothing prevents a user from typing garbage into the grade column. Every variant crashes or silently corrupts your analytics.

Example — the real issue:

grades.txt

```
001, Database, 92
002, Database, 92.5
003, Database, 92,5      ← comma used as decimal separator
004, Database, ninety-two ← text instead of number
005, Database, N/A       ← missing value sentinel
006, Database,           ← empty field
```

```
avg = sum(int(row.split(",")[2]) for row in open("grades.txt")) / 6
# ValueError: invalid literal for int() with base 10: ' 92.5'
```

One malformed record kills the whole report.

! Problem 6 — Crashes leave corrupt files

Power is cut in the middle of a batch update. Half the file is in the new format; half is in the old. There is no recovery log.

Example — the real issue:

grades.txt (mid-write, power cut here)

```
001, Database, 92      ← old row, untouched
002, Database, 78      ← old row, untouched
003, Database, 90      ← partially rewritten
003, Database, 9       ← truncated! file ends here
```

The file is now permanently inconsistent. No transaction log, no rollback, no way to know which rows were supposed to change.

“OK, I Will Write a Program to Handle All That”

You can. In fact, that is exactly what engineers did for 20 years before the database was invented — custom file-management programs for every application. The result:

- Every new question required a new program.
- No shared query language — each programmer invented their own.
- Security was enforced only by OS file permissions (all-or-nothing).
- No reuse: the inventory system could not share data with the payroll system even when they stored the same employee records.

Consider the contrast. Answering “*find all employees earning more than 65,000 who work in the IT department*” in a file system requires ~50 lines of parsing code. In a database, it is one line:

```
SELECT Fname, Lname FROM Employee WHERE Salary > 65000 AND Dno = 6;
```

That one line is the reason the database query language was invented.

The Art of Solution: The DBMS

Every one of the six problems above has a specific, engineered solution inside a **Database Management System (DBMS)**:

Text-file problem	DBMS solution	Covered in
Line-by-line scan — $O(n)$	B+ tree index — $O(\log n)$ lookup	Chapter 10
Inconsistency across files	Foreign keys + single authoritative schema	Chapter 5
Lost concurrent updates	Transactions + locking	Chapter 12
Manual join code	SQL JOIN with referential integrity	Chapter 7

Text-file problem	DBMS solution	Covered in
No type enforcement	Typed schema via DDL (INT, DATE, CHECK)	Chapter 7
Corrupt files after crash	Write-Ahead Log (WAL) + recovery	Chapter 12
Bespoke query programs	SQL — declarative, reusable, standard	Chapter 7

💡 Numbers That Put This in Perspective

- **Search speed:** Scanning 1 million rows sequentially → ~1–3 seconds. Finding the same row via a B+ tree index → ~0.1 milliseconds. That is a **10,000× speedup**.
- **Integrity:** A foreign-key violation is caught at INSERT time — not discovered in a bug report two weeks later.
- **Code volume:** One SQL SELECT with a WHERE clause replaces approximately 50 lines of Python file-parsing code.

This engineered solution — the DBMS — is what this book is about. Every chapter builds one part of that solution, starting with the most fundamental question: what *is* a database, and how is it structured?

What Is a Database?

A **database** is an organized collection of interrelated data that models some aspect of the real world. A **Database Management System (DBMS)** is the software layer that manages, stores, retrieves, and controls access to that data.

i Note

The combination of a database and its DBMS is called a **database system**. In everyday speech, people often say “database” when they mean the entire system.

The Database Is Just Files — The DBMS Is the *Container*

! A database is, literally, a set of files on disk

When you run `CREATE DATABASE company;` in MySQL, the server simply creates a folder on disk named `company/`. When you run `CREATE TABLE employee (...)`, MySQL writes files like `employee.ibd` into that folder. Your rows, indexes, and even the data dictionary all live on disk as **bytes in files** — nothing magical.

Proof you can verify on your own laptop:

```
# On a default MySQL install (Linux)
$ ls /var/lib/mysql/company/
db.opt  employee.ibd  department.ibd  project.ibd

# On Windows
C:\ProgramData\MySQL\MySQL Server 8.0\Data\company\
```

So *if* a database is just files, why did we spend the whole Notepad Scenario arguing against files? Because **raw files are dead bytes**. Making those bytes behave like a reliable, fast, multi-user, crash-safe system requires an entire **runtime/container** around them — that runtime is the DBMS.

Think of it exactly like programs and operating systems:

- A .exe file on disk is just bytes — it cannot “run” anything by itself. The **operating system** loads it into memory, gives it CPU time, manages its files, and protects it from other programs.
- A .ibd / .frm / .mdf database file on disk is just bytes — it cannot “query” anything by itself. The **DBMS** loads pages into a buffer cache, schedules queries, isolates transactions, and protects data from other users.

What the DBMS adds on top of the raw files:

Concern	What “just files” give you	What the DBMS gives you
OS integration	Raw read/write syscalls	Buffered, page-aligned I/O; direct-I/O for WAL; fsync for durability
Memory / cache	Nothing — every read hits disk	Buffer pool — hot pages stay in RAM (thousands of times faster than disk)
Concurrency	OS advisory file locks (all-or-nothing)	Row/page/table locks, MVCC, isolation levels, deadlock detection
User management	OS file permissions only	CREATE USER, GRANT/REVOKE at database / table / column / row level
Security	Bytes on disk, readable by anyone with file access	Authentication, encryption-at-rest, audit logs, TLS for client connections
Crash safety	Half-written file after a crash	Write-Ahead Log + rollback/redo → ACID durability

Concern	What “just files” give you	What the DBMS gives you
Query processing	You write the scan / join / sort code	SQL parser → optimizer → executor picks the best plan automatically
Self-description	You remember what columns exist	Data dictionary — the database describes itself
Networking	Local access only	TCP listener, connection pool, client drivers (JDBC/ODBC)



The one-sentence summary

A **database** is where your data *lives* (on disk). A **DBMS** is the runtime container that turns those dead bytes into a reliable, fast, multi-user, queryable service — exactly like an operating system turns a .exe file into a running program.

Key properties of a database

- **Self-describing:** The database stores a description of its own structure — called the **catalog** (or *data dictionary*) — alongside the data itself. Unlike a text file, the database knows what columns exist, what types they have, and what constraints apply.
- **Insulation between programs and data:** Application programs do not need to know *how* data is physically stored. Changing storage details does not require rewriting application code (**data abstraction**).
- **Support for multiple views:** Different users see different tailored views of the same underlying data. The payroll clerk sees salary columns; the student portal sees only name and grade.
- **Sharing and multiuser transaction processing:** Multiple users and programs access data concurrently while the DBMS guarantees that their actions do not interfere with each other.



Real-World Analogy

Think of the DBMS as a well-organized university library. The catalog (data dictionary) describes every book. The librarians (transaction manager) ensure two people cannot check out the same physical copy simultaneously. The restricted-archive rules (access control) determine who can read sensitive records. You never touch the shelves directly — you always go through the library system.

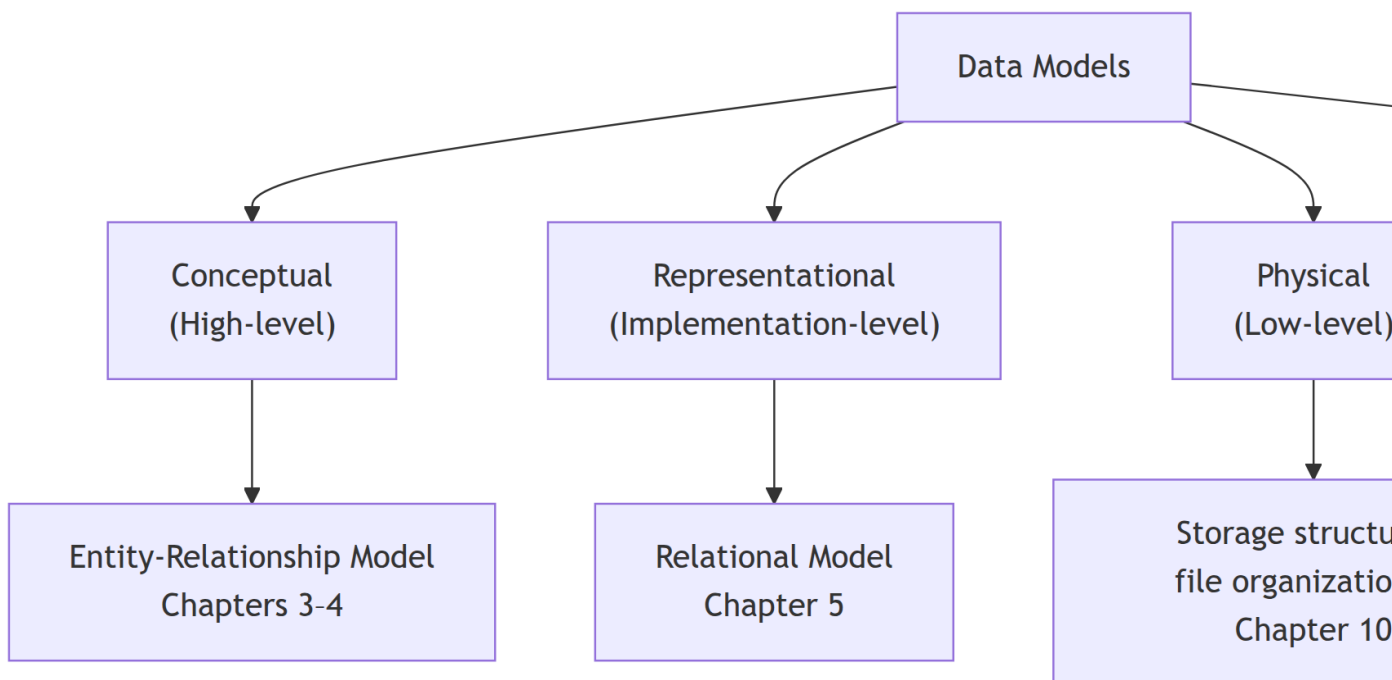
The database model introduced in this chapter becomes your foundation. All the tools you will build later — ER diagrams, relational algebra, SQL, indexes, transactions — are ways of working with precisely this structure.

Data Models

A **data model** is a collection of concepts for describing the structure of a database — including the data types, relationships among data, and constraints that must hold on the data.

Think of a data model as a *language* for talking about data structure. Different problems call for different languages, just as blueprints, circuit diagrams, and prose all describe the same building from different perspectives.

Classification of Data Models



The Relational Model (Briefly)

Proposed by **E.F. Codd in 1970** [4], the relational model represents data as a collection of **tables (relations)**, each with named columns of fixed types. It remains the dominant model in enterprise software more than 50 years later.

i Fun Fact

Codd's 1970 paper, "*A Relational Model of Data for Large Shared Data Banks*", is one of the most cited papers in computer science. He later defined **Codd's 12 Rules** as a checklist for what makes a system truly relational — only a handful of commercial databases satisfy all 12.

The relational model is elegant because it is mathematically grounded in **set theory** and **first-order logic**. That foundation is what makes SQL queries predictable and composable — and what makes relational algebra (Chapter 6) a rigorous optimization tool.

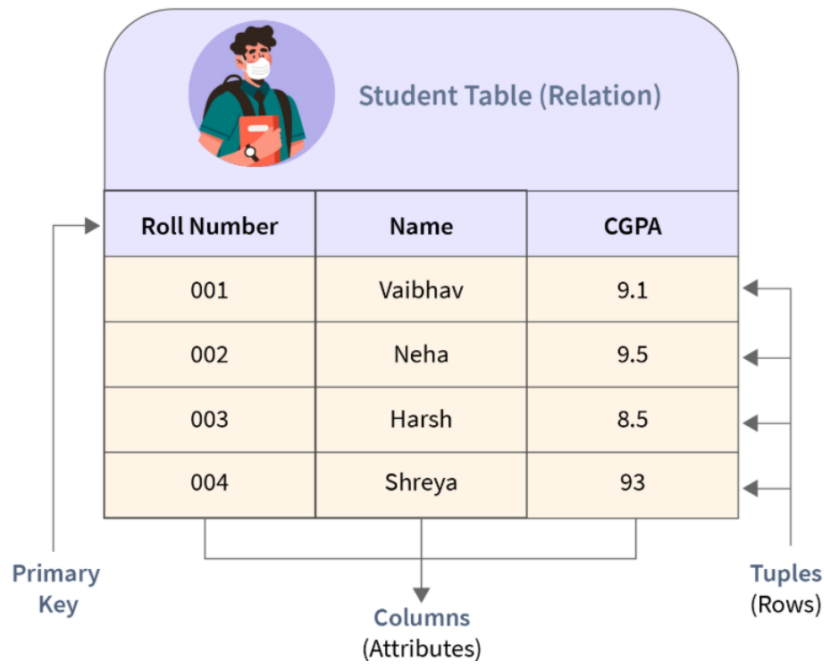


Figure 1: Anatomy of a relation: rows are *tuples*, columns are *attributes*, and the primary key uniquely identifies each row.

The relational model is not the only data model in use. A wider ecosystem of database *families* has grown up around it — hierarchical and network models that predate it, object-oriented and object-relational extensions that generalize it, and the NoSQL and NewSQL families that answer modern scale and flexibility requirements.

You do not need a complete tour of those families to follow the rest of this book. A short preview follows here; the full guided tour — with industry examples, advantages, and disadvantages of every family — lives in **Appendix B: A Guided Tour of Database Families** (@sec-appB).

i Quick preview

- **Relational** (MySQL, PostgreSQL, Oracle) — tables + foreign keys + SQL. Dominant in enterprise software; the model used in Parts III–VII of this book.
- **Document** (MongoDB) — JSON-shaped records with flexible schemas. Covered in Chapter 15.
- **Key-value** (Redis, DynamoDB) — fast get/put on a distributed hash table.
- **Column-family** (Cassandra, HBase) — huge sparse tables optimized for write throughput.
- **Graph** (Neo4j) — nodes and edges; fast relationship traversal.
- **NewSQL** (Spanner, CockroachDB) — SQL + ACID at global scale.

Real systems often combine several of these — called **polyglot persistence**.

Database Schema vs. Instance

Every database has two faces at any moment in time: its *structure* and its *current data*.

Concept	Definition	Analogy
Schema (intension)	The description/structure of the database — table names, column names, types, constraints	A class definition in object-oriented programming
Instance (extension)	The actual data stored at a specific point in time	An object instantiated from that class

A schema changes rarely — only when the structure of the problem changes (a new column is added, a table is split). An instance changes constantly — with every INSERT, UPDATE, and DELETE.

! Important

Common mistake: Students sometimes say “the database has 30 employees” when describing the *schema*. The count 30 is an *instance* fact. The schema only says: “there is a table called Employee with columns Ssn, Fname, Salary, ...”. Tomorrow the instance might have 31 employees; the schema is unchanged.

Worked Example

Schema (permanent structure):

```
Employee(Ssn, Fname, Lname, Salary, Dno)
Department(Dnumber, Dname, Mgr_ssn)
```

Instance (data at one point in time):

```
Employee: ('001', 'Ahmed', 'Khalidi', 62000, 1)
          ('002', 'Sara', 'Hasan', 59000, 2)
          ...30 rows total...
```

If we add a column Email to Employee, the *schema* changes. The existing rows get a NULL email until populated — the *instance* changes only when we insert or update data.

The distinction between schema and instance is the first of several **abstraction layers** in database systems. A DBMS actually separates *three* layers — the user’s view, the logical tables, and the physical storage — so that each can evolve independently. This three-schema architecture and the **data independence** it

provides are central to how databases are designed, so they are treated in full in [Chapter 2, §Three-Schema Architecture](#).

To communicate with a DBMS — to define those schemas and manipulate that data — we need a language.

Database Languages

A DBMS provides structured languages for every interaction. Modern SQL unifies all of these into a single language, but the conceptual separation matters for understanding what each statement is doing.

Language	Abbreviation	Purpose	SQL examples
Data Definition Language	DDL	Define and modify schemas, tables, constraints	CREATE TABLE, ALTER TABLE, DROP TABLE
Data Manipulation Language	DML	Insert, retrieve, update, delete data	SELECT, INSERT, UPDATE, DELETE
Data Control Language	DCL	Grant and revoke access privileges	GRANT, REVOKE
Transaction Control Language	TCL	Manage transaction boundaries	COMMIT, ROLLBACK, SAVEPOINT

Note

In SQL, all four sub-languages coexist in the same client session. CREATE TABLE is DDL; SELECT is DML; GRANT SELECT ON Employee TO alice is DCL; COMMIT is TCL.

With a language defined, we need to understand who uses it.

Database Users and Administrators

Different people interact with a database system in fundamentally different ways, with different tools, different technical knowledge, and different responsibilities.

Role	Description	Typical tool
Naive / parametric users	Use pre-built forms and apps; unaware of DBMS internals	Web form, mobile app
Application programmers	Write and maintain programs that access the DB via SQL embedded in code	Python, Java, PHP
Sophisticated users	Write ad-hoc SQL queries directly; data analysts, researchers	SQL client, Jupyter
DBA (Database Administrator)	Manages the entire database system	Command-line, admin console

DBA Responsibilities

- **Schema definition** — creates the initial structure using DDL
- **Storage structure decisions** — decides which indexes to build, which storage engine to use
- **Authorization management** — grants and revokes privileges to users and roles
- **Routine maintenance** — backups, performance monitoring, query tuning
- **Schema evolution** — modifies the schema safely as requirements change

With users and roles defined, we can look inside the DBMS to understand the software components that serve all of them.

Where Does the DBMS Run?

Before diving into the internals, it is worth asking a simpler question: *where does the DBMS actually run relative to the user?* Your MySQL CLI session on your laptop, MySQL Workbench connecting to a lab server, and a student logging into the AAUP portal from Chrome are three very different arrangements — classically called **1-tier**, **2-tier**, and **3-tier** deployments. Each adds a network hop, a security boundary, and an independent scaling point.

Because deployment architecture is really an engineering trade-off rather than a foundational concept, the full treatment — with AAUP lab / Workbench / portal walkthroughs, trade-offs, connection pooling, and cloud/serverless variants — lives in [Chapter 2, §DBMS Deployment Architecture](#).

Advantages of Using a DBMS

💡 Key Advantages (exam-ready list)

1. **Controlling data redundancy** — one authoritative copy; changes propagate automatically via constraints
2. **Data sharing** — concurrent access by multiple users and applications with isolation guarantees
3. **Enforcing integrity constraints** — centrally in the DBMS, not scattered across application code
4. **Restricting unauthorized access** — fine-grained user roles and privileges at table, column, and row level
5. **Providing persistent storage** — data survives program termination, process crashes, and server reboots
6. **Providing backup and recovery** — automatic crash recovery via write-ahead logging
7. **Multiple user interfaces** — forms, SQL, REST APIs, embedded queries, natural-language interfaces
8. **Representing complex relationships** — foreign keys, joins, recursive references

Summary

! Chapter 1 — Key Takeaways

- **The core problem:** text files fail at scale — they are slow to search ($O(n)$), vulnerable to inconsistency, broken by concurrent access, and offer no type safety or crash recovery.
- A **database** is a self-describing, organized collection of interrelated data; a **DBMS** manages it with a data dictionary, query processor, transaction manager, and buffer manager.
- **File systems** suffer from seven classic problems: redundancy, inconsistency, isolation, integrity, atomicity, concurrency, security. A DBMS solves each with a specific mechanism.
- A **data model** (ER, relational, NoSQL) provides the vocabulary for describing data structure.
- The **three-schema architecture** (external $\rightarrow$ conceptual $\rightarrow$ internal) enables **data independence**: change one level without rewriting the others.
- **Logical independence** isolates applications from schema changes; **physical independence** isolates schemas from storage changes. Physical is easier to achieve.
- DBMS sub-languages: DDL (define), DML (manipulate), DCL (control), TCL (transactions). Modern SQL unifies all four.
- **DBMS deployment** comes in three tiers: 1-tier (all on one machine, single user), 2-tier (client + DB server via JDBC/ODBC), and 3-tier (client + app server + DB). Most modern web applications use 3-tier for security and scalability. Internal DBMS components (Query

Processor, Storage Manager) are detailed in Chapter 2.

Review Questions

Every question below includes a collapsible **Show Answer** box so you can self-check after attempting the question.

Conceptual

Q1. Define the term **database system**. Distinguish it from a *database* and from a *DBMS*, giving a concrete example of each.

Show Answer

- **Database** — the *collection of data itself* (the rows, tables, documents). Example: the AAUP Student, Enrollment, and Course tables stored on disk.
- **DBMS** — the *software* that creates, manages, queries, and protects the database. Example: MySQL, PostgreSQL, Oracle, MongoDB.
- **Database system** — the **combination of the database + DBMS + application programs + users**. Example: the entire AAUP academic information system (MySQL + portal + staff + students).

Q2. What is the **data dictionary** (system catalog)? What information does it store, and which DBMS component maintains it?

Show Answer

The **data dictionary** (a.k.a. system catalog) is a special set of tables inside the database that describes **the database's own structure** — table names, column names, data types, constraints, indexes, views, user privileges. It is maintained by the **DBMS itself** (specifically the DDL processor updates it whenever CREATE, ALTER, or DROP runs) and consulted by every query during parsing and optimization. This is what makes a database **self-describing**, unlike a plain text file.

Q3. Explain the difference between **logical data independence** and **physical data independence**. Which is harder to achieve in practice, and why?

Show Answer

- **Logical data independence** — the ability to change the **conceptual schema** (add a column, rename a table, split a table) **without rewriting external views or application programs**.
- **Physical data independence** — the ability to change the **internal/storage schema** (add an index, switch storage engine, re-partition) **without changing the conceptual schema**.

Logical independence is harder. Adding a column is safe, but splitting Student into Student + StudentContact or renaming gpa → cumulative_gpa forces every view, query, and application that referenced the old structure to be updated. Physical independence is easier because the DBMS already hides storage details behind SQL.

Q4. List the seven classic problems of using plain files instead of a DBMS, and match each one to the specific DBMS mechanism that solves it.

Show Answer

Problem	DBMS solution
Data redundancy	Normalization + single authoritative schema
Data inconsistency	Foreign keys + referential integrity
Data isolation	SQL joins across tables
Integrity constraints scattered in code	Centralized CHECK, NOT NULL, UNIQUE, FK constraints
Atomicity problems on crash	Transactions + write-ahead log (WAL)
Concurrent access anomalies	Locking / MVCC (transaction isolation)
Security below file level	Role-based access control (GRANT / REVOKE)

Notepad Scenario (Section recap)

Q5. In the Notepad Scenario, why does a **linear scan** of grades.txt become impractical as the university grows? Give the Big-O cost and a concrete timing estimate for 1,000,000 rows.

Show Answer

A file scan reads every line sequentially, so cost is $O(n)$. At roughly 1 μ s per row on a modern laptop, scanning 1 million rows takes **$\approx 1\text{--}3$ seconds per query**. With an index the same lookup becomes $O(\log n) \approx 0.1$ ms — about a **10,000 $\times$ speedup**.

Q6. Two secretaries edit grades.txt simultaneously; the second save silently erases the first. Name this anomaly and name the DBMS mechanism that prevents it.

Show Answer

This is the **Lost Update** anomaly. A DBMS prevents it with **transactions** providing **isolation** via locking (or MVCC) — the second writer either waits for the first to commit or is rolled back and retried.

Data Models

Q7. Classify each of the following as *conceptual*, *representational/implementation*, or *physical* data model: (a) Entity-Relationship (ER) diagram, (b) Relational model, (c) B+ tree file organization, (d) JSON document model, (e) Storage block layout.

Show Answer

- (a) ER — **conceptual** (high-level, for humans)
- (b) Relational — **representational / implementation** (tables and rows)
- (c) B+ tree — **physical** (how records are stored)
- (d) JSON document — **self-describing / representational** (NoSQL)
- (e) Block layout — **physical**

Three-Schema Architecture

Q8. Compare the **conceptual level** and the **external level** of the three-schema architecture. Who works at each level, and what do they see? Answer in the AAUP context.

Show Answer

- **Conceptual level** — the **DBA** in the AAUP IT Center. Sees the *entire* logical schema: every table (Student, Course, Enrollment, Instructor, ...), every column, every FK, every constraint.
- **External level** — each end user or application. A **student** sees `my_grades_view` (her rows only); an **instructor** sees `my_sections_view`; the **Registrar** sees `enrollment_stats_view`. Each is a customized projection of the conceptual schema.

Deployment Tiers

Q9. Classify each of the following as **1-tier**, **2-tier**, or **3-tier** and justify:

- (a) Running `mysql -u root -p` in the Windows CMD on your own laptop against a local MySQL server.
- (b) Opening **MySQL Workbench** on your laptop and connecting to `localhost:3306`.
- (c) Sara opening `https://portal.aaup.edu` in Chrome to check her grades.
- (d) A SQLite-backed mobile note-taking app.

Show Answer

- (a) **1-tier** — CLI + server + data files all on one machine, one user. (The Database Lab setup.)
- (b) **2-tier** — Workbench is a separate client process that speaks the MySQL protocol over TCP (port 3306) to `mysqld`. Same architecture whether the server is on the same laptop or a remote lab server.
- (c) **3-tier** — Browser (Presentation) → AAUP web/app server (Application, enforces auth + business rules) → MySQL database (Data, firewalled from the Internet).
- (d) **1-tier** — SQLite is a library linked into the app; no separate server process.

Q10. Why is a 3-tier architecture preferred over 2-tier for a public-facing system like the AAUP portal? Give **three reasons**.

Show Answer

1. **Security** — the database is never exposed to the Internet; the attacker must first compromise the app server.
2. **Scalability** — app servers can be replicated horizontally behind a load balancer independently of the DB.
3. **Centralized authorization** — business rules (“students see only their rows”) live in one trusted tier, not scattered across every client.

(Also acceptable: maintainability, DB portability, clean separation of UI / logic / data.)

Data Languages

Q11. Match each statement to the correct sub-language (**DDL / DML / DCL / TCL**): (a) `CREATE TABLE Student(...)`, (b) `SELECT * FROM Student`, (c) `GRANT SELECT ON Student TO dr_smith`, (d) `COMMIT`, (e) `UPDATE Student SET gpa = 3.9 WHERE id = '001'`, (f) `ROLLBACK`.

Show Answer

- (a) **DDL** — defines structure
- (b) **DML** — reads data
- (c) **DCL** — grants privileges
- (d) **TCL** — commits a transaction
- (e) **DML** — modifies data
- (f) **TCL** — undoes uncommitted work

True / False (with explanation)

Q12. “A database schema changes every time a new row is inserted.”

Show Answer

False. Inserting a row changes the **data**, not the **schema**. The schema (tables, columns, types, constraints) only changes when you run DDL such as `ALTER TABLE`.

Q13. “Physical data independence means that application programs do not need to know which disk the data is stored on.”

Show Answer

True, but the phrasing is narrow. Physical independence means the **entire internal storage level** (file organization, indexes, disk location, block size) can change without affecting the conceptual schema or applications — the disk name is just one example.

Q14. “A naive user can interact with a DBMS only by writing SQL queries.”

Show Answer

False. Naive (end) users typically interact through **forms, menus, mobile apps, or canned reports** that issue SQL on their behalf. They rarely write SQL themselves — that is the job of application programmers and DBAs.

Q15. “The data dictionary is stored separately from the database it describes.”

Show Answer

False. The data dictionary is stored **inside the same database** as a set of special system tables (e.g., `information_schema.tables` in MySQL, `pg_catalog` in PostgreSQL). This is exactly what makes the DB *self-describing*.

Multiple Choice

Q16. Which DBMS sub-language is used to define tables, columns, and constraints?

- (a) DML (b) DCL (c) DDL (d) TCL

Show Answer

(c) DDL — Data Definition Language.

Q17. Which property of a database distinguishes it from a plain text file?

- (a) It can store text
- (b) It is self-describing
- (c) It requires an internet connection
- (d) It can only be accessed by one user at a time

Show Answer

(b) It is self-describing. The database stores its own schema (the data dictionary) alongside the data — a plain text file does not.

Q18. Physical data independence means you can change:

- (a) application programs without changing the schema
- (b) external views without changing the conceptual schema
- (c) the internal storage structure without changing the conceptual schema
- (d) the conceptual schema without changing external views

Show Answer

(c) — change the internal (storage) layer without touching the conceptual schema. Option (d) would be *logical* independence.

Q19. Which component of a DBMS is responsible for ensuring that a partially completed transaction is rolled back after a crash?

- (a) Query optimizer
- (b) Buffer manager
- (c) Transaction/recovery manager
- (d) Data dictionary

Show Answer

(c) Transaction / recovery manager — uses the write-ahead log (WAL) to undo incomplete transactions after a crash.

Design Exercise

Q20. A small clinic stores patient records in three CSV files: `patients.csv`, `appointments.csv`, and `doctors.csv`. Identify **three specific anomalies or risks** and, for each, name the DBMS feature that would eliminate it.

Show Answer

Any three of the following:

1. **Referential integrity violated** — an appointment can reference a non-existent patient id. Fix: **foreign key** from `appointments.patient_id` → `patients.id`.
2. **Lost updates** — two receptionists edit `appointments.csv` simultaneously and one save overwrites the other. Fix: **transactions with locking / MVCC**.
3. **No type enforcement** — date column can contain "2026-04-17", "17/4/26", or "N/A". Fix: **typed schema** (`DATE NOT NULL`).
4. **Slow search** — linear scan through 100,000 appointments to find one doctor's schedule. Fix: **B+ tree index** on `doctor_id`.
5. **No security granularity** — OS file permissions are all-or-nothing. Fix: **GRANT/REVOKE** at table, column, or row level.
6. **Crash corruption** — power cut mid-write leaves a half-written CSV. Fix: **write-ahead logging + crash recovery**.

SQL / Query Preview

Q21. Given a table `Grades(StudentId, StudentName, Course, Grade)`, write a SQL query to retrieve the names of all students with a grade above 80. Explain in one sentence why this replaces tens of lines of file-parsing code.

Show Answer

```
SELECT DISTINCT StudentName
FROM   Grades
WHERE  Grade > 80;
```

SQL is **declarative** — you state *what* you want; the DBMS's query optimizer decides *how* to fetch it (scan vs. index, which order, which join method). All the file-opening, line-splitting, type-parsing, filtering, and de-duplication that you would hand-write in Python is handled once, correctly, by the engine.

Q22. What SQL sub-language command would you use to:

- (a) Create the Grades table?
- (b) Insert a new grade record?
- (c) Allow user dr_smith to read from Grades but not modify it?
- (d) Undo all changes made in the current session?

Show Answer

- (a) **DDL** — CREATE TABLE Grades (...);
- (b) **DML** — INSERT INTO Grades VALUES (...);
- (c) **DCL** — GRANT SELECT ON Grades TO dr_smith;
- (d) **TCL** — ROLLBACK;

Database System Concepts & Architecture

Chapter 2

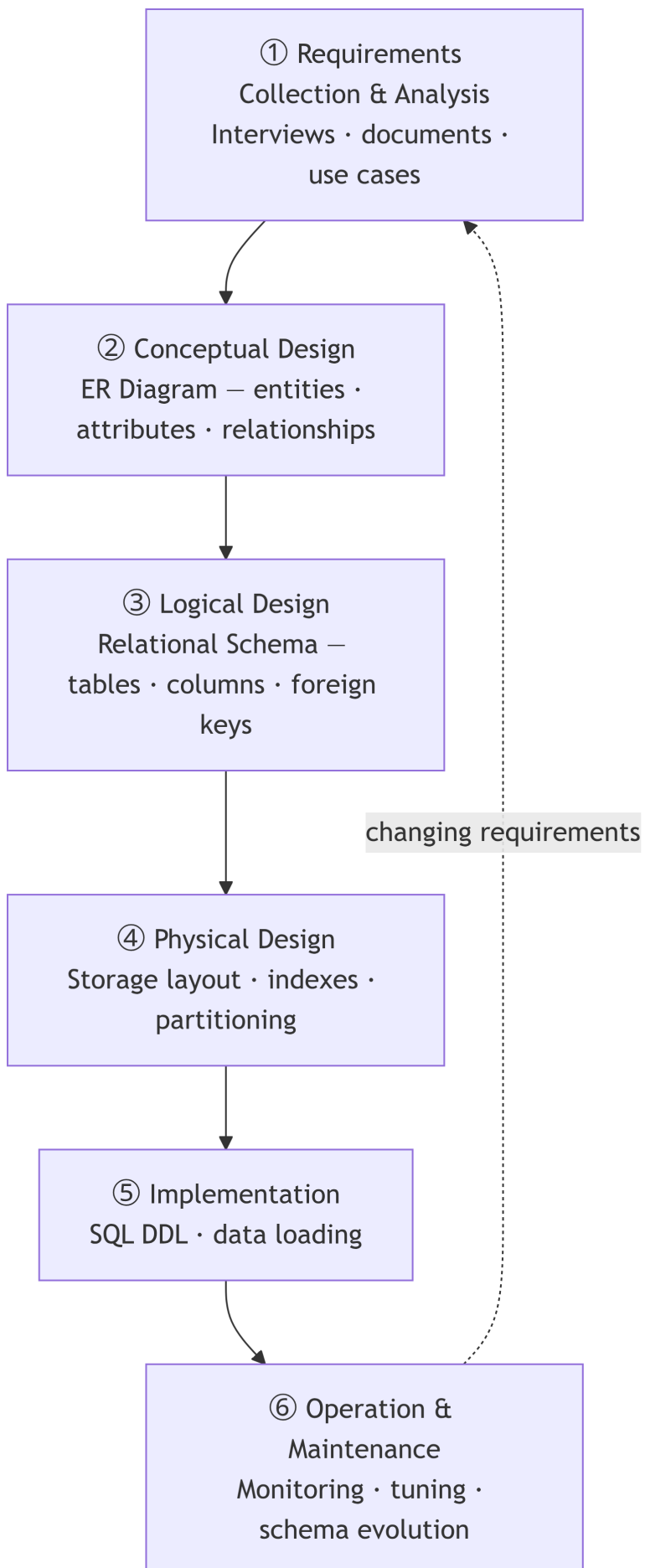
The purpose of abstracting is not to be vague,
but to create a new semantic level in which
one can be absolutely precise.

—Edsger W. Dijkstra

Chapter 1 answered *why* we use a database. This chapter answers *how* we build and structure one — tracing the full journey from a blank page to a running system, and then examining the internal machinery that makes that system work reliably.

The Database Design Lifecycle

No database should be designed from thin air. A well-established **design lifecycle** guides the process from understanding what users need to deploying a production system.



Notice the feedback arrow: maintenance often uncovers new requirements, restarting the cycle.

Phase Deliverables

Phase	Key Questions	Deliverable
Requirements	What data must be stored? What queries must be answered?	Data requirements document
Conceptual	What entities and relationships exist in the domain?	ER diagram (Chapters 3–4)
Logical	How do entities translate to tables? Are they normalized?	Relational schema (Chapters 5, 9)
Physical	Which indexes? What storage engine? How to partition?	Annotated schema + storage spec (Chapter 10)
Implementation	Create the tables; load the initial data	Running database with SQL DDL
Maintenance	Is performance degrading? Do new fields need to be added?	Tuning reports; migration scripts

Note

This book follows the lifecycle order: Chapters 3–4 cover conceptual design (ER), Chapters 5–6 cover logical design (relational model and algebra), Chapters 7–8 cover SQL (implementation language), Chapter 9 covers normalization (logical refinement), and Chapters 10–13 cover the physical and operational layers.

With the lifecycle established, we now look more closely at one of its most important concepts: the three levels of abstraction that make databases manageable.

Three-Schema Architecture (ANSI/SPARC)

Chapter 1 introduced the *schema vs. instance* distinction. A full DBMS actually separates **three** schemas, not two — the user’s view, the logical tables, and the physical storage. This section describes each level, shows an AAUP example, and explains the **data independence** the separation buys us.

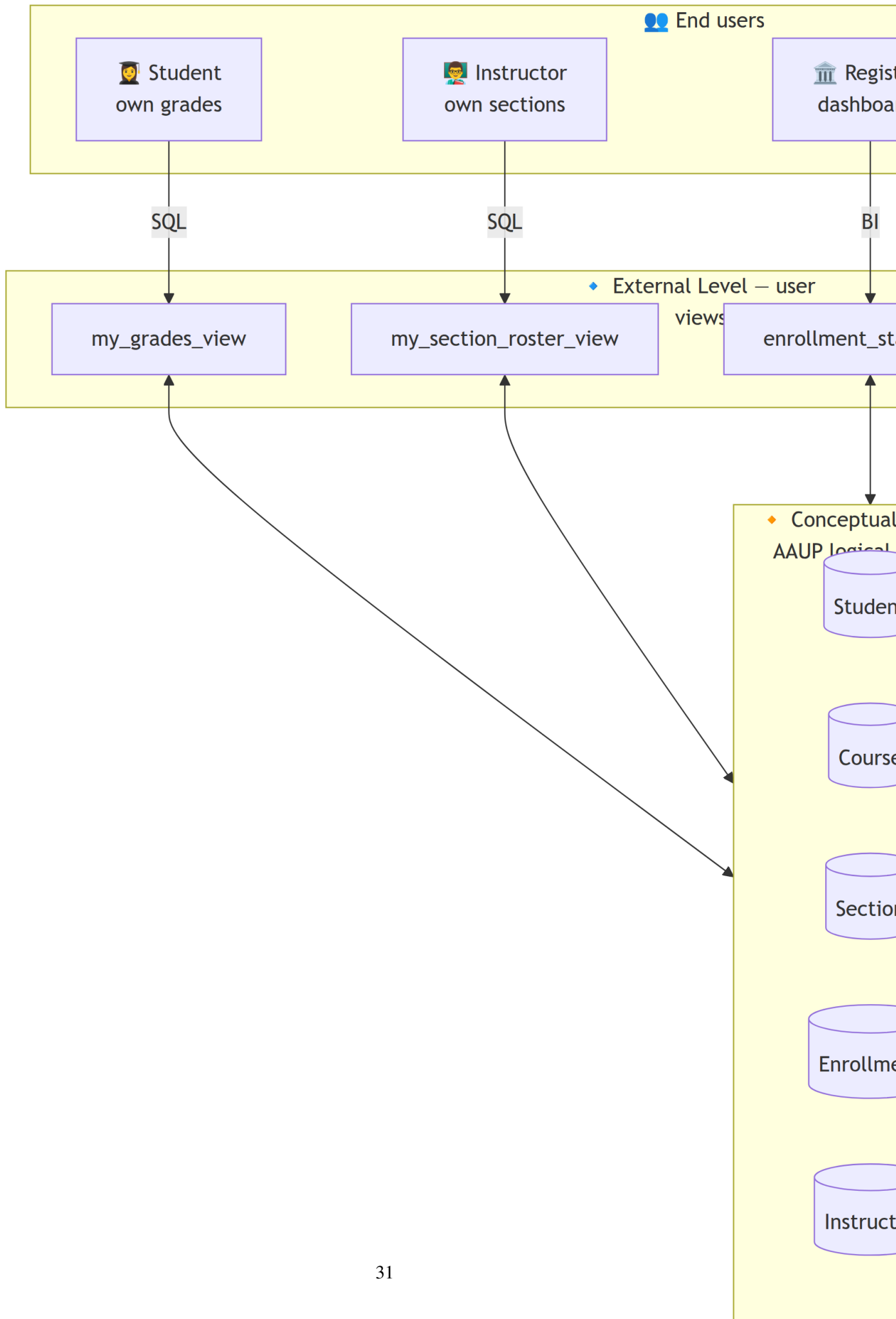
As a database grows, different people interact with it differently: end users care about a grade-entry form; the application developer cares about the logical table structure; the storage engineer cares about disk blocks and indexes. The **three-schema architecture** separates these concerns.

i Scenario — The AAUP Academic Information System

Picture the **Arab American University – Palestine (AAUP)** academic information system (the one you use through the student portal and SIS). Thousands of people touch the *same* underlying database every day, but each of them sees it very differently:

- **A student in the Computer Engineering Department** logs into the portal to check her grades for *Database Systems* and register for next semester's courses. She should only see **her own** courses, grades, GPA, and schedule — never another student's record.
- **A course instructor** opens the faculty portal to enter grades for *Database Systems — Section 1*. He should see the list of students enrolled in **his sections only**, plus the assessment columns (midterm, project, final) — but **not** salary information or other instructors' sections.
- **The Registrar's office** needs a dashboard of enrollment counts per faculty (Engineering, Medicine, Law, Arts) and graduation statistics across the whole university.
- **The Finance department** generates tuition invoices and payroll — they see student fees and staff salaries, but not academic grades.
- **The database administrator (DBA)** in the IT Center at Jenin campus is responsible for the entire logical schema — every table, every foreign key, every constraint.
- **The system engineer** tunes physical storage: which tables live on SSD, which columns are indexed, how backups are scheduled across the Jenin and Ramallah data centers.

One database. Six very different views. The **three-schema architecture** is what makes this possible.



Two ANSI/SPARC mappings — **external conceptual** (one per view) and **conceptual internal** (managed by the DBMS) — are what give us data independence.

The Three Levels

- **External level** — *what each user sees*. At AAUP this is the set of **views** behind every portal screen: `my_grades_view`, `my_sections_view`, `enrollment_stats_view`, `billing_view`. Each view exposes only the rows and columns that user is authorized to see.
- **Conceptual level** — *the complete logical schema maintained by the AAUP DBA*. All tables (Student, Course, Section, Enrollment, Instructor, Faculty), all columns, all primary/foreign keys, all constraints. This is the single source of truth. Table definitions, foreign keys, and constraints live here; this is the level DBAs and application developers work in.
- **Internal level** — *how the data is physically stored*. Tablespaces on SSD at the Jenin campus data center, B+ tree indexes on `student_id` and `course_code`, partitioning of Enrollment by semester, and nightly replication to the Ramallah site. File organizations, record formats, and access paths live here; users never see this level.

Conceptual schema in detail:

- Student(`student_id`, name, faculty, department, gpa)
- Course(`course_code`, title, credits)
- Section(`section_id`, `course_code`, `instructor_id`, semester)
- Enrollment(`student_id`, `section_id`, grade)
- Instructor(`instructor_id`, name, department, salary)
- Faculty(`faculty_id`, name, campus)

Tip

Why views matter beyond simplicity: Views are the main mechanism for implementing **column-level access control**. By granting SELECT on a view but not on the base table, you can share a salary report that omits the Ssn (social security number) column entirely.

Data Independence — An AAUP Example

The three-level separation provides **data independence** — the ability to change one level without rewriting the level(s) above it.

Type	Definition	Example at AAUP
Logical data independence	Change the <i>conceptual</i> schema without rewriting external views or application programs	The Registrar adds a new <code>mobile_number</code> column to <code>Student</code> so the university can send SMS notifications. Ahmed still logs in and sees his grades exactly as before — the student portal was not touched.
Physical data independence	Change the <i>internal</i> schema (e.g., add an index, switch storage engine) without changing the conceptual schema	The IT team adds a B+ tree index on <code>Enrollment(section_id)</code> so grade-sheet queries run 100× faster. No SQL query, no portal, no report has to be rewritten.

 Tip

Physical independence is relatively easy to achieve (most DBMSs do it well). Logical independence is harder — adding a column is safe, but splitting `Student` into `Student` + `StudentContact`, or renaming `gpa` to `cumulative_gpa`, forces every view and application that referenced the old structure to be updated.

The abstraction levels describe the *design* of the database. When the database is actually *deployed*, it runs inside a larger **system architecture** that determines how clients connect to it.

DBMS Deployment Architecture

A DBMS does not always run on the same machine as the application that uses it. **Deployment architecture** describes how many independent layers (*tiers*) exist between the end user and the DBMS engine. More tiers increase security and scalability at the cost of added complexity.

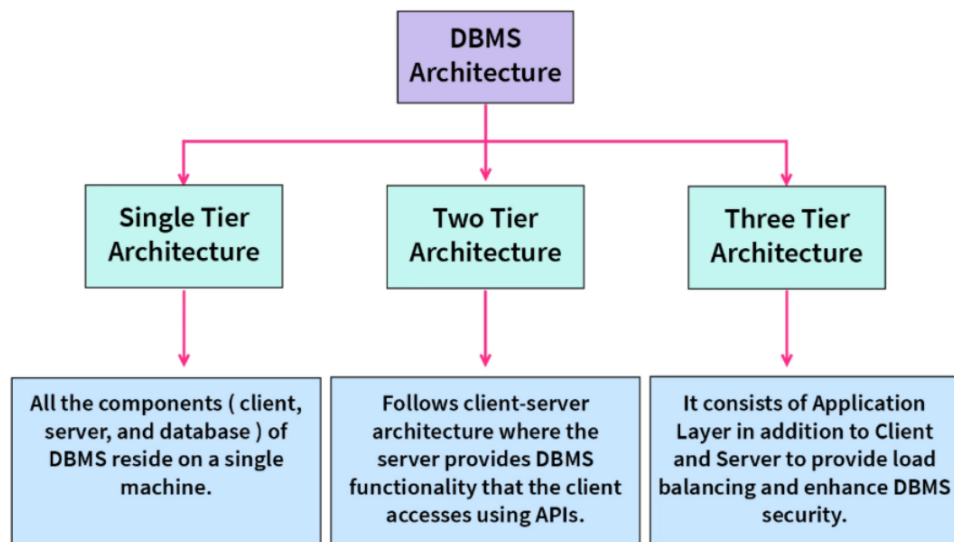


Figure 1: DBMS Tier Architecture Overview

1-Tier (Single-Tier) Architecture

All components — user interface, application logic, and the DBMS — reside on a **single machine**. The user interacts directly with the database with no network involvement.

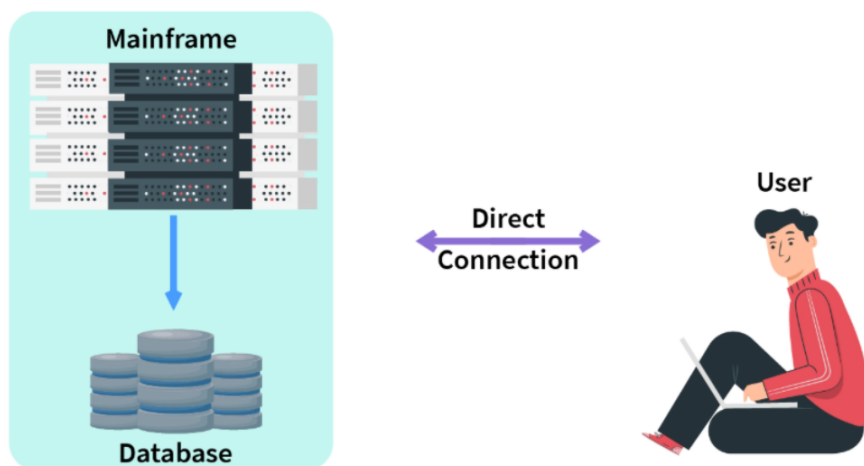


Figure 2: 1-Tier (Single-Tier) Architecture

Example of Single Tier DBMS Architecture

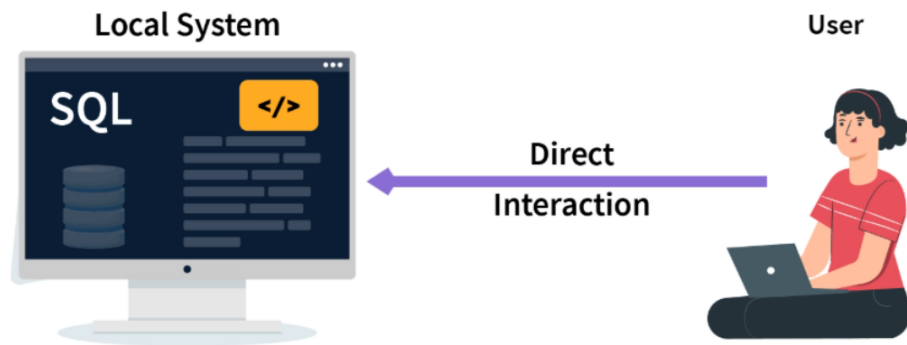


Figure 3: Example of Single Tier DBMS Architecture

💡 You have already used a 1-tier setup — in the Database Lab!

Remember the **Database Systems lab at AAUP**? When you installed MySQL Server on your own laptop and then opened the MySQL shell from the Windows Command Prompt to run your first `SELECT` — **that was a 1-tier architecture**.

```
c:\> mysql -u root -p
Enter password: *****
mysql> USE company;
mysql> SELECT * FROM employee;
```

On your laptop at that moment:

- **The client** (the `mysql` shell) — running on your machine
- **The application logic** (you typing SQL) — running on your machine
- **The DBMS server** (`mysqld`) + the data files — also running on your machine

No network, no other users, no separate server — everything on one box. That is exactly what *single-tier* means. Any program that uses SQLite (which is just a library embedded inside your application) is 1-tier by design.

Note: once you connect from one laptop to a **MySQL server running on a different machine** (e.g., `mysql -h lab-server -u student -p`), you have crossed into **2-tier** territory — covered next.

When to use: Learning environments, local development, single-user desktop tools (e.g., a SQLite-backed app, MS Access on a single laptop, or your MySQL lab setup).

Advantage	Disadvantage
Simple — no network needed	Single user only
No network latency	No isolation between app and DB
Full direct control	Cannot scale

2-Tier (Client-Server) Architecture

The **client** runs the user interface and application logic; the **DBMS** runs on a separate **server**. Communication uses standard APIs (JDBC, ODBC, native drivers) over a TCP network.

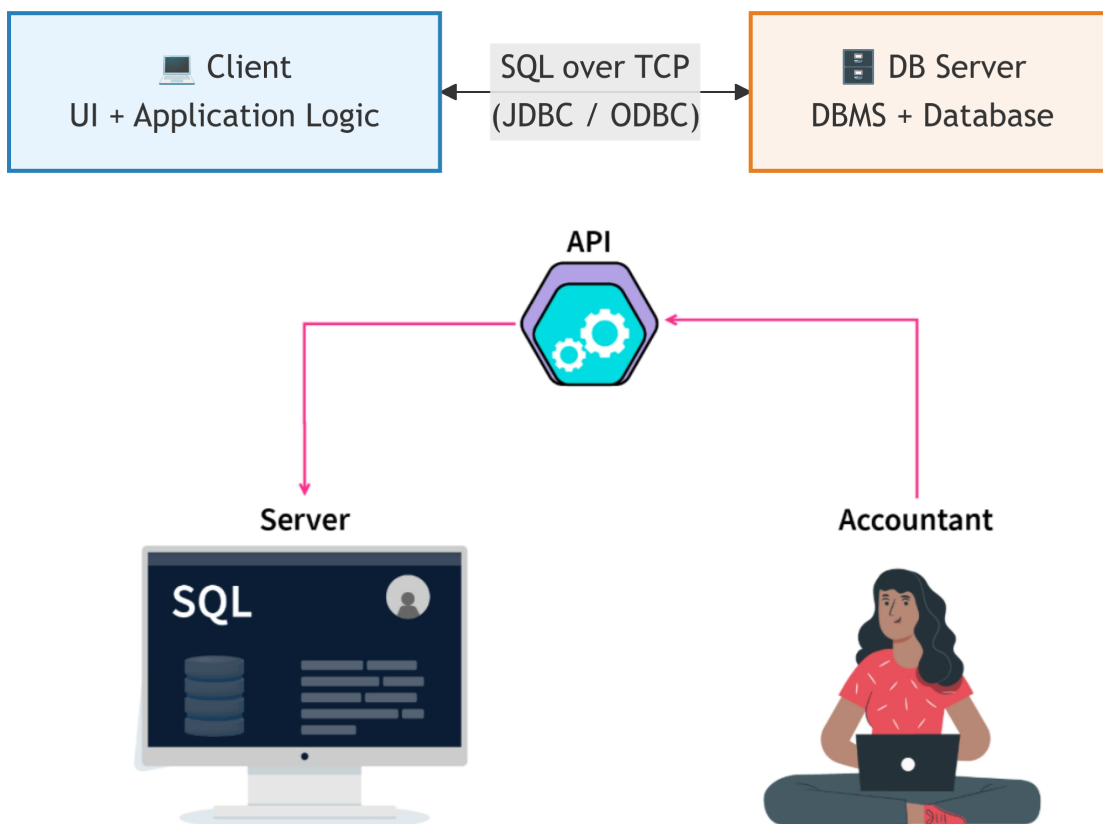


Figure 4: Example of 2-Tier (Client-Server) Architecture

💡 You have used 2-tier too — every time you open MySQL Workbench

When you launch **MySQL Workbench** and click a connection to run a query, you are using a textbook 2-tier architecture.

- **Client (Tier 1)** — `MySQLWorkbench.exe` on your laptop: the query editor, schema inspector, EER diagram designer, and result grid. *All the UI and application logic runs here.*
- **DB Server (Tier 2)** — `mysqld` listening on TCP port **3306**, executing your SQL and sending rows back.

- **The wire between them** — the **MySQL native protocol** (the same role JDBC / ODBC plays for Java / .NET apps).

This stays 2-tier even when Workbench and the server are on the *same* laptop (connecting to `localhost:3306`). What defines the tier is the *process / protocol* separation, not physical location. Connect to a remote server instead (e.g., the lab server `mysql-lab.aaup.edu`) and you simply change the hostname — the architecture is identical.

Rule of thumb

- `mysql` CLI on the same machine as the server → taught as **1-tier** (all-in-one bundled lab setup)
- Workbench `mysqld` over TCP (local or remote) → **2-tier**
- Browser → web server → database → **3-tier** (see next section)

When to use: Departmental applications, internal tools where the developer controls all clients (e.g., a bank teller terminal, a clinic management desktop app).

Advantage	Disadvantage
Multiple concurrent users	Business logic lives on client — hard to update
Server handles query processing	Direct DB exposure is a security risk
Better performance than 1-tier	Server becomes bottleneck as clients grow

3-Tier Architecture

An **application server** (middle tier) is inserted between the client and the DBMS. The client never communicates directly with the database — it only talks to the application server.

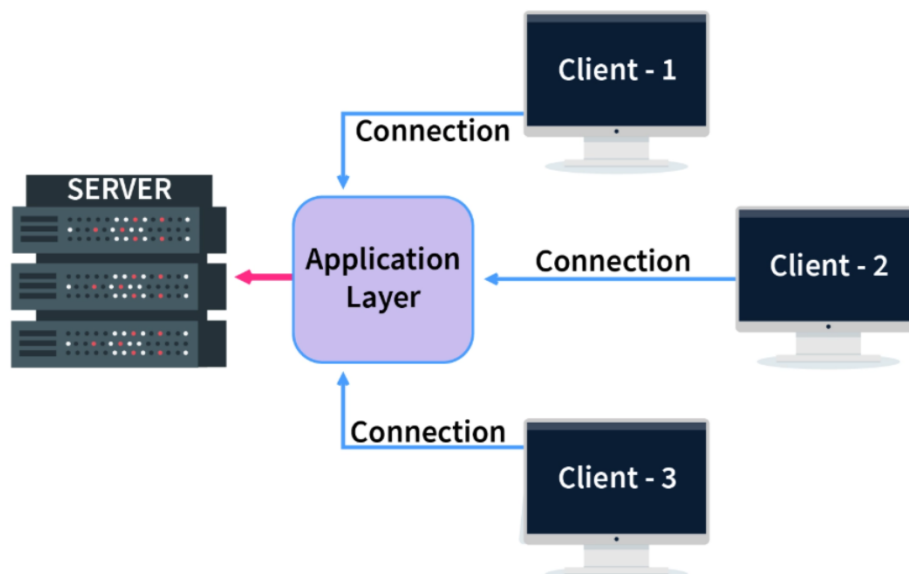
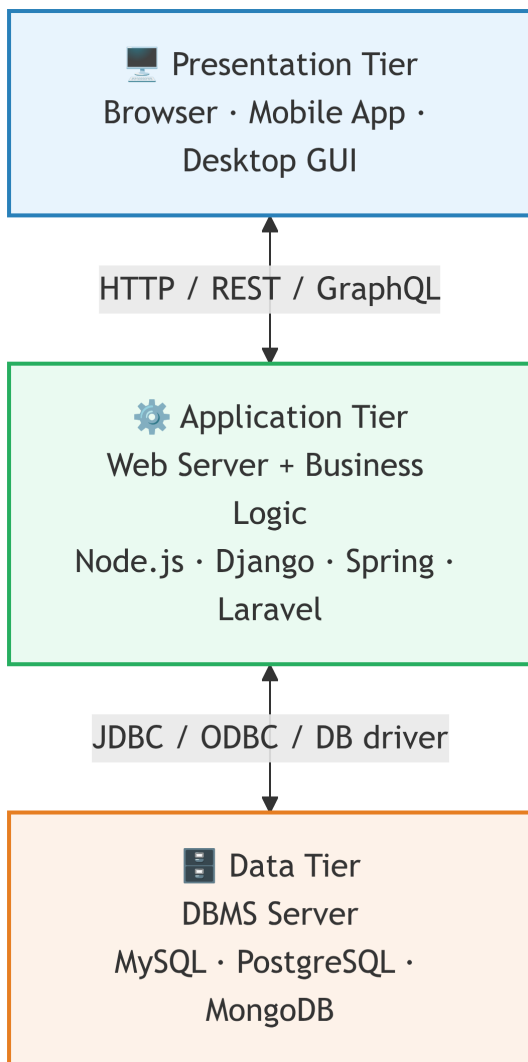


Figure 5: Example of 3-Tier Architecture

 Scenario — AAUP student opens the portal from her laptop

Sara opens Chrome on her laptop in the café and visits `https://portal.aaup.edu` to check her Database Systems grade. What she thinks is “one click” is actually a 3-tier architecture in action:

- **Presentation Tier (Sara’s laptop)** — Chrome renders HTML, CSS, and JavaScript. It knows **nothing** about tables, SQL, or the database. It only speaks **HTTPS** to the portal.
- **Application Tier (AAUP web/app server)** — running somewhere in the Jenin campus data center (e.g., Node.js / Django / Laravel). It:
 1. Receives Sara’s HTTPS request.
 2. Checks her session / JWT token — is she logged in? is she Sara?
 3. Applies business rules (students can see **their own** grades only).
 4. Translates the request into SQL and forwards it to the DBMS.
- **Data Tier (AAUP database server)** — MySQL / PostgreSQL on a protected server, reachable only from the app server, never from the public Internet. It runs:

```
SELECT course_code, grade
FROM Enrollment
WHERE student_id = '202012345';
```

and returns the rows to the app tier, which renders them into the HTML page Sara sees.

Why three tiers instead of two?

Concern	What 3-tier gives AAUP
Security	The database is behind a firewall; Sara’s browser cannot talk to it directly. Even if her laptop is compromised, the attacker cannot run raw SQL.
Scalability	When registration opens and 10,000 students hit the portal at once, AAUP adds more app servers behind a load balancer without touching the DB.
Authorization	“Students see only their rows” is enforced by the app tier — a single trusted place — not by every desktop client.
Maintainability	The portal team can redesign the UI, or swap Laravel for Node.js, without ever touching the database schema.

Now compare this to Sara running `mysql -u sara -p` on her laptop (1-tier) or opening Workbench against the lab server (2-tier). Same data, completely different architectures — each fit for a different purpose.

When to use: Virtually all modern web applications — e-commerce, banking portals, social media, ERP systems.

Concern	How 3-tier addresses it
Security	DBMS is never directly reachable from the Internet
Scalability	App servers can be load-balanced independently of the DB
Maintainability	Business logic changes in the app tier leave the DB untouched
DB portability	Swapping MySQL for PostgreSQL requires changes only in the app tier
Complexity	More layers = harder to debug and deploy

Tier Architecture at a Glance

	1-Tier	2-Tier	3-Tier
Con-current users	1	Dozens–hundreds	Millions
Security	None	Low	High
Scalability	None	Limited	High
Complexity	Minimal	Moderate	High
Typical example	SQLite on a laptop	Bank teller app	Amazon.com

Why Tier Count Matters

Every tier boundary you add provides three benefits and one cost:

What a tier boundary adds	Effect
A network hop	Latency — but also decoupling

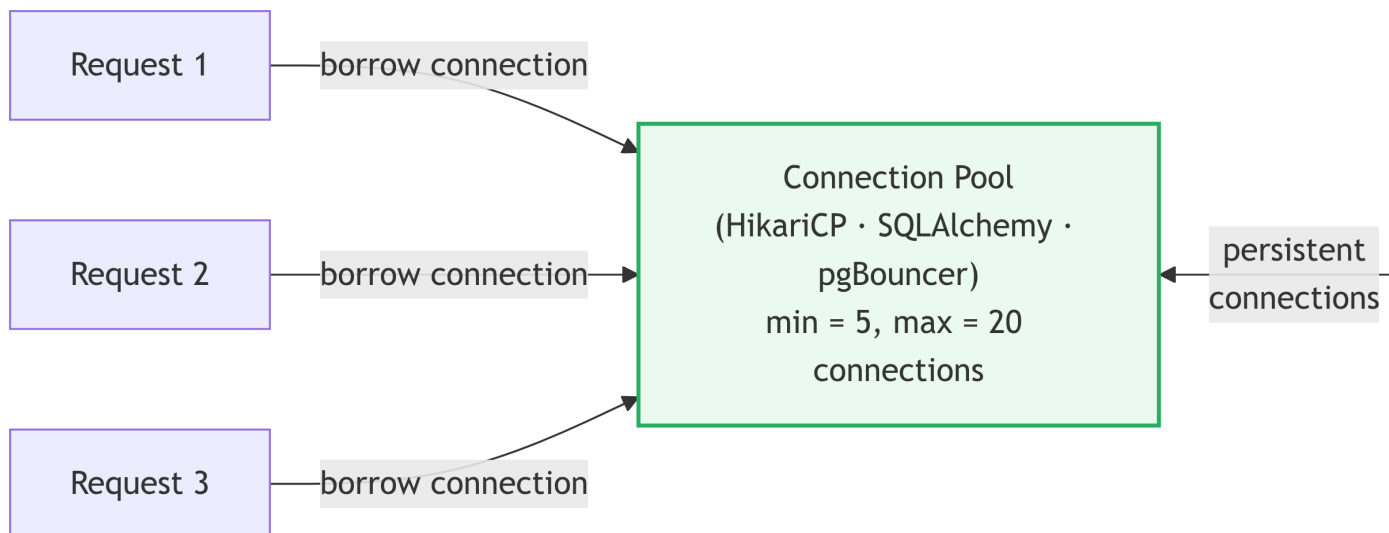
What a tier boundary adds	Effect
An authentication checkpoint	Security layer between Internet and your data
An independent scaling unit	You can scale the app tier without touching the DB
Deployment complexity	More services to deploy, monitor, and debug

This trade-off is why enterprise applications almost universally choose 3-tier despite its complexity: the security and scalability gains far outweigh the operational overhead at scale.

Connection Pooling — A Critical Optimisation for 2-Tier and 3-Tier

Opening a database connection is expensive: it requires a TCP handshake, authentication, and session initialisation. In any multi-user deployment, re-opening a connection for every request would dominate query execution time.

Connection pooling solves this: the application server maintains a **pool** of pre-opened, reusable connections to the DBMS. Incoming requests borrow a connection, use it, and return it.



Without pooling, 1 000 web requests = 1 000 connection open/close cycles — often slower than the queries themselves. Most modern frameworks configure pooling by default.

Beyond 3-Tier: Distributed and Cloud Deployments

Real-world systems at scale often add further tiers or distribute the data tier itself:

Deployment type	Description	Example
Distributed DBMS	Data spread across multiple servers globally; appears to applications as a single DB	Google Spanner, CockroachDB
Federated DB	Separate autonomous databases cooperate via a shared query interface	Legacy enterprise integration
Cloud DB (DBaaS)	Fully managed DBMS — no server to provision, pay per query or storage	AWS RDS, Azure SQL Database, Google Cloud SQL
Serverless DB	Scales to zero when idle; provisions capacity on demand per request	AWS Aurora Serverless, PlanetScale

Tip

Cloud databases are still 3-tier. The cloud provider simply manages the data tier on your behalf. Your application tier (running in containers or VMs) still communicates via JDBC/ODBC to an endpoint that happens to be managed by AWS or Azure instead of a server in your own rack.

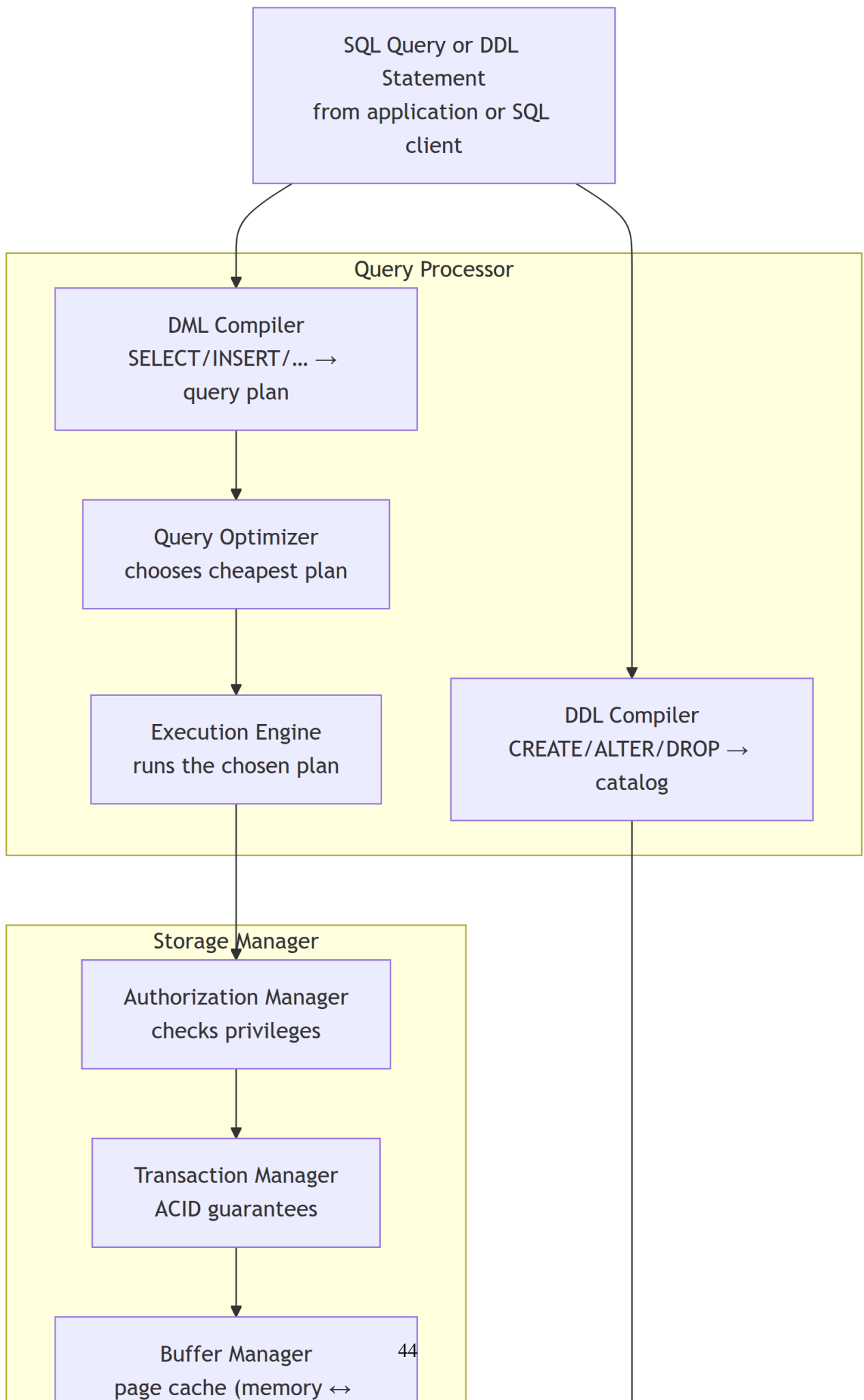
Choosing the Right Architecture

Scenario	Recommended tier
Learning SQL / prototype on a laptop	1-tier (SQLite)
Internal company tool, ≤ 50 users	2-tier
Public web application, any user count	3-tier
Global service, high availability required	3-tier + distributed data tier
No ops team, variable load	3-tier + DBaaS / serverless

Understanding *where* the DBMS runs in the deployment topology clarifies *what* must be inside the DBMS itself — the subject of the next section.

DBMS Component Architecture (Detailed)

A DBMS is a layered system of cooperating subsystems. Every SQL statement you write passes through each layer below:



Key Sub-Components

Component	Input	Output
DDL Compiler	CREATE TABLE ...	Updates system catalog
DML Compiler	SELECT ... FROM ... WHERE ...	Relational algebra expression tree
Query Optimizer	Algebra expression + catalog stats	Chosen execution plan (cheapest cost)
Execution Engine	Execution plan	Tuples / result set
Transaction Manager	BEGIN, COMMIT, ROLLBACK	ACID-compliant state transitions
Buffer Manager	Page requests from execution engine	Pages loaded from disk or found in cache
Authorization Manager	Operation + user identity	Allow or deny

A Single SELECT — The Full Journey

When you run:

```
SELECT Fname, Salary FROM Employee WHERE Dno = 5;
```

this is what happens internally:

1. **Parser:** Checks syntax; verifies that Employee, Fname, Salary, Dno exist in the catalog.
2. **Authorization Manager:** Confirms your account has SELECT privilege on Employee.
3. **DML Compiler:** Translates this to a relational algebra expression: $\pi_{Fname, Salary}(\sigma_{Dno=5}(Employee))$.
4. **Query Optimizer:** Checks catalog statistics. If an index on Dno exists, uses index scan (Chapter 10); otherwise, full table scan.
5. **Execution Engine:** Executes the chosen plan, retrieving pages via the Buffer Manager.
6. **Buffer Manager:** Returns cached pages directly; fetches uncached pages from disk.
7. **Result:** Rows are assembled and returned to the client.

Note

The optimizer's switch between *index scan* and *full table scan* is entirely transparent to you. The same SQL statement produces the same answer regardless of which physical plan is chosen — that is physical data independence in action.

All of these components rely on metadata stored in the system catalog.

The System Catalog (Data Dictionary)

The **system catalog** is a special database-within-the-database that stores metadata about every object in the system. It is the backbone of the query optimizer and the DDL compiler.

What the Catalog Stores

Metadata category	Examples
Schema objects	Table names, column names, data types, NOT NULL constraints
Integrity constraints	Primary keys, foreign keys, unique constraints, check constraints
Index definitions	Which columns are indexed, index type (B+ tree, hash)
User accounts & privileges	Username, roles, granted permissions
Optimizer statistics	Row counts, column cardinalities, histogram data
View definitions	SQL text of each CREATE VIEW statement

Note

In MySQL, the catalog is accessible through the `information_schema` virtual database:

```
-- List all tables in the current schema
SELECT table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'company_db';

-- Inspect columns of Employee
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'Employee'
ORDER BY ordinal_position;
```

Why Catalog Statistics Matter

The query optimizer does not execute every possible plan to find the fastest one — that would take longer than the query itself. Instead, it uses **cost formulas** driven by catalog statistics:

- $|R|$ = number of rows in table R (stored in `table_rows`)
- $|\text{distinct}(R.A)|$ = number of distinct values in column A
- Selectivity of a predicate = fraction of rows that satisfy it

For `WHERE Dno = 5`, if the catalog says `Dno` has 8 distinct values over 30,000 rows, the selectivity is approximately $\frac{1}{8}$ and the estimated result is ~3,750 rows — cheap enough to justify an index scan. Without catalog statistics, the optimizer is effectively blind.

Types of Database Systems

Databases can be classified along three independent axes — the data model they use, where they run, and how many users they serve. A full tour of the data-model axis (relational, document, key-value, wide-column, graph, NewSQL, ...) lives in [Appendix B: A Guided Tour of Database Families](#); here we look at the other two axes, since they directly affect the deployment and internal architecture just covered.

By Deployment Location

Type	Description	When to use
Centralized	One physical site	Small to medium applications
Distributed	Data spread across sites; appears as one DB	High availability, geographic distribution
Federated	Autonomous DBs cooperate via a common interface	Legacy integration
Cloud	Fully managed by a cloud provider (RDS, Cloud SQL)	On-demand scaling, reduced operational cost

By Number of Users

Type	Users	Typical product
Single-user	1 desktop	SQLite
Workgroup	A few dozen	MySQL Community
Enterprise	Thousands concurrent	Oracle, SQL Server Enterprise

Requirements Collection and Analysis

Before a single table can be designed, the analyst must understand exactly what the system must do. Skipping this phase is the single most common cause of costly database redesigns six months into a project.

What to Gather

1. **Entities and attributes:** What real-world objects must be represented? What properties do they have?
2. **Relationships:** How do entities relate to one another? What are the cardinalities?
3. **Functional requirements:** What queries, reports, and updates must the system support?
4. **Constraints and business rules:** What rules must always be true? (e.g., “A student cannot enroll in a course they have already passed.”)
5. **Non-functional requirements:** Performance targets, availability, security classification.

Techniques

Technique	Best for
Structured interviews	Eliciting hidden requirements from domain experts
Reviewing existing documents	Discovering implicit constraints in forms and reports
Observation	Understanding manual workflows to automate
Use-case analysis	Translating processes into data operations
Prototyping	Validating understanding with stakeholders early

Worked Example: University Registration System

After a series of interviews with the Registrar, you capture the following requirements:

- The university offers **courses**, each with a course code, title, and number of credit hours.
- **Students** have a student ID, name, date of birth, and declared major.
- A student may **enroll** in many courses per semester, and each course may have many enrolled students.
- A course may have **prerequisites**: other courses that must be completed first.
- Each course section is **taught by** one faculty member in a given room at a given time.
- A student can only be enrolled in a course once per semester.

From this paragraph, a trained analyst identifies: 4 entities (*Student, Course, Section, Faculty*), at least 3 relationships (*enrolls-in, teaches, has-prerequisite*), and several integrity constraints (*unique enrollment per semester, prerequisite completion*). The conceptual design phase (Chapter 3) translates these into an ER diagram.

Summary

! Chapter 2 — Key Takeaways

- The **database design lifecycle** has six phases: requirements → conceptual → logical → physical → implementation → maintenance. It is iterative, not linear.
 - The three **abstraction levels** (physical, logical, view) hide complexity; the view level is implemented via SQL views and is the primary mechanism for column-level access control.
 - Modern applications use a **three-tier architecture** (presentation → application → data) for security, scalability, and maintainability.
 - The DBMS internally consists of a **query processor** (DDL compiler, DML compiler, optimizer, execution engine) and a **storage manager** (transaction, buffer, file, authorization managers).
 - A single SELECT passes through at least 7 internal steps before a result is returned.
 - The **system catalog** stores all metadata; optimizer statistics in the catalog determine which execution plan is chosen.
-

Review Questions

Conceptual

1. Name the six phases of the database design lifecycle and identify the *deliverable* produced at the end of each phase.
2. Explain in detail what happens inside a DBMS when you execute the query `SELECT * FROM Employee WHERE Salary > 70000`. Name each internal component involved, in order.

Short-Answer / Compare

3. Compare **two-tier** and **three-tier** client-server architectures. Under what circumstances would a team choose two-tier despite its limitations?

4. Explain the difference between **logical data independence** and **physical data independence** using the following scenario: a DBA adds a B+ tree index on the Salary column of Employee. Which type of independence does this change demonstrate? Why?

True / False (with explanation)

5. “The system catalog is separate from the database it describes — it lives in a special OS file that user queries cannot read.” — True or False? Explain.
6. “The query optimizer always chooses the plan with the lowest number of disk reads.” — True or False? Explain what cost model it actually uses.
7. “In a three-tier architecture, the application tier connects directly to the Internet without going through the data tier.” — True or False? Explain the security implication.

Multiple Choice

8. Which DBMS component is responsible for ensuring that two concurrent transactions do not leave the database in an inconsistent state?
 - (a) Buffer Manager
 - (b) Query Optimizer
 - (c) **Transaction Manager**
 - (d) DDL Compiler
9. Which of the following is **not** stored in the system catalog?
 - (a) Column names and data types
 - (b) Number of rows in each table
 - (c) User privilege grants
 - (d) **The actual row data of each table**

Design Exercise

10. You are hired to build a database for a hospital that tracks patients, doctors, wards, and appointments. List **five specific data requirements** you would gather from the hospital staff, and for each, explain whether it is a functional requirement, a data requirement, or a constraint.

SQL / Query Exercise

11. Using `information_schema.columns` in MySQL (syntax shown in §2.5), write a SQL query that lists the column names and data types of the Department table in the `company_db` schema. Does this query access *data* or *metadata*? Explain the difference.

Part II: Data Modeling

Entity-Relationship (ER) Modeling

Chapter 3

The conceptual schema is the key to database design; all else is implementation detail.

—Peter Chen, 1976

Chapter 2 placed ER modeling in the design lifecycle as the **conceptual design** step — the phase where we translate a written problem description into a precise graphical model, before touching any SQL. This chapter teaches you the full vocabulary of ER modeling: what it is, how it is drawn, and how to apply it to a real requirements document.

What Is an ER Diagram?

An **Entity-Relationship (ER) diagram** is a graphical representation of a **conceptual schema** — a high-level, implementation-independent description of the data a system must store.

Conceptual schema: A description of the *structure* of a database at a level that is independent of any specific DBMS, storage format, or programming language. It answers: “*What data exists, and how is it connected?*” — not “*How is it stored?*”

ER modeling was introduced by Peter Chen in 1976. Its core contribution was giving database designers a single, shared language to communicate with both business stakeholders (who understand the problem domain) and software engineers (who will implement the solution).

Why ER First?

If you start with...	Risk
SQL tables directly	Miss relationships; duplicate data; poor constraints

If you start with...	Risk
Code first	Logic drives storage; hard to change later
ER diagram first	Clean conceptual model; easy to review with users; maps to any DBMS

An ER diagram is always drawn **before** writing any SQL. It is then converted to a relational schema using a seven-step mechanical mapping process covered in [Chapter 5 — The Relational Model](#).

ER Notations Overview

An ER diagram is a graphical language, and like any language it comes in different dialects. Three notations are widely used; you will encounter all three in textbooks and professional tools.

Notation	Introduced by	Where you see it
Chen	Peter Chen, 1976	University textbooks, academic exams, this course
Min-Max (Structural Constraints)	Various authors	Formal specifications needing exact minimum and maximum counts
Crow's Foot	Industry convention	Design tools — MySQL Workbench, Lucidchart, draw.io, Visio, ERwin

All three notations express the same concepts — entities, attributes, relationships, cardinality, and participation — using different visual symbols. You do not need to master all three right now. The important thing is to know they exist so you can recognise a diagram regardless of the notation it uses.

i Notation used in this chapter

All diagrams from here through the Worked Example are drawn in **Chen notation**. After the Worked Example, dedicated sections introduce Min-Max and Crow's Foot in detail and show how each notation represents the same Company Database.

Chen Notation — Symbol Reference

Chen notation is the academic standard you will use for all diagrams in this course. The table below is your reference for every symbol.

Concept	Shape	Description
Strong Entity	Single rectangle	An object with independent existence and its own key
Weak Entity	Double rectangle	An object that depends on another entity for identification
Relationship	Single diamond	An association between two or more entities
Identifying Relationship	Double diamond	The relationship that identifies a weak entity
Attribute	Ellipse (oval)	A property of an entity or relationship
Key Attribute	Ellipse with underlined name	Uniquely identifies each entity instance
Multivalued Attribute	Double ellipse	Can hold multiple values per instance
Derived Attribute	Dashed ellipse	Computed from another stored attribute
Composite Attribute	Ellipse → child ellipses	Built from smaller sub-attributes
Total Participation	Double line	Every instance of the entity must participate
Partial Participation	Single line	Some instances may not participate
Cardinality label	1, N, or M on the line	Placed near each entity; shows the maximum

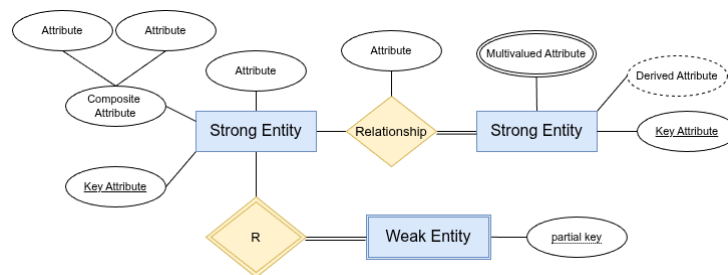


Figure 1: Chen Notation — Shape Reference

Entities

i All diagrams in this chapter use Chen notation

The sections below (Entities, Attributes, Relationships) describe and illustrate concepts using **Chen notation**: single rectangles for strong entities, double rectangles for weak entities, diamonds for relationships, and ovals for attributes. Cardinality is shown with 1, N, or M labels on the lines. If a diagram you encounter elsewhere uses pairs of numbers like $(1, N)$ or crow's foot line endings,

those are Min-Max and Crow's Foot notations — both are covered in full detail after the Worked Example.

Strong Entity

A **strong entity type** (or simply *entity type*) is a real-world object or concept that:

1. Has an **independent existence** — it does not depend on another object to exist in the database.
2. Has a set of **attributes** that describe it.
3. Has at least one attribute (or combination) that **uniquely identifies** each instance — its **key attribute**.

Notation: A **single rectangle** labeled with the entity name (all caps by convention).

Examples: EMPLOYEE, DEPARTMENT, PROJECT, STUDENT, PRODUCT

Entity type vs. Entity instance:

Term	Meaning	Example
Entity type	The template / class	EMPLOYEE
Entity set	All current instances	All 500 employees in the DB
Entity instance	One specific occurrence	Ahmed Ali, SSN=001

Example — STUDENT entity with its attributes in Chen notation:

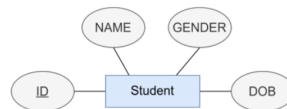


Figure 2: Entity STUDENT with attributes ID (key), NAME, GENDER, and DOB

In this example: Student is the entity (rectangle), ID is the key attribute (underlined), and NAME, GENDER, DOB are simple attributes (ovals connected by lines).

Weak Entity

A **weak entity type** has **no key attribute of its own** — it cannot be uniquely identified without reference to another entity (its **owner**).

Feature	Strong Entity	Weak Entity
Has own key?	Yes	No — only a partial key
Identified by	Own primary key	Owner's key + partial key
Rectangle notation	Single	Double

Feature	Strong Entity	Weak Entity
Identifying relationship	Single diamond	Double diamond
Participation in identifying rel.	May vary	Always total

Partial key: The attribute(s) of a weak entity that, combined with the owner's key, uniquely identify a weak entity instance. Shown with a **dashed underline**.

! Important

Example: DEPENDENT is weak under EMPLOYEE.

- DEPENDENT has a partial key: Name
- Two dependents of *different* employees may share the same name — perfectly fine.
- Two dependents of the *same* employee may **not** share the same name.
- Full identifier: (SSN, Name) — a composite key where SSN comes from the owner.

If an employee is deleted, all their dependents are deleted too (CASCADE). A dependent record cannot exist without an employee.

Example — Hotel (strong) and Room (weak entity) in Chen notation:

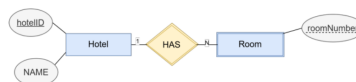


Figure 3: Room is a weak entity identified by Hotel (owner) + roomNumber (partial key, dashed underline). The HAS relationship is the identifying relationship (double diamond in full Chen notation).

- Hotel is the **strong entity** — it has its own key (hotelID, underlined).
- Room is the **weak entity** — roomNumber alone is not unique across all hotels (two hotels can both have room 101). It is identified by the combination (hotelID, roomNumber).
- HAS is the **identifying relationship** (1:N — one hotel has many rooms).

Attributes

Attribute Types

An **attribute** is a property that describes an entity type or relationship type. Quarto supports the following types:

Type	Description	Chen Shape	Example
Simple	Atomic, cannot be divided further	Plain oval	Salary, Gender
Composite	Built from smaller sub-attributes	Oval feeding child ovals	Name → (Fname, Minit, Lname)
Multivalued	Can hold multiple values per entity instance	Double oval	Phone_Numbers, Dept_Locations
Derived	Computed from another stored attribute; not physically stored	Dashed oval	Age derived from Bdate
Stored	Permanently stored; may be a source for derived attributes	Plain oval	Bdate
Key	Uniquely identifies each entity instance	Oval with underlined text	Ssn, Dnumber
Partial key	Identifies weak entity instances within the owner; not globally unique	Oval with dashed underline	Name (of DEPENDENT)
Complex	Both composite and multivalued	Nested double oval	{Address(Street, City, Zip)}
NULL	Value is unknown or not applicable	—	Super_ssn for a top-level manager

Simple (Atomic) Attribute

A **simple** attribute cannot be divided into smaller meaningful parts. It holds a single indivisible value per entity instance. All attributes in the example below — Id, name, price, and quantity — are simple; each stores one value that has no sub-components.

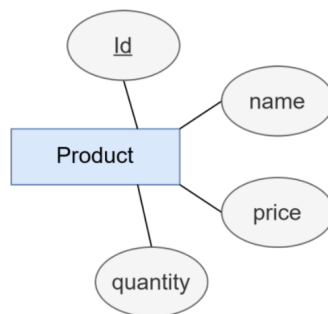


Figure 4: Simple attributes — Product entity: every oval holds one indivisible value

Composite Attribute

A **composite** attribute is made up of smaller, meaningful sub-attributes. The sub-attributes are shown as child ovals branching from the parent oval. In the example, FullName breaks down into FirstName and

LastName; Address breaks down into Street, City, and State.

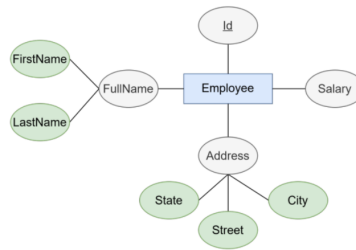


Figure 5: Composite attributes — Employee FullName and Address each decompose into sub-attributes (shown in green)

Single-Valued Attribute

A **single-valued** attribute holds exactly one value for each entity instance. Most attributes are single-valued — for example, every employee has exactly one FullName and one Address. This contrasts with multi-valued attributes that can hold several values at once.

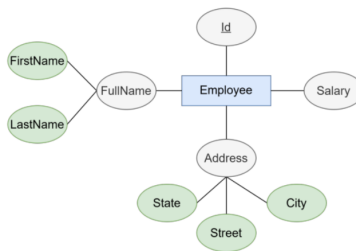


Figure 6: Single-valued attributes — each Employee has exactly one FullName and one Address

Multi-Valued Attribute

A **multi-valued** attribute can hold more than one value for the same entity instance. It is drawn as a **double oval**. In the example, an employee can have multiple Addresses and multiple Skills.

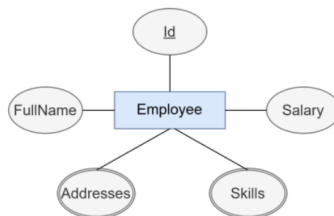


Figure 7: Multi-valued attributes — Addresses and Skills drawn as double ovals because one employee may have several

Derived Attribute

A **derived** attribute can be computed from another stored attribute; it is drawn as a **dashed oval**. Because the value can always be recalculated, it is not physically stored in the database. In the example, Age

(dashed oval) is derived from the stored DOB attribute.

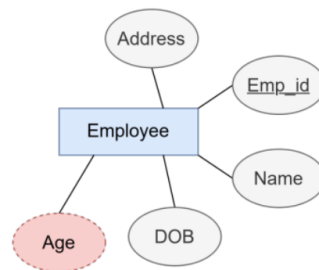


Figure 8: Derived attribute — Age (dashed oval) is computed from the stored DOB attribute

Key Attribute

A **key** attribute uniquely identifies every instance of an entity type. It is drawn as an oval with its name **underlined**. In the example, hotelID is underlined, meaning no two Hotel records share the same hotelID.



Figure 9: Key attribute — hotelID (underlined oval) uniquely identifies each Hotel entity

💡 Tip

When is something an attribute vs. an entity?

Ask: “Do I need to store multiple facts *about* this, or just one value?” - If multiple facts → it is an entity. - If one value → it is an attribute.

Example: Gender → one value per employee → **attribute**. Example: Department → has a name, number, location, manager → **entity**.

Keys in ER Diagrams

Before moving to relationships, it is essential to understand the **key rule** — the single most important constraint in ER design:

! The ER Key Rule

Every strong entity type must have exactly one key attribute (or one composite key attribute group) that uniquely identifies each instance. You cannot draw a valid ER diagram for a strong entity without specifying its key.

Weak entities do not have their own key — instead they have a **partial key** that, combined with the owner entity's key, forms a composite identifier.

Key Attribute (Strong Entity)

A **key attribute** is drawn as an oval with its name **underlined**. It guarantees that no two instances of the entity have the same value for that attribute.

- One entity can have **only one key attribute** (or one composite key group) at the ER level.
- If two candidate identifiers exist (e.g., both SSN and Email are unique), you underline **one** in the ER diagram — the one you choose as the primary identifier. The other becomes an alternate key at the relational level (Chapter 5).
- The key attribute in the ER diagram maps directly to the **primary key** column(s) when the entity becomes a relational table.

ER concept	What it means	Maps to (Ch 5)
Key attribute (underlined)	Unique identifier for the entity	Primary key
Composite key attribute	Two+ attributes together form the key	Composite primary key

Partial Key (Weak Entity)

A **partial key** is drawn with a **dashed underline**. It is *not* globally unique — it is only unique within the set of instances owned by the same owner entity.

- The full identifier of a weak entity instance is: **owner's primary key + partial key**.
- This combination maps to a **composite primary key** in the relational table (Chapter 5, Step 2 of the mapping).

Example — DEPENDENT under EMPLOYEE:

EMPLOYEE.SSN	DEPENDENT.Name	Unique?
001	Ali	unique as (001, Ali)
002	Ali	different owner — allowed
001	Ali	duplicate — not allowed

i What about Foreign Keys?

Foreign keys do **not** appear in ER diagrams. They are a relational concept — a column in one table that references the primary key of another table. Foreign keys are introduced in Chapter 5 and created automatically by the ER-to-relational mapping rules.

In an ER diagram, a **relationship line** plays the role that a foreign key plays in a relational table. When the mapping converts a 1:N relationship to SQL, it adds a foreign key column. You do not draw it; it emerges from the mapping.

Relationships

A **relationship type** defines an association among two or more entity types. A **relationship instance** is one specific pairing of entity instances.

Notation: A **single diamond** labeled with the relationship name (verb or verb phrase).

Degree of a Relationship

Degree	Name	Example
1	Unary (recursive)	EMPLOYEE SUPERVISION EMPLOYEE
2	Binary	EMPLOYEE WORKS_FOR DEPARTMENT
3	Ternary	EMPLOYEE Supplies PROJECT with PART

Most real-world relationships are **binary**. Before using a ternary relationship, ask whether it can be decomposed into two binary ones without losing information.

Cardinality Ratios

For a binary relationship, the **cardinality ratio** specifies how many instances of one entity can be related to one instance of the other. In Chen notation, the label (1, N, or M) is written on the relationship line **near the entity it constrains**.

One-to-One (1:1)

Each instance of entity A is associated with **at most one** instance of entity B, and vice versa.

- Each person has **at most one** passport — not everyone has a passport.
- Each passport belongs to **exactly one** person — every passport has an owner.

Reading: “A person may have zero or one passport; a passport must be owned by exactly one person.”

Entity-Relationship (ER) Modeling

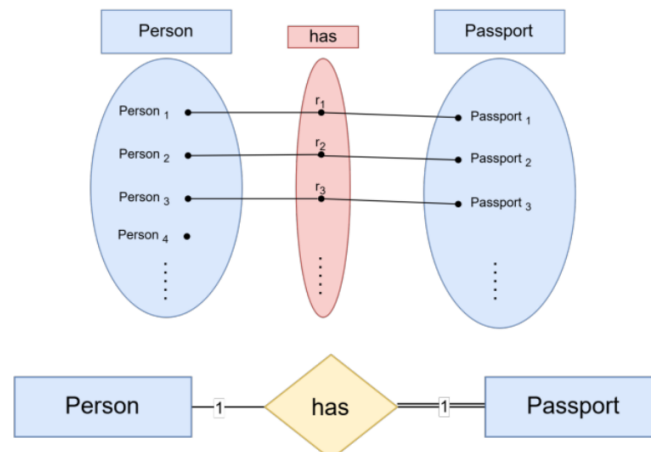


Figure 10: 1:1 relationship — each Person has at most one Passport; each Passport belongs to exactly one Person

One-to-Many (1:N)

One instance of entity A is associated with **many** instances of entity B, but each B instance is associated with at most one A.

- A department can exist **without** employees (e.g., a newly created department with no staff yet) → **partial** participation of Department in the relationship.
- An employee **must** be assigned to a department → **total** participation of Employee in the relationship.

Left → Right (Department → Employee): One department employs zero or more employees.

Right → Left (Employee → Department): Each employee works for exactly one department.

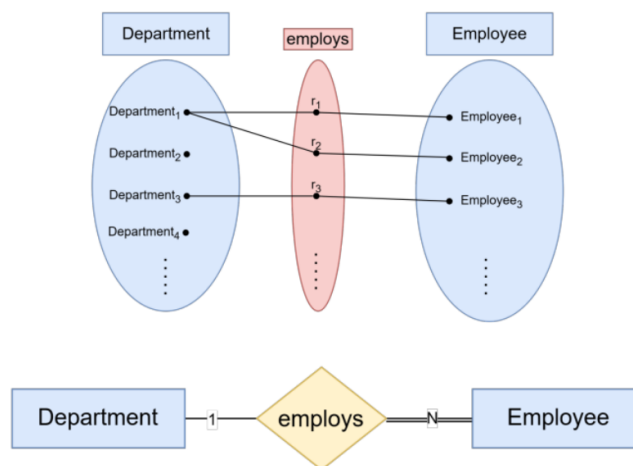


Figure 11: 1:N relationship — one Department employs many Employees; each Employee belongs to exactly one Department

Many-to-Many (M:N)

Each instance of entity A can be associated with **many** instances of entity B, and each B can be associated with many A's.

Left → Right (Student → Course): A student may enroll in zero or more courses.

Right → Left (Course → Student): A course may have zero or more students.

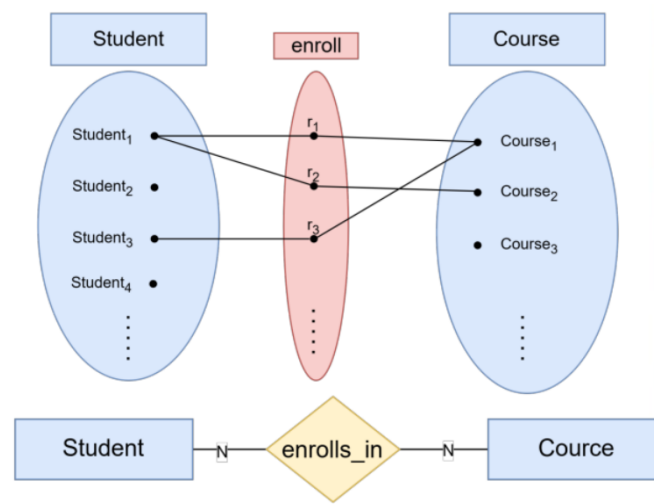


Figure 12: M:N relationship — many Students enroll in many Courses; many Courses have many Students

Participation Constraints

Participation constraints specify whether **every** instance of an entity must participate in a relationship, or whether participation is optional.

Line type	Name	Meaning
Single line –	Partial participation	Some instances may not participate
Double line =	Total participation	<i>Every</i> instance must participate

Example — Department managedBy Employee:

- **Department** side → **double line (total)**: every department *must* be managed by exactly one employee.
- **Employee** side → **single line (partial)**: not every employee is a manager — an employee *may* manage a department, but doesn't have to.



Figure 13: Participation constraints — Department (total, double line) must have a manager; Employee (partial, single line) may or may not be a manager

i Note

Total participation means the entity **cannot exist** in the database without being connected to this relationship. A department record cannot be inserted unless it has a managing employee assigned to it.

Relationship Attributes

Some relationships carry their own data — a value that belongs to the **pairing** of two entities, not to either entity alone.

! Important

Hours in *WORKS_ON* is a **relationship attribute**: it is the number of hours *this employee* spends on *this project*.

- Storing it with EMPLOYEE is wrong: hours on which project?
- Storing it with PROJECT is wrong: hours by which employee?
- It belongs to the *combination* (Employee, Project) — i.e., the relationship.

In Chen notation, relationship attributes are drawn as ovals attached to the diamond.

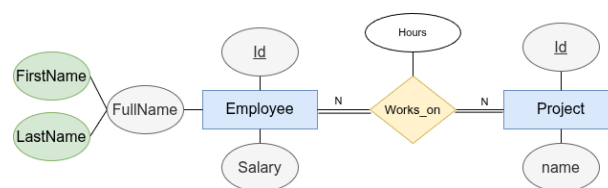


Figure 14: WORKS_ON relationship attribute — Hours is attached to the WORKS_ON diamond because it belongs to the (Employee, Project) pairing, not to either entity alone

Recursive (Unary) Relationships

A relationship where the **same entity type** participates more than once, in different **roles**.

The diagram below shows two versions of the same idea — the top one is **wrong**, the bottom one is **correct**:

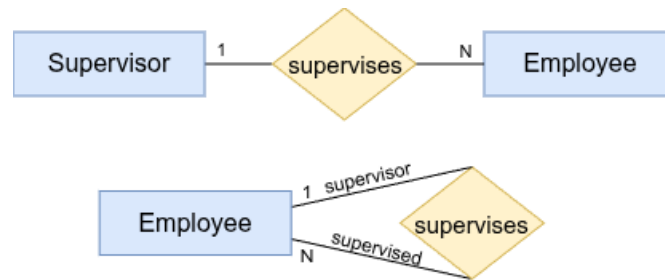


Figure 15: Recursive relationship — top diagram (wrong): Supervisor drawn as a separate entity; bottom diagram (correct): Supervisor is a role of Employee, shown as a unary relationship back to the same entity

Why the first diagram is wrong: Supervisor is not a different kind of thing from Employee — a supervisor *is* an employee who happens to manage others. Drawing two separate entity boxes implies two different tables in the database, which is incorrect.

Why the second diagram is correct: A single Employee entity with a **recursive** SUPERVISION relationship back to itself, with two labelled roles: Supervisor (1 side) and Supervisee (N side). One entity maps to one table — the role is just a label on the line.

Why both sides are partial (single line):

Role	Min	Reason
Supervisor (1 side)	0	Not every employee supervises others — a regular worker never plays the supervisor role
Supervisee (N side)	0	Not every employee has a supervisor — the top-level manager has no one above them

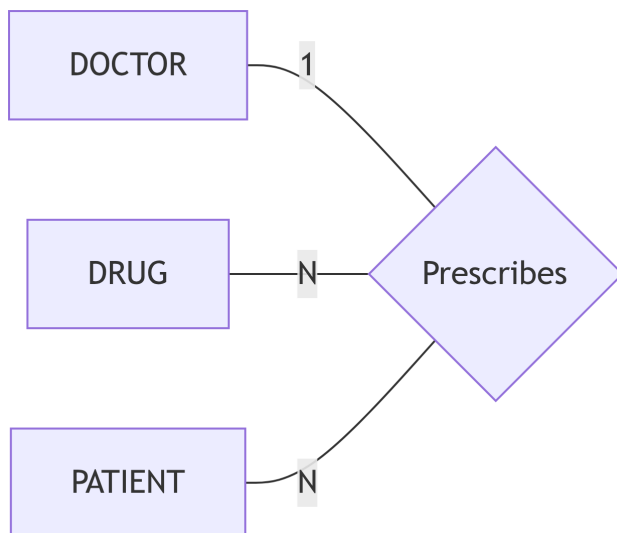
Both sides are $(0, N)$ and $(0, 1)$ respectively — **partial participation**, shown as single lines on both ends.

Ternary Relationships

Involve three entity types simultaneously. A SUPPLIER **supplies** a PART to a PROJECT — no pair alone captures the full meaning.

Where the 1 / N Labels Go

In a ternary relationship there is one diamond connected to **three** entities by three lines. A cardinality label (1 or N) is placed on each line **at the entity end of the line — close to the entity box itself, far from the diamond**, exactly like binary relationships.



Ternary relationship — a DOCTOR prescribes a DRUG for a PATIENT. Label near each entity answers: ‘given one specific pair of the other two, how many of me?’

How to Read the Labels — the “Fix Two, Count the Third” Rule

For each entity, mentally fix one specific instance of the other two and ask: “How many instances of this entity can participate?” The label answers that question.

Label	Fixed	Question	Answer
1 near DOCTOR	a specific (Drug, Patient) pair	How many doctors prescribed <i>this</i> drug to <i>this</i> patient?	1 — exactly one doctor per prescription
N near DRUG	a specific (Doctor, Patient) pair	How many drugs does <i>this</i> doctor prescribe to <i>this</i> patient?	N — many drugs possible
N near PATIENT	a specific (Doctor, Drug) pair	How many patients receive <i>this</i> drug from <i>this</i> doctor?	N — many patients

Full reading of the diagram: “Each (Drug, Patient) combination is handled by exactly one Doctor; each (Doctor, Patient) pair can involve many Drugs; each (Doctor, Drug) pair can be prescribed to many Patients.”

💡 Tip

Quick rule: The 1 label means “at most one” when you fix the other two. The N label means “potentially many.”

i Note

Before using a ternary relationship, verify that the three-way combination is truly needed. If any pair of the three entities fully determines the third, a ternary can usually be replaced by two binary relationships.

Worked Example: The Company Database

This section builds the **Company Database ER diagram** incrementally — one requirement at a time. For each step, the relevant sentence from the requirements is highlighted, followed by the updated diagram and an explicit note on any assumption made.

Requirements

*“The company has several **departments**. Each department has a unique **name** and a **number**. A department is managed by exactly one **employee**, who is called the **manager**. The manager has a **start date** for their managerial role. A department may have several **locations**.*

*Each employee has a unique **social security number (SSN)**, a **first name**, **middle initial**, **last name**, **date of birth**, **address**, **gender**, and **salary**. An employee’s **age** can be derived from their date of birth and does not need to be stored separately. Each employee works in exactly one department but may work on several **projects**. We track the **number of hours** per week each employee works on each project. Each employee may be supervised by at most one other employee.*

*Each project has a unique **name**, a **number**, and a single **location**. Each project is controlled by exactly one department.*

*Employees may have **dependents**. Each dependent has a **name**, **gender**, **date of birth**, and **relationship** to the employee.”*

Step 1 — EMPLOYEE Entity

*“Each employee has a unique **social security number (SSN)**, a **first name**, **middle initial**, **last name**, **date of birth**, **address**, **gender**, and **salary**. An employee’s **age** can be derived from their date of birth and does not need to be stored separately.”*

We start with the most described entity: EMPLOYEE.

- SSN → **key attribute** (underlined oval) — uniquely identifies each employee
- Name → **composite attribute** — decomposes into Fname, Minit, Lname
- Bdate → stored attribute — the value is physically kept in the database
- Age → **derived attribute** (dashed oval) — computed from Bdate; never stored
- Address, Gender, Salary → simple, single-valued attributes

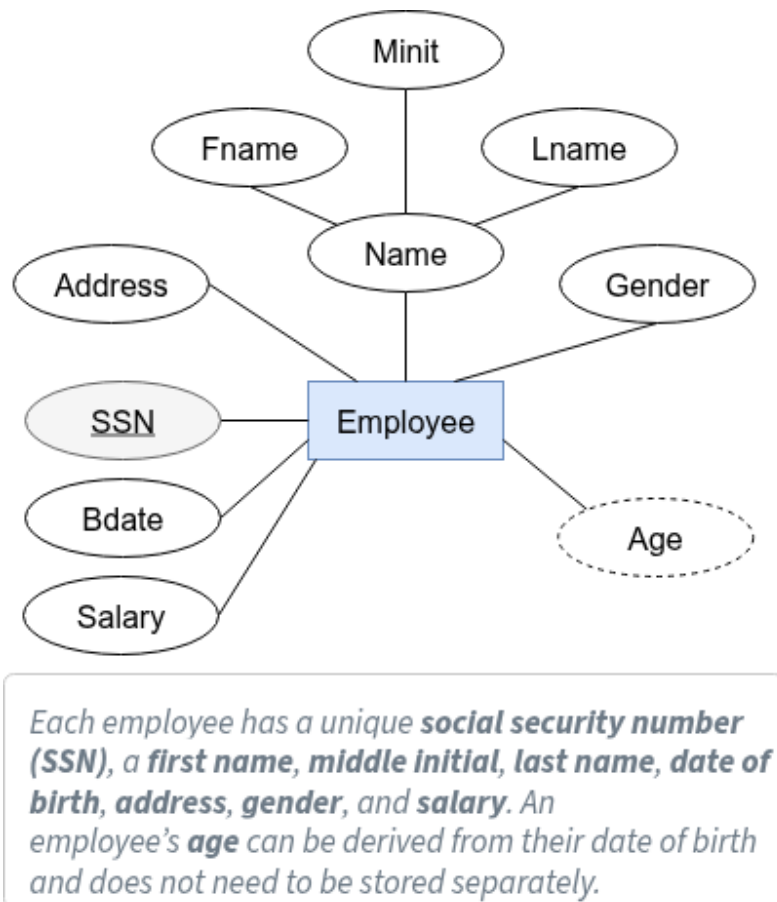


Figure 16: Step 1 — EMPLOYEE entity with all its attributes

Step 2 — DEPARTMENT Entity + MANAGES Relationship

*“The company has several **departments**. Each department has a unique **name** and a **number**. A department may have several **locations**. A department is managed by **exactly one** employee, who is called the **manager**. The manager has a **start date** for their managerial role.”*

- DEPARTMENT is a strong entity; Number is its key attribute
- Locations → **multivalued attribute** (double oval) — “may have several”
- Start_date → **relationship attribute** on MANAGES — it belongs to the (Employee, Department) pairing, not to either entity alone

- **Department side of MANAGES:** “*exactly one*” → **total** participation, max = 1 (double line, label 1)
- **Employee side of MANAGES:** The requirement does not say every employee is a manager. “Manager” is a *role* played by some employees, not a separate entity type.

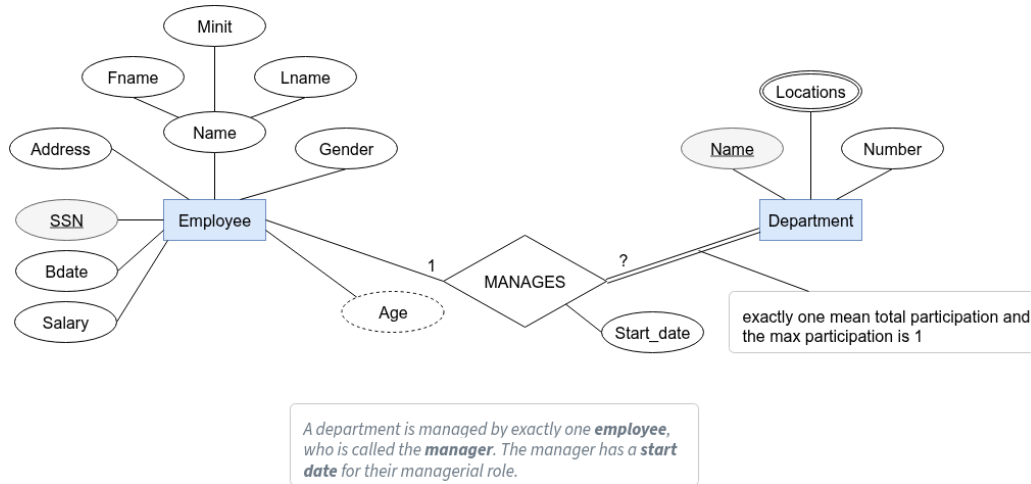


Figure 17: Step 2a — DEPARTMENT added with MANAGES. The “?” marks the open question on the Employee side.

i Assumption — MANAGES (Employee side)

Stated: A department must have exactly one manager → Department side is **total**, max = 1.

Reasoning from requirements: “Manager” is a role — not every employee manages a department → Employee side is **partial**.

Assumption: An employee can manage at most one department at a time → max = 1.

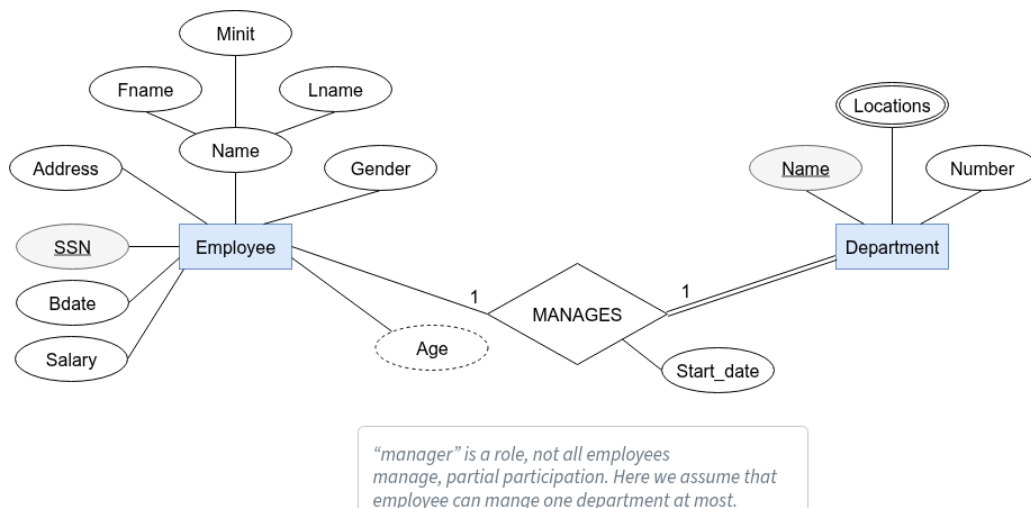


Figure 18: Step 2b — Resolved: Employee side of MANAGES is partial (single line), max = 1

Step 3 — WORKS_FOR Relationship

*“Each employee works in **exactly one** department.”*

WORKS_FOR connects EMPLOYEE and DEPARTMENT with cardinality N:1.

- **Employee side:** “*exactly one*” → **total** participation (double line), max = 1
- **Department side:** The requirement only states where employees go; it says nothing about whether a department must have employees.

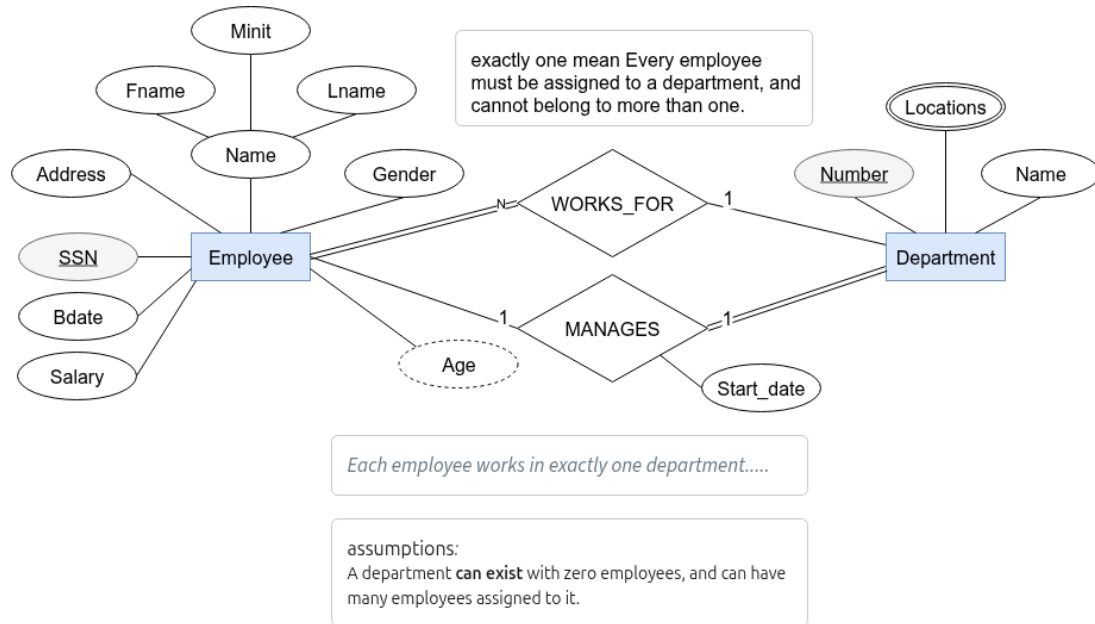


Figure 19: Step 3 — WORKS_FOR added. Employee side is total (double line, N); Department side resolved below.

i Assumption — WORKS_FOR (Department side)

Stated: Every employee must belong to exactly one department → Employee side is **total**.

Assumption: The diagram shows a **single line** on the Department side, meaning a department can exist with zero employees assigned to it → Department side is **partial**, min = 0, max = N.

This is a design choice. You could require every department to have at least one employee by making their side total (double line).

Step 4 — PROJECT Entity

*“Each project has a unique **name**, a **number**, and a single **location**.”*

PROJECT is a strong entity with three attributes.

- Number → **key attribute** (underlined oval)
- Name → simple attribute (unique per requirements, but Number is the diagrammatic key)
- Location → simple, single-valued attribute — “*a single location*” confirms no double oval needed

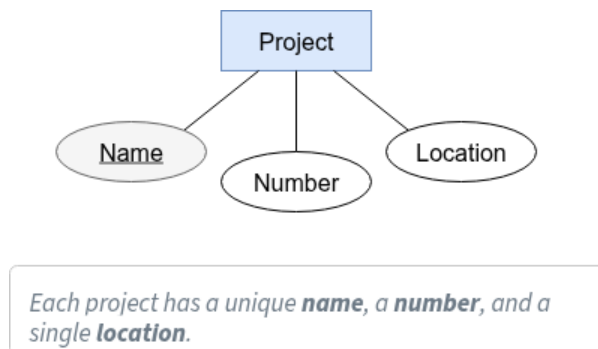


Figure 20: Step 4 — PROJECT entity with its three attributes

Step 5 — CONTROLS Relationship

“Each project is controlled by **exactly one** department.”

CONTROLS connects PROJECT and DEPARTMENT with cardinality N:1.

- **Project side:** “*exactly one*” → **total** participation (double line), max = 1
- **Department side:** The requirement says nothing about whether a department must control any projects.

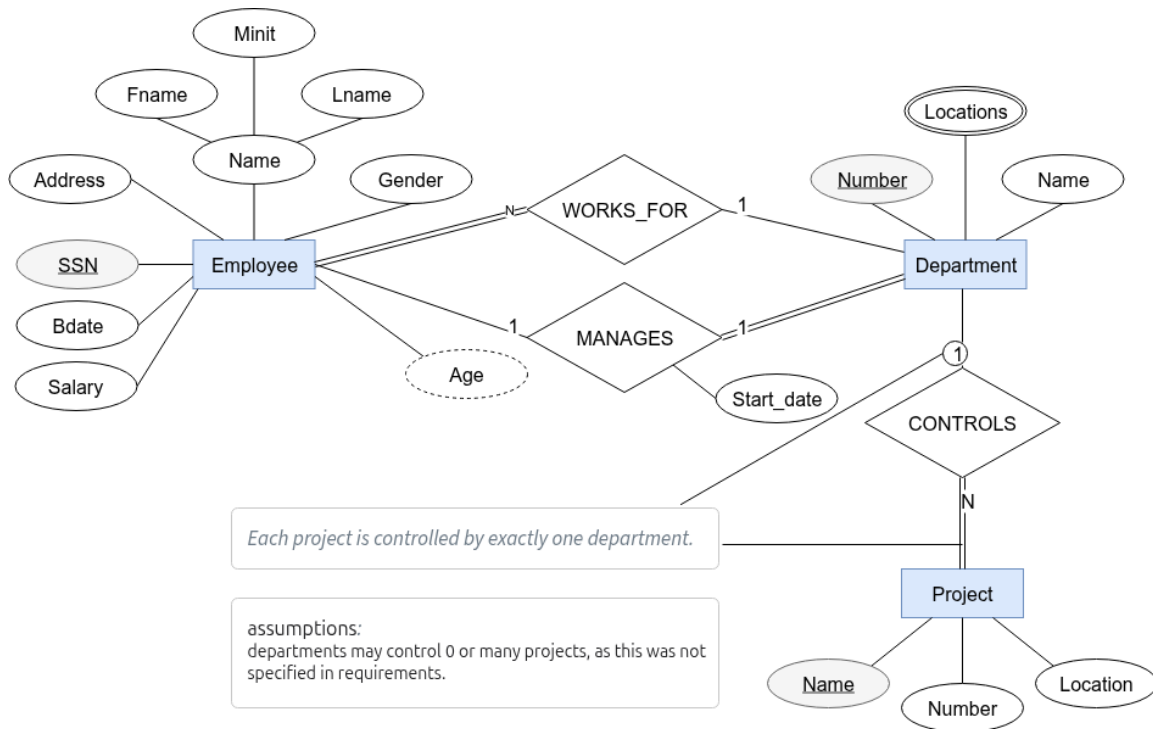


Figure 21: Step 5 — CONTROLS added. Project side is total (double line, 1); Department side resolved below.

i Assumption — CONTROLS (Department side)

Stated: Every project is controlled by exactly one department → Project side is **total**.

Assumption: A department may control zero or many projects → Department side is **partial**, min = 0, max = N.

The requirements do not require every department to control a project.

Step 6 — WORKS_ON Relationship

*“Each employee **may** work on several **projects**. We track the **number of hours** per week each employee works on each project.”*

WORKS_ON connects EMPLOYEE and PROJECT with cardinality M:N.

- Hours → **relationship attribute** on WORKS_ON — it belongs to the (Employee, Project) pairing, not to either entity alone
- **Employee side:** The word “**may**” explicitly makes this **partial** (min = 0), max = N
- **Project side:** Not stated — see assumption below.

Entity-Relationship (ER) Modeling

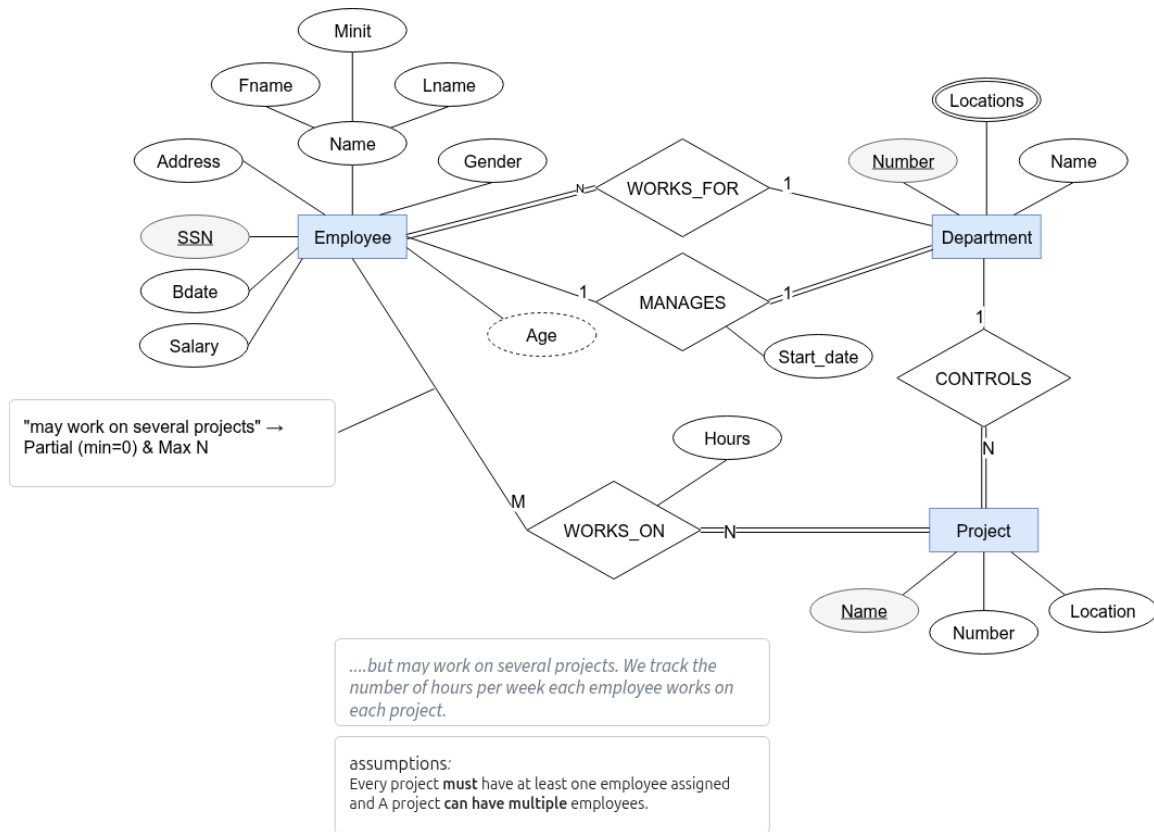


Figure 22: Step 6 — WORKS_ON added with Hours as a relationship attribute. Employee side is partial (single line, M).

i Assumption — WORKS_ON (Project side)

Stated: “May work on” → Employee participation is **partial** — directly from requirements.

Assumption: A project that has no employees assigned is not yet operational → Project side is **total**, min = 1, max = N.

This assumption must be confirmed with stakeholders — it is not stated in the text.

Step 7 — SUPERVISION Relationship

“Each employee **may** be supervised by **at most one** other employee.”

SUPERVISION is a **recursive (unary)** relationship — EMPLOYEE related to itself, with two roles: **Supervisor** and **Supervisee**.

- **Supervisee side:** “may be supervised” → **partial** (min = 0); “at most one” → max = 1
- **Supervisor side:** Not stated — see assumption below.

Entity-Relationship (ER) Modeling

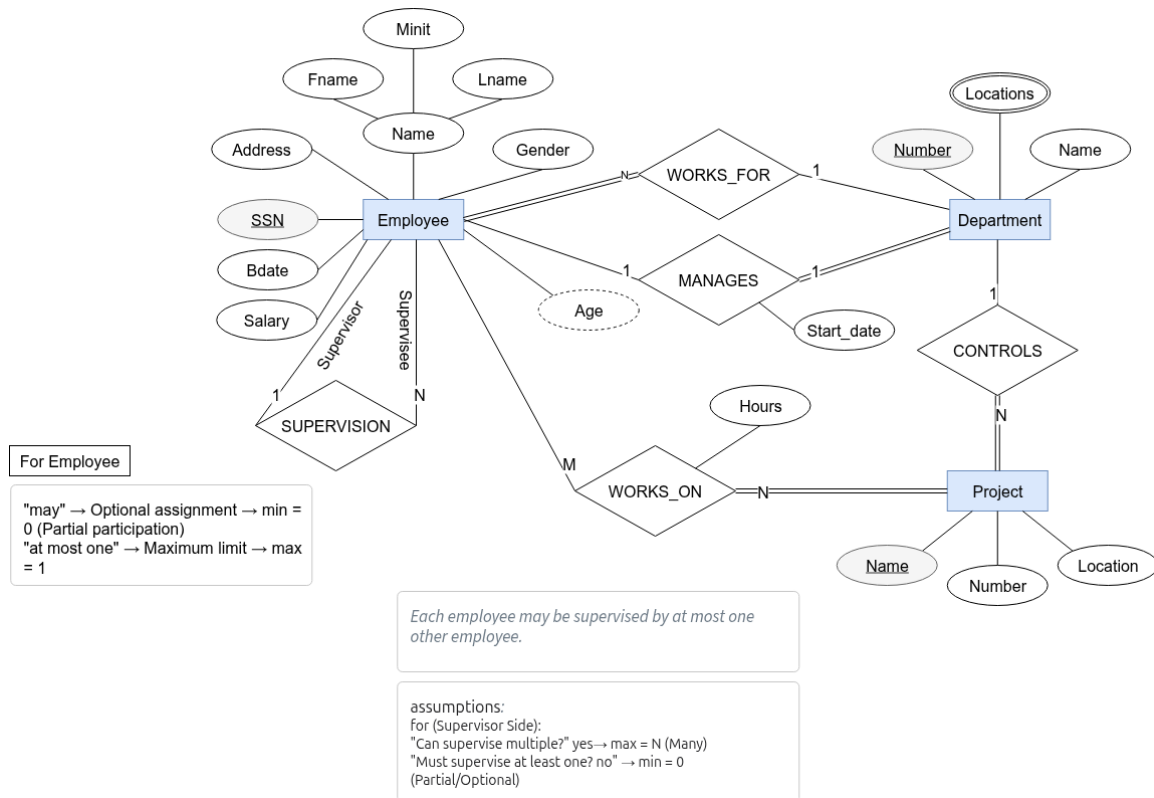


Figure 23: Step 7 — SUPERVISION recursive relationship added. Supervisee: partial, max = 1.

i Assumption — SUPERVISION (Supervisor side)

Stated: Each employee *may* be supervised by at most one → Supervisee side is **partial, max = 1**.

Assumption: Not every employee supervises anyone (e.g., a regular worker has no one under them; the top-level manager also has no supervisor) → Supervisor side is **partial**, min = 0, max = N.

Step 8 — DEPENDENT Weak Entity + HAS (Full Diagram)

*“Employees **may** have **dependents**. Each dependent has a **name**, **gender**, **date of birth**, and **relationship** to the employee.”*

DEPENDENT is a **weak entity** — it has no key of its own and cannot exist without its owning employee.

- Name → **partial key** (dashed-underline oval) — unique within one employee’s dependents, not globally
- Gender, Bdate, Relationship → simple attributes
- HAS is the **identifying relationship** (double diamond) — the combination (SSN, Name) uniquely identifies each dependent
- **Employee side:** “*may have*” → **partial** (min = 0, max = N)

- **Dependent side:** A dependent cannot exist without an employee → **total** (min = 1) — this follows structurally from the weak entity definition, not from an assumption

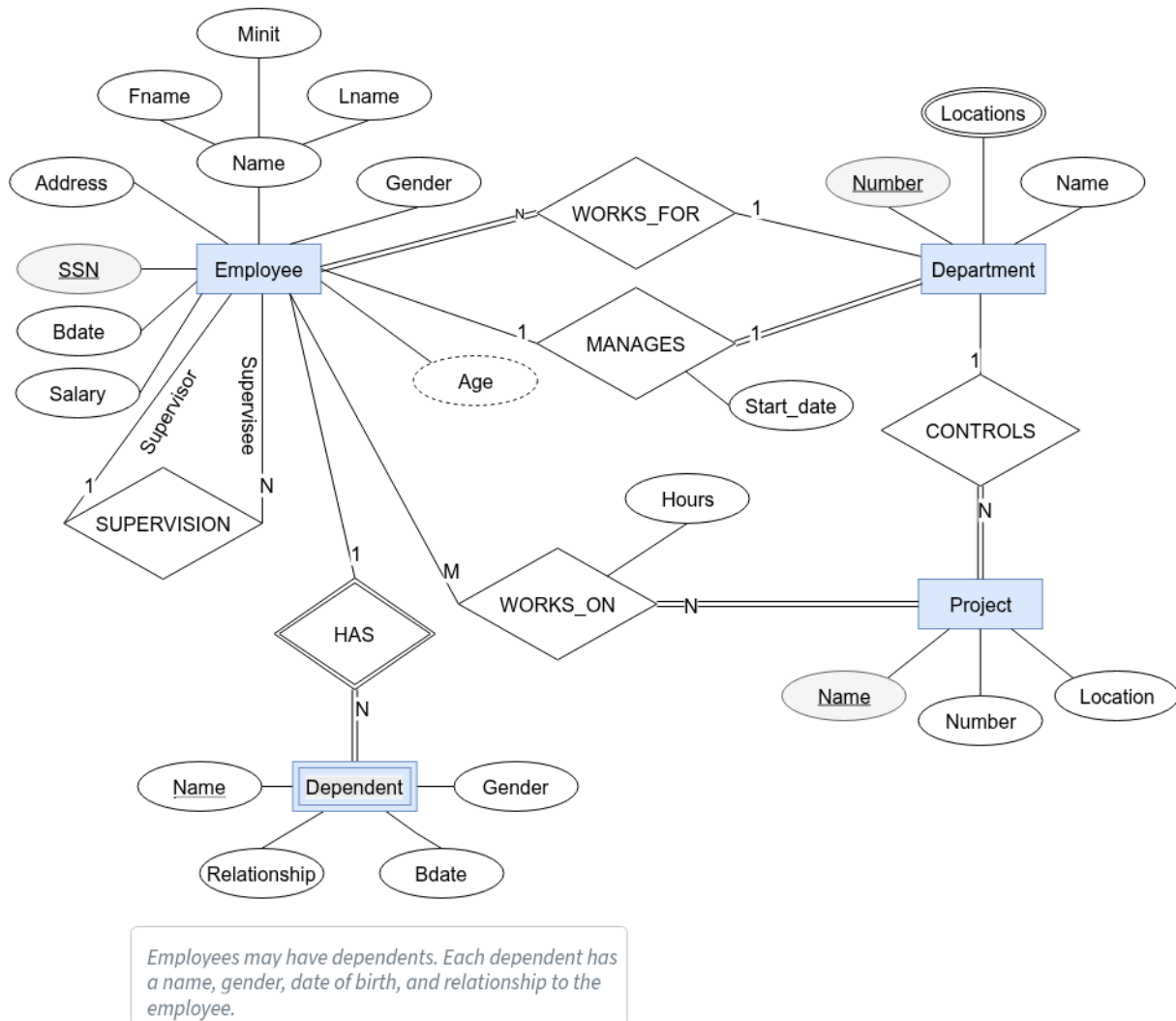


Figure 24: Full Diagram — All entities, attributes, and relationships complete

Summary

Step	Added	Key requirement	Assumption made
1	EMPLOYEE + attributes	SSN key; composite Name; derived Age from Bdate	—
2	DEPARTMENT + MANAGES (1:1)	Dept must have exactly one manager → Dept: total, max = 1	Employee side: partial, max = 1 (manager is a role)

Step	Added	Key requirement	Assumption made
3	WORKS_FOR (N:1)	“ <i>exactly one department</i> ” → Employee: total, max = 1	Dept side: partial (a dept can exist with zero employees)
4	PROJECT + attributes	Number key; single Location	—
5	CONTROLS (N:1)	“ <i>exactly one department</i> ” → Project: total, max = 1	Dept side: partial (a dept may control no projects)
6	WORKS_ON (M:N) + Hours	“ <i>may work on</i> ” → Employee: partial , min = 0	Project side: total (a project must have staff)
7	SUPERVI- SION (recursive 1:N)	“ <i>may ... at most one</i> ” → Supervisee: partial, max = 1	Supervisor side: partial (not all employees supervise)
8	DEPENDENT (weak) + HAS (1:N)	“ <i>may have dependents</i> ” → Employee: partial	Dependent total is structural (weak entity rule)

! Stated vs. Assumed Constraints — Quick Reference

Directly from the requirements text:

- Employee **total** in WORKS_FOR — “*works in exactly one department*”
- Department **total** in MANAGES — “*managed by exactly one employee*”
- Project **total** in CONTROLS — “*controlled by exactly one department*”
- Employee **partial** in MANAGES — “manager” is a role, not all employees manage
- Employee **partial** in WORKS_ON — “*may work on*”
- Employee **partial** in HAS — “*may have dependents*”
- Supervisee **partial** in SUPERVISION — “*may be supervised by at most one*”
- Dependent **total** in HAS — structural (weak entity cannot exist without owner)

Design assumptions (reasonable operational defaults, not stated in text):

- Department **partial** in WORKS_FOR — a department can exist with zero employees (diagram shows single line)
- Department **partial** in CONTROLS — a department need not control any project
- Project **total** in WORKS_ON — a project without any assigned staff is not operational
- Supervisor **partial** in SUPERVISION — not every employee supervises anyone

Min-Max Notation

Chen notation tells us *whether* a participation is total or partial and *what* the cardinality ratio is (1:1, 1:N, M:N), but it stops short of answering a frequent business question: *exactly how many?* “At most three active orders,” “between two and five team members,” “exactly one department chair” — these rules all collapse into Chen’s coarse labels. **Min-Max notation** was introduced precisely to close that gap: it keeps the same diagram shape but replaces each line-end label with a numeric pair (min, max) that pins down both bounds explicitly.

Min-Max notation (also called *structural constraints notation*) extends the basic Chen diagram by annotating each entity’s side of a relationship line with a pair of numbers: (min, max) .

What Do min and max Mean?

- **min** — the minimum number of relationship instances this entity *must* participate in:
 - $min = 0 \rightarrow$ participation is **optional** (equivalent to a single line in Chen)
 - $min \geq 1 \rightarrow$ participation is **mandatory** (equivalent to a double line in Chen)
- **max** — the maximum number of relationship instances this entity *can* participate in:
 - $max = 1 \rightarrow$ at most one
 - $max = N$ (or ∞) $\rightarrow$ no upper limit

(min, max) pair	English reading	Chen equivalent
$(0, 1)$	“zero or one”	Partial, cardinality 1
$(1, 1)$	“exactly one”	Total, cardinality 1
$(0, N)$	“zero or more”	Partial, cardinality N
$(1, N)$	“one or more”	Total, cardinality N
$(4, N)$	“at least four”	Total, cardinality N

Placement Rule

The (min, max) pair is written on the line **next to the entity it constrains**. The general form is:



Figure 25: General form of Min-Max placement on a binary relationship.

- An entity of type E_1 may be related to **at least** min_1 and **at most** max_1 entities of type E_2 .
- Likewise, min_2 is the minimum number and max_2 is the maximum number of E_1 entities to which an E_2 entity is related.

Illustrative Example — EMPLOYEE and DEPARTMENT

The diagram below shows Min-Max annotation on a generic EMPLOYEE–DEPARTMENT relationship. The values here are **chosen to illustrate specific constraint patterns** and are independent of the Company Database worked example.



Figure 26: Min-Max notation on an EMPLOYEE–DEPARTMENT relationship (illustrative values).

Pair	Placed next to	Meaning
(1, 1)	EMPLOYEE	Every employee is associated with exactly one department — mandatory and singular.
(4, N)	DEPARTMENT	Every department has at least 4 employees — a specific minimum business rule.

i Note

The (4, N) on the DEPARTMENT side is intentionally a specific-value constraint to show that Min-Max is not limited to the four standard combinations. The Company Database uses (1, N) for this side — see the table in the next section.

Company Database in Min-Max Notation

The table below re-expresses every relationship from the Worked Example using (min, max) pairs, consistent with the Full Constraint Summary and the Chen diagram.

Relationship	Entity A	(min, max) on A	Entity B	(min, max) on B
WORKS_FOR	EMPLOYEE	(1, 1)	DEPARTMENT	(0, N)
MANAGES	EMPLOYEE	(0, 1)	DEPARTMENT	(1, 1)
WORKS_ON	EMPLOYEE	(0, N)	PROJECT	(1, N)
CONTROLS	PROJECT	(1, 1)	DEPARTMENT	(0, N)

Relationship	Entity A	(<i>min</i> , <i>max</i>) on A	Entity B	(<i>min</i> , <i>max</i>) on B
SUPERVISION	EMPLOYEE (Supervisee)	(0, 1)	EMPLOYEE (Supervisor)	(0, N)
HAS	EMPLOYEE	(0, N)	DEPENDENT	(1, 1)

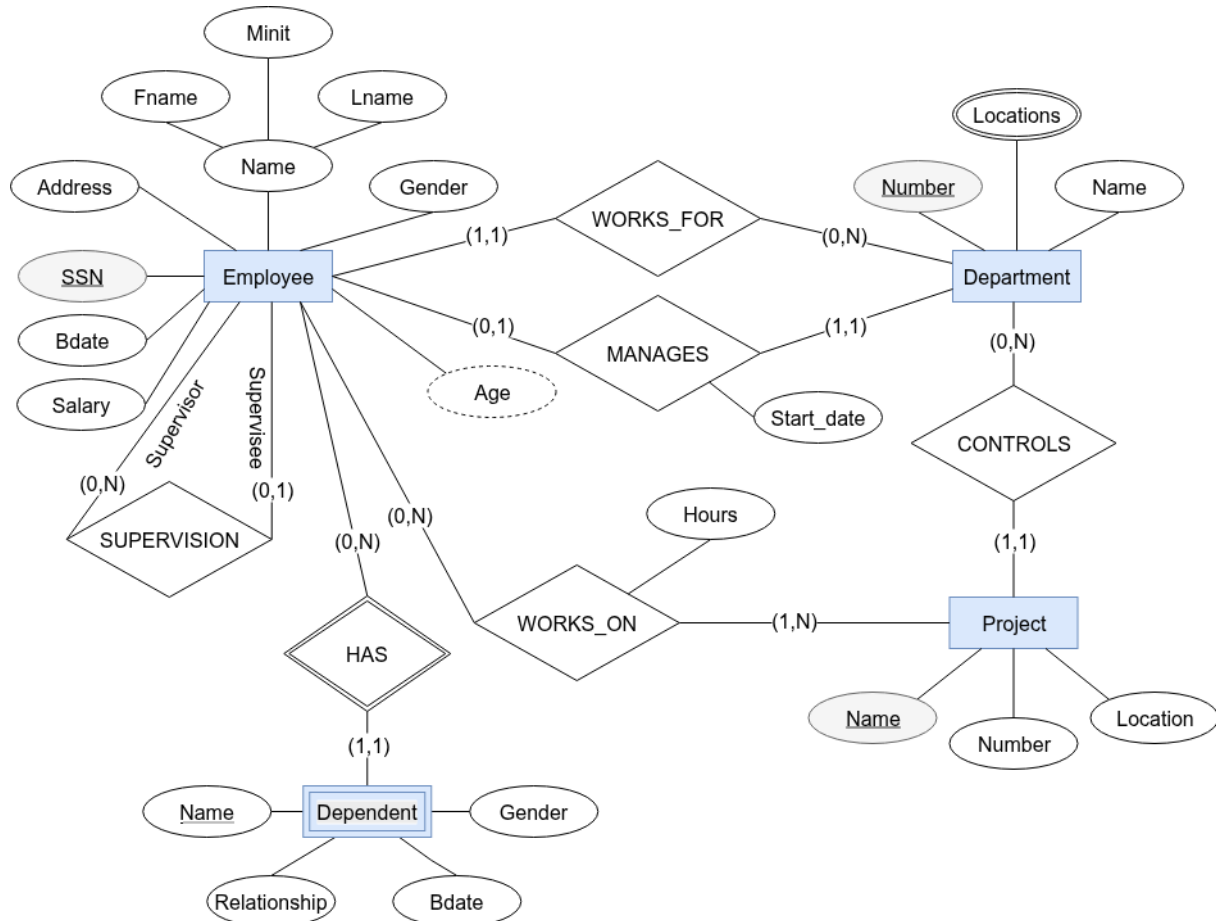


Figure 27: Full Company ER diagram annotated with Min-Max notation.

Tip

Why use Min-Max instead of Chen labels?

Chen notation labels each relationship line with 1, N, or M — this captures the *maximum* cardinality but not the *minimum*. Min-Max adds the minimum, making it immediately clear whether participation is optional or mandatory without relying on single-vs-double lines. It is especially useful when writing precise business rules for a development team to implement.

Chen vs Min-Max: Side-by-Side Examples

Each example below shows the **same relationship drawn twice** — once in Chen notation (top) and once in Min-Max notation (bottom) — so you can see exactly how the two representations map onto each other.

Example 1 — Professor CHAIRS Department (One-to-One, partial / total)

Business Requirements

The university has several departments. Each department must have exactly one chairperson who must be a professor. A professor can chair at most one department. Some professors may not chair any department — the chairperson role is optional for professors. If a professor is appointed as chair, they must chair exactly one department. No department can exist without a chairperson.

Solution

Relationship: CHAIRS

Property	Entity A — Professor	Entity B — Department
Participation	Partial (optional)	Total (mandatory)
(min, max)	$(0, 1)$	$(1, 1)$
Chen cardinality label	1	1

Diagram (Chen top / Min-Max bottom):

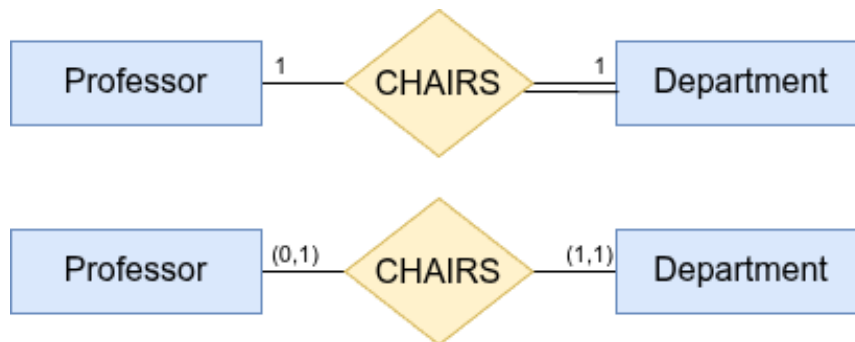


Figure 28: Chen notation (top) vs Min-Max notation (bottom) for CHAIRS.

Reading the relationship:

- **Left → Right (LTR):** Each Professor chairs **0 or 1** Department.
- **Right → Left (RTL):** Each Department is chaired by **exactly 1** Professor.

How to read — Chen (top):

- The 1 on the Professor side and the 1 on the Department side tell us this is a **1:1** relationship.
- The **double line** on the Department side means Department participation is **total** (mandatory).
- The **single line** on the Professor side means Professor participation is **partial** (optional).

How to read — Min-Max (bottom):

Pair	Placed next to	Meaning
(0, 1)	Professor	A professor chairs at most one department — but is not required to chair any.
(1, 1)	Department	Every department has exactly one chair — mandatory, and only one.

Key insight: Chen told us the *ratio* (1:1) but left participation implicit in the line style. Min-Max spells out both the minimum and maximum in a single, self-documenting pair.

Example 2 — Student HOLDS_PERMIT Parking_Permit (One-to-One, both partial)

Business Requirements

Students may obtain a parking permit from the university. A student can have at most one parking permit. Some students don't have cars and therefore have no permit. Each parking permit is assigned to exactly one student, but permits can exist in the system before being assigned. If a student leaves, their permit becomes available for reassignment.

 Solution

Relationship: HOLDS_PERMIT

Property	Entity A — Student	Entity B — Parking_Permit
Participation	Partial (optional)	Partial (optional)
(min, max)	$(0, 1)$	$(0, 1)$
Chen cardinality label	1	1

Assumption: Permits can exist unassigned; not all students need permits.

Diagram (Chen top / Min-Max bottom):

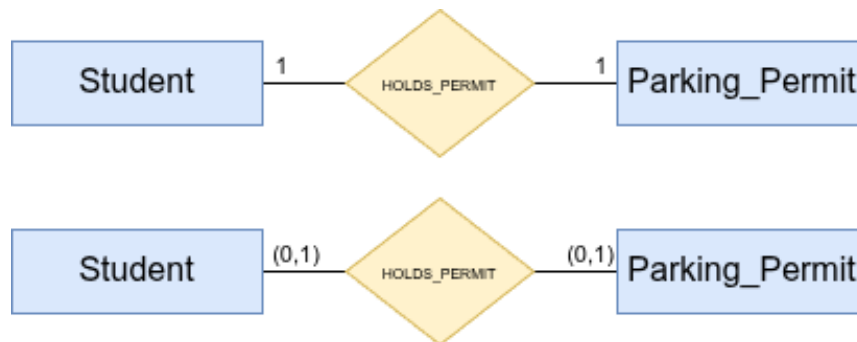


Figure 29: Chen notation (top) vs Min-Max notation (bottom) for HOLDS_PERMIT.

Reading the relationship:

- **Left → Right (LTR):** Each Student holds **0 or 1** Parking Permit.
- **Right → Left (RTL):** Each Parking Permit is held by **0 or 1** Student.

How to read — Chen (top):

- 1 on both sides → **1:1** relationship.
- **Single line** on both sides → participation is **partial** on both ends (neither student nor permit is required to be matched).

How to read — Min-Max (bottom):

Pair	Placed next to	Meaning
(0, 1)	Student	A student holds at most one parking permit — but may hold none.
(0, 1)	Parking_Permit	A parking permit is held by at most one student — but may be unassigned.

Key insight: Both sides are optional. Min-Max makes this unambiguous without having to check whether lines are single or double.

i Note

The full implementation of reassignment — ON DELETE SET NULL, status trigger, and the design discussion SET NULL vs. CASCADE — lives in [Appendix C §C.1](#).

Example 3 — Department OFFERS Course (One-to-Many, both mandatory)**Business Requirements**

Every department in the university must offer at least one course. A course must be offered by exactly one department. Departments can offer multiple courses. Courses cannot exist without being associated with a department. New departments must create at least one course within their first semester.

 Solution**Relationship: OFFERS**

Property	Entity A — Department	Entity B — Course
Participation	Total (mandatory)	Total (mandatory)
(min, max)	$(1, N)$	$(1, 1)$
Chen cardinality label	N	1

Assumption: Every department must offer courses; cross-listing handled separately.

Diagram (Chen top / Min-Max bottom):

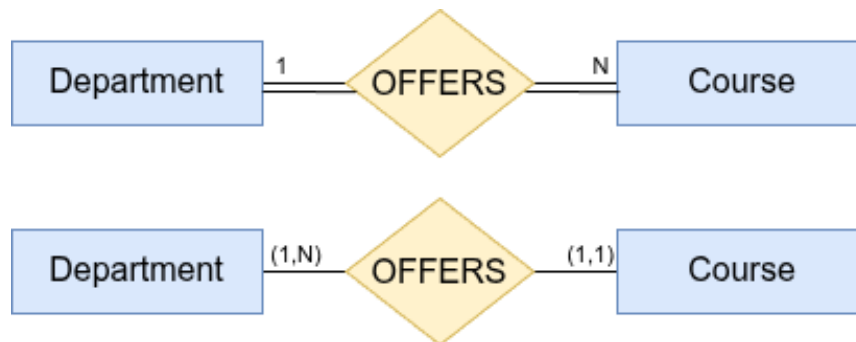


Figure 30: Chen notation (top) vs Min-Max notation (bottom) for OFFERS.

Reading the relationship:

- **Left → Right (LTR):** Each Department offers **1 or more** Courses.
- **Right → Left (RTL):** Each Course is offered by **exactly 1** Department.

How to read — Chen (top):

- 1 on Department, N on Course → **1:N** relationship.
- **Double line** on both sides → participation is **total** on both ends.

How to read — Min-Max (bottom):

Pair	Placed next to	Meaning
(1, N)	Department	Every department offers at least one course (no upper limit).
(1, 1)	Course	Every course is offered by exactly one department.

Key insight: Chen's double lines on both sides simply said "mandatory on both ends."

Min-Max adds the upper bound: a department can offer *many* courses, but a course belongs to *only one* department.

⚠ Warning

The static ER diagram cannot capture the *timing* rule "within the first semester." The enforcement trigger and its design alternatives are worked out in [Appendix C §C.2](#).

Example 4 — Professor ADVISES Student (One-to-Many, both partial)**Business Requirements**

Professors may serve as academic advisors to students. A professor can advise multiple students. Some professors choose not to advise any students. A student may have at most one academic advisor. Some students, particularly freshmen, may not have an assigned advisor yet. Each advisor-student relationship is formalized through the advising system.

 Solution**Relationship: ADVISES**

Property	Entity A — Professor	Entity B — Student
Participation	Partial (optional)	Partial (optional)
(min, max)	$(0, N)$	$(0, 1)$
Chen cardinality label	N	1

Assumption: Advising is voluntary for professors; freshmen and other students may be unassigned until they select an advisor.

Diagram (Chen top / Min-Max bottom):

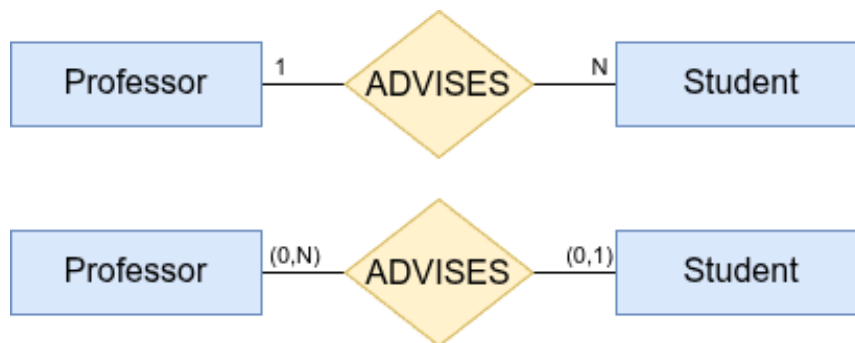


Figure 31: Chen notation (top) vs Min-Max notation (bottom) for ADVISES.

Reading the relationship:

- **Left → Right (LTR):** Each Professor advises **0 or more** Students.
- **Right → Left (RTL):** Each Student is advised by **0 or 1** Professor.

How to read — Chen (top):

- 1 on Professor, N on Student → **1:N** relationship.
- **Single line** on both sides → participation is **partial** on both ends.

How to read — Min-Max (bottom):

Pair	Placed next to	Meaning
$(0, N)$	Professor	A professor may advise any number of students — or none at all.
$(0, 1)$	Student	A student may have at most one advisor — but is not required to have one.

Key insight: Chen indicated a 1:N ratio with optional participation, but gave no upper bound on the N side. Min-Max confirms there truly is no cap on how many students a professor can advise, while also confirming a student cannot have more than one.

i Relationship Attributes: The ADVISES Relationship Has More to Say

The phrase “*Each advisor-student relationship is formalized through the advising system*” is a strong hint that **ADVISES is not a simple binary connection** — it carries descriptive information of its

own. In ER modelling, when a relationship has data associated with it (not belonging to either entity alone), that data is modelled as **relationship attributes**.

Candidate attributes for ADVISES:

Attribute	Type	Meaning
assignment_date	DATE	When the advising relationship was established
status	ENUM(ACTIVE, INACTIVE, PENDING)	Current state of the advisory relationship
end_date	DATE (nullable)	When the relationship was formally terminated
notes	TEXT	Free-text notes entered by advisor or registrar

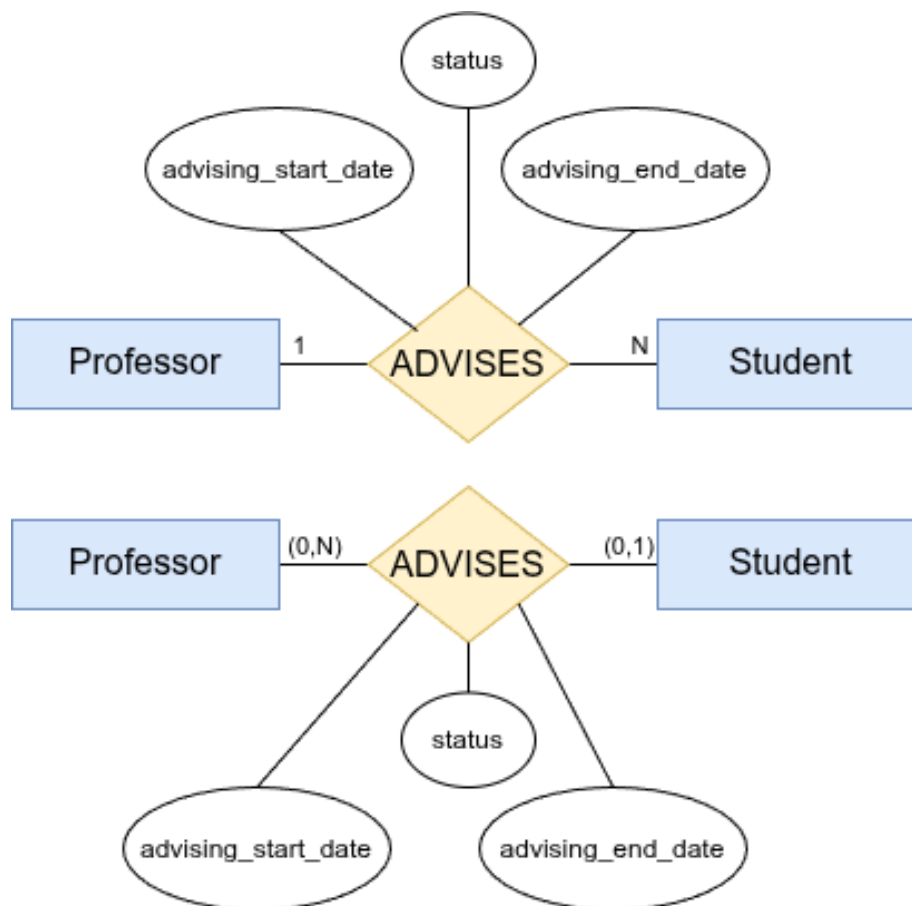
Revised ER diagram with relationship attributes:

Figure 32: ADVISES relationship with attributes (assignment_date, status, end_date).

Design implication: When a relationship has attributes, it is often a candidate for promotion to an **associative entity** (also called a relationship entity or junction table) — especially if other entities need to reference it. For example, if the advising record itself needs to be referenced by a report or an audit trail, ADVISES would become its own entity (e.g., AdvisingRecord) with a surrogate key.

i Note**Reading template for Min-Max**

Given a pair (min, max) placed next to entity E :

“Each [E] participates in this relationship **at least** min **time(s)** and **at most** max **time(s)**.”

When $min = 0$ the participation is **optional**; when $min \geq 1$ it is **mandatory**. When $max = N$ there is **no upper limit**; when $max = 1$ the entity participates in **at most one** instance.

Specific Value Constraints: $(0, 3)$, $(2, 5)$, and Beyond

Not every business rule fits neatly into the four standard patterns $(0, 1)$, $(1, 1)$, $(0, N)$, $(1, N)$. Min-Max notation allows **any non-negative integers** as the min and max values.

Example	English meaning	Typical business rule
$(0, 3)$	zero to three	A customer may place at most 3 active orders at a time
$(2, 5)$	two to five	A project team must have between 2 and 5 members
$(1, 4)$	one to four	An employee manages at least 1, at most 4 direct reports
$(3, 3)$	exactly three	A committee seat is always filled by exactly 3 co-chairs

These constraints are **perfectly valid** at the conceptual level — they are precise expressions of real business rules that the common patterns simply cannot capture. Write them on your ER diagram exactly as you would any other (min, max) pair.

Warning

Conceptual validity $\neq$ automatic SQL enforcement

Standard SQL foreign-key constraints only know how to enforce “must exist” (referential integrity). They have no mechanism to *count* how many rows a related entity participates in before accepting or rejecting a change. Specific value constraints such as $(0, 3)$ or $(2, 5)$ therefore **cannot be enforced by foreign keys alone**.

Enforcing specific value constraints in the database requires either subquery-CHECKs (PostgreSQL), triggers, stored procedures, or application logic. The four strategies, with trade-offs and ready-to-run SQL, are compared in [Appendix C §C.3](#).

Crow’s Foot Notation

Crow’s Foot notation is an ER diagramming notation used to represent entities, attributes, and relationships. It is the standard notation adopted by most professional database design tools — MySQL Workbench, Lucidchart, draw.io, Visio, and ERwin — and is applicable at the **conceptual, logical, and**

physical design stages, though it is most commonly encountered during logical and physical design in practice.

Entities are drawn as plain rectangular boxes; the relationship line carries small tick and arc symbols at each end. Those end symbols are called **crow's foot markers** because three diverging lines resemble a bird's foot.

Symbol Reference

Each line end carries **two markers**. The marker *closest to the entity box* gives the **minimum**; the marker *furthest from the entity box* gives the **maximum**.

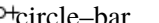
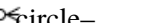
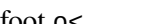
Minimum markers (inner — nearest the entity box):

Symbol	Meaning
(single vertical bar)	Mandatory — must participate ($min \geq 1$)
o (small open circle)	Optional — may not participate ($min = 0$)

Maximum markers (outer — away from the entity box):

Symbol	Meaning
(single vertical bar)	At most one ($max = 1$)
< (three diverging lines = crow's foot)	Many — no upper limit ($max = N$)

Common combinations at a line end:

Symbol	Name	(min, max)	Chen equivalent	Plain English
	bar-bar	(1, 1)	Total, cardinality 1	Exactly one
	\ \			
	circle-bar	(0, 1)	Partial, cardinality 1	Zero or one
	o\			
	bar-crow's	(1, N)	Total, cardinality N	One or many
	foot \ <			
	circle-crow's	(0, N)	Partial, cardinality N	Zero or many
	foot o<			

Reading Rule

Read the symbols at **entity B's end** of the line—they describe how many B instances can be associated with one A instance:

“Each [Entity A] has [outer marker reading] [inner marker reading] [Entity B].”

Example: DEPARTMENT —|{ EMPLOYEE

“Each Department has **one or many** Employees.” ($min = 1, max = N$)

Crow's Foot Examples

Each example below shows a Crow's Foot diagram (top) alongside its Chen equivalent (bottom).

Example 1 — Person HAS Passport (One-to-One, both mandatory)

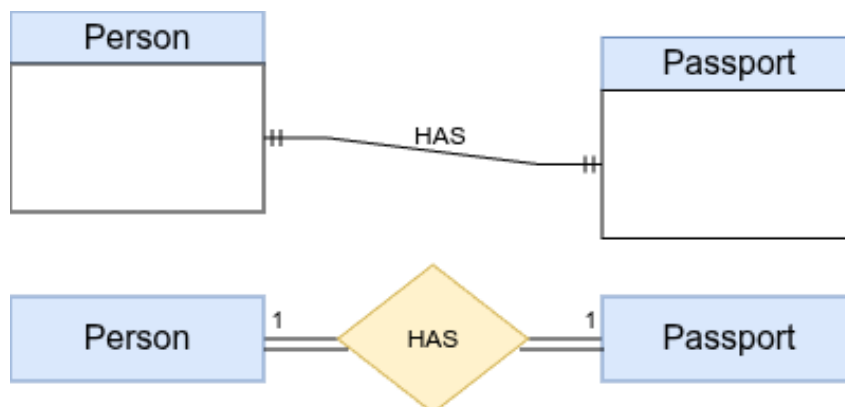


Figure 33: Crow's Foot (top) vs Chen (bottom): Person–HAS–Passport.

End	Symbol	Reading
Person side	(bar–bar)	Every person holds exactly one passport.
Passport side	(bar–bar)	Every passport belongs to exactly one person.

Both participations are total (mandatory) and the cardinality is 1:1 on both sides.

Example 2 — Employee IS ASSIGNED Office (One-to-One, Employee optional)

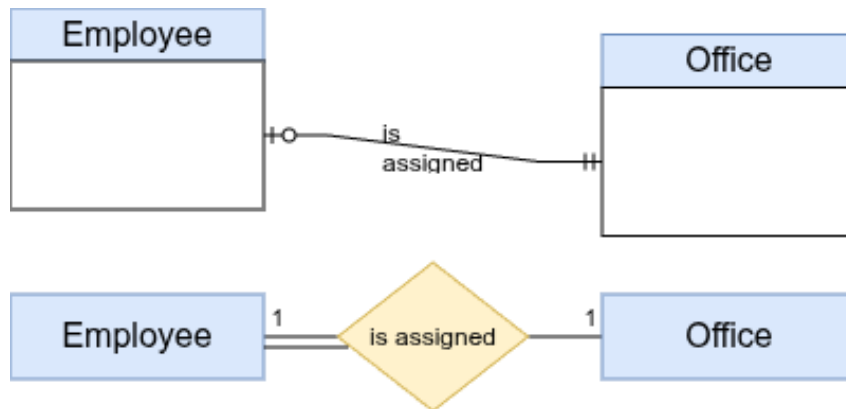


Figure 34: Crow's Foot (top) vs Chen (bottom): Employee–is assigned–Office.

End	Symbol	Reading
Employee side	o (circle–bar)	An employee may be assigned to zero or one office.
Office side	(bar–bar)	Every office is assigned to exactly one employee.

Employee participation is partial (optional); Office participation is total (mandatory).

Example 3 — Customer PLACES Order (One-to-Many, both mandatory)

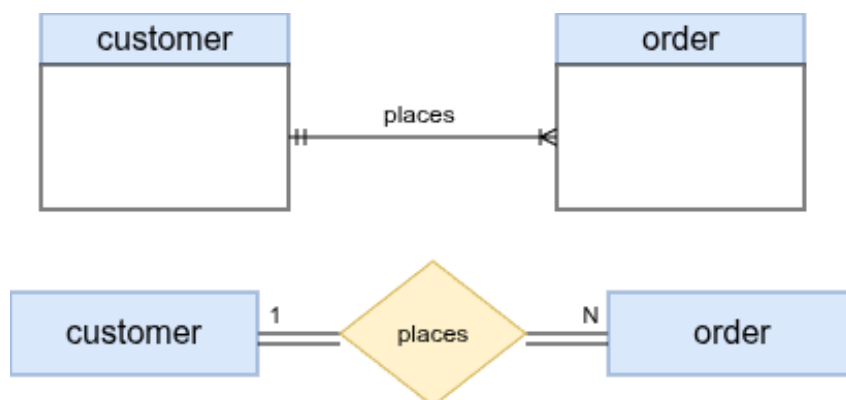


Figure 35: Crow's Foot (top) vs Chen (bottom): Customer–places–Order.

End	Symbol	Reading
Customer side	(bar–bar)	Every order is placed by exactly one customer.
Order side	< (bar–crow’s foot)	Every customer places one or many orders.

Both participations are mandatory; cardinality is 1:N.

Textual Note — Extending Crow’s Foot for Specific Upper Bounds

Standard Crow’s Foot symbols can only express four combinations: $(0, 1)$, $(1, 1)$, $(0, N)$, $(1, N)$. When a business rule demands a **specific upper bound** (e.g., at most 3), the symbol alone is insufficient.

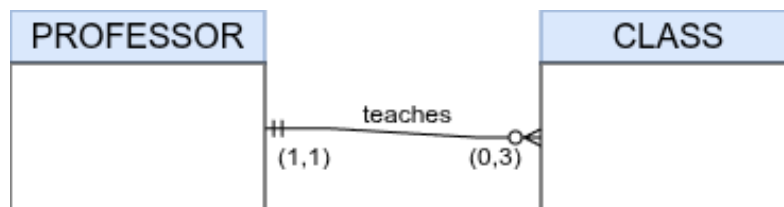


Figure 36: Crow’s Foot with a textual note for PROFESSOR–teaches–CLASS.

In the diagram above, the CLASS side carries:

- The **circle–crow’s foot** symbol $\circ<$ — meaning *zero or many* in standard Crow’s Foot.
- A **textual annotation** $(0, 3)$ written next to the symbol to narrow the upper bound to **at most 3**.

i Note

Why a textual note?

The standard $\circ<$ symbol means *unbounded many* ($max = N$). There is no dedicated Crow’s Foot symbol for a finite upper limit such as 3. The widely adopted convention is to keep the closest-matching symbol ($\circ<$ for zero-or-many) and add a parenthesised note — $(0, 3)$ — directly beside it to record the actual maximum. The $(1, 1)$ on the PROFESSOR side is read normally: every teaching instance belongs to exactly one professor.

This approach is also used in tools like ERwin and Visio when constraints exceed what the four standard symbols can express.

Crow's Foot Across Design Stages — Conceptual vs. Logical

Crow's Foot notation is used at **multiple design stages**, and what appears inside each entity box changes as you move from one stage to the next. The Customer–Order example below illustrates the key differences between a **conceptual** diagram and a **logical** diagram drawn with the same Crow's Foot symbols.

Conceptual schema — what the business means, not how it is stored:

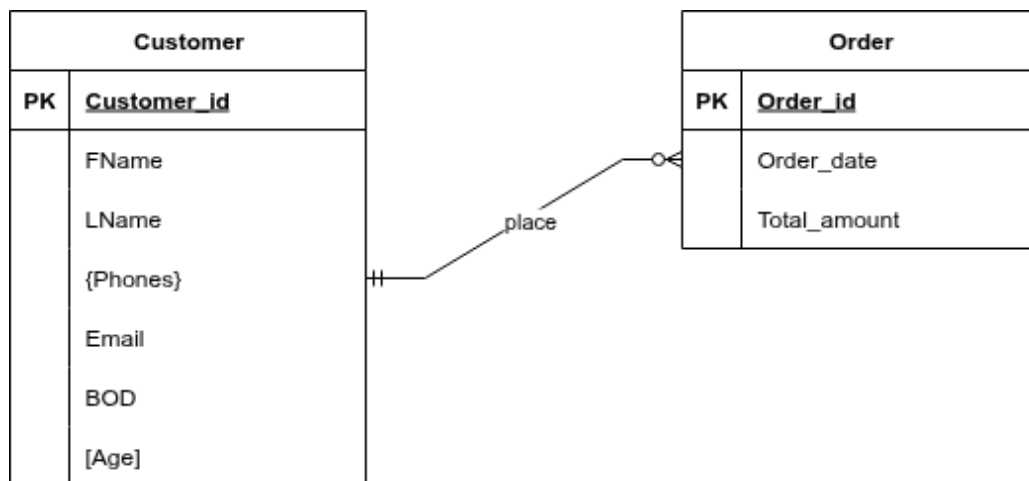


Figure 37: Conceptual Crow's Foot diagram: Customer places zero-or-many Orders.

At this stage:

- **Multi-valued attribute** {Phones} is shown inside the Customer box using curly braces — it is recorded as a fact about the entity but not yet split into a separate table.
- **Derived attribute** [Age] is shown using square brackets — it is computable from BOD and is included only for documentation; it will not become a stored column.
- **No foreign key** appears in the Order box — FK columns are a relational implementation detail, not a conceptual concept.
- The relationship line carries the crow's foot markers only; the line represents the *association*, not a column.

Logical schema — how the data will be structured in a relational database:

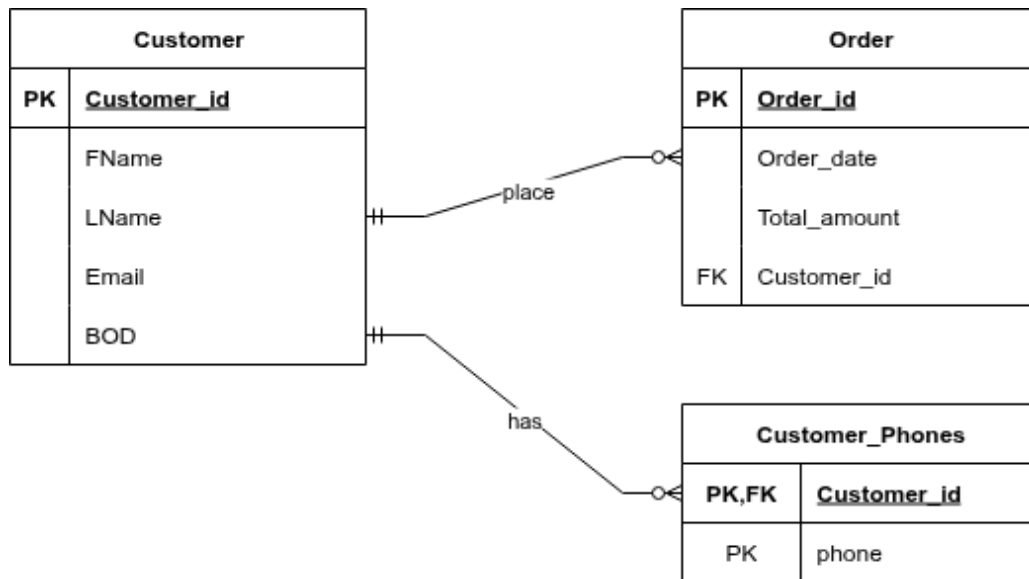


Figure 38: Logical Crow's Foot diagram: FK added to Order; Customer_Phones resolved as a separate table.

At this stage, three transformations have been applied:

Conceptual element	Logical transformation	Reason
{Phones} multi-valued attribute	Becomes a new table Customer_Phones(Customer_id, phone) PK/FK, phone PK)	Relational columns are atomic — a cell cannot hold a list of values
[Age] derived attribute	Dropped — not stored	Age is computed from BOD at query time; storing it would create update anomalies
No FK in Order	Customer_id FK added to Order	The many-side table must hold the FK that references the one-side PK

i Note

Reading the FK relationship in the logical diagram

The crow's foot line between Customer and Order still reads the same way — one customer places zero-or-many orders. The difference is that the Order table now contains a Customer_id column (marked FK) that enforces this link at the database level. The line in the logical diagram connects the Customer_id PK column in Customer to the Customer_id FK column in Order, making the referential constraint explicit.

The new Customer_Phones table has a **composite primary key** (Customer_id, phone) — both columns together identify a unique phone number per customer — and Customer_id is simultaneously a FK back to Customer.

 Tip

When to use which notation?

Situation	Best choice
University course, exam, textbook exercise	Chen
Specifying precise business rules (exact min and max counts)	Min-Max
Design tool, project documentation, professional work	Crow's Foot

All three notations encode the same information. Once you understand the underlying concepts, switching between notations is straightforward.

ER-to-Relational Mapping and Final Schema

With the ER / EER diagram now finalised, the next step is to translate it into concrete relational tables. That translation is a seven-step algorithm that depends on relational vocabulary — candidate keys, foreign keys, referential actions, and NULL semantics — which is developed in full in Chapter 5. To avoid teaching those concepts twice, the complete mapping walkthrough for this Company case study, together with the final SQL script, lives at the end of Chapter 5. See [Chapter 5, §ER-to-Relational Mapping](#).

Summary

 Chapter 3 — Key Takeaways

1. **ER diagram** = graphical conceptual schema drawn before any SQL or implementation decisions.
2. **Three notations**: Chen (academic standard, used in this book), Min-Max (precise constraints), Crow's Foot (industry tools).
3. **Strong entity**: independent existence, has its own key. **Weak entity**: no key of own; identified by owner's key + partial key via identifying relationship; always total participation.
4. **Seven attribute types**: simple, composite, multivalued (double oval), derived (dashed oval), stored, key (underlined), partial key (dashed underline).
5. **Cardinality ratios**: 1:1, 1:N, M:N — written as labels near entities in Chen notation.
6. **Participation**: total (double line) = entity cannot exist without this relationship; partial (single

line) = optional.

7. **Min-Max** (min, max) gives exact lower and upper bounds — more precise than cardinality ratio alone.
8. **Relationship attributes** belong to the relationship when their value depends on the combination of both participating entities.
9. **7-step mapping** (covered in Chapter 5): Strong $\rightarrow$ table (1); Weak $\rightarrow$ table with composite PK (2); 1:1 $\rightarrow$ FK on total side (3); 1:N $\rightarrow$ FK on N-side (4); M:N $\rightarrow$ junction table (5); Multivalued $\rightarrow$ separate table (6); N-ary $\rightarrow$ new table (7). The full walkthrough and SQL script for the Company Database appear in [Chapter 5, §ER-to-Relational Mapping](#).

Review Questions

Conceptual

1. Define **entity type**, **entity instance**, and **key attribute**. Give a concrete example of each from the Company Database.
2. Explain the difference between a **composite attribute** and a **multivalued attribute**. How does each appear in the ER diagram? (*Their effect on the relational mapping is covered in Chapter 5.*)
3. What is a **weak entity type**? What distinguishes it visually in Chen notation? Give a real-world example other than DEPENDENT.
4. What is the difference between **Chen notation**, **Min-Max notation**, and **Crow's Foot notation**? In what context is each used?

Short-Answer / Compare

5. Compare **cardinality ratio** and (min, max) notation. Write the (min, max) constraints for WORKS_FOR and explain each value.
6. Explain what **total participation** means for EMPLOYEE in WORKS_FOR. What SQL constraint enforces this?
7. Describe the difference between a **relationship attribute** and an **entity attribute**. Why does Hours belong to WORKS_ON and not to EMPLOYEE or PROJECT?
8. Why is Start_date a relationship attribute of MANAGES rather than an attribute of EMPLOYEE or DEPARTMENT?

True / False (with explanation)

9. “A many-to-many relationship can safely be represented by adding a FK to one of the two entity tables.” — True or False?

10. “Derived attributes must be stored in the database to avoid recomputing them.” — True or False?
11. “A weak entity always has total participation in its identifying relationship.” — True or False?

Multiple Choice

12. In Chen notation, a **double diamond** represents:
- (a) A ternary relationship
 - (b) A many-to-many relationship
 - (c) **An identifying relationship for a weak entity**
 - (d) A recursive relationship
13. In Crow’s Foot notation, —o{— on the right end of a line means:
- (a) Exactly one — (b) One or many — (c) **Zero or many** — (d) Zero or one

Design Exercise

14. A car rental company stores: **Vehicles** (plate number, make, model, year, mileage), **Customers** (customer ID, name, driving licence), and **Rentals** (pickup date, return date, total cost). Each rental involves exactly one vehicle and one customer. A vehicle can be rented many times (but not simultaneously).
- (a) Draw a complete ER diagram in Chen notation with entity types, attribute types, key attributes, relationship types, cardinality ratios, and participation constraints.

i Mapping & SQL Questions — See Chapter 5

The questions below require relational-model vocabulary (foreign keys, referential integrity, NULL semantics) and the 7-step ER-to-Relational algorithm, all developed in **Chapter 5 — The Relational Model**. Review them here as a forward-preview of what mapping entails.

- **Design Exercise (continued)** — For the car rental ER diagram you drew above:
 - (b) Apply the 7-step mapping to produce a relational schema.
 - (c) Identify which FKs may be NULL and which must be NOT NULL. Justify each.
- **True / False** — “In the 7-step mapping, a multivalued attribute of a strong entity becomes a separate table.”
- **Multiple Choice** — Which step of the 7-step mapping handles M:N relationships?
 - (a) Step 3 (b) Step 4 (c) **Step 5** (d) Step 6
- **Multiple Choice** — For the *MANAGES* (1:1) relationship, the FK is placed in the DEPARTMENT table because:
 - (a) DEPARTMENT has more rows

- (b) Every **DEPARTMENT** must have a manager (total participation), minimizing **NULLs**
- (c) SQL does not support 1:1 keys on **EMPLOYEE**
- (d) Managers are employees, so the FK must point outward
- **SQL** — Write the **CREATE TABLE** statement for the **WORKS_ON** table (Step 5 mapping), including the composite PK and both FKs. Which SQL keyword enforces **NOT NULL** on FK columns?

Enhanced Entity-Relationship (EER) Modeling

Chapter 4

A good model is a simplification of reality that retains exactly those details that matter for the question at hand.

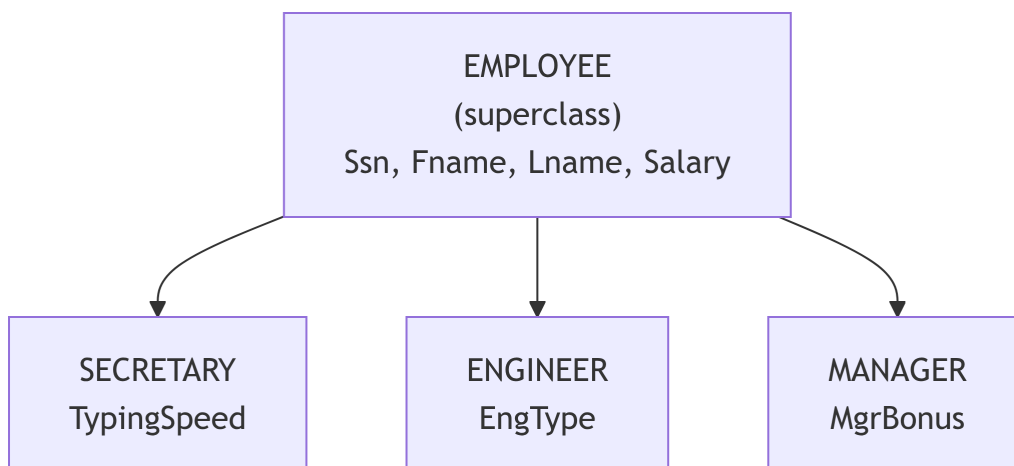
—George E. P. Box (adapted)

The ER model of Chapter 3 handles most real-world data structures, but it lacks one critical capability: **type hierarchies**. When some employees are engineers, some are managers, and a few are engineer-managers, the basic ER model forces you to bundle everything into one bloated entity type or fragment the design artificially. The Enhanced ER (EER) model adds three mechanisms — *specialization/generalization*, *aggregation*, and *union types* — to handle these situations cleanly.

Subclasses, Superclasses, and Inheritance

An entity type S is a **subclass** of entity type C if: - Every entity instance of S is *also* an instance of C - S has attributes or relationships that are not meaningful for all of C

C is called the **superclass** of S , and the pair forms a **superclass/subclass (IS-A) relationship**.



Type Inheritance

A subclass entity **inherits** all attributes and participation in relationships of its superclass. It additionally has its own **specific attributes** and may participate in its own **specific relationships**.

Note

Inheritance is *implicit* in the EER model — you do not explicitly copy attributes from superclass to subclass in the diagram. When you map to relational tables (§4.7), you choose how to physically represent shared attributes.

With the IS-A relationship established, we can now distinguish between two design processes that produce it.

Specialization

Specialization is a **top-down** design process: starting with a superclass, the designer defines subclasses that represent better-defined subsets of the superclass.

Rationale: Some entities in EMPLOYEE have attributes irrelevant to others — not every employee has a TypingSpeed, but every secretary does. Specializing EMPLOYEE into SECRETARY keeps the schema clean.

Constraints on Specialization

A specialization carries two independent constraints, each with two options:

1. Disjointness Constraint

Constraint	Symbol	Meaning
Disjoint (d)	Circle with d	Each entity belongs to at most one subclass
Overlapping (o)	Circle with o	An entity may belong to multiple subclasses simultaneously

Disjoint example: An employee is either HOURLY_EMPLOYEE or SALARIED_EMPLOYEE — never both. Payroll would not compute correctly if an employee appeared in both.

Overlapping example: A university PERSON can simultaneously be a STUDENT and an EMPLOYEE (e.g., a teaching assistant). Excluding one membership would lose data.

2. Completeness Constraint

Constraint	Notation	Meaning
Total	Double line from superclass to circle	Every superclass entity must belong to at least one subclass
Partial	Single line from superclass to circle	A superclass entity may belong to no subclass

Total example: Every SHAPE must be either a CIRCLE, RECTANGLE, or TRIANGLE — there are no “generic shapes” stored.

Partial example: Not every EMPLOYEE is a manager — most employees have no subclass membership.

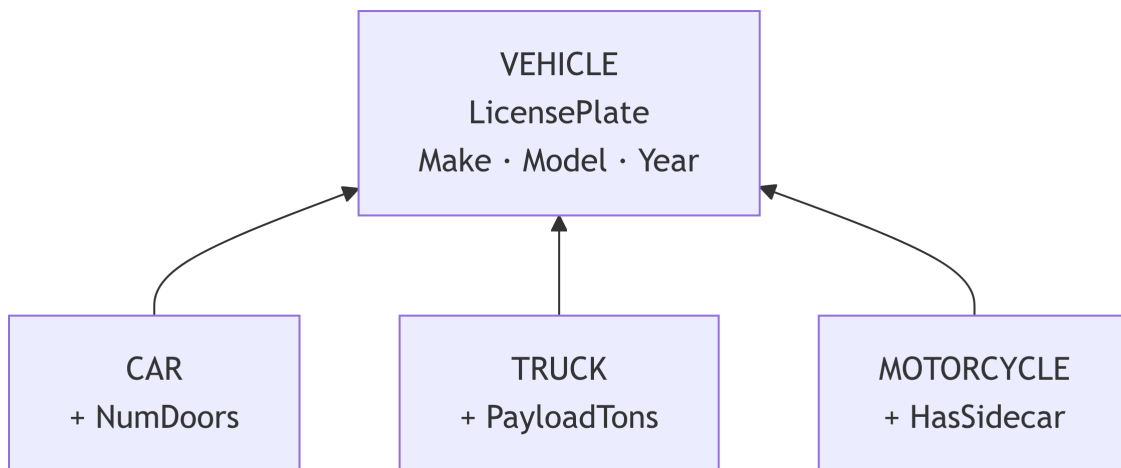
Combining the Two Constraints

Combination	Practical meaning	Example
Total + Disjoint	Every superclass entity is in exactly one subclass	EMPLOYEE → {HOURLY, SALARIED}
Total + Overlapping	Every entity is in at least one, possibly many subclasses	PERSON → {STUDENT, EMPLOYEE}
Partial + Disjoint	Some entities are in no subclass; those that are fall in at most one	VEHICLE → {CAR, TRUCK} (a bicycle doesn't fit)
Partial + Overlapping	Some entities are in no subclass; some are in multiple	PRODUCT → {DIGITAL, PHYSICAL} (can be both for a bundled product)

Generalization

Generalization is the **bottom-up** process: the designer observes that several entity types share common attributes and abstracts those into a new superclass.

Example: Designing independently, you create CAR, TRUCK, and MOTORCYCLE all with LicensePlate, Make, Model, Year. Noticing the overlap, you generalize them into VEHICLE, extracting the shared attributes:



💡 Tip

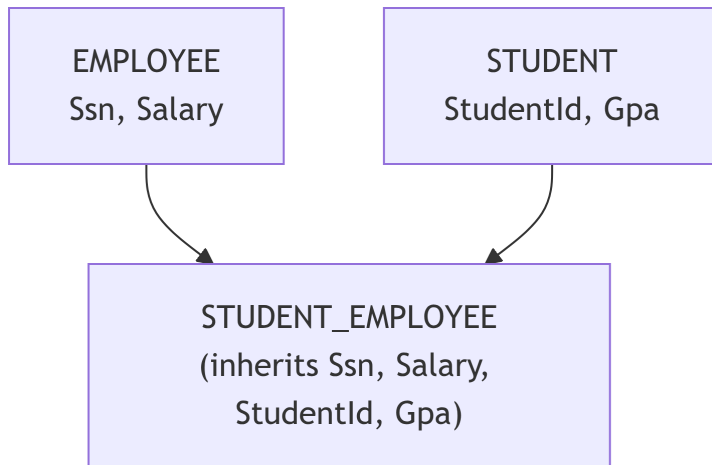
Specialization vs. Generalization produce identical EER diagrams. The distinction is purely in the *decision process* (top-down design vs. bottom-up abstraction). In practice, real designs mix both: you may generalize two existing entities and then specialize the resulting superclass further.

With hierarchies defined, the next structural concept addresses how entities organize into trees versus networks.

Specialization Hierarchies and Lattices

Structure	Description	Constraint
Specialization hierarchy	Each subclass has exactly one direct superclass	Produces a tree
Specialization lattice	A subclass can have more than one direct superclass	Produces a DAG (directed acyclic graph)

In a **lattice**, a subclass inherits from **all** its superclasses — this is **multiple inheritance**.



! Important

A **STUDENT_EMPLOYEE** entity must simultaneously be an instance of both **EMPLOYEE** *and* **STUDENT**. This is the key distinction between a lattice node (shared subclass) and a union type (§4.6), where membership in *only one* superclass is required.

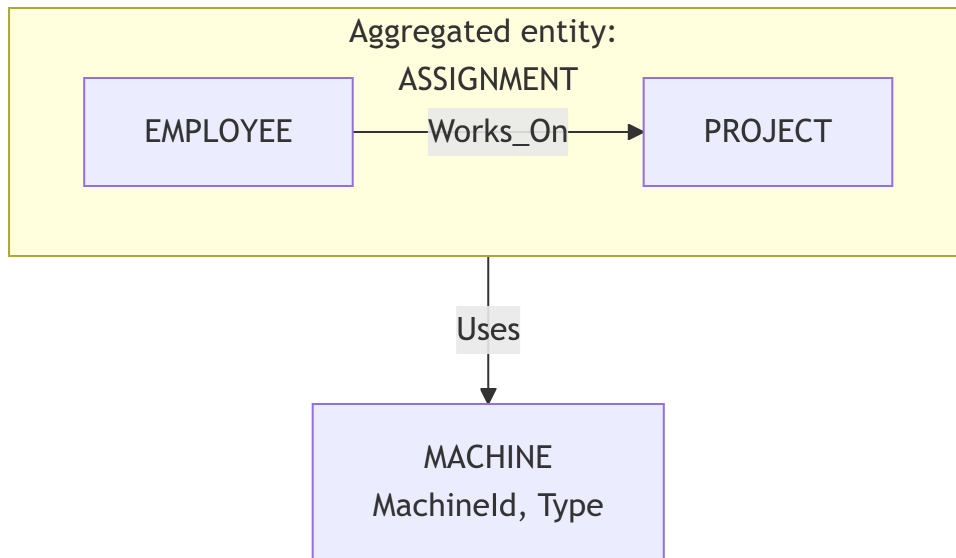
Aggregation

Aggregation promotes an existing *relationship type* to a higher-level entity, allowing that relationship to participate in another relationship.

Why Aggregation?

Consider: “An **EMPLOYEE** **Works_On** a **PROJECT**. For some assignments, a specific **MACHINE** is **Used** during that assignment.”

You might try to add **MACHINE** as a third participant in **Works_On** (making it ternary). But that implies that every combination of (Employee, Project, Machine) is an assignment — whereas the real constraint is: *given an existing (Employee, Project) assignment*, a machine may be used. Aggregation captures this:



The aggregated entity (ASSIGNMENT) is a *virtual entity* made from the Works_On relationship. It can now participate in Uses with MACHINE.

i Note

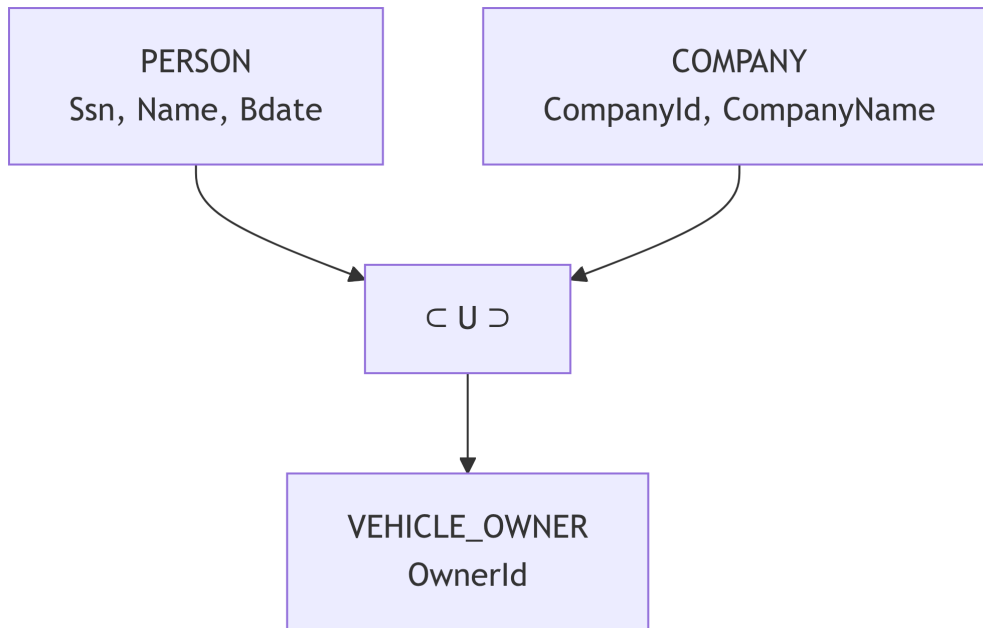
When mapped to relational tables, the aggregation maps to its junction table (WORKS_ON) plus an additional USES table that references WORKS_ON's primary key.

Category (UNION) Types

A **category** (or *Union type*) is a subclass that can come from **different, unrelated superclass types**. The key difference from a lattice node:

Feature	Lattice node (shared subclass)	Category (Union)
Superclasses	Multiple, related IS-A hierarchy	Multiple, different types
Membership	Entity must be in all superclasses	Entity is in exactly one superclass
Symbol	Arrows from multiple parents	U circle

Example: A VEHICLE_OWNER can be either a PERSON or a COMPANY. A VEHICLE_OWNER instance is exactly one of these — not both.



 Tip

Categories require a **surrogate key** (OwnerId) because the natural keys of different superclasses are incompatible types (e.g., Ssn vs. CompanyId).

EER-to-Relational Mapping

Mapping Specialization — Four Options

Given a superclass C with subclasses $S_1, S_2, \dots, S_k$:

Option	Schema design	Best when
A — Single table	One table for all subclasses; NULL for inapplicable columns; add a Type discriminator column	Few subclasses, few subclass-specific attributes. Simple queries but wastes space with NULLs.
B — Subclass tables only	One table per subclass; each table includes <i>all inherited</i> superclass attributes (no shared superclass table)	Total and disjoint. Fast subclass-only queries; no joins needed. But super-class-wide queries require UNION.

Option	Schema design	Best when
C — Superclass + subclass tables	Table for superclass with its attributes; each subclass table stores only subclass-specific attributes + FK to superclass	Most common. Works for overlapping or partial. Superclass-wide query uses superclass table alone; subclass-specific query joins two tables.
D — Superclass table with all attributes	Single table, same as Option A but no separate subclass tables at all	Legacy or read-only schemas; maximum NULL tolerance.

 Tip
Default choice: Option C.

```

EMPLOYEE(Ssn PK, Fname, Lname, Salary)
SECRETARY(Ssn FK→EMPLOYEE, TypingSpeed)
ENGINEER(Ssn FK→EMPLOYEE, EngType)
MANAGER(Ssn FK→EMPLOYEE, MgrBonus)

```

A query asking for all engineers joins EMPLOYEE with ENGINEER. A query asking for all employees' names goes to EMPLOYEE alone.

Mapping Aggregation

1. Map the aggregated relationship to a junction table (Step 5 from Ch 3).
2. Add a new table for the outer relationship that includes the junction table's PK as a foreign key.

Example:

```

WORKS_ON(Essn, Pno, Hours)          -- from aggregated Works_On
USES(Essn, Pno, MachineId, StartTime) -- new table; (Essn,Pno) FK→WORKS_ON

```

Mapping Categories

1. Assign a **surrogate key** to the category that is type-neutral.
2. Create the category table using only the surrogate key plus category-specific attributes.
3. Add the surrogate key as a FK to each superclass table.

Example:

```
VEHICLE_OWNER(OwnerId PK, LicenseCount)
PERSON(Ssn PK, Name, Bdate, OwnerId FK→VEHICLE_OWNER)
COMPANY(CompanyId PK, CompanyName, OwnerId FK→VEHICLE_OWNER)
```

EER vs. Object-Oriented Mapping

The EER specialization/generalization hierarchy closely resembles inheritance in object-oriented languages. The correspondences are:

OOP concept	EER concept
Superclass / parent class	Superclass entity type
Subclass / child class	Subclass entity type
Single inheritance	Specialization hierarchy (tree)
Multiple inheritance	Specialization lattice
Abstract class	Total specialization superclass (no bare instances)
Concrete class	Partial specialization superclass (bare instances allowed)

The critical difference: OOP inheritance is about *behavior* (methods); EER inheritance is about *data* (attributes and relationships only).

Summary

! Chapter 4 — Key Takeaways

- **EER** extends ER with superclass/subclass type hierarchies, enabling **type inheritance**.
- **Specialization** (top-down) divides a superclass into subclasses. **Generalization** (bottom-up) abstracts common attributes into a new superclass. Both produce the same diagram.
- Two constraints must be declared for every specialization: **disjointness** (d/o) and **completeness** (total/partial).
- **Specialization hierarchy**: each subclass has exactly one superclass (tree). **Specialization lattice**: a subclass can have multiple superclasses (DAG) — multiple inheritance.
- **Aggregation** promotes a relationship to an entity so that it can itself participate in a further relationship.
- **Category (union type)**: a subclass whose instances can come from *different* superclass types

(union); contrast with lattice nodes (intersection).

- Mapping to relational: four options for specialization; **Option C** (superclass table + subclass tables with FK) is the standard default.

Review Questions

Conceptual

1. Define the **IS-A relationship** in EER modeling. Why is it fundamentally different from a regular ER relationship?
2. What is **type inheritance** in the context of EER? If **ENGINEER** is a subclass of **EMPLOYEE**, what attributes does an **ENGINEER** entity have?
3. Explain the difference between a **specialization hierarchy** and a **specialization lattice**. When does a lattice arise?

Short-Answer / Compare

4. Compare **total** and **partial** completeness constraints with **disjoint** and **overlapping** disjointness constraints. Give one real-world example for each of the four possible combinations.
5. Why is **aggregation** needed? Can the same modeling objective always be achieved with a ternary relationship instead? Give a scenario where aggregation is strictly necessary.
6. Compare a **category (union type)** to a **lattice node (shared subclass)**. Which requires a surrogate key, and why?

True / False (with explanation)

7. “In a total, disjoint specialization, a superclass entity may exist without belonging to any subclass.” — True or False? Explain.
8. “Type inheritance means that subclass tables in Option C must repeat all superclass attribute definitions.” — True or False? Explain.
9. “An overlapping specialization means every entity must belong to multiple subclasses.” — True or False? Explain.
10. “Aggregation and ternary relationships always model the same information, just with different notation.” — True or False? Explain.

Multiple Choice

11. Which mapping option for specialization is best when the specialization is **total and disjoint** and queries almost always target individual subclasses?
- (a) Option A **(b) Option B** (c) Option C (d) Option D
12. A STUDENT_EMPLOYEE that inherits from both STUDENT and EMPLOYEE is an example of:
- (a) A union type
 - (b) A category
 - **(c) A shared subclass in a specialization lattice**
 - (d) An aggregation

Design Exercise

13. A hospital manages three kinds of patients: **Inpatients** (admitted, have a bed number and ward), **Outpatients** (have scheduled appointments), and some patients who are both inpatients and outpatients (continuity-of-care cases).
- (a) Model this using EER notation: superclass, subclasses, disjointness constraint, completeness constraint.
 - (b) Is the disjointness constraint **disjoint** or **overlapping**? Is the completeness **total** or **partial**? Justify.
 - (c) Map the EER model to relational tables using Option C. Write the table schemas with primary and foreign keys.

SQL / Query Exercise

14. Using Option C mapping for EMPLOYEE → {SECRETARY, ENGINEER, MANAGER}:

```
EMPLOYEE(Ssn, Fname, Lname, Salary)
ENGINEER(Ssn, EngType, LicenseNum)
```

Write a SQL query that retrieves the full name, salary, and engineering type for all engineers. Then write a second query that returns *all* employees including non-engineers (NULLs where engineering data is absent). Which SQL keyword enables the second result?

Part III: The Relational Model

The Relational Data Model

Chapter 5

Future users of large data banks must be protected from having to know how the data is organized in the machine.

—E. F. Codd, 1970

A Brief History — Why the Relational Model?

The relational model was introduced in **1970** by the English computer scientist **Edgar F. Codd** in his landmark paper “*A Relational Model of Data for Large Shared Data Banks*”. To appreciate the model’s design choices, it helps to understand the problems it was solving.

The Problems Before 1970

Before the relational model, databases were based on **hierarchical** (tree-shaped) and **network** (graph-shaped) models. These models forced application programmers to:

- **Know the physical layout of data** — any change to file structure or storage required rewriting every application that touched that data. Codd called this *physical data dependence*.
- **Navigate the “Connection Trap”** — to retrieve related data in network models, a programmer had to walk explicit pointer paths. Missing a pointer or traversing in the wrong direction produced wrong results silently.
- **Accept inconsistency** — there was no mathematical foundation to define what “consistent” even meant, so the same fact could be stored contradictorily in different parts of the database.

What Codd Set Out to Fix

Codd’s 1970 paper targeted three specific goals:

Goal	Problem it Solved
Data independence	Programs should work unchanged even when physical storage changes
Resolving the Connection Trap	Data relationships should be expressed declaratively (by value), not by physical pointers
Mathematical consistency	A sound basis — set theory and predicate logic — to define, query, and reason about data

The key insight was to represent all data as **tables of values**, where relationships between tables are expressed by **matching values** (foreign keys) rather than by pointers. A query language can then declare *what* data is needed, not *how* to navigate to it — this is the foundation that SQL is built on.

i Note

From Chapter 3 to Chapter 5. Chapters 3 and 4 produced an ER/EER diagram — Codd’s conceptual-design layer, free of implementation detail. This chapter makes the transition to the relational layer: the precise mathematical structure that every ER diagram is eventually mapped into.

! Important

Design order matters. In this course, relational tables are derived from ER diagrams using the 7-step mapping covered at the end of this chapter. We do **not** design tables from scratch in this chapter. If you need to define a new table, first draw the ER diagram, then map. Skipping the ER step leads to schemas with redundancy and missing constraints.

From ER to Relational — Vocabulary Bridge

Before diving into theory, it helps to see how the concepts you already know from Chapter 3 translate into relational terminology. Every ER construct has an exact counterpart in the relational world.

ER World (Ch 3)	Relational World (Ch 5)	Notes
Entity	Relation / Table	One rectangle → one table
Entity instance	Tuple / Row	One real-world object → one row
Attribute	Attribute / Column	Same word, same idea

ER World (Ch 3)	Relational World (Ch 5)	Notes
Key attribute (underlined oval)	Primary key	The chosen unique identifier
Partial key (dashed-underline oval)	Part of composite PK	Combined with owner's FK to form the PK
Composite attribute	Multiple columns	Each sub-attribute becomes its own column
Derived attribute (dashed oval)	Not stored — computed in queries	Never becomes a column
Multivalued attribute (double oval)	Separate table with FK	Cannot be a single column
Strong entity	Table with its own PK	Stands alone
Weak entity	Table with composite PK (partial key + owner FK)	Cannot exist without owner
1:N relationship	Foreign key column added to the N side	No new table needed
1:1 relationship	Foreign key column on the total-participation side	No new table needed
M:N relationship	New junction table with composite PK	Always creates a new table
Relationship attribute	Column in the junction or relationship table	Moves with the relationship

 Tip

The key taxonomy you will learn in this chapter — superkey, candidate key, primary key, alternate key, foreign key, composite key — is **relational model vocabulary**. In the ER world you only had “key attribute” and “partial key”. The full taxonomy only makes sense once you have a table with rows.

Where Do Tables Come From?

Before we dive into the mathematical definition of tables, domains, and keys, we must answer a fundamental question: **Where do the tables come from?**

We don't just invent tables randomly. We design them based on how we perceive the real world. This process starts with two core concepts: **Entities** and **Relationships**.

Database design is usually a three-step process:

1. **Conceptual Design (The “What”)**: We create an abstract model of the real world. We identify the things that matter (entities) and how they are connected (relationships). This is often done using an Entity-Relationship (ER) diagram. This model is independent of any specific database software.
2. **Logical Design (The “How”)**: We take our conceptual model and map it to a specific data model, such as the Relational Model. We define tables (relations), columns (attributes), keys, and constraints. This is the main subject of this chapter.
3. **Physical Design (The “Where”)**: We implement our logical design on a specific Database Management System (DBMS) like MySQL, PostgreSQL, or Oracle, deciding on file organizations and indexes.

Mathematical Foundation

The relational model was proposed by **E.F. Codd** in 1970 and grounded in **set theory** and **first-order predicate logic**.

A **relation** R of degree n is a subset of the Cartesian product of n domains:

$$R \subseteq D_1 \times D_2 \times \dots \times D_n$$

Each element of R is a **tuple** $t = (v_1, v_2, \dots, v_n)$ where $v_i \in D_i$.

A **domain** D_i is a set of **atomic** (indivisible) values of the same type. For example: - $D_{Salary} = \{x \in \mathbb{R} \mid x \geq 0\}$ - $D_{Gender} = \{'M', 'F', 'N'\}$

Domain Specification Components

Saying “a domain is a data type” is only a starting point. A complete domain specification in a real system has seven components. These map directly to the SQL constraints you write on each column.

Component	What it Specifies	SQL Mechanism	Example
1. Data Type	The kind of values (integer, text, date ...)	INT, VARCHAR, DATE, DECIMAL, BOOLEAN	Salary DECIMAL(10, 2)
2. Length / Precision	Maximum characters, total digits, decimal places	VARCHAR(n), CHAR(n), DECIMAL(p, s)	Name VARCHAR(50), GPA DECIMAL(3, 2)
3. Date Format	How date/time values are interpreted	DATE (ISO 8601: YYYY-MM-DD internally)	Bdate DATE stores 1995-08-20
4. Range	Lower and upper bounds on numeric or date values	CHECK(col BETWEEN a AND b)	CHECK(GPA BETWEEN 0.0 AND 4.0)
5. Constraints	Rules that restrict allowed values beyond type	NOT NULL, UNIQUE, CHECK(expr), PRIMARY KEY, FOREIGN KEY	CHECK(Gender IN ('M', 'F', 'N'))
6. NULL Support	Whether the attribute may be absent	NULL (default) or NOT NULL	Middle_Name VARCHAR(30) — nullable
7. Default Value	Value used when none is supplied on INSERT	DEFAULT expr	Status VARCHAR(10) DEFAULT 'Active'

i Note

Date formats. SQL stores DATE values in ISO 8601 format (YYYY-MM-DD) internally regardless of how an application displays them. You may see DD/MM/YYYY in European interfaces or MM/DD/YYYY in American ones — these are *display* formats, not storage formats. Always insert dates as '2026-02-07'.

Example: Domain Specification for a Student Table Column

```
-- Component 1 (Data Type) + Component 2 (Precision): DECIMAL(3,2)
-- Component 4 (Range): CHECK(GPA BETWEEN 0.0 AND 4.0)
-- Component 5 (Constraint): NOT NULL
-- Component 7 (Default): DEFAULT 0.0
GPA DECIMAL(3,2) NOT NULL DEFAULT 0.0 CHECK(GPA BETWEEN 0.0 AND 4.0)

-- Component 1 (Data Type): VARCHAR(50)
```

```
-- Component 2 (Length):      max 50 chars
-- Component 6 (NULL):        nullable (middle name may not exist)
Middle_Name VARCHAR(50)

-- Component 1: DATE
-- Component 3 (Date Format): always stored YYYY-MM-DD
-- Component 5: NOT NULL (every student has a birth date)
Bdate DATE NOT NULL
```

! Important

NULL has three meanings — even though it looks like one thing, NULL in a column can represent three different real-world situations:

Reason	Example
Unknown — the value exists but we don't know it	Bdate NULL for a historical record where birth date is unrecorded
Not applicable — the attribute makes no sense for this entity	Spouse_Name NULL for an unmarried employee
Does not exist — the entity genuinely has no value	Second_Phone NULL for a person who has only one phone number

The database cannot distinguish these three meanings from NULL alone. Use documentation or separate boolean flags when the distinction matters.

Key Terminology

Formal Term	SQL Term	Meaning
Relation	Table	A set of tuples; no duplicates
Tuple	Row / Record	One entity instance
Attribute	Column / Field	A named domain variable
Domain	Data type	Allowed values for an attribute
Degree	Arity	Number of attributes
Cardinality	Row count	Number of tuples
Relation schema	Table definition	$R(A_1 : D_1, \dots, A_n : D_n)$
Relation instance	Table contents	The current set of tuples

Formal Term	SQL Term	Meaning
Database schema	All table definitions	Collection of all relation schemas
Database state	All table contents	Snapshot of all current tuples

Properties of Relations

Property	Formal Relation	SQL Table
No duplicate tuples	Required by set theory	Enforced only with PK or UNIQUE
Tuple ordering	Irrelevant	Undefined without ORDER BY
Attribute ordering	Irrelevant	Fixed by column position
Atomic attribute values	Required (1NF)	Required in normalized tables
NULL values	Not allowed (formal model)	Allowed in ANSI SQL

Note

SQL tables are **multisets** (bags), not sets. Without a primary key, SQL allows duplicate rows. The formal relational model does not. Always define a primary key.

Attribute Types

An **attribute** represents a property or characteristic of an entity. Not all attributes behave the same way, and understanding their differences directly affects how you design a relational schema.

Classification Overview

Attributes

- ├ By Structure
 - | ├ Simple (Atomic) – cannot be subdivided
 - | └ Composite – can be subdivided into sub-attributes
- ├ By Multiplicity

	— Single-Valued	– exactly one value per entity
	— Multi-Valued	– multiple values per entity → always needs a separate table
—	By Storage	
	— Stored	– explicitly stored in the database
	— Derived	– computed from stored attributes → never stored as a column
	— NULL	– value is absent (unknown / not applicable / does not exist)

1. Simple (Atomic) Attributes

Cannot be subdivided into smaller, meaningful parts. Each value is a single indivisible unit.

Attribute	Why Atomic
Ssn CHAR(9)	Used as one 9-character block; splitting it has no meaning
Age INT	A single integer, indivisible
ISBN VARCHAR(17)	Treated as one code, not split by segment
EmployeeID INT	Single surrogate integer

In the relational model: simple attributes map directly to a single column. This is the normal case.

2. Composite Attributes

Can be logically divided into sub-attributes that each carry independent meaning.

Composite Attribute	Sub-attributes
Name	Fname, Minit, Lname
Address	Street, City, State, ZIP, Country
Date_of_Birth	Day, Month, Year
Phone	Country_Code, Area_Code, Number

In the relational model: composite attributes are **flattened** — each sub-attribute becomes its own column. The composite name itself does not appear as a column (Step 1 of ER-to-Relational Mapping).

```
-- Composite 'Name' → flattened into three columns
EMPLOYEE(Ssn,
          Fname NN,    -- sub-attribute
          Minit,       -- sub-attribute (nullable)
          Lname NN)    -- sub-attribute
-- 'Name' itself is NOT an attribute in the schema
```

 Tip

Flatten or not? Flatten when users will search, sort, or display sub-attributes independently. If Address will always be displayed as one string and never filtered by City, a single VARCHAR column may be acceptable — but it sacrifices flexibility.

3. Single-Valued Attributes

Have **exactly one value** per entity at any given time. This is the default assumption for relational attributes.

Example	Reason
StudentID INT	Each student has exactly one ID
Bdate DATE	Each person has one birth date
Current_GPA DECIMAL(3,2)	One current GPA per student
Department VARCHAR(50)	Employee belongs to one department at a time

In the relational model: single-valued attributes map directly to a standard column — no special handling needed.

4. Multi-Valued Attributes

Can have **multiple values** for a single entity.

Example	Multiple Values
Phone_Numbers	{+1-800-555-0101, +1-800-555-0102}
Email_Addresses	{alice@work.com, alice@home.com}
Skills	{Python, Java, SQL}
Dept_Locations	{Houston, Stafford, Bellaire}

In the relational model: multi-valued attributes **cannot** be a single column (that would violate 1NF — atomicity). They are always mapped to a **separate table** with a FK back to the owner (Step 6 of ER-to-Relational Mapping).

```
-- Multi-valued 'Skills' → separate table
EMPLOYEE_SKILLS(Ssn NN→EMPLOYEE(Ssn) {ON DELETE CASCADE ON UPDATE CASCADE},
                Skill NN)
PK: (Ssn, Skill)    -- composite PK: one row per (employee, skill)
```

! Important

Never store multiple values in one column. Comma-separated lists ('Python, Java, SQL') or multiple numbered columns (Skill1, Skill2, Skill3) both violate 1NF and break queries, indexes, and foreign keys. Always use a separate table.

5. Stored Attributes

Values are **explicitly written to and read from** the database. Every column that is not derived is a stored attribute.

Example	Stored As
Bdate DATE	The actual birth date value 1995-08-20
Salary DECIMAL(10,2)	The exact salary figure 75000.00
Hire_Date DATE	The calendar date the employee was hired

6. Derived Attributes

Values can be **computed from other stored attributes** at query time. They are **never stored as columns** — storing them creates redundancy and risks inconsistency (the stored value can go stale).

Derived Attribute	Computed From	Query Expression
Age	Bdate	EXTRACT(YEAR FROM AGE(Bdate))
Years_of_Experience	Hire_Date	EXTRACT(YEAR FROM AGE(Hire_Date))
Total_Salary	Base_Salary + Bonus	Base_Salary + Bonus
Total_Price	Quantity × Unit_Price	Quantity * Unit_Price

In the relational model: derived attributes are **omitted** from the schema entirely (Step 1 rule). Compute them in SELECT queries.

```
-- WRONG: Age stored as a column → goes stale on every birthday
EMPLOYEE(Ssn, Fname NN, Lname NN, Bdate, Age, ...)

-- CORRECT: Age omitted from schema; computed on demand
EMPLOYEE(Ssn, Fname NN, Lname NN, Bdate, ...)

-- Query: derive Age at read time – always current, no storage cost
SELECT Fname, Lname,
       EXTRACT(YEAR FROM AGE(Bdate)) AS Age
FROM Employee;
```

7. NULL Attributes

NULL represents the **absence of a value**. Recall the three meanings from §Domain Specification Components:

Reason	Attribute Example	How to Declare
Unknown — value exists but not recorded	Bdate DATE — person alive but date unknown	Bdate DATE (nullable, omit NOT NULL)
Not applicable — attribute doesn't apply	Spouse_Name VARCHAR(50) — unmarried person	Spouse_Name VARCHAR(50) (nullable)
Does not exist — entity has no such value	Second_Phone CHAR(10) — person has one phone	Second_Phone CHAR(10) (nullable)

Design rule: Mark a column NN when the business rule requires a value to always be present. Leave it without NN when the value is genuinely optional.

```
EMPLOYEE(Ssn,
          Fname NN,           -- required
          Minit,              -- optional (middle name may not exist)
          Lname NN,           -- required
          Salary NN CK(Salary > 0), -- required, with range check
          Super_ssn →EMPLOYEE(Ssn) -- nullable: CEO has no supervisor
          {ON DELETE SET NULL ON UPDATE CASCADE})
```

Summary: Attribute Types at a Glance

Type	Stored as Column?	Special Handling
Simple	Yes — one column	None
Composite	No — flatten to sub-attribute columns	Step 1 mapping
Single-Valued	Yes — one column	None
Multi-Valued	No — separate table with FK	Step 6 mapping
Stored	Yes — one column	None
Derived	No — omit; compute in query	Step 1 mapping
NULL	Same column, NULL/NOT NULL declaration	Nullability constraint

Formal Schema Notation

A **relation schema** defines the structure of a table: its name, attributes, data types, and constraints. There are three common ways to write the same schema. This book uses the **Formal Notation** (Style 1) exclusively — learn it thoroughly, then use the other styles to read SQL textbooks or exam questions.

Style 1 — Formal Notation (Used in This Book)

This is the compact, implementation-independent shorthand used throughout this course.

Symbol	Meaning	Example
First attribute(s) in the list	Primary Key (PK) — listed first; underlined in diagrams	EMPLOYEE(<u>Ssn</u> , ...)
NN	NOT NULL	Fname NN
U	UNIQUE	Dname NN U
CK(expr)	CHECK constraint	Salary CK(Salary >= 0)
→TABLE(pk)	Foreign Key referencing TABLE's pk column	Dno NN→DEPARTMENT(Dnumber)
{ON DELETE X ON UPDATE Y}	Referential action for that FK	Dno NN→DEPARTMENT(Dnumber) {ON DELETE RESTRICT ON UPDATE CASCADE}

Symbol	Meaning	Example
PK: (a, b) below the relation	Composite primary key (two or more attributes)	PK: (Essn, Pno)

Example — Student relation:

```
STUDENT(StudentID,
        Fname NN, Lname NN,
        Email NN U,
        Phone,
        GPA CK(GPA BETWEEN 0.0 AND 4.0),
        Major NN-DEPARTMENT(DeptCode) {ON DELETE SET NULL ON UPDATE CASCADE})
```

Reading it: StudentID is the PK (listed first). Fname and Lname must not be NULL. Email must be unique and not null. Phone is optional. GPA has a range check. Major is a FK to DEPARTMENT with cascading updates and NULL on department deletion.

Style 2 — Shorthand Notation

Used in many textbooks and exam questions for quick sketches. Attributes are listed without data types; the primary key is **underlined**.

```
STUDENT(StudentID, Fname, Lname, Email, Phone, GPA, Major)
```

```
ENROLLMENT(StudentID, CourseID, Semester, Grade)
```

Where: a dashed underline or arrow indicates a foreign key in many textbook diagrams.

When to use it: Quick whiteboard sketches, exam answers where full notation is not required. You lose all constraint information — this is a *naming* notation only.

Style 3 — SQL CREATE TABLE (Full Constraint Notation)

The most verbose and most precise. Used for actual database implementation. The same constraints expressed in Style 1 formal notation become CONSTRAINT clauses.

```
CREATE TABLE Student (
    StudentID      INT          PRIMARY KEY AUTO_INCREMENT,
    Fname          VARCHAR(50)  NOT NULL,
    Lname          VARCHAR(50)  NOT NULL,
    Email          VARCHAR(100) NOT NULL UNIQUE,
    Phone          CHAR(10),
    GPA            DECIMAL(3,2)  CHECK (GPA BETWEEN 0.0 AND 4.0),
    Major          VARCHAR(10),
    FOREIGN KEY (Major) REFERENCES Department(DeptCode)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Comparing the Three Styles

The following table shows the **same** ENROLLMENT relation written in all three styles.

Logical facts: - PK is composite: (StudentID, CourseID, Semester) - StudentID → FK to STUDENT - CourseID → FK to COURSE - Grade is optional (not yet assigned at enrollment) - Semester must not be NULL

Style	Representation
Formal (Book)	ENROLLMENT(StudentID NN→STUDENT(StudentID) {ON DELETE CASCADE ON UPDATE CASCADE}, CourseID NN→COURSE(CourseID) {ON DELETE CASCADE ON UPDATE CASCADE}, Semester NN, Grade) PK: (StudentID, CourseID, Semester)
Shorthand SQL	ENROLLMENT(<u>StudentID</u> , <u>CourseID</u> , <u>Semester</u> , Grade) CREATE TABLE Enrollment (StudentID INT NOT NULL, CourseID INT NOT NULL, Semester VARCHAR(20) NOT NULL, Grade CHAR(2), PRIMARY KEY (StudentID, CourseID, Semester), FOREIGN KEY (StudentID) REFERENCES Student(StudentID) ON DELETE CASCADE, FOREIGN KEY (CourseID) REFERENCES Course(CourseID) ON DELETE CASCADE);

! Important

Throughout this book, always use Style 1 (Formal Notation). It is language-independent and captures all constraints concisely. SQL (Style 3) is used when implementing schemas; Shorthand (Style 2) is used only in quick diagrams.

Company Database — Logical Schema

! From ER Diagram (Chapter 3) → Relational Schema (This Chapter)

The six relations below are derived directly from the Company Database ER diagram built step-by-step in **Chapter 3** using the **7-step ER-to-Relational mapping** (covered in §ER-to-Relational Mapping at the end of this chapter). Every attribute, key, and foreign key traces back to a specific ER construct — we do **not** invent tables from scratch.

Check Chapter 3 for the definitive ER diagram and attribute names.

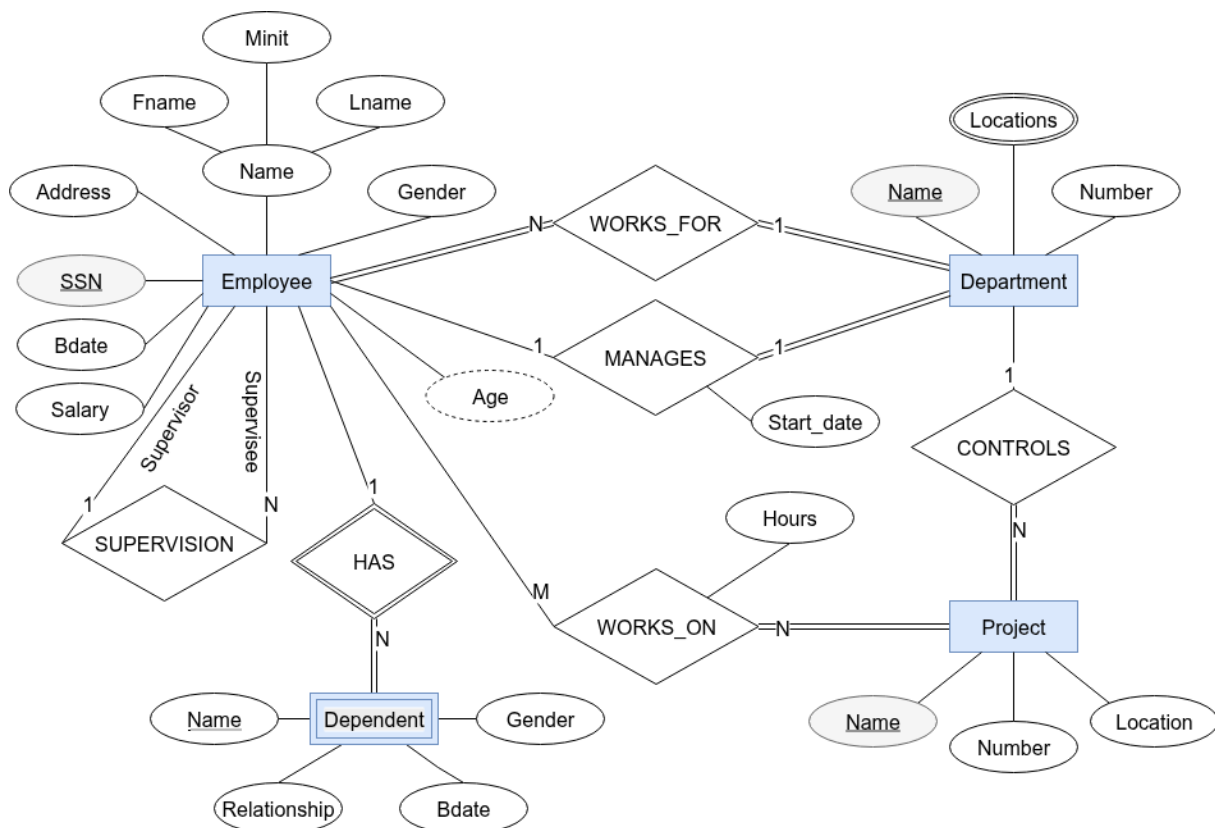


Figure 1: Company Database — ER Diagram (Chen Notation), as designed in Chapter 3

Logical Schema Tables (Full Detail)

Each table below shows the complete picture: every column, its data type, integrity constraints, which columns are primary or foreign keys, and the referential actions that keep the database consistent when rows are updated or deleted.

EMPLOYEE — strong entity EMPLOYEE (ER Step 1)

Column	Data Type	Key	Constraints	FK → Table(col)	ON UPDATE	ON DELETE
Ssn	CHAR(9)	PK	NOT NULL	—	—	—
Fname	VARCHAR(15)		NOT NULL	—	—	—
Minit	CHAR(1)			—	—	—
Lname	VARCHAR(15)		NOT NULL	—	—	—
Bdate	DATE			—	—	—
Address	VARCHAR(100)			—	—	—
Gender	CHAR(1)		CHECK (IN ('M', 'F', 'N'))	—	—	—
Salary	DECIMAL(10,2)		NOT NULL, CHECK (> 0)	—	—	—
Super_ssn	CHAR(9)		nullable	EMPLOYEE(Ssn)	CASCADE	SET NULL
Dno	INT	FK	NOT NULL	DEPART- MENT(Dnumber)	CASCADE	RESTRICT

Super_ssn is nullable — top-level managers have no supervisor (SUPERVISION is partial on both sides).

ON DELETE SET NULL on Super_ssn — if a supervisor is deleted, subordinates stay but their supervisor reference is cleared.

ON DELETE RESTRICT on Dno — a department with assigned employees cannot be deleted; reassign employees first.

DEPARTMENT — strong entity DEPARTMENT (ER Step 2)

Column	Data Type	Key	Constraints	FK → Table(col)	ON UPDATE	ON DELETE
Dnumber	INT	PK	NOT NULL	—	—	—

Column	Data Type	Key	Constraints	FK → Table(col)	ON UPDATE	ON DELETE
Dname	VARCHAR(25)		NOT NULL, UNIQUE	—	—	—
Mgr_ssn	CHAR(9)	FK	NOT NULL	EMPLOYEE(Ssn)	CASCADE	RESTRICT
Mgr_startdate	DATE			—	—	—

Mgr_ssn is NOT NULL — every department must have exactly one manager (DEPARTMENT is total in MANAGES).

ON DELETE RESTRICT on Mgr_ssn — you cannot delete an employee who is currently a department manager; reassign first.

DEPT_LOCATIONS — multivalued attribute Locations of DEPARTMENT (ER Step 2 / Mapping Rule 6)

Column	Data Type	Key	Constraints	FK → Table(col)	ON UPDATE	ON DELETE
Dnumber	INT	PK + FK	NOT NULL	DEPART- MENT(Dnumber)	CASCADE	CASCADE
Dlocation	VARCHAR(50)	PK	NOT NULL	—	—	—

Composite PK: (Dnumber, Dlocation) — a department can appear many times, once per location.

ON DELETE CASCADE — when a department is deleted, all its location rows are removed automatically.

PROJECT — strong entity PROJECT (ER Step 4)

Column	Data Type	Key	Constraints	FK → Table(col)	ON UPDATE	ON DELETE
Pnumber	INT	PK	NOT NULL	—	—	—
Pname	VARCHAR(25)		NOT NULL, UNIQUE	—	—	—
Plocation	VARCHAR(50)			—	—	—
Dnum	INT	FK	NOT NULL	DEPART- MENT(Dnumber)	CASCADE	RESTRICT

Dnum is NOT NULL — every project is controlled by exactly one department (PROJECT is total in CONTROLS).

ON DELETE RESTRICT — a department that controls active projects cannot be deleted.

WORKS_ON — M:N relationship WORKS_ON with attribute Hours (ER Step 6)

Column	Data Type	Key	Constraints	FK → Table(col)	ON UPDATE	ON DELETE
Essn	CHAR(9)	PK + FK	NOT NULL	EMPLOYEE(Ssn)	CASCADE	CASCADE
Pno	INT	PK + FK	NOT NULL	PROJECT(Pnumber)	CASCADE	CASCADE
Hours	DECIMAL(5,1)		CHECK (>= 0)	—	—	—

Composite PK: (Essn, Pno) — one row per (employee, project) pair.

ON DELETE CASCADE on both FKs — removing an employee or project automatically deletes the related work assignments.

DEPENDENT — weak entity DEPENDENT, owned by EMPLOYEE via HAS (ER Step 8)

Column	Data Type	Key	Constraints	FK → Table(col)	ON UPDATE	ON DELETE
Essn	CHAR(9)	PK + FK	NOT NULL	EMPLOYEE(Ssn)	CASCADE	CASCADE
Dependent_name	VARCHAR(50)	PK	NOT NULL	—	—	—
Gender	CHAR(1)		CHECK (IN ('M', 'F', 'N'))	—	—	—
Bdate	DATE			—	—	—
Relationship	VARCHAR(20)			—	—	—

Composite PK: (Essn, Dependent_name) — the partial key Dependent_name is unique only within one employee's dependents.

ON DELETE CASCADE — a dependent is a weak entity and cannot exist without its owning employee.

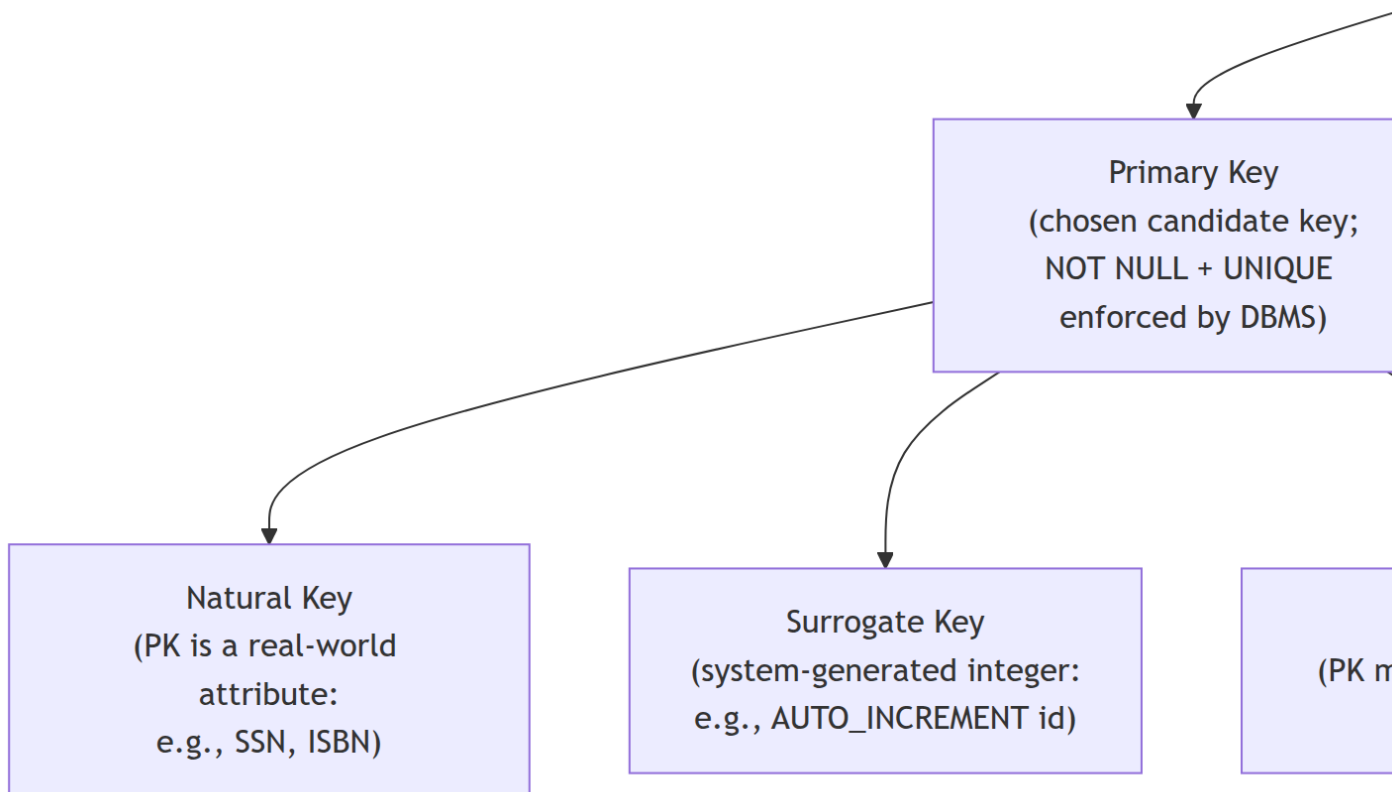
Compact Schema (Quick Reference)

The same six relations using the formal shorthand (identical convention used throughout §ER-to-Relational Mapping). PK = first/leading attribute(s); NN = NOT NULL; U = UNIQUE; CK () = CHECK; →TABLE(pk) = FK; {...} = referential actions; PK: (a, b) below = composite PK.

```
EMPLOYEE(Ssn,  
          Fname NN, Minit, Lname NN, Bdate, Address,  
          Gender CK(Gender IN ('M','F','N')),  
          Salary NN CK(Salary > 0),  
          Super_ssn →EMPLOYEE(Ssn)          {ON DELETE SET NULL    ON UPDATE CASCADE},  
          Dno      NN→DEPARTMENT(Dnumber)    {ON DELETE RESTRICT    ON UPDATE CASCADE})  
  
DEPARTMENT(Dnumber, Dname NN U,  
           Mgr_ssn    NN→EMPLOYEE(Ssn)      {ON DELETE RESTRICT    ON UPDATE CASCADE},  
           Mgr_start_date)  
  
DEPT_LOCATIONS(Dnumber→DEPARTMENT(Dnumber) {ON DELETE CASCADE    ON UPDATE CASCADE},  
               Dlocation NN)  
PK: (Dnumber, Dlocation)  
  
PROJECT(Pnumber, Pname NN U, Plocation,  
        Dnum NN→DEPARTMENT(Dnumber)        {ON DELETE RESTRICT    ON UPDATE CASCADE})  
  
WORKS_ON(Essn→EMPLOYEE(Ssn)                {ON DELETE CASCADE    ON UPDATE CASCADE},  
        Pno→PROJECT(Pnumber)                {ON DELETE CASCADE    ON UPDATE CASCADE},  
        Hours CK(Hours >= 0))  
PK: (Essn, Pno)  
  
DEPENDENT(Essn→EMPLOYEE(Ssn)                {ON DELETE CASCADE    ON UPDATE CASCADE},  
        Dependent_name NN,  
        Gender CK(Gender IN ('M','F','N')),  
        Bdate, Relationship)  
PK: (Essn, Dependent_name)
```

Complete Key Taxonomy

The word “key” is overloaded in database terminology. Here is the complete hierarchy:



Definitions

i Note

Coming from ER diagrams? The key attribute (underlined oval) in an ER diagram corresponds to the **primary key** here. A partial key (dashed-underline) on a weak entity becomes part of a **composite primary key** in the relational table. The rest of this taxonomy — superkey, candidate key, alternate key, foreign key — is relational-model vocabulary that has no direct ER equivalent.

Foreign keys specifically are created by the mapping rules in §ER-to-Relational Mapping at the end of this chapter.

Superkey: A set of one or more attributes $\{A_1, \dots, A_k\}$ such that no two tuples in any valid relation instance share the same values for all k attributes.

Candidate key: A *minimal* superkey — removing any attribute from the set destroys its uniqueness property.

Primary key: The candidate key chosen by the designer as the official identifier. The DBMS creates an *implicit* unique, not-null index for the PK.

Alternate key: Any candidate key that was not chosen as the PK. These should still be declared as UNIQUE NOT NULL (or UNIQUE if NULLs are possible) to prevent duplicate non-null values.

Unique key: A UNIQUE constraint placed on any column or column combination that is **not** the primary key. It guarantees no two non-NULL values are the same, but — unlike primary keys — it **permits** NULL values (typically more than one, since NULLs are considered distinct in most SQL engines). Every alternate key is implemented as a unique key, but not every unique key is a candidate key.

! Important

Primary Key vs Alternate Key vs Unique Key — the distinction:

	Primary Key	Alternate Key	Unique Key
Is it a candidate key?	Yes	Yes	Not necessarily
NULL allowed?	Never	Depends (usually NOT NULL)	Yes (NULLs permitted)
How many per table?	Exactly one	Zero or more	Zero or more
SQL declaration	PRIMARY KEY	UNIQUE NOT NULL	UNIQUE

Ex- StudentID Email (declared UNIQUE NOT NULL) Phone (declared UNIQUE, may be NULL)

In practice: declare your chosen PK with PRIMARY KEY; declare all other candidate keys with UNIQUE NOT NULL; declare non-key uniqueness requirements (e.g., usernames that may be left blank during registration) with UNIQUE alone.

```
CREATE TABLE Student (
    StudentID INT PRIMARY KEY, -- Primary Key: NOT NULL + UNIQUE enforced
    Ssn CHAR(9) UNIQUE NOT NULL, -- Alternate Key: also a candidate key, no
    Email VARCHAR(100) UNIQUE NOT NULL, -- Alternate Key: also a candidate key
    Phone CHAR(10) UNIQUE, -- Unique Key: uniqueness enforced, but NU
    Fname VARCHAR(50) NOT NULL,
    Lname VARCHAR(50) NOT NULL
);
```

Foreign key (FK): Attribute(s) in relation R_1 that reference the PK of relation R_2 . Enforces **referential integrity** between tables.

Natural key: A real-world attribute used as PK (SSN, passport number, ISBN). Pros: meaningful to users; Cons: changes in the real world break referential integrity.

Surrogate key: A system-generated integer (e.g., id INT AUTO_INCREMENT) used as PK. Pros: stable, compact, never changes; Cons: no business meaning.

Composite key: A PK made of two or more attributes. Required for M:N junction tables and weak entities.

Choosing a Primary Key

When multiple candidate keys exist, use this guide:

Criterion	Recommendation
Stability	Choose the candidate key most unlikely to change (SSNs change if government reassigns; id never changes)
Compactness	Prefer fewer bytes — integer (4 bytes) over UUID (16 bytes) over long string; smaller PK = faster indexes

Criterion	Recommendation
Natural vs. surrogate	Prefer surrogate for entities that are internal (orders, log entries); natural is acceptable for lookup tables (country code, ISO currency)
Composite vs. single-column	Single-column PKs are easier to reference via FKs in multiple tables; composite is fine for junction tables
Uniqueness guarantee	If no real-world attribute reliably guarantees uniqueness (people can share names, addresses), use a surrogate

Rule of Thumb

Use `id INT AUTO_INCREMENT` (or `BIGINT` for very large tables) as PK for most tables. Keep natural keys as `UNIQUE NOT NULL` alternate keys. This gives you: stability, compactness, and a meaningful constraint on the business attribute.

```
CREATE TABLE Employee (
    id          INT          AUTO_INCREMENT PRIMARY KEY,
    Ssn         CHAR(9)      NOT NULL UNIQUE,    -- alternate key
    Fname       VARCHAR(15)  NOT NULL,
    Salary      DECIMAL(10,2) CHECK (Salary > 0),
    Dno         INT          NOT NULL REFERENCES Department(id)
);
```

Key Analysis — Worked Examples

The following examples build key-identification skill systematically. For each example, the method is:

1. Identify **superkeys** — any attribute set that guarantees uniqueness based on business rules (not just current data).
2. Find the **minimal** superkeys → these are the **candidate keys**.
3. Choose one candidate key as the **primary key** (using the selection guide above).
4. Remaining candidate keys → **alternate keys** (declared `UNIQUE NOT NULL`).
5. Note any **unique keys** (`UNIQUE` but `NULL` permitted, or non-minimal).
6. Identify **foreign keys** from context.

Important

Never infer keys from sample data alone. If a column happens to have no duplicates in the current rows, that does not make it a key. A key must be justified by a *business rule* or a stated functional

dependency that holds for all past, present, and future valid instances.

Example 1 — Students Table (Multiple Candidate Keys, Nullable Alternate Key)

StudentID	Name	Email	Phone	Major	AdvisorID
1001	Alice	alice@uni.edu	555-0101	CS	501
1002	Bob	bob@uni.edu	555-0102	Math	502
1003	Alice	NULL	555-0103	CS	501

Semantic rules: StudentID is auto-generated and always unique. Email is university-assigned and unique when present, but may be NULL for new students. Name, Major, AdvisorID can repeat.

Superkey analysis:

Attribute Set	Unique in All Future Instances?	Verdict
{StudentID}	Yes — auto-generated, enforced	Superkey
{Email}	Yes for non-NULL values (NULLs are distinct in SQL)	Superkey
{Name}	No — two students named Alice already	Not a superkey
{Major}	No — CS appears twice	Not a superkey
{StudentID, Name}	Yes — but Name is redundant	Superkey, not minimal
{StudentID, Email}	Yes — but each alone already works	Superkey, not minimal

Results:

Key Type	Value	SQL Declaration
Candidate keys	{StudentID}, {Email}	—
Primary key	StudentID	PRIMARY KEY
Alternate key	Email	UNIQUE (NULL permitted for new students)
Unique key	Email	UNIQUE
Foreign key	AdvisorID → ADVISOR(AdvisorID)	FOREIGN KEY ... ON DELETE SET NULL

Formal schema notation:

```
STUDENT(StudentID,
        Name NN,
        Email U,
        Phone,
        Major,
        AdvisorID →ADVISOR(AdvisorID) {ON DELETE SET NULL ON UPDATE CASCADE})
```

Example 2 — Scoreboard (Composite Candidate Key)

A racing game records scores. Each player has at most one score per year.

Player	Year	Score
Mario	2023	9800
Luigi	2023	8750
Luigi	2024	9100
Mario	2024	9800

Semantic rule: One score per player per year. The same score value can appear for different players or in different years.

Superkey analysis:

Attribute Set	Unique?	Reason
{Player}	No	Luigi appears twice
{Year}	No	2023 appears twice
{Score}	No	9800 appears twice
{Player, Year}	Yes	Business rule: one score per player per year
{Player, Score}	No	Same player could score the same in different years
{Score, Year}	No	Different players could tie in the same year
{Player, Year, Score}	Yes	Contains {Player, Year}

Results:

Key Type	Value
Candidate key	{Player, Year}
Primary key	(Player, Year) — composite
Alternate keys	None
Foreign keys	None

! Important

The Superset Principle. Once {Player, Year} is a superkey, every superset — {Player, Year, Score}, etc. — is automatically also a superkey. Only the *minimal* superkeys are candidate keys. Listing every possible superkey is unnecessary.

Formal schema notation:

SCOREBOARD(Player NN, Year NN, Score NN)
 PK: (Player, Year)

Example 3 — Employee / Department / Project (Multi-Table)

Employee table — semantic rules: EmployeeID system-generated, unique. Email unique per employee. DepartmentID repeats.

Key Type	EMPLOYEE	DEPARTMENT	PROJECT
Candidate keys	{EmployeeID}, {Email}	{DepartmentID}, {DepartmentName}	{ProjectID}
Primary key	EmployeeID	DepartmentID	ProjectID
Alternate keys	Email	DepartmentName	None
Unique keys	Email	DepartmentName	None
Foreign keys	DepartmentID → DEPARTMENT	None	ManagerID → EMPLOYEE

Formal schema notation:

EMPLOYEE(EmployeeID,
 FirstName NN, LastName NN,
 Email NN U,
 DepartmentID NN→DEPARTMENT(DepartmentID) {ON DELETE RESTRICT ON UPDATE CASCADE})

DEPARTMENT(DepartmentID, DepartmentName NN U)


```
PROJECT(ProjectID,
        ProjectName NN,
        ManagerID NN-EMPLOYEE(EmployeeID) {ON DELETE RESTRICT ON UPDATE CASCADE})
```

Example 4 — Online Store Orders (Surrogate Key, Composite Trap)

An online store records orders. *OrderID* is always unique. A customer may order the same product twice on the same day.

OrderID	CustomerID	OrderDate	ProductID
1001	C1	2026-01-15	PRD-A
1002	C1	2026-01-15	PRD-B
1003	C2	2026-01-16	PRD-A

Why {CustomerID, OrderDate, ProductID} is not a candidate key: The business allows a customer to order the same product twice on the same day. Future rows could duplicate that triple — so it is not a reliable superkey.

Results:

Key Type	Value
Candidate key	{OrderID} only
Primary key	OrderID (surrogate)
Alternate keys	None
Foreign keys	CustomerID → CUSTOMERS, ProductID → PRODUCTS

Formal schema notation:

```
ORDER(OrderID,
       CustomerID NN-CUSTOMERS(CustomerID) {ON DELETE RESTRICT ON UPDATE CASCADE},
       OrderDate NN,
       ProductID NN-PRODUCTS(ProductID) {ON DELETE RESTRICT ON UPDATE CASCADE})
```

Example 5 — University Students (Nullable Candidate Keys)

Semantic rules: `StudentID` — never NULL, always unique. `Ssn` — unique when present, NULL for international students. `Email` — unique when present, NULL during registration. `Phone` and `Name` can repeat.

Results:

Key Type	Value	SQL
Candidate keys	{ <code>StudentID</code> }, { <code>Ssn</code> }, { <code>Email</code> }	—
Primary key	<code>StudentID</code> — never NULL, stable	PRIMARY KEY
Alternate keys	<code>Ssn</code> , <code>Email</code>	UNIQUE (NULLs permitted)
Foreign keys	None in this table	—

Formal schema notation:

```
STUDENT(StudentID,
        Fname NN, Lname NN,
        Ssn U,
        Email U,
        Phone)
```

i Note

`Ssn U` and `Email U` use the U (UNIQUE) symbol without NN, reflecting that NULLs are permitted. In SQL: UNIQUE without NOT NULL. Multiple NULLs are allowed because `NULL ≠ NULL` in SQL — each NULL is considered distinct and does not violate UNIQUE.

Example 6 — Self-Referencing FK (Employee Hierarchy)

Each employee may have a manager who is also an employee. The CEO has no manager.

EmpID	Name	ManagerID	DeptID
E001	Nour	NULL	D1
E002	Ahmed	E001	D1
E003	Sara	E001	D2
E004	Omar	E002	D1

Results:

Key Type	Value
Candidate key	{EmpID}
Primary key	EmpID
Alternate keys	None
Foreign keys	ManagerID → EMPLOYEE(EmpID) (self-ref), DeptID → DEPARTMENT

Formal schema notation:

```
EMPLOYEE(EmpID,
        Name NN,
        ManagerID →EMPLOYEE(EmpID)    {ON DELETE SET NULL  ON UPDATE CASCADE},
        DeptID NN →DEPARTMENT(DeptID)  {ON DELETE RESTRICT  ON UPDATE CASCADE})
```

Note

Self-referencing FK. The FK ManagerID points back to the same table (EMPLOYEE). ManagerID is nullable — the root of the hierarchy (CEO) has no manager. ON DELETE SET NULL means: if a manager row is deleted, their direct reports' ManagerID is set to NULL rather than the reports being deleted.

Integrity Constraints

Domain Constraint

Every attribute value must belong to the attribute's domain.

```
Gender  CHAR(1) CHECK (Gender IN ('M', 'F', 'N'))
Salary DECIMAL(10,2) CHECK (Salary >= 0)
```

Entity Integrity Constraint

No component of a primary key may be NULL.

The PK is the tuple's identifier. A NULL PK would mean “we don't know which entity this is” — making the row unidentifiable and violating the relational model.

Referential Integrity Constraint

Let R_1 have a foreign key FK referencing R_2 's primary key PK . The constraint is:

$$\forall t_1 \in r_1 : t_1[FK] = \text{NULL} \vee \exists t_2 \in r_2 : t_2[PK] = t_1[FK]$$

Plain English: Every non-NULL FK value in R_1 must match an existing PK value in R_2 .

Key Uniqueness Constraint

Every candidate key must hold distinct non-NULL values across all tuples.

! Constraint Violations — Quick Reference

Constraint	Violation example	Error message type
Domain	INSERT Salary = -5000	CHECK constraint failed
Entity integrity	INSERT INTO Employee (Ssn) VALUES (NULL)	Column 'Ssn' cannot be null
Referential integrity	INSERT Employee Dno=99 where Dept 99 doesn't exist	Foreign key constraint failed
Key uniqueness	Two rows with same Ssn	Duplicate entry for key 'Ssn'

Foreign Key Actions

When a referenced PK is deleted or updated, the DBMS executes the declared **referential action**:

Action	On DELETE	On UPDATE	Meaning
RESTRICT / NO ACTION	Default	Default	Reject the operation if matching FK rows exist
CASCADE	Delete matching FK rows	Update FK values to match new PK	Propagate automatically
SET NULL	Set FK to NULL	Set FK to NULL	Detach the referencing rows
SET DEFAULT	Set FK to default value	Set FK to default	Fill with a placeholder

Example 1 — What Each Action Does

Setup. The schema below declares `EMPLOYEE.Dno` as a FK referencing `DEPARTMENT`:

`DEPARTMENT(Dnumber: INT NN, Dname: VARCHAR(20) NN U)`

PK: (Dnumber)

`EMPLOYEE(Ssn: CHAR(9) NN, Fname: VARCHAR(15) NN,`

`Dno: INT → DEPARTMENT(Dnumber) {ON DELETE <action> ON UPDATE <action>})`

PK: (Ssn)

Initial data:

DEPARTMENT

Dnumber	Dname
5	Research
4	Administration

EMPLOYEE

Ssn	Fname	Dno
123456789	Alice	5
333445555	Bob	5
987654321	Carol	4

Case A — DELETE FROM DEPARTMENT WHERE Dnumber = 5

(Alice and Bob both reference Dnumber = 5)

Action on FK	What the DBMS does	EMPLOYEE after
RESTRICT	Operation rejected. Alice and Bob still reference Dno = 5.	Unchanged
CASCADE	Alice's row and Bob's row are deleted automatically .	Only Carol remains
SET NULL	Alice.Dno ← NULL , Bob.Dno ← NULL .	All 3 rows kept; Dno is NULL for Alice & Bob
SET DEFAULT	Alice.Dno ← default value (e.g., 1), Bob.Dno ← default.	All 3 rows kept; Dno = 1 for Alice & Bob

Case B — UPDATE DEPARTMENT SET Dnumber = 7 WHERE Dnumber = 5

Action on FK	What the DBMS does	EMPLOYEE after
RESTRICT	Operation rejected. Alice and Bob still hold Dno = 5.	Unchanged
CASCADE	Alice.Dno ← 7, Bob.Dno ← 7 automatically.	Dno = 7 for Alice & Bob
SET NULL	Alice.Dno ← NULL, Bob.Dno ← NULL.	All 3 rows kept; Dno is NULL
SET DEFAULT	Alice.Dno ← default , Bob.Dno ← default .	Dno = default for Alice & Bob

 Tip

Choosing the right action: - **CASCADE** is appropriate for weak entity owners (deleting a department deletes all employees in it). - **SET NULL** is appropriate when the relationship is optional (an employee can temporarily have no department). - **RESTRICT** is the safest default — it forces you to handle the dependency explicitly before deleting/updating.

Example 2 — Multiple FK References: One RESTRICT Blocks Everything

A single parent table can be referenced by **multiple child tables**, each with its own declared action. The critical rule is:

 Important

All FK constraints on the parent row are checked before the operation is allowed. If even **one** referencing table uses RESTRICT and has matching rows, the entire operation is rejected — regardless of what the other child tables declare.

Setup.

```
DEPARTMENT(Dnumber: INT NN, Dname: VARCHAR(20) NN U)
    PK: (Dnumber)
```

```
EMPLOYEE(Ssn: CHAR(9) NN, Fname: VARCHAR(15) NN,
          Dno: INT → DEPARTMENT(Dnumber) {ON DELETE CASCADE ON UPDATE CASCADE})
    PK: (Ssn)
```

```
PROJECT(Pnumber: INT NN, Pname: VARCHAR(15) NN,
        Dnum: INT → DEPARTMENT(Dnumber) {ON DELETE RESTRICT ON UPDATE RESTRICT})
    PK: (Pnumber)
```

Initial data:

DEPARTMENT

Dnumber	Dname
5	Research
4	Admin

EMPLOYEE

Ssn	Fname	Dno
123456789	Alice	5
333445555	Bob	5

PROJECT

Pnumber	Pname	Dnum
10	ProductX	5
20	Automation	4

Scenario A — Try to delete Research (Dnumber = 5):

The DBMS checks **both** child tables:

Child table	Action declared	Matching rows?	Result
EMPLOYEE	ON DELETE CASCADE	Alice, Bob → Dno = 5	Would cascade-delete them
PROJECT	ON DELETE RESTRICT	ProductX → Dnum = 5	BLOCKS the operation

→ **DELETE is rejected.** Neither the department row nor the employee rows are removed. The application must first delete or reassign PROJECT rows that reference Dnumber = 5, then retry.

Scenario B — Try to delete Admin (Dnumber = 4):

Child table	Action declared	Matching rows?	Result
EMPLOYEE	ON DELETE CASCADE	Carol → Dno = 4	Would cascade-delete Carol

Child table	Action declared	Matching rows?	Result
PROJECT	ON DELETE RESTRICT	Automation → Dnum = 4	BLOCKS the operation

→ **DELETE is also rejected** because Automation references Dnumber = 4.

Scenario C — Try to delete a department with NO child references (e.g., add Dnumber = 9 with no employees or projects):

Child table	Matching rows?	Result
EMPLOYEE	None	OK
PROJECT	None	OK

→ **DELETE succeeds** because no FK constraint is violated.

Note

Key takeaway: When designing a schema with multiple FK references to the same parent, think of RESTRICT as a *veto*. Any RESTRICT FK with live referencing rows will block the operation for the entire parent row, even if every other FK says CASCADE.

NULL Values and Three-Valued Logic

NULL represents *unknown*, *missing*, or *not applicable*. Different NULLs express different real-world meanings:

Reason for NULL	Example
Unknown	Bdate of an employee not on record
Not applicable	Super_ssn for the CEO (no supervisor)
Not yet known	Mgr_start_date during a transition period

Three-Valued Logic

Standard two-valued logic (TRUE/FALSE) does not work with NULLs. SQL uses **three-valued logic** (TRUE / FALSE / UNKNOWN):

P	Q	P AND Q	P OR Q
T	UNKNOWN	UNKNOWN	T
F	UNKNOWN	F	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Comparisons involving NULL always yield UNKNOWN:

`NULL = NULL` → UNKNOWN (not TRUE!)

`NULL <> NULL` → UNKNOWN

`NULL IS NULL` → TRUE (use `IS NULL` / `IS NOT NULL`)

! Important

Aggregate functions (SUM, AVG, MAX, MIN) **ignore NULL values**. `COUNT(*)` counts all rows; `COUNT(column)` counts only non-NULL values.

Business Rules: DB Constraints vs. Application Logic

A **business rule** is a statement about the organization that must always be true.

Where to enforce	Examples	Mechanism
Database (DBMS)	“Salary must be positive”; “Every employee must belong to a department”; “SSN is unique”	CHECK, NOT NULL, FOREIGN KEY, UNIQUE
Application code	“Managers must have 3 years experience”; “Enrollment requires prerequisite completion”	Business logic layer, triggers
Both	“Age must be ≥ 18”	CHECK in DB + client-side validation

💡 Tip

Prefer DB-level constraints wherever possible. Application code changes frequently; different applications may access the same database and accidentally skip validation. A DB constraint cannot

be bypassed from any application.

The exception: complex rules that require multi-table logic or dynamic thresholds are better expressed in **stored procedures** or **triggers** (or at the business logic layer for maintainability).

Handling Constraint Violations in Applications

The DBMS will reject any INSERT, UPDATE, or DELETE that violates a constraint — but simply letting the error crash your application is not acceptable. This section explains what each operation can violate and how to handle it properly.

What Each Operation Can Violate

Operation	Possible Violations
INSERT	PK duplicate; FK value not in parent table; NOT NULL column omitted; UNIQUE duplicate; CHECK fails; domain type mismatch
UPDATE	PK changed to a duplicate; FK changed to non-existent parent; NOT NULL set to NULL; UNIQUE creates duplicate; CHECK fails; referential integrity on parent PK update
DELETE	Referential integrity — deleting a parent row that has matching FK children (default: RESTRICT rejects the delete)

Note

DELETE does not violate PK, UNIQUE, NOT NULL, or CHECK constraints — those apply to the *values* in a row, not to removing rows.

Referential Actions Recap

When a parent row's PK is deleted or updated, the DBMS applies the declared action to the child FK rows:

Action	On DELETE	On UPDATE
RESTRICT / NO ACTION	Reject if child rows exist	Reject if child rows exist
CASCADE	Delete child rows	Propagate new PK value to FKs
SET NULL	Set FK to NULL	Set FK to NULL

Action	On DELETE	On UPDATE
SET DEFAULT	Set FK to its declared default	Set FK to its declared default

Application-Level Response Strategies

Knowing the DBMS will reject invalid data is not enough — your application must handle the error gracefully. The following strategies apply regardless of programming language or framework:

Strategy 1 — Pre-validate before issuing SQL

Check incoming data against business rules before sending the query. This avoids unnecessary round-trips to the database for obviously invalid input.

```
-- Example: check uniqueness before inserting
SELECT COUNT(*) FROM Student WHERE Email = 'newstudent@uni.edu';
-- If 0, proceed with INSERT; otherwise inform the user
```

Caveat: Pre-validation has a race condition in concurrent environments — another transaction may insert the same email between your check and your insert. Always combine pre-validation with constraint enforcement at the DB level.

Strategy 2 — Use transactions for multi-statement operations

Wrap related INSERT/UPDATE/DELETE statements in a single transaction. If any statement causes a violation, roll back the entire unit of work so no partial changes persist.

```
BEGIN;
INSERT INTO Department (Dnumber, Dname, Mgr_ssn, Mgr_start_date)
VALUES (5, 'Robotics', 'E999', '2026-01-01'); -- may fail if E999 doesn't exist
INSERT INTO Employee (Ssn, Fname, Lname, Salary, Dno)
VALUES ('E999', 'Laila', 'Nasser', 90000, 5);
COMMIT;
-- If Employee insert fails, the Department insert is rolled back
```

Strategy 3 — Catch and translate DBMS error codes

Every DBMS returns a specific error code for each constraint violation. Catch these and convert them into user-friendly messages.

Constraint Violated	MySQL Error	PostgreSQL Error	User Message (example)
PRIMARY KEY duplicate	1062	23505	“A student with this ID already exists.”
FOREIGN KEY fails	1452	23503	“The selected department does not exist.”

Constraint Violated	MySQL Error	PostgreSQL Error	User Message (example)
NOT NULL fails	1048	23502	“Name is required and cannot be blank.”
CHECK fails	3819	23514	“GPA must be between 0.0 and 4.0.”
UNIQUE fails	1062	23505	“This email is already registered.”

Strategy 4 — Use deferrable constraints for circular dependencies

Some schemas have circular foreign-key dependencies (e.g., Employee references Department, Department references Employee as manager). Inserting the first row of either table temporarily violates the other FK. The solution: declare FK constraints as DEFERRABLE INITIALLY DEFERRED and insert both rows in a single transaction.

```
-- PostgreSQL example: circular Employee ↔ Department dependency
ALTER TABLE Department
  ADD CONSTRAINT fk_dept_manager
  FOREIGN KEY (Mgr_ssn) REFERENCES Employee(Ssn)
  DEFERRABLE INITIALLY DEFERRED;    -- checked at COMMIT, not at INSERT

BEGIN;
  INSERT INTO Department (Dnumber, Dname, Mgr_ssn) VALUES (1, 'Research', 'E001');
  INSERT INTO Employee (Ssn, Fname, Lname, Dno) VALUES ('E001', 'Hana', 'Saeed', 1);
COMMIT;
-- Both FKs checked together at COMMIT – both rows exist, no violation
```

MySQL does not support DEFERRABLE. Use the two-phase ALTER TABLE DDL approach (create tables without the circular FK, then add it) as shown in §Final Company Database Schema.

Strategy 5 — Test edge cases explicitly

Write tests that deliberately trigger each constraint and verify the application responds correctly:

Test Case	Expected Behavior
Insert with duplicate PK	Application shows “ID already in use”
Insert with NULL in NOT NULL column	Application shows “Field is required”
Delete parent with children and RESTRICT	Application shows “Cannot delete — dependencies exist”

Test Case	Expected Behavior
Delete parent with children and CASCADE	Child rows deleted silently
FK to non-existent parent	Application shows “Referenced record not found”
CHECK constraint violated	Application shows business-rule-specific message

! Important

Design constraints carefully from the start. Changing a NOT NULL column to nullable (or vice versa) on a table with millions of rows is an expensive, sometimes risky migration. Decide at design time which attributes are truly required and which are optional, and encode that decision as a DB constraint — not as a comment or documentation note.

ER-to-Relational Mapping

Chapters 3 and 4 produced a complete ER / EER diagram for the Company domain. Now that we have the full relational vocabulary — relations, keys, foreign keys, referential actions, and NULL semantics — we can mechanically translate that diagram into a working set of SQL tables. A **seven-step algorithm** covers every legal ER construct; the rest of this section walks through each step on the Company case study and ends with the complete schema in one place.

i Algorithm Overview

Step	ER Construct	Mapping Rule
1	Strong entity type	One table per entity; flatten composites; omit derived; multivalued deferred to Step 6
2	Weak entity type	Composite PK = owner PK + partial key; FK to owner with ON DELETE CASCADE
3	Binary 1:1 relationship	FK on the total -participation side (NOT NULL); both-partial → FK nullable
4	Binary 1:N relationship	FK on the N-side ; NOT NULL if N-side is total, nullable if partial
5	Binary M:N relationship	Junction table; composite PK = both FKs; include relationship attributes

6	Multivalued attribute	New table; composite PK = owner FK + attribute value
7	N-ary relationship ($n > 2$)	New table; FKs from all n participating entities; PK = all FKs

i Schema Notation Shorthand (Recap)

Each step shows the mapping result in **formal schema notation** — a compact, implementation-independent description — before the SQL. The shorthand introduced at the start of this chapter (§Formal Schema Notation) is used throughout:

Symbol	Meaning	Example
First attribute(s)	Primary Key (PK) — underlined in textbook diagrams	EMPLOYEE(<u>Ssn</u> , ...)
NN	NOT NULL	Fname NN
U	UNIQUE	Dname NN U
CK(expr)	CHECK constraint	Salary CK(Salary >= 0)
→TABLE(pk)	Foreign Key referencing TABLE's pk attribute	Dno NN→DEPARTMENT(Dnumber)
{ON DELETE X ON UPDATE Y}	Referential action for that specific FK	Essn→EMPLOYEE(Ssn) {ON DELETE CASCADE ON UPDATE CASCADE}
Composite PK	Two or more leading FK attributes	WORKS_ON(Essn→EMPLOYEE(Ssn), Pno→PROJECT(Pnumber), ...)

Step 1 — Strong Entity Types

Rule: For each strong entity type E , create one relation. **Flatten** composite attributes (e.g., Name → Fname, Minit, Lname). **Omit** derived attributes — compute them at query time. Multivalued attributes are handled in Step 6.

ER Source — Company Database:

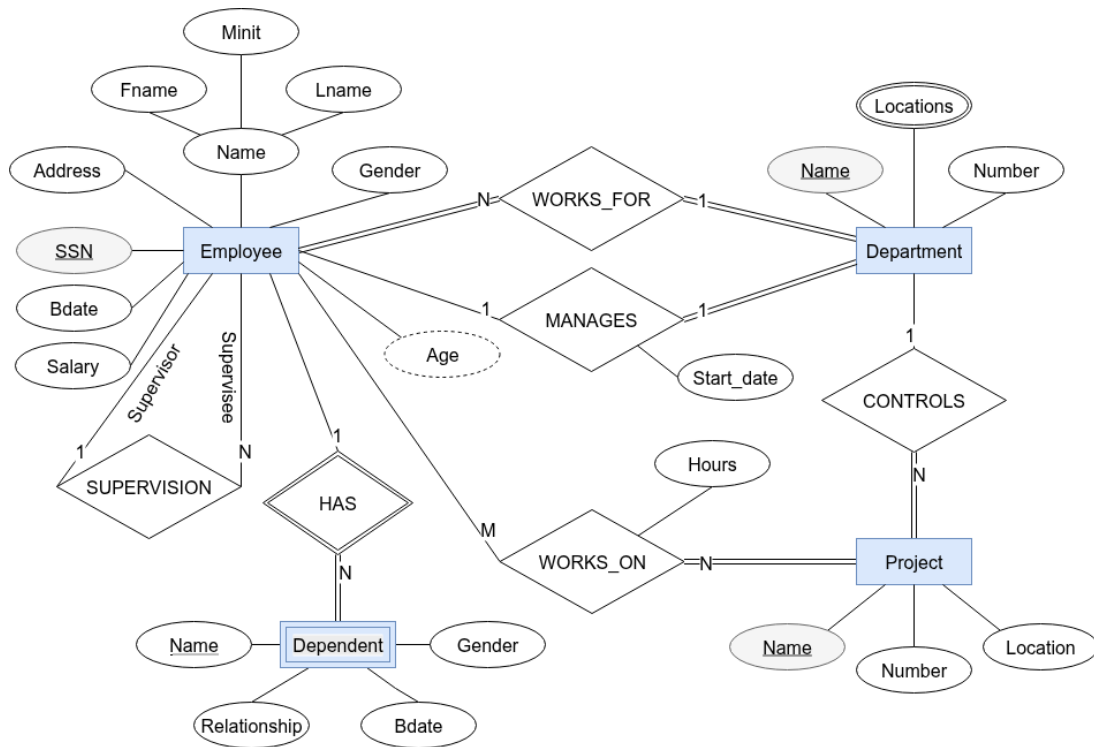


Figure 2: Full Company Database ER Diagram.

Formal Schema

```
EMPLOYEE(Ssn, Fname NN, Minit, Lname NN, Bdate,
        Address,
        Gender CK(Gender IN ('M','F')),
        Salary NN CK(Salary >= 0))
-- Super_ssn added in Step 4b | Dno added in Step 4a
```

```
DEPARTMENT(Dnumber CK(Dnumber > 0), Dname NN U)
-- Mgr_ssn and Mgr_start_date added in Step 3a
```

```
PROJECT(Pnumber CK(Pnumber > 0), Pname NN U, Plocation,
        Dnum NN→DEPARTMENT(Dnumber))
```

SQL Implementation

EMPLOYEE

```
CREATE TABLE Employee (
    ssn          CHAR(9)          PRIMARY KEY
                CHECK (ssn ~ '^d{9}$'),          -- fixed-length national ID, 9 digits
    fname        VARCHAR(15)      NOT NULL,
```

```
minit          CHAR(1),                -- nullable: not everyone has a mid
lname          VARCHAR(15)      NOT NULL,
-- composite 'Name' flattened into fname, minit, lname above ↑
bdate          DATE,                -- stored; derived 'Age' is omitted
address        VARCHAR(100),        -- single simple attribute as shown
gender         CHAR(1)      CHECK (gender IN ('M', 'F')),
salary         NUMERIC(10, 2) NOT NULL CHECK (salary >= 0)
-- super_ssn (self-referencing FK) added in Step 4b
-- dno (FK to Department)      added in Step 4a
);
```

Derived attribute rule: Age is derived from bdate — do **not** store it. Compute on demand:

```
EXTRACT(YEAR FROM AGE(bdate)) AS age
```

DEPARTMENT

```
CREATE TABLE Department (
  dnumber      INT          PRIMARY KEY CHECK (dnumber > 0),
  dname        VARCHAR(30)  NOT NULL UNIQUE
  -- mgr_ssn and mgr_start_date added in Step 3a
);
```

PROJECT

```
CREATE TABLE Project (
  pnumber      INT          PRIMARY KEY CHECK (pnumber > 0),
  pname        VARCHAR(25)  NOT NULL UNIQUE,
  plocation    VARCHAR(30),
  dnum         INT          NOT NULL,                -- total participation → NOT NULL
  FOREIGN KEY (dnum) REFERENCES Department(dnumber)
  ON DELETE RESTRICT                                -- cannot delete dept with active pr
  ON UPDATE CASCADE
);
```

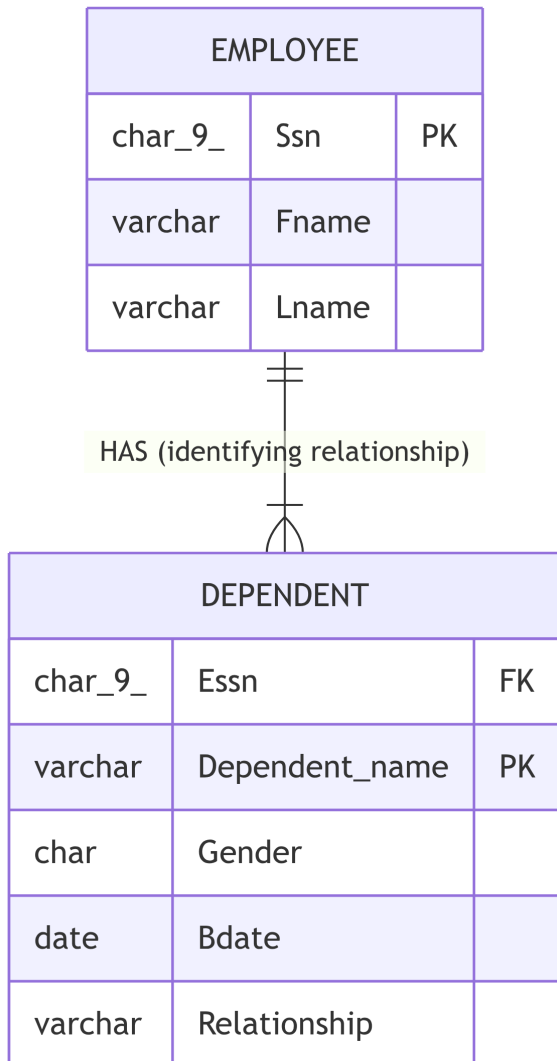
Step 2 — Weak Entity Types

Rule: For each weak entity type W with owner E :

- Create table R ; include all attributes of W .
- Add owner's PK as a FK column (NOT NULL).

- PK of R = **owner FK + partial key** of W (composite).
- Use ON DELETE CASCADE — the weak entity has no independent existence.

ER Source — DEPENDENT (weak entity, owner: EMPLOYEE):



Formal Schema

```
DEPENDENT(Essn NN-EMPLOYEE(Ssn) {ON DELETE CASCADE ON UPDATE CASCADE},
    Dependent_name NN,
    Gender CK(Gender IN ('M', 'F')),
    Bdate,
    Relationship NN CK(Relationship IN
        ('Spouse', 'Son', 'Daughter', 'Father', 'Mother', 'Sibling')))
PK: (Essn, Dependent_name)      -- composite: owner FK + partial key
```

SQL Implementation

```

CREATE TABLE Dependent (
    essn          CHAR(9)          NOT NULL,          -- owner FK (part of composite
    dependent_name VARCHAR(15)      NOT NULL,          -- partial key
    gender        CHAR(1)          CHECK (gender IN ('M', 'F')),
    bdate         DATE,
    relationship   VARCHAR(10)      NOT NULL
    CHECK (relationship IN
           ('Spouse', 'Son', 'Daughter', 'Father', 'Mother', 'Sibling')),
    PRIMARY KEY (essn, dependent_name),                -- composite PK
    FOREIGN KEY (essn) REFERENCES Employee(ssn)
    ON DELETE CASCADE                                -- employee deleted → all dependen
    ON UPDATE CASCADE
);

```

 Warning

ON DELETE CASCADE is mandatory for weak entities. A dependent cannot exist without its owner employee. Omitting it violates the identifying relationship semantics.

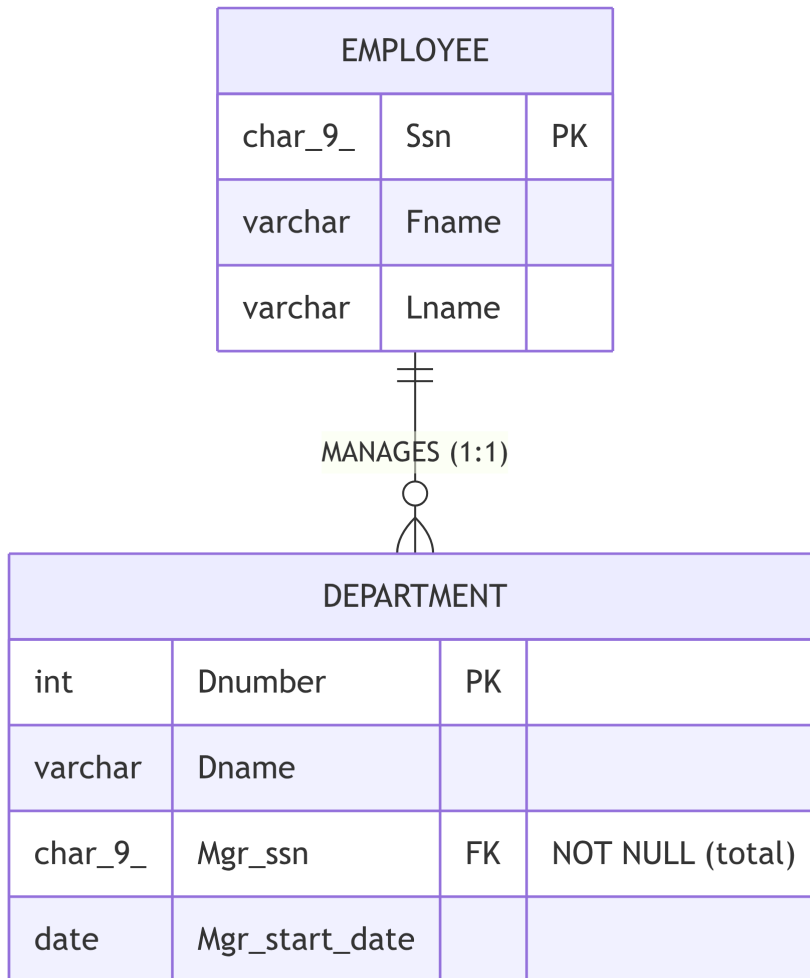
Step 3 — Binary 1:1 Relationships

Three participation scenarios produce different mapping decisions:

Scenario	Mapping rule	FK nullable?
(a) One total / one partial	FK on the total side; include relationship attrs	NOT NULL
(b) Both total	Merge tables or cross-FKs with DEFERRABLE	Both NOT NULL
(c) Both partial	FK on more stable entity	NULL allowed

(a) MANAGES — Department (total) : Employee (partial)

ER Source:



Formal Schema — FK and relationship attribute added to DEPARTMENT (total side):

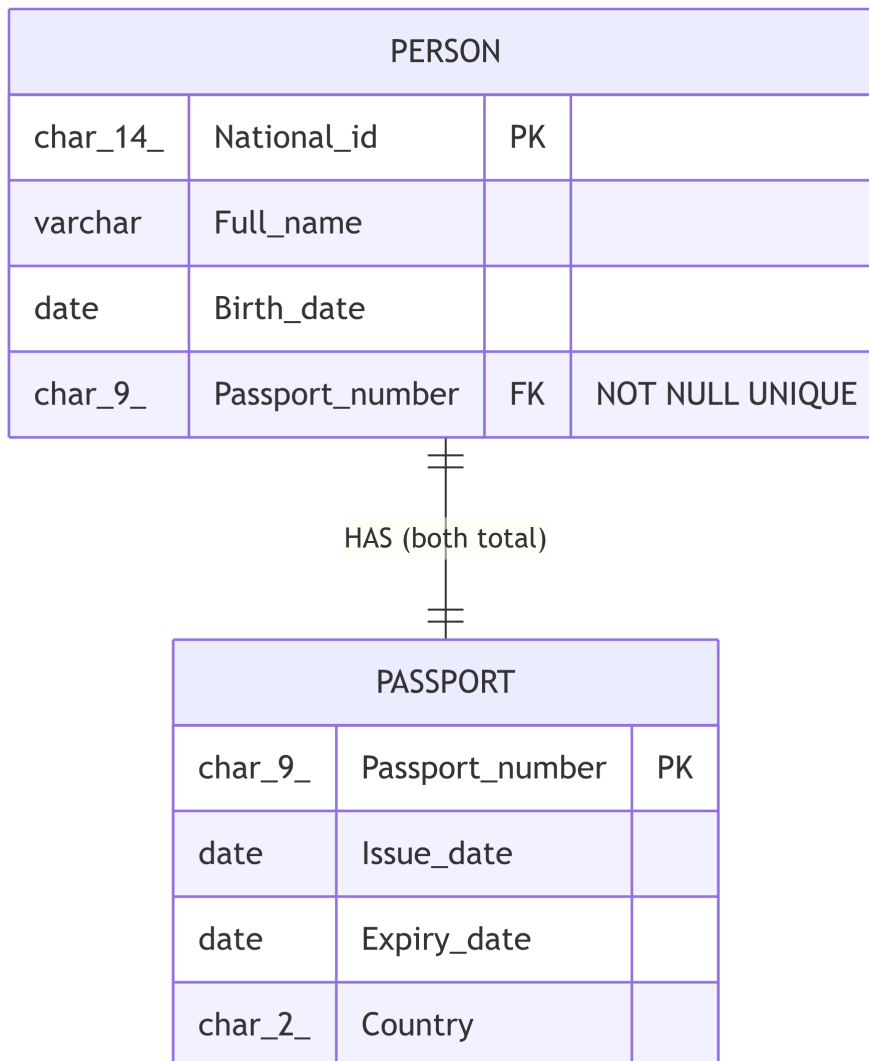
```
DEPARTMENT(Dnumber CK(Dnumber > 0), Dname NN U,
            Mgr_ssn NN→EMPLOYEE(Ssn),
            Mgr_start_date NN)
-- Mgr_ssn NOT NULL: every department must have exactly one manager (total participation)
-- Mgr_start_date is a relationship attribute of MANAGES
```

SQL:

```
ALTER TABLE Department
  ADD COLUMN mgr_ssn          CHAR(9)  NOT NULL,          -- total → NOT NULL
  ADD COLUMN mgr_start_date   DATE      NOT NULL DEFAULT CURRENT_DATE,
  ADD CONSTRAINT fk_dept_manager
    FOREIGN KEY (mgr_ssn) REFERENCES Employee(ssn)
    ON DELETE RESTRICT                                     -- cannot delete an active manager
    ON UPDATE CASCADE;
```

(b) Both-Total — Person : Passport

ER Source:



Formal Schema:

```
PASSPORT(Passport_number, Issue_date NN,
          Expiry_date NN CK(Expiry_date > Issue_date),
          Country NN)
```

```
PERSON(National_id, Full_name NN, Birth_date NN,
        Passport_number NN U→PASSPORT)
```

```
-- NN + UNIQUE together enforce 1:1
```

```
-- DEFERRABLE needed: Person and Passport must be inserted in a single transaction
```

SQL:

```
CREATE TABLE Passport (
    passport_number CHAR(9) PRIMARY KEY,
```

```

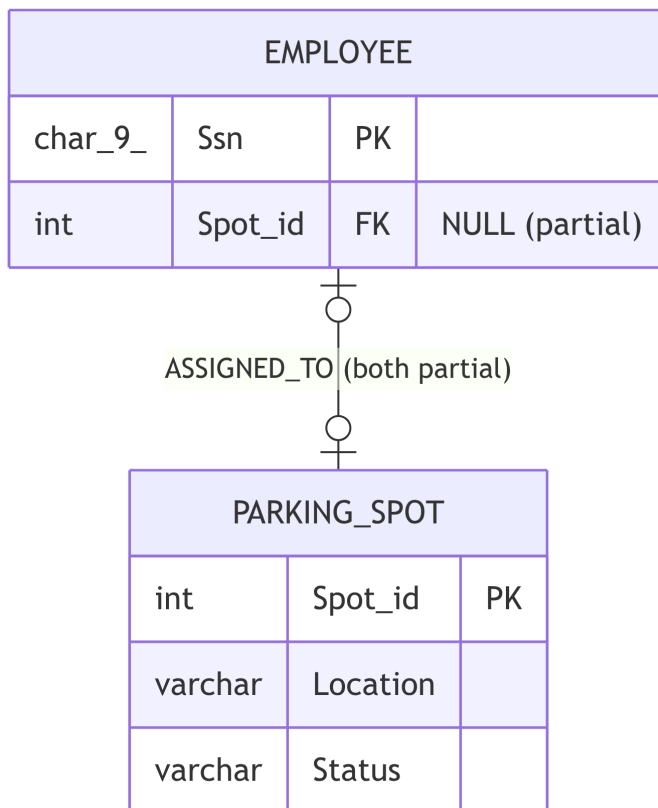
issue_date      DATE      NOT NULL,
expiry_date     DATE      NOT NULL CHECK (expiry_date > issue_date),
country         CHAR(2)    NOT NULL          -- ISO 3166-1 alpha-2
);

CREATE TABLE Person (
  national_id    CHAR(14)   PRIMARY KEY,
  full_name      VARCHAR(60) NOT NULL,
  birth_date     DATE       NOT NULL,
  passport_number CHAR(9)   NOT NULL UNIQUE,          -- total → NOT NULL; UNIQUE enf
  FOREIGN KEY (passport_number) REFERENCES Passport(passport_number)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
    DEFERRABLE INITIALLY DEFERRED          -- allow inserting both in one
);

```

(c) Both-Partial — Employee : ParkingSpot

ER Source:



Formal Schema:

```
EMPLOYEE(..., Spot_id→PARKING_SPOT)
-- Spot_id nullable: partial participation on both sides
-- ON DELETE SET NULL: spot freed automatically when employee departs
```

SQL:

```
ALTER TABLE Employee
  ADD COLUMN spot_id INT NULL, -- partial → nullable
  ADD CONSTRAINT fk_emp_parking
    FOREIGN KEY (spot_id) REFERENCES ParkingSpot(spot_id)
    ON DELETE SET NULL -- spot freed when employee lea
    ON UPDATE CASCADE;
```

Step 4 — Binary 1:N Relationships

Rule: Add the **1-side's PK** as a FK column to the **N-side** table; include any relationship attributes on the N-side.

N-side participation	FK nullability	Typical FK action
Total	NOT NULL	RESTRICT (protect parent)
Partial	NULL allowed	SET NULL or SET DEFAULT

(a) WORKS_FOR — Department 1 : N Employee (Employee total)

ER Source:



Figure 3: EMPLOYEE–DEPARTMENT relationship with Min-Max notation.

Formal Schema — FK added to EMPLOYEE (N-side, total):

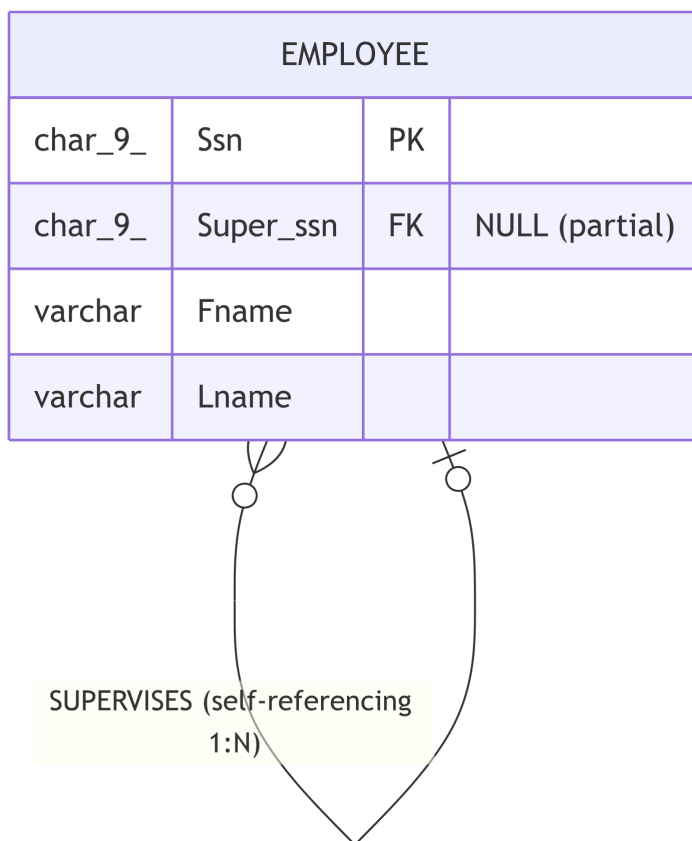
```
EMPLOYEE(Ssn, Fname NN, Minit, Lname NN, Bdate,
  Address,
  Gender CK(Gender IN ('M', 'F')),
  Salary NN CK(Salary >= 0),
  Super_ssn →EMPLOYEE(Ssn),
  Dno NN→DEPARTMENT(Dnumber))
-- Dno NOT NULL: every employee must belong to exactly one department (total)
```

SQL:

```
ALTER TABLE Employee
  ADD COLUMN dno INT NOT NULL,           -- total → NOT NULL
  ADD CONSTRAINT fk_emp_dept
    FOREIGN KEY (dno) REFERENCES Department(dnumber)
    ON DELETE RESTRICT                  -- cannot remove dept with empl
    ON UPDATE CASCADE;
```

(b) SUPERVISES — Employee self-references itself (both sides partial)

ER Source:



Formal Schema:

```
EMPLOYEE(..., Super_ssn→EMPLOYEE(Ssn))
  -- Super_ssn nullable: top executive has no supervisor (partial on both sides)
  -- ON DELETE SET NULL: subordinates become unmanaged when supervisor is removed
```

SQL:

```
ALTER TABLE Employee
  ADD COLUMN super_ssn CHAR(9) NULL,    -- partial → nullable
  ADD CONSTRAINT fk_emp_supervisor
```

```
FOREIGN KEY (super_ssn) REFERENCES Employee(ssn)
ON DELETE SET NULL -- supervisor deleted → subordi
ON UPDATE CASCADE;
```

i Note

Self-referencing FK — the FK points back to the same table (Employee references Employee).
The root of the reporting hierarchy has super_ssn = NULL.

(c) CONTROLS — Department 1 : N Project (Project total)

Already handled in Step 1: Project.dnum NOT NULL was declared with its FK. **No further action needed.**

Step 5 — Binary M:N Relationships

Rule: Create a **junction table** *R*. Add both entity PKs as FK columns. PK of *R* = combination of both FKs (both are NOT NULL since they form the key). Include relationship attributes as additional columns.

WORKS_ON — Employee M:N Project (relationship attribute: Hours)

ER Source:

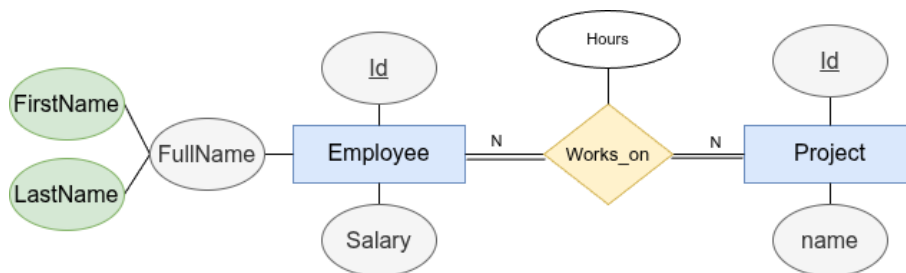


Figure 4: WORKS_ON M:N relationship with Hours attribute.

Formal Schema:

```
WORKS_ON(Essn→EMPLOYEE(Ssn) {ON DELETE CASCADE ON UPDATE CASCADE},
        Pno→PROJECT(Pnumber) {ON DELETE CASCADE ON UPDATE CASCADE},
        Hours CK(Hours >= 0 AND Hours <= 168))
PK: (Essn, Pno)
-- Both FKs are NOT NULL (they form the composite PK)
```

SQL:


```
CREATE TABLE Works_On (
    essn    CHAR(9)          NOT NULL,
    pno     INT              NOT NULL,
    hours   NUMERIC(5, 1)    CHECK (hours >= 0 AND hours <= 168),    -- max realistic work ho
    PRIMARY KEY (essn, pno),
    FOREIGN KEY (essn) REFERENCES Employee(ssn)
        ON DELETE CASCADE                                           -- employee leaves → remove ass
        ON UPDATE CASCADE,
    FOREIGN KEY (pno) REFERENCES Project(pnumber)
        ON DELETE CASCADE                                           -- project cancelled → remove a
        ON UPDATE CASCADE
);
```

Note

CASCADE vs RESTRICT on junction tables:

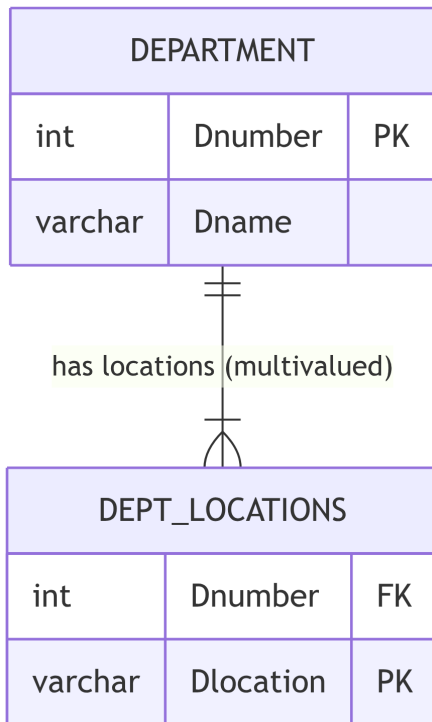
Junction row has ...	Recommended action
No independent meaning (pure association)	CASCADE — remove the link when either entity is removed
Independent business meaning (e.g., a contract, a grade)	RESTRICT — preserve the record even after one entity is deleted

Step 6 — Multivalued Attributes

Rule: For each multivalued attribute A on entity E , create a new table R . PK of R = (owner FK + attribute value). Use ON DELETE CASCADE.

DEPT_LOCATIONS — multivalued Locations on Department

ER Source:



Formal Schema:

```

DEPT_LOCATIONS(Dnumber→DEPARTMENT(Dnumber) {ON DELETE CASCADE ON UPDATE CASCADE},
               Dlocation NN)
PK: (Dnumber, Dlocation)    -- composite PK prevents duplicate locations per dept
    
```

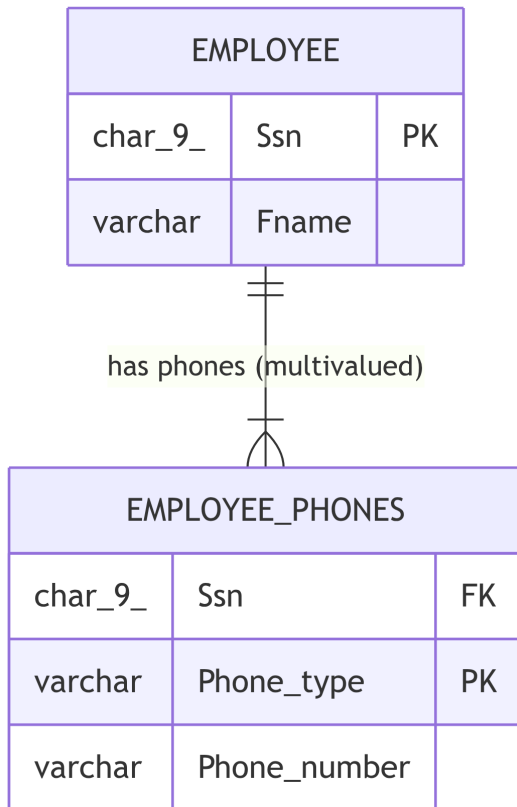
SQL:

```

CREATE TABLE Dept_Locations (
    dnumber      INT          NOT NULL,
    dlocation    VARCHAR(30)  NOT NULL,
    PRIMARY KEY (dnumber, dlocation),           -- composite PK prevents duplic
    FOREIGN KEY (dnumber) REFERENCES Department(dnumber)
        ON DELETE CASCADE                       -- department deleted → all loc
        ON UPDATE CASCADE
);
    
```

EMPLOYEE_PHONES — multivalued Phone_numbers on Employee

ER Source:



Formal Schema:

```

EMPLOYEE_PHONES(Ssn-EMPLOYEE(Ssn) {ON DELETE CASCADE ON UPDATE CASCADE},
    Phone_type NN CK(Phone_type IN ('Mobile', 'Home', 'Work')),
    Phone_number NN CK(Phone_number ~ '^\\+?[\\d\\s\\-\\(\\)]{7,20}$'))
PK: (Ssn, Phone_type)          -- one number per type per employee
    
```

SQL:

```

CREATE TABLE Employee_Phones (
    ssn          CHAR(9)          NOT NULL,
    phone_type   VARCHAR(10)      NOT NULL CHECK (phone_type IN ('Mobile', 'Home', 'Work')),
    phone_number VARCHAR(20)      NOT NULL
        CHECK (phone_number ~ '^\\+?[\\d\\s\\-\\(\\)]{7,20}$'),
    PRIMARY KEY (ssn, phone_type),
    FOREIGN KEY (ssn) REFERENCES Employee(ssn)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
    
```

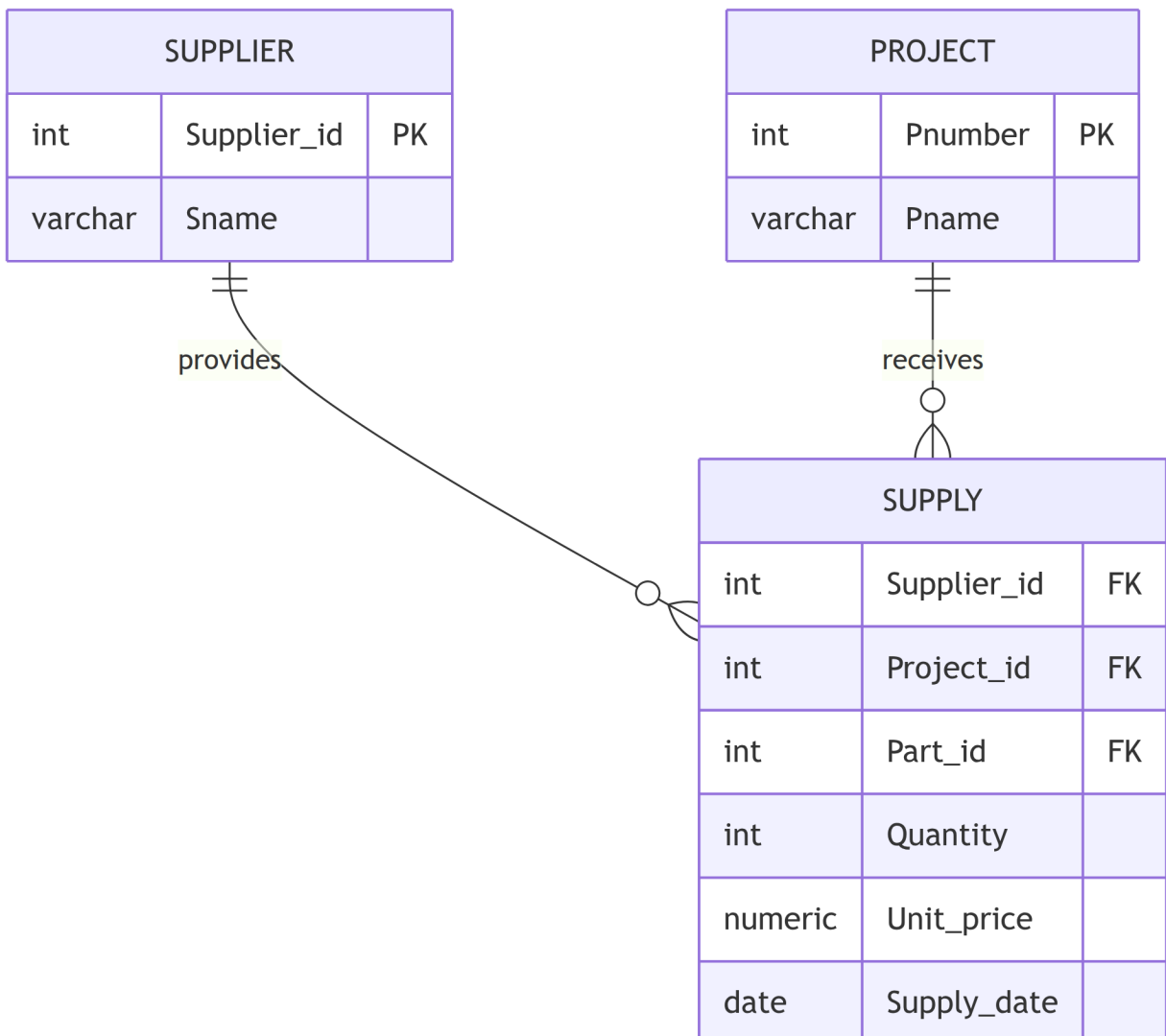
Step 7 — N-ary Relationships ($n > 2$)

Rule: Create a new table R . Add FKs from all n participating entity tables. PK of R = combination of all FKs. Include any relationship attributes as additional columns.

SUPPLY — ternary (Supplier $\times$ Project $\times$ Part)

A supplier supplies a specific part to a specific project.

ER Source:



Formal Schema:

```
SUPPLY(Supplier_id→SUPPLIER {ON DELETE RESTRICT ON UPDATE CASCADE},
      Project_id→PROJECT(Pnumber) {ON DELETE RESTRICT ON UPDATE CASCADE},
      Part_id→PART {ON DELETE RESTRICT ON UPDATE CASCADE},
      Quantity NN CK(Quantity > 0),
      Unit_price NN CK(Unit_price >= 0),
```

```

Supply_date NN)
PK: (Supplier_id, Project_id, Part_id)
-- All three FKs are NOT NULL (they form the composite PK)

```

SQL:

```

CREATE TABLE Supply (
    supplier_id    INT          NOT NULL,
    project_id     INT          NOT NULL,
    part_id        INT          NOT NULL,
    quantity       INT          NOT NULL CHECK (quantity > 0),
    unit_price     NUMERIC(10, 2) NOT NULL CHECK (unit_price >= 0),
    supply_date    DATE         NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (supplier_id, project_id, part_id),           -- all three FKs form composite
    FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (project_id) REFERENCES Project(pnumber)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (part_id) REFERENCES Part(part_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

```

FK Referential Action Reference

When a referenced row is deleted or its PK is updated, the DBMS applies the declared action:

Action	On DELETE	On UPDATE	Best for
RESTRICT (default)	Block delete if child rows exist	Block update	Records with independent business meaning
CASCADE	Delete child rows automatically	Propagate the new PK value	Weak entities; purely associative junction rows
SET NULL	Set FK column(s) to NULL	Set FK column(s) to NULL	Optional (partial) relationships
SET DEFAULT	Set FK column(s) to their declared default	Set FK column(s) to default	Reassign to a fallback/placeholder entity
NO ACTION	Like RESTRICT but checked at end of transaction	Same	Deferred constraint checking

NULL Summary

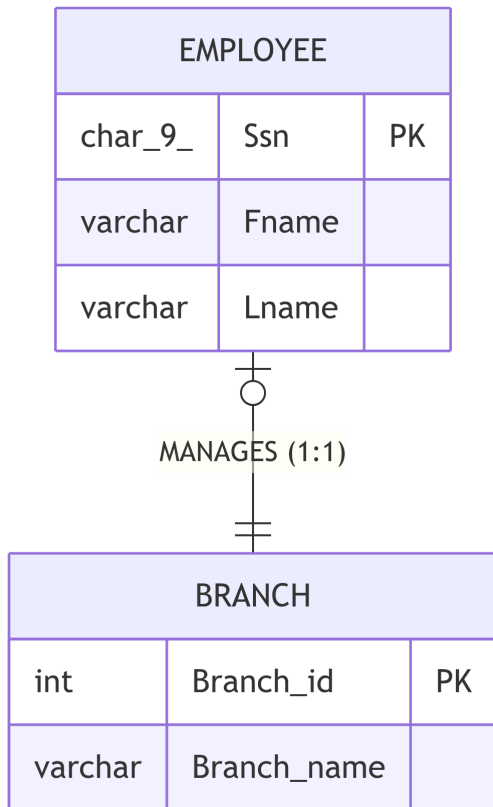
Situation	FK nullable?	Typical referential action
Weak entity FK to owner	NOT NULL (part of PK)	CASCADE
1:1 — FK on total-participation side	NOT NULL	RESTRICT
1:1 — FK on partial-participation side	NULL allowed	SET NULL
1:N — N-side total participation	NOT NULL	RESTRICT
1:N — N-side partial participation	NULL allowed	SET NULL
M:N junction table FKs	NOT NULL (part of PK)	CASCADE
N-ary junction table FKs	NOT NULL (part of PK)	RESTRICT
Self-referencing FK (supervisor)	NULL allowed	SET NULL

Solved Examples

Each example below gives a natural-language scenario and an ER diagram. Work through the mapping yourself, then expand the **Solution** panel to check your answer. Where more than one mapping is technically correct the solution discusses *all* options and justifies the preferred choice.

Example 1 — 1:1 · Total / Partial · No Relationship Attributes

Scenario: Every *Branch* must have exactly one *Manager* (an Employee). An Employee may manage at most one Branch — but most Employees are not managers.



💡 Solution

Participation summary:

Side	Cardinality	Participation	(<i>min</i> , <i>max</i>)
EMPLOYEE	1	Partial — most employees are not managers	(0, 1)
BRANCH	1	Total — every branch has exactly one manager	(1, 1)

Mapping rule (Step 3a): For a 1:1 relationship with one **total** and one **partial** side, place the FK on the **total** side. The FK is NOT NULL because that entity must always be connected.

Why FK on BRANCH, not EMPLOYEE?

Placing the FK on EMPLOYEE would mean most rows would hold NULL (partial side) — unnecessary NULLs in every non-manager row. Placing it on BRANCH keeps the column non-null and compact.

Formal Schema:

```
EMPLOYEE(Ssn, Fname NN, Lname NN)
```

```
BRANCH(Branch_id, Branch_name NN,
       Manager_ssn NN→EMPLOYEE(Ssn) {ON DELETE RESTRICT ON UPDATE CASCADE})
-- Manager_ssn NOT NULL: total participation – a branch must always have a manager
```

SQL:

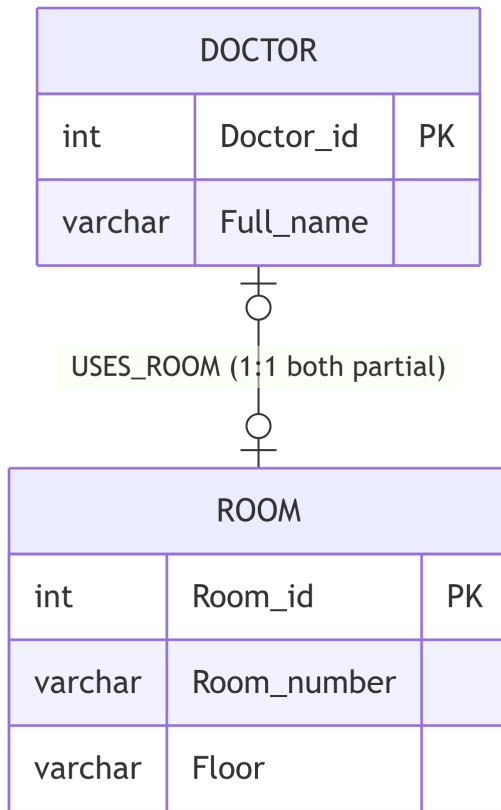
```
CREATE TABLE Employee (
    ssn    CHAR(9)    PRIMARY KEY,
    fname  VARCHAR(30) NOT NULL,
    lname  VARCHAR(30) NOT NULL
);
```

```
CREATE TABLE Branch (
    branch_id    INT        PRIMARY KEY,
    branch_name  VARCHAR(50) NOT NULL,
    manager_ssn  CHAR(9)    NOT NULL,          -- total → NOT NULL
    FOREIGN KEY (manager_ssn) REFERENCES Employee(ssn)
        ON DELETE RESTRICT                    -- cannot remove an active manager
        ON UPDATE CASCADE
);
```

Key insight: The FK always migrates to the **total** side. NOT NULL there is mandatory — it enforces the “must have” constraint at the database level.

Example 2 — 1:1 · Both Partial · No Relationship Attributes (two valid solutions)

Scenario: A *Doctor* may be assigned a *Clinic* room to use. A room may be assigned to at most one doctor. Neither assignment is mandatory — some doctors do rounds only; some rooms are shared conference spaces.



💡 Solution

Participation summary:

Side	Cardinality	Participation	(<i>min</i> , <i>max</i>)
DOCTOR	1	Partial — some doctors have no assigned room	(0, 1)
ROOM	1	Partial — some rooms are not assigned	(0, 1)

Mapping rule (Step 3c): Both sides partial → FK can go on **either** table. Choose the side that minimises NULLs and reflects the natural business direction.

Option A — FK on DOCTOR (*preferred*)

Reasoning: A doctor is created first (the doctor *exists* independently of having a room); the room assignment is a property of the doctor, not the room. Most queries will start from a doctor and ask “which room?”, not the reverse.

```
DOCTOR(Doctor_id, Full_name NN,
        Room_id→ROOM {ON DELETE SET NULL ON UPDATE CASCADE})
-- Room_id nullable: partial participation
```

```
ALTER TABLE Doctor
```

```
ADD COLUMN room_id INT NULL,
```

```
ADD CONSTRAINT fk_doc_room
```

```
FOREIGN KEY (room_id) REFERENCES Room(room_id)
```

```
ON DELETE SET NULL
```

```
-- room removed → doctor simply loses a room
```

```
ON UPDATE CASCADE;
```

Option B — FK on ROOM (*also valid, slightly worse*)

Reasoning: If rooms are assigned-to far less frequently than doctors exist, ROOM rows would have more NULLs. But technically correct.

```
ROOM(Room_id, Room_number NN, Floor NN,
      Doctor_id→DOCTOR {ON DELETE SET NULL ON UPDATE CASCADE})
```

Option C — Separate junction table (*over-engineered for 1:1*)

Avoids NULLs entirely but adds a third table and join for a simple 1:1. Only warranted if the assignment itself carries extra attributes (e.g., assignment date, duration).

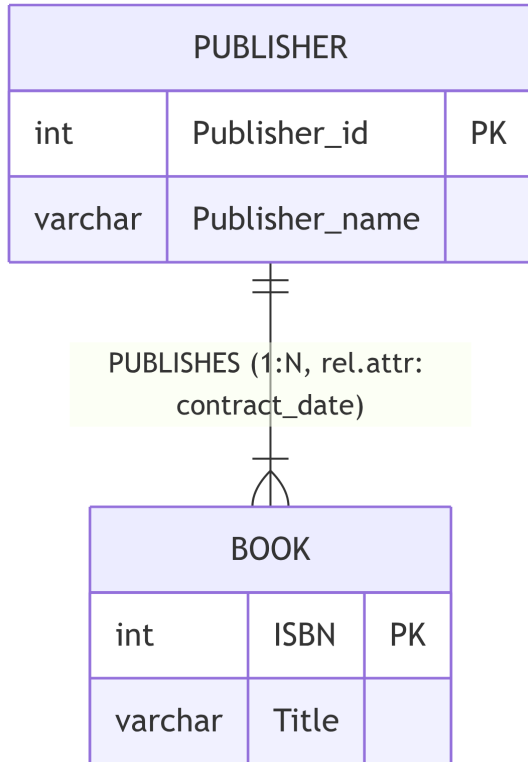
Decision guide for both-partial 1:1:

Criterion	Prefer FK on ...
One entity “owns” the other conceptually	The owning entity
One entity is queried far more often	The entity used as the starting point in queries
One entity exists before the other	The entity that is created second (it references what already exists)
NULLs would be rare on one side	The side where NULLs are rarest

Bottom line: FK on DOCTOR. Doctors exist before room assignments; the room column is logically a doctor attribute; most queries say “find this doctor’s room”, not “find this room’s doctor”.

Example 3 — 1:N · N-side Total · Relationship Attribute

Scenario: A *Publisher* produces many *Books*. Every Book must belong to exactly one Publisher (total on Book). The relationship records the `contract_date` when the publishing agreement was signed.



💡 Solution

Participation summary:

Side	Cardinality	Participation	(<i>min</i> , <i>max</i>)
PUB-LISHER	1	Partial — a publisher may have no books yet	(0, <i>N</i>)
BOOK	N	Total — every book must have a publisher	(1, 1)

Mapping rule (Step 4): FK migrates to the **N-side** (BOOK). N-side is total → FK is NOT NULL. Relationship attribute contract_date travels with the FK to the N-side row.

Formal Schema:

PUBLISHER(Publisher_id, Publisher_name NN U)

BOOK(ISBN, Title NN,
 Publisher_id NN→PUBLISHER {ON DELETE RESTRICT ON UPDATE CASCADE},
 Contract_date NN)
 -- Publisher_id NOT NULL: book cannot exist without a publisher (total)
 -- Contract_date is a relationship attribute – stored on the N-side row

SQL:

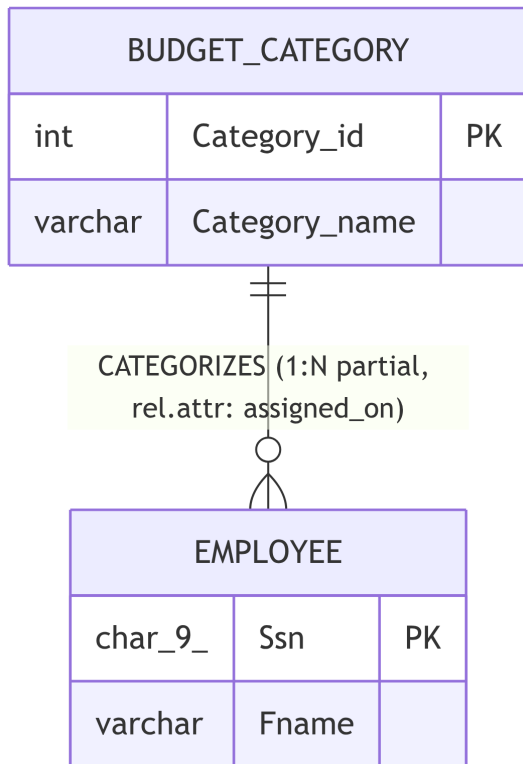
```
CREATE TABLE Publisher (
    publisher_id    INT          PRIMARY KEY,
    publisher_name  VARCHAR(80)  NOT NULL UNIQUE
);
```

```
CREATE TABLE Book (
    isbn            CHAR(13)     PRIMARY KEY,
    title           VARCHAR(200) NOT NULL,
    publisher_id    INT          NOT NULL,           -- total → NOT NULL
    contract_date   DATE         NOT NULL,          -- relationship attribute on N-side
    FOREIGN KEY (publisher_id) REFERENCES Publisher(publisher_id)
        ON DELETE RESTRICT                          -- cannot remove publisher with active
        ON UPDATE CASCADE
);
```

Key insight: Relationship attributes in 1:N relationships always travel to the N-side row — there is no need for a separate table.

Example 4 — 1:N · N-side Partial · Relationship Attribute

Scenario: A *Department* may optionally assign a *Budget_Category* to each *Employee* for expense tracking. Not every employee has a budget category (e.g., interns are excluded). If a budget category is deleted, affected employees lose the assignment rather than being deleted themselves. The relationship records the assigned_on date.



💡 Solution

Participation summary:

Side	Cardinality	Participation	(<i>min</i> , <i>max</i>)
BUD- GET_CAT- EGORY	1	Partial — a category may have no employees yet	(0, <i>N</i>)
EM- PLOYEE	<i>N</i>	Partial — some employees have no category	(0, 1)

Mapping rule (Step 4): FK on the N-side (EMPLOYEE). N-side is **partial** → FK is NULL allowed. Relationship attribute assigned_on goes on the same row, but must be NULL when the FK is NULL (no assignment = no date).

Formal Schema:

```
EMPLOYEE(Ssn, Fname NN,
          Category_id→BUDGET_CATEGORY {ON DELETE SET NULL ON UPDATE CASCADE},
          Assigned_on)
-- Category_id nullable: partial participation
-- Assigned_on nullable: meaningless without a category assignment
-- CK(Category_id IS NULL AND Assigned_on IS NULL
--    OR Category_id IS NOT NULL AND Assigned_on IS NOT NULL) ← enforce consistency
```

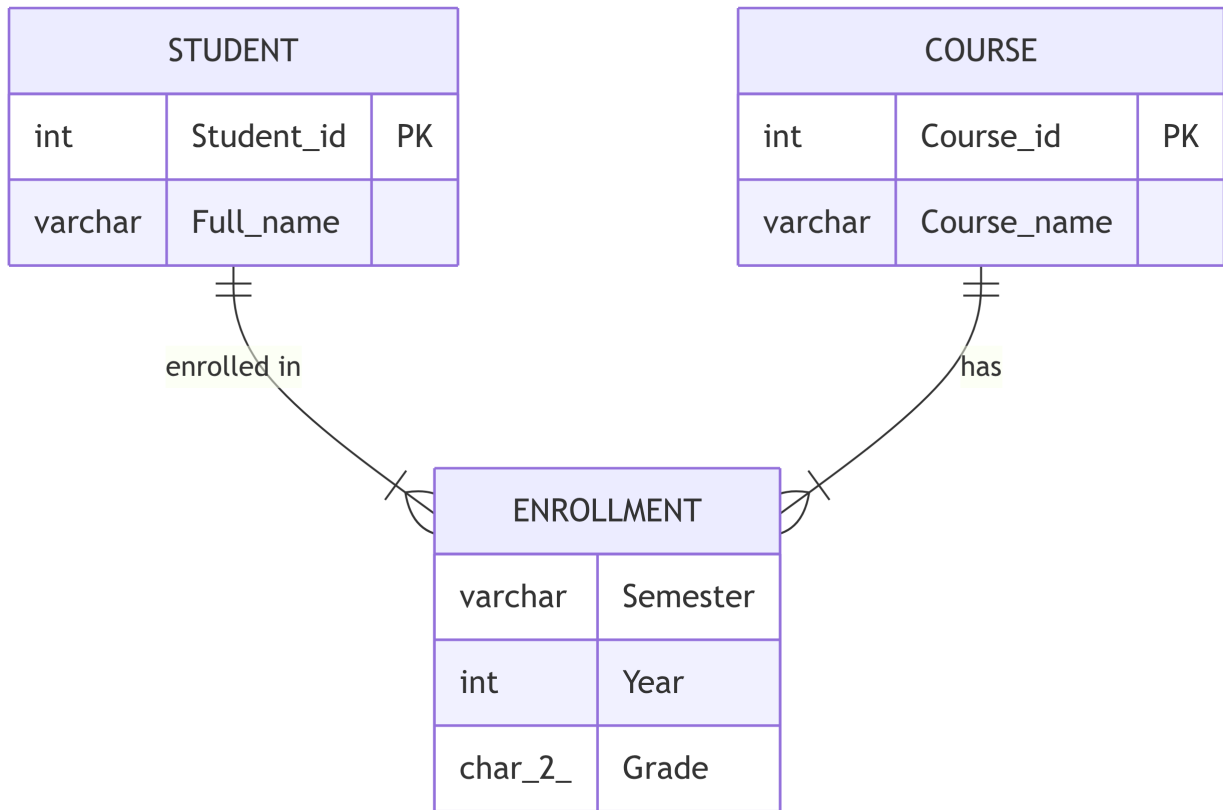
SQL:

```
CREATE TABLE Employee (
    ssn          CHAR(9)    PRIMARY KEY,
    fname        VARCHAR(30) NOT NULL,
    category_id  INT         NULL,           -- partial → nullable
    assigned_on  DATE        NULL,          -- nullable when no category
    CONSTRAINT chk_category_consistency
        CHECK (
            (category_id IS NULL AND assigned_on IS NULL) OR
            (category_id IS NOT NULL AND assigned_on IS NOT NULL)
        ),
    FOREIGN KEY (category_id) REFERENCES Budget_Category(category_id)
        ON DELETE SET NULL                -- category removed → employees lose a
        ON UPDATE CASCADE
);
```

Key insight: When a relationship attribute accompanies a nullable FK, add a CHECK constraint to keep them consistent — either *both* filled in or *both* null.

Example 5 — M:N · Both Total · Relationship Attribute

Scenario: Students enroll in courses; each student takes many courses, and each course has many students. Both participations are total (a student must be enrolled in at least one course; a course must have at least one student). The enrollment records the semester, year, and grade (nullable until graded).

**💡 Solution****Participation summary:**

Side	Cardinality	Participation	(min, max)
STUDENT	M	Total — every student is enrolled in at least one course	$(1, N)$
COURSE	N	Total — every course has at least one student	$(1, N)$

Mapping rule (Step 5): Create a **junction table** ENROLLMENT. Both FKs are NOT NULL (they form the composite PK). Relationship attributes semester, year, grade become regular columns on the junction table.

Formal Schema:

STUDENT(Student_id, Full_name NN)

COURSE(Course_id, Course_name NN)

ENROLLMENT(Student_id NN→STUDENT {ON DELETE CASCADE ON UPDATE CASCADE},
Course_id NN→COURSE {ON DELETE CASCADE ON UPDATE CASCADE},
Semester NN CK(Semester IN ('Fall', 'Spring', 'Summer')),
Year NN CK(Year >= 2000),
Grade CK(Grade IN ('A+', 'A', 'A-', 'B+', 'B', 'B-', 'C+', 'C', 'C-', 'D', 'F')))

PK: (Student_id, Course_id)

-- Grade is nullable: not yet assigned until the course ends

SQL:

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    full_name VARCHAR(60) NOT NULL  
);
```

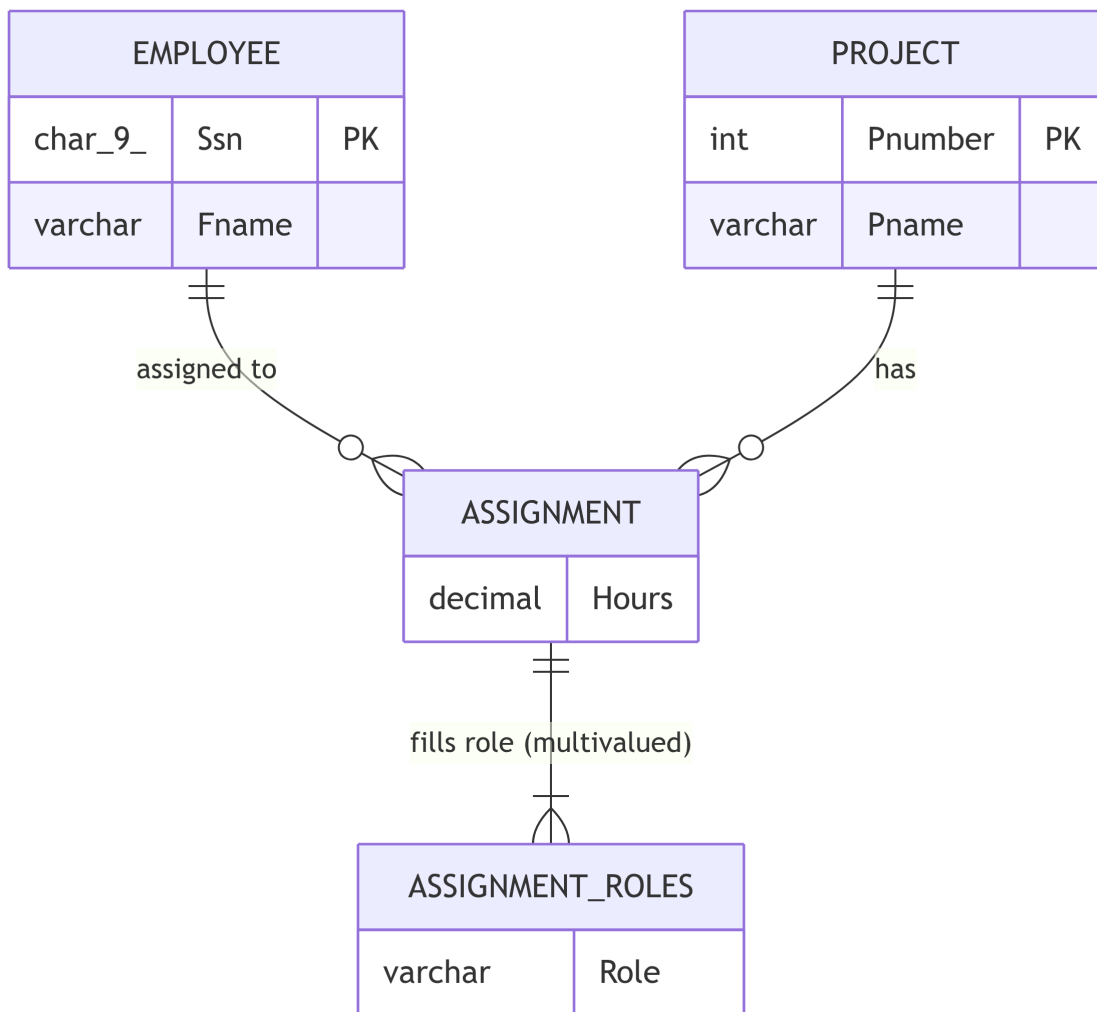
```
CREATE TABLE Course (  
    course_id INT PRIMARY KEY,  
    course_name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Enrollment (  
    student_id INT NOT NULL,  
    course_id INT NOT NULL,  
    semester VARCHAR(10) NOT NULL CHECK (semester IN ('Fall', 'Spring', 'Summer')),  
    year INT NOT NULL CHECK (year >= 2000),  
    grade CHAR(2) NULL  
    CHECK (grade IN ('A+', 'A', 'A-', 'B+', 'B', 'B-', 'C+', 'C', 'C-', 'D', 'F')),  
    PRIMARY KEY (student_id, course_id),  
    FOREIGN KEY (student_id) REFERENCES Student(student_id)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (course_id) REFERENCES Course(course_id)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);
```


Key insight: In M:N mappings the junction table is **always** created regardless of participation — and both FK columns are always NOT NULL because they form the composite primary key.

Example 6 — M:N with Multivalued Relationship Attribute

Scenario: Employees work on projects (M:N). For each assignment, the employee may fill multiple *roles* on that project (e.g., 'Developer', 'Tester', 'Lead'). The set of roles is multivalued and belongs to the *relationship*, not to the employee or project individually.



💡 Solution

Step 1 — Map the M:N relationship (Step 5):

Create the junction table ASSIGNMENT for the M:N relationship. Hours is a scalar relationship attribute → goes directly on ASSIGNMENT.

```
ASSIGNMENT(Essn NN→EMPLOYEE(Ssn) {ON DELETE CASCADE ON UPDATE CASCADE},
           Pno NN→PROJECT(Pnumber) {ON DELETE CASCADE ON UPDATE CASCADE},
           Hours CK(Hours >= 0))
PK: (Essn, Pno)
```

Step 2 — Map the multivalued relationship attribute (Step 6 applied to a relationship):

Roles is multivalued on the *relationship* (not a single entity). Create a separate table ASSIGNMENT_ROLES. Its PK is the *full* junction key plus the attribute value.

```
ASSIGNMENT_ROLES(Essn→EMPLOYEE(Ssn) {ON DELETE CASCADE ON UPDATE CASCADE},
                 Pno→PROJECT(Pnumber) {ON DELETE CASCADE ON UPDATE CASCADE},
                 Role NN CK(Role IN ('Developer', 'Tester', 'Lead', 'Analyst', 'DBA'))))
PK: (Essn, Pno, Role)
FK: (Essn, Pno) → ASSIGNMENT(Essn, Pno) {ON DELETE CASCADE ON UPDATE CASCADE}
```

SQL:

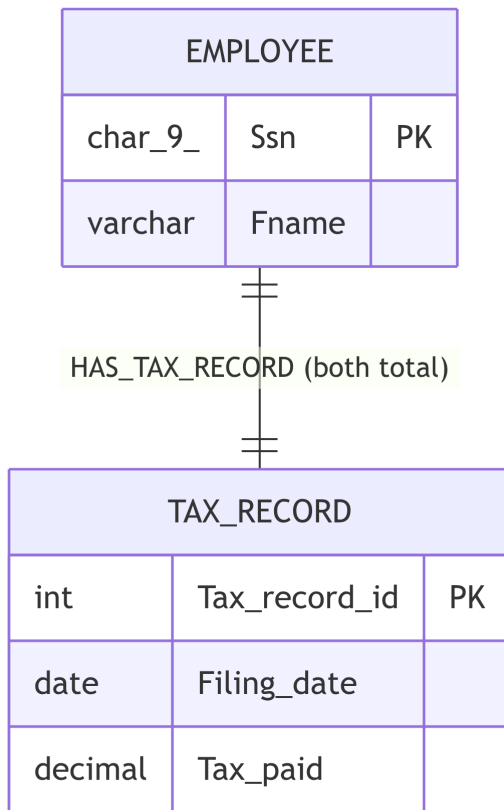
```
CREATE TABLE Assignment (
    essn    CHAR(9)      NOT NULL,
    pno     INT          NOT NULL,
    hours   NUMERIC(5,1) CHECK (hours >= 0),
    PRIMARY KEY (essn, pno),
    FOREIGN KEY (essn) REFERENCES Employee(ssn)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (pno) REFERENCES Project(pnumber)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
CREATE TABLE Assignment_Roles (
    essn    CHAR(9)      NOT NULL,
    pno     INT          NOT NULL,
    role    VARCHAR(20)  NOT NULL
        CHECK (role IN ('Developer', 'Tester', 'Lead', 'Analyst', 'DBA')),
    PRIMARY KEY (essn, pno, role),
    FOREIGN KEY (essn, pno) REFERENCES Assignment(essn, pno)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

Key insight: A multivalued attribute on a *relationship* requires its own table whose PK extends the junction table's composite key with the attribute value. The composite FK pointing back to the junction table enforces referential integrity between the two tables.

Example 7 — 1:1 · Both Total · Multiple Solutions Compared

Scenario: Every *Employee* must have exactly one *Tax_Record*, and every *Tax_Record* belongs to exactly one *Employee*. Both entities must exist before the relationship can be established (corporate regulation requires simultaneous creation).



💡 Solution

Participation summary:

Side	Cardinality	Participation	(<i>min</i> , <i>max</i>)
EM- PLOYEE	1	Total — every employee must have a tax record	(1, 1)
TAX_RECORD		Total — every tax record belongs to exactly one employee	(1, 1)

Three mappings are valid. All three are evaluated below.

Option A — FK on EMPLOYEE (*preferred*)

TAX_RECORD(Tax_record_id, Filing_date NN, Tax_paid NN)

EMPLOYEE(Ssn, Fname NN,
 Tax_record_id NN U→TAX_RECORD {ON DELETE RESTRICT ON UPDATE CASCADE})
 -- NN + UNIQUE together enforce the 1:1 at the DB level

```
CREATE TABLE Tax_Record (
    tax_record_id  INT          PRIMARY KEY,
    filing_date    DATE          NOT NULL,
    tax_paid       NUMERIC(12,2) NOT NULL
);

CREATE TABLE Employee (
    ssn            CHAR(9)      PRIMARY KEY,
    fname          VARCHAR(30)  NOT NULL,
    tax_record_id  INT          NOT NULL UNIQUE,    -- NN + UNIQUE enforces 1:1
    FOREIGN KEY (tax_record_id) REFERENCES Tax_Record(tax_record_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);
```

Why preferred: Tax records are government-issued entities that exist independently; an employee is created *referencing* the record. The FK lives on EMPLOYEE (the entity created second, after the record is ready).

Option B — FK on TAX_RECORD (*also valid*)

EMPLOYEE(Ssn, Fname NN)

TAX_RECORD(Tax_record_id, Filing_date NN, Tax_paid NN,
 Owner_ssn NN U→EMPLOYEE {ON DELETE CASCADE ON UPDATE CASCADE})

Why less preferred: A tax record referencing an employee suggests the *record* is created after the *employee* — the reverse of how tax systems actually work.

Option C — Merge into one table (*only when always co-created and co-deleted*)

EMPLOYEE_TAX(Ssn, Fname NN, Filing_date NN, Tax_paid NN)

Eliminates the FK and join entirely. Valid when employee and tax record always exist together and are managed as a unit. Loses the ability to query or manage tax records independently.

Decision guide for both-total 1:1:

Question	Preferred option
Which entity is created first?	FK on the entity created second (it references what already exists)
Are the entities always co-created and co-deleted?	Consider merging into one table
Do both entities need to be queried independently?	Keep as separate tables ; FK on the less central entity
Is there a “parent” and a “dependent” entity?	FK on the dependent side

Enforcing both-total with a circular dependency:

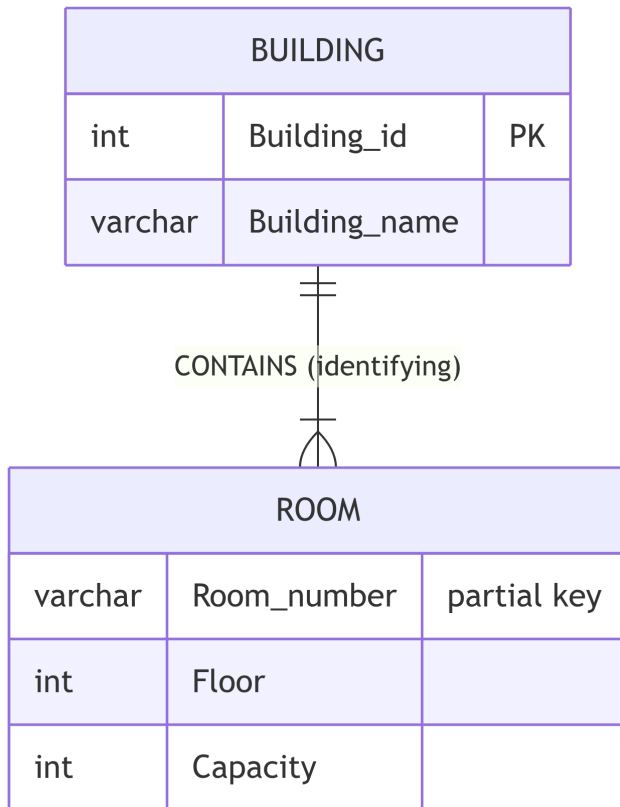
When both FKs must be NOT NULL (true mutual total participation), a **chicken-and-egg** problem arises: inserting Employee requires Tax_Record to exist, and vice versa. Solutions:

1. Wrap both INSERT statements in a single transaction with DEFERRABLE INITIALLY DEFERRED constraints.
2. Temporarily allow NULL, insert both rows, then update and tighten the constraint.
3. Merge into one table (Option C).

Bottom line: Choose Option A. Tax records exist before employees reference them; NOT NULL UNIQUE on Employee.tax_record_id enforces the 1:1 constraint; and DEFERRABLE handles simultaneous creation.

Example 8 — Weak Entity · Total / Identifying Relationship

Scenario: A *Building* contains many *Rooms*. Rooms are identified by their room number *within a building* — the same number can exist in different buildings. If a building is demolished, all its rooms cease to exist.



💡 Solution

Participation summary:

Side	Role	(<i>min</i> , <i>max</i>)
BUILDING	Owner (strong)	(0, <i>N</i>) — may have no rooms yet
ROOM	Weak entity	(1, 1) — must belong to exactly one building

Mapping rule (Step 2): Weak entity → separate table; PK = owner FK + partial key; ON DELETE CASCADE is mandatory.

Formal Schema:

BUILDING(Building_id, Building_name NN)

ROOM(Building_id NN→BUILDING {ON DELETE CASCADE ON UPDATE CASCADE},
 Room_number NN,
 Floor NN,
 Capacity CK(Capacity > 0))
 PK: (Building_id, Room_number)
 -- Building_id is NOT NULL and part of PK: total participation by definition
 -- Room_number is a partial key: unique only within one building

SQL:

```
CREATE TABLE Building (
    building_id    INT          PRIMARY KEY,
    building_name  VARCHAR(80) NOT NULL
);
```

```
CREATE TABLE Room (
    building_id    INT          NOT NULL,           -- owner FK (part of composite PK)
    room_number    VARCHAR(10) NOT NULL,           -- partial key
    floor          INT          NOT NULL,
    capacity       INT          CHECK (capacity > 0),
    PRIMARY KEY (building_id, room_number),         -- composite PK
    FOREIGN KEY (building_id) REFERENCES Building(building_id)
        ON DELETE CASCADE                          -- building demolished → all rooms del
        ON UPDATE CASCADE
);
```

Key insight: Weak entities **always** use ON DELETE CASCADE because they have no existence independent of their owner. The composite PK prevents a room number from colliding across buildings.

Example 9 — Quick-Reference Card

The table below consolidates every scenario covered in Examples 1–8 into a single reference.

Scenario	Cardinality	Participation	FK location	FK nullable?	ON DELETE action
1:1 — one total / one partial	1:1	Total + Partial	Total side	NOT NULL	RESTRICT
1:1 — both partial (entity A more stable)	1:1	Partial + Partial	More stable entity	NULL	SET NULL
1:1 — both total (A created first)	1:1	Total + Total	Entity created second	NOT NULL UNIQUE	RESTRICT
1:N — N-side total	1:N	Any + Total	N-side	NOT NULL	RESTRICT
1:N — N-side partial	1:N	Any + Partial	N-side	NULL	SET NULL
M:N (any participation)	M:N	Any	Junction table (new)	NOT NULL (PK)	CASCADE
Weak entity	—	Identifying	Weak entity table	NOT NULL (part of PK)	CASCADE
Multivalued attribute on entity	—	—	New table (owner FK + value)	NOT NULL (PK)	CASCADE
Multivalued attribute on relationship	—	—	New table (junction FK + value)	NOT NULL (PK)	CASCADE

Final Company Database Schema

Formal Schema (Complete)

```
EMPLOYEE(Ssn, Fname NN, Minit, Lname NN, Bdate,
         Address,
         Gender CK(Gender IN ('M','F')),
         Salary NN CK(Salary >= 0),
         Super_ssn →EMPLOYEE(Ssn)          {ON DELETE SET NULL  ON UPDATE CASCADE},
         Dno NN→DEPARTMENT(Dnumber) {ON DELETE RESTRICT  ON UPDATE CASCADE})

DEPARTMENT(Dnumber CK(Dnumber > 0), Dname NN U,
          Mgr_ssn NN→EMPLOYEE(Ssn)          {ON DELETE RESTRICT  ON UPDATE CASCADE},
          Mgr_start_date NN)

DEPT_LOCATIONS(Dnumber→DEPARTMENT(Dnumber) {ON DELETE CASCADE  ON UPDATE CASCADE},
              Dlocation NN)
PK: (Dnumber, Dlocation)

PROJECT(Pnumber CK(Pnumber > 0), Pname NN U, Plocation,
        Dnum NN→DEPARTMENT(Dnumber)          {ON DELETE RESTRICT  ON UPDATE CASCADE})

WORKS_ON(Essn→EMPLOYEE(Ssn)                  {ON DELETE CASCADE  ON UPDATE CASCADE},
        Pno→PROJECT(Pnumber)                  {ON DELETE CASCADE  ON UPDATE CASCADE},
        Hours CK(Hours >= 0 AND Hours <= 168))
PK: (Essn, Pno)

DEPENDENT(Essn→EMPLOYEE(Ssn)                  {ON DELETE CASCADE  ON UPDATE CASCADE},
        Dependent_name NN,
        Gender CK(Gender IN ('M','F')),
        Bdate,
        Relationship NN CK(Relationship IN
        ('Spouse','Son','Daughter','Father','Mother','Sibling'))
        PK: (Essn, Dependent_name)
```

Full SQL Script (dependency order)

```
-- =====
-- Step 1b: Strong Entity – Department (created first; no FK deps)
-- =====
CREATE TABLE Department (
```

```

dnumber      INT          PRIMARY KEY CHECK (dnumber > 0),
dname        VARCHAR(30)  NOT NULL UNIQUE,
mgr_ssn      CHAR(9)      NOT NULL,          -- Step 3a: MANAGES (total) → NOT N
mgr_start_date DATE      NOT NULL DEFAULT CURRENT_DATE
-- FK to Employee added via ALTER after Employee is created (circular dep)
);

-- =====
-- Step 1a: Strong Entity – Employee (depends on Department)
-- =====
CREATE TABLE Employee (
    ssn        CHAR(9)      PRIMARY KEY
                CHECK (ssn ~ '^d{9}$'),
    fname      VARCHAR(15)  NOT NULL,
    minit      CHAR(1),
    lname      VARCHAR(15)  NOT NULL,
    bdate      DATE,        -- 'Age' is derived → not stored
    address    VARCHAR(100), -- single simple attribute as shown
    gender     CHAR(1)      CHECK (gender IN ('M', 'F')),
    salary     NUMERIC(10, 2) NOT NULL CHECK (salary >= 0),
    super_ssn  CHAR(9)      NULL,          -- Step 4b: self-ref, partial → nul
    dno        INT          NOT NULL,      -- Step 4a: WORKS_FOR, total → NOT
    FOREIGN KEY (super_ssn) REFERENCES Employee(ssn)
                ON DELETE SET NULL ON UPDATE CASCADE,
    FOREIGN KEY (dno) REFERENCES Department(dnumber)
                ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Close the circular dependency:
ALTER TABLE Department
    ADD CONSTRAINT fk_dept_manager
        FOREIGN KEY (mgr_ssn) REFERENCES Employee(ssn)
        ON DELETE RESTRICT ON UPDATE CASCADE;

-- =====
-- Step 1c: Strong Entity – Project (depends on Department)
-- =====
CREATE TABLE Project (
    pnumber    INT          PRIMARY KEY CHECK (pnumber > 0),
    pname      VARCHAR(25)  NOT NULL UNIQUE,
    plocation  VARCHAR(30),

```

```
dnum          INT          NOT NULL,          -- Step 4a: CONTROLS, total → NOT
FOREIGN KEY (dnum) REFERENCES Department(dnumber)
ON DELETE RESTRICT ON UPDATE CASCADE
);

-- =====
-- Step 2: Weak Entity – Dependent (owner: Employee)
-- =====

CREATE TABLE Dependent (
    essn        CHAR(9)      NOT NULL,          -- owner FK (part of composite PK)
    dependent_name VARCHAR(15) NOT NULL,          -- partial key
    gender       CHAR(1)      CHECK (gender IN ('M', 'F')),
    bdate        DATE,
    relationship  VARCHAR(10)  NOT NULL
    CHECK (relationship IN
        ('Spouse', 'Son', 'Daughter', 'Father', 'Mother', 'Sibling')),
    PRIMARY KEY (essn, dependent_name),
    FOREIGN KEY (essn) REFERENCES Employee(ssn)
    ON DELETE CASCADE ON UPDATE CASCADE          -- weak entity: cascade is mandatory
);

-- =====
-- Step 5: M:N Junction – Works_On (Employee × Project)
-- =====

CREATE TABLE Works_On (
    essn        CHAR(9)      NOT NULL,
    pno         INT          NOT NULL,
    hours       NUMERIC(5, 1) CHECK (hours >= 0 AND hours <= 168),
    PRIMARY KEY (essn, pno),
    FOREIGN KEY (essn) REFERENCES Employee(ssn)
    ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (pno) REFERENCES Project(pnumber)
    ON DELETE CASCADE ON UPDATE CASCADE
);

-- =====
-- Step 6: Multivalued – Dept_Locations (Department.locations)
-- =====

CREATE TABLE Dept_Locations (
    dnumber      INT          NOT NULL,
    dlocation    VARCHAR(30)  NOT NULL,
```

```
PRIMARY KEY (dnumber, dlocation),
FOREIGN KEY (dnumber) REFERENCES Department(dnumber)
ON DELETE CASCADE ON UPDATE CASCADE
);
```

⚠ Circular FK: Employee Department

Employee.dno → Department and Department.mgr_ssn → Employee form a circular foreign-key dependency. You cannot create both constraints simultaneously in a single CREATE TABLE sequence. Two solutions:

1. **Two-phase DDL** — create Department without mgr_ssn FK, create Employee, then ALTER TABLE Department ADD CONSTRAINT ... to add the missing FK (as shown above).
2. **Deferred constraints** — declare both FKs as DEFERRABLE INITIALLY DEFERRED and wrap the initial inserts in a single transaction. PostgreSQL checks deferred constraints only at COMMIT, so both rows can be inserted before either FK is validated.

Summary

! Chapter 5 — Key Takeaways

- **History:** Codd (1970) introduced the relational model to solve physical data dependence, the Connection Trap, and data inconsistency in older hierarchical/network models.
- A **relation** is a mathematical set of tuples. Properties: no duplicates, no ordering, atomic values (1NF).
- **Domain specification** has 7 components: data type, length/precision, date format, range, constraints, NULL support, and default value.
- **Attribute types:** simple (atomic), composite (flatten to sub-columns), single-valued, multi-valued (separate table), stored, derived (never stored — compute in query), NULL.
- **Schema notation — three styles:** Formal (this book: NN, U, CK(), →TABLE(pk)), Shorthand (underline PK), SQL (CREATE TABLE). Always use Formal notation in this course.
- **Schema diagrams:** rectangles for relations, underlines for PK, arrows for FK; convey structure at a glance.
- **Key taxonomy:** superkey → candidate key → {primary key, alternate key, unique key} → {natural / surrogate / composite}. Unique key = UNIQUE constraint, NULLs permitted; Alternate key = candidate key not chosen as PK, declared UNIQUE NOT NULL.
- **Key identification:** a candidate key is a minimal set of attributes that uniquely identifies every tuple — justified by business rules, never inferred from sample data alone (the Superset Principle: adding attributes to a superkey always yields another superkey). Full FD theory is

covered in Chapter 9.

- **PK selection guide:** prefer surrogate integers for stability and compactness; keep natural attributes as UNIQUE NOT NULL alternate keys.
- **Four integrity constraints:** domain, entity integrity (PK $\neq$ NULL), referential integrity (FK $\rightarrow$ PK), key uniqueness.
- **FK actions:** RESTRICT (safe default), CASCADE (weak entities, junction rows), SET NULL (optional relationships), SET DEFAULT, NO ACTION.
- **NULL** uses three-valued logic (T/F/UNKNOWN); comparisons involving NULL yield UNKNOWN; aggregates ignore NULLs; three meanings: Unknown / Not Applicable / Does Not Exist.
- **Business rules:** enforce at DB level with constraints when possible; use application code for complex multi-table logic.
- **Handling violations:** pre-validate input, use transactions, catch error codes, use deferrable constraints for circular FKs, test edge cases.

Review Questions

Conceptual

1. Define **relation**, **tuple**, **attribute**, and **domain** using set-theory language. Give a concrete example from the Company Database for each.
2. Explain the difference between a **candidate key**, a **primary key**, and an **alternate key**. Can a relation have more than one candidate key? Give an example.
3. What is the **entity integrity constraint**? Give a precise statement of why a NULL primary key violates it.

Short-Answer / Compare

4. Compare **natural keys** and **surrogate keys**. For the EMPLOYEE table in the Company Database, which would you choose as PK? Justify using at least two of the criteria from §5.7.
5. Explain the difference between ON DELETE CASCADE and ON DELETE SET NULL. For the DEPENDENT weak entity referenced by EMPLOYEE, which action is more appropriate? Why?
6. The WORKS_ON table has a composite PK (Essn, Pno). Explain why neither Essn alone nor Pno alone qualifies as a candidate key, using the concept of functional dependency.

True / False (with explanation)

7. “If no two rows in the current *Employee* table have the same *Lname*, then *Lname* is a candidate key.” — True or False? Explain the principle involved.
8. “*NULL = NULL* evaluates to *TRUE* in *SQL*.” — True or False? What expression should be used instead?
9. “Foreign keys can always be *NULL*.” — True or False? Explain when a FK must be NOT NULL.
10. “The *RESTRICT* action on a foreign key prevents deleting a parent row that has matching children.” — True or False? What is the alternative to *CASCADE* in this case?

Multiple Choice

11. Which of the following is **not** a property of a formal mathematical relation?
 - (a) No duplicate tuples
 - (b) Column order is irrelevant
 - (c) **Rows are stored in insertion order**
 - (d) All attribute values are atomic
12. An alternate key is:
 - (a) A foreign key that references an alternate table
 - (b) **A candidate key that was not chosen as the primary key**
 - (c) A composite key with more than three attributes
 - (d) A key that allows NULL values

Design Exercise

13. Given the following business rules for a university course enrollment system:
 - A student is uniquely identified by their student number AND (separately) by their national ID.
 - A student may enroll in many courses; a course may have many enrolled students.
 - Every enrollment must record the semester, the grade (may be null if not yet graded), and the status (ENROLLED, WITHDRAWN, COMPLETED).
 - Dropping a student from the system should set their enrollments to reflect the deletion but not delete the enrollment history.
- (a) Identify all candidate keys for the Enrollment relation.
- (b) Specify the FK action for the student’s deletion.
- (c) Write the formal schema notation for Student and Enrollment using the conventions in §5.4.

SQL Exercise

14. Write the complete CREATE TABLE statements for DEPARTMENT and EMPLOYEE from the Company Database, enforcing:

- All structural constraints from §5.4
- Salary > 0
- Gender restricted to 'M', 'F', or 'N'
- ON DELETE SET NULL ON UPDATE CASCADE for the Dno FK
- ON DELETE CASCADE for the Super_ssn FK (deleting an employee also removes direct reports — debate whether this is the right choice)

Relational Algebra

Chapter 6

SQL is a *declarative* language: you say *what* you want; the DBMS decides *how* to compute it. The “how” is determined by translating your SQL into **relational algebra** — a *procedural* formal language where every operation is an explicit step. Understanding relational algebra means understanding how the query optimizer thinks, why certain queries are fast, and what is happening beneath every JOIN you write.

Interactive RELAX — Try Every Example Yourself!

This chapter includes **live RELAX widgets** so you can run every expression directly in the page. You can also use the full **interactive calculator** in this book at [Appendix D — RELAX Calculator](#), which includes **5 different database schemas** to practise with:

Schema	Domain
Enhanced Company DB	Employees, departments, projects
University DB	Students, faculty, courses, enrolment
E-Commerce DB	Customers, products, orders, reviews
Hospital DB	Patients, doctors, admissions
Library DB	Members, books, loans, fines

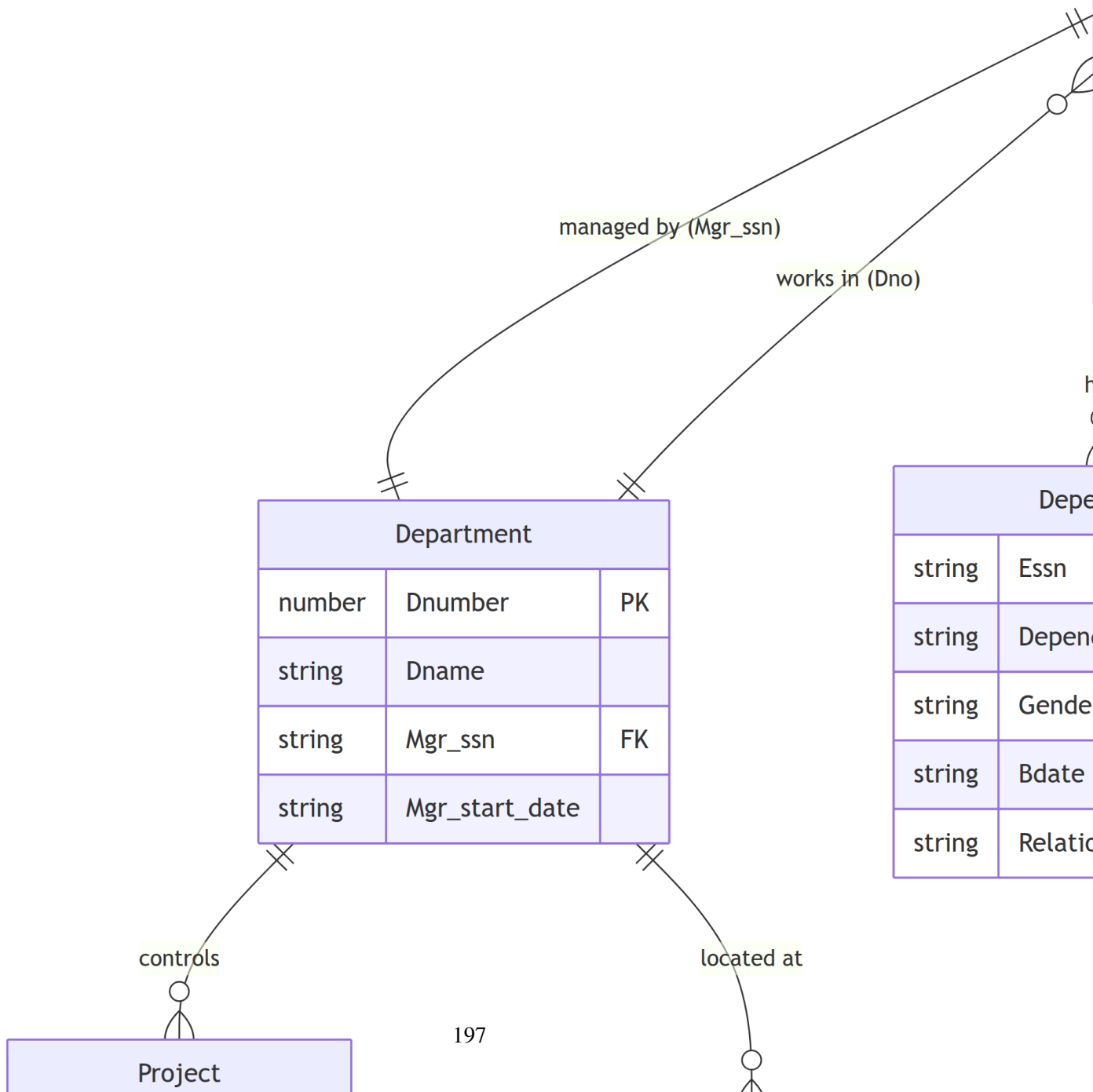
For the external standalone calculator, visit: <https://nireas.iee.ihu.gr/relax/calc.htm> — paste the **RELAX schema** from the collapsible box below (§6.1) into the *Group Editor*, then type your expression in the query box.

Company Database Schema for RELAX

The following schema loads the complete Enhanced Company Database — 30 employees, 8 departments, 12 projects — directly into RELAX.

 Click to expand: RELAX inline-relation schema (paste into Group Editor)

Entity-Relationship Overview



What Is Relational Algebra?

Relational algebra is a **procedural formal query language** for relations. Each *operation* takes one or two relations as input and returns a new relation as output.

Closure property: The result of every relational algebra operation is itself a relation. This means operations can be freely composed — the output of one becomes the input of the next.

$$\text{Expression} = \text{Op}_k(\dots \text{Op}_2(\text{Op}_1(R_1, R_2)) \dots)$$

There are **6 fundamental operations** (from which all others can be derived) and several convenient **additional (derived) operations**.

Group	Operations
Fundamental	σ select, π project, $\cup$ union, $-$ difference, $\times$ Cartesian product, ρ rename
Derived	$\cap$ intersection, $\bowtie$ natural join, $\bowtie_\theta$ theta join, $\bowtie$ outer joins, $\div$ division

Result Schema Notation

For every operation in this chapter, we track the **result schema** — the column names and degree of the output relation. This is exactly what the DBMS query optimizer computes before execution.

Format used:

Result: RelationName(col1, col2, ...)

Degree: k Cardinality: m rows (estimated)

Column names in a result use **qualified notation** Table.Column when there is ambiguity (e.g., after a Cartesian product that produces two Ssn columns).

How Results Are Presented in This Chapter

The diagrams below show the four ways you will see a query result presented:

Fname	Salary	Dno
string	number	number
Ali	73000	4
Mohammed	67000	2
Hussein	71000	null
... more rows ...		

Note

In the **Solution** boxes throughout this chapter, clicking **Run** executes the expression live and shows **all four views** — formula, tree, table, and row count — just like [Appendix D](#).

Fundamental Operations

1. Select — σ (Filter Rows)

$$\sigma_{\text{condition}}(R)$$

Returns the subset of tuples in R that satisfy the *predicate*. Equivalent to a SQL WHERE clause.

Properties:

Property	Value
Input	Relation R
Output columns	Same as R (degree unchanged)
Output rows	All tuples where condition is true (cardinality $\leq R $)
Condition operators	$=, \neq, <, >, \leq, \geq$, combined with $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT)

Syntax:

Format	Notation
LaTeX	$\sigma_{\text{condition}}(\text{Relation})$
RELAX	σ condition (Relation)
SQL	SELECT * FROM Relation WHERE condition

Exercise 6.1 — Simple Selection

List all employees whose salary is greater than 65,000.

Write the relational algebra expression using the σ operator on the Employee relation.

LaTeX:

$$\sigma_{\text{Salary} > 65000}(\text{Employee})$$

RELAX syntax:

σ Salary > 65000 (Employee)

Solution — Click to expand

Exercise 6.2 — Compound Selection

List all employees who work in department 1 AND earn more than 60,000.

LaTeX:

$$\sigma_{\text{Dno}=1 \wedge \text{Salary} > 60000}(\text{Employee})$$

RELAX syntax:

σ Dno = 1 and Salary > 60000 (Employee)

Solution — Click to expand

SQL equivalent:

SELECT * **FROM** Employee **WHERE** Dno = 1 **AND** Salary > 60000;

2. Project — π (Filter Columns)

$$\pi_{A_1, A_2, \dots, A_k}(R)$$

Returns a relation containing only the listed attributes. Duplicate tuples are **eliminated** (set semantics).

Properties:

Property	Value
Input	Relation R
Output columns	Only $A_1, \dots, A_k$ (degree = k)
Output rows	Cardinality $\leq R $ — duplicates removed

Syntax:

Format	Notation
LaTeX	$\pi_{A_1, A_2, \dots}(\text{Relation})$
RELAX	$\pi \text{ A1, A2, ... } (\text{Relation})$
SQL	SELECT DISTINCT A1, A2, ... FROM Relation

Exercise 6.3 — Simple Projection

List the first name, last name, and salary of every employee.

LaTeX:

$$\pi_{\text{Fname, Lname, Salary}}(\text{Employee})$$

RELAX syntax:

$\pi \text{ Fname, Lname, Salary } (\text{Employee})$

Solution — Click to expand

Exercise 6.4 — Combined Select + Project

List the first name and salary of all female employees earning above 60,000.

Hint: compose π outside σ .

LaTeX:

$$\pi_{\text{Fname, Salary}}(\sigma_{\text{Gender}='F' \wedge \text{Salary}>60000}(\text{Employee}))$$

RELAX syntax:

$\pi \text{ Fname, Salary } (\sigma \text{ Gender} = 'F' \text{ and Salary} > 60000 (\text{Employee}))$

Solution — Click to expand

 Tip

The SQL keyword DISTINCT corresponds to projection in the formal model. Without DISTINCT, SQL is a **bag** (multiset) operation; with it, SQL matches formal **set** semantics.

3. Union — $\cup$ (Combine Two Relations)

$$R \cup S$$

Returns all tuples in R , all tuples in S , with **duplicates removed**.

Set Compatibility Rules

$\cup$, $\cap$, and $-$ all require **union-compatible** operands:

1. R and S must have the **same degree** (number of attributes)
2. Corresponding attributes must have **compatible domains** (same or convertible types)
3. The **result takes the attribute names from the left operand** (R)

Syntax:

Format	Notation
LaTeX	$R \cup S$
RELAX	$(\text{expr1}) \sqcup (\text{expr2})$
SQL	$\dots \text{UNION } \dots$

Exercise 6.5 — Union

Find all SSNs that appear in either the Employee table or the Works_On table.

LaTeX:

$$\pi_{\text{Ssn}}(\text{Employee}) \cup \pi_{\text{Essn}}(\text{Works_On})$$

RELAX syntax:

$$(\pi \text{ Ssn } (\text{Employee})) \sqcup (\pi \text{ Essn } (\text{Works_On}))$$

Solution — Click to expand

SQL equivalent: UNION removes duplicates; UNION ALL keeps all rows (bag semantics).

4. Set Difference — — (In One But Not the Other)

$$R - S$$

Returns tuples that are in R but **not** in S . Requires union compatibility.

Syntax:

Format	Notation
LaTeX	$R - S$
RELAX	<code>(expr1) - (expr2)</code>
SQL	<code>... EXCEPT ... or NOT EXISTS / NOT IN</code>

Exercise 6.6 — Set Difference

Find the SSNs of employees who are NOT currently assigned to any project.

The SSNs in Employee minus the SSNs in Works_On.

LaTeX:

$$\pi_{\text{Ssn}}(\text{Employee}) - \pi_{\text{Essn}}(\text{Works_On})$$

RELAX syntax:

`( $\pi$  Ssn (Employee)) - ( $\pi$  Essn (Works_On))`

Solution — Click to expand

SQL equivalent:

```
SELECT Ssn FROM Employee
EXCEPT
SELECT Essn FROM Works_On;
```

5. Cartesian Product — $\times$ (All Combinations)

$$R \times S$$

Returns every possible combination of one tuple from R with one tuple from S .

Properties:

Property	Value
Output degree	$\text{degree}(R) + \text{degree}(S)$ — shared names qualified as <code>R.col</code> and <code>S.col</code>
Output cardinality	$ R \times S $

Syntax:

Format	Notation
LaTeX	$R \times S$
RELAX	<code>R × S</code>
SQL	<code>FROM R CROSS JOIN S</code>

Exercise 6.7 — Cartesian Product

Produce all combinations of Employee and Department rows.

Notice the resulting degree (columns) and cardinality (rows).

LaTeX:

$\text{Employee} \times \text{Department}$

RELAX syntax:

`Employee × Department`

Solution — Click to expand

! Important

The raw Cartesian product generates **$30 \times 8 = 240$ rows** but most are meaningless combinations. It is almost always followed by a σ to keep only matching rows — that combination is precisely a **join**.

Exercise 6.8 — Why `Employee × Employee` Fails (Column Name Conflict)

Try to pair every employee with every other employee using a Cartesian product.

When you write `Employee × Employee`, both copies of the table bring the **same 10 column names** (`Fname`, `Lname`, `Ssn`, ...).

A relation must have **unique column names** — there is no way to tell `Fname` (left copy) from `Fname` (right copy).

The formal model resolves this by **qualifying** shared names with the relation name:

Employee × Employee ⇒ columns: $\underbrace{\text{Employee.Fname, Employee.Ssn, ...}}_{\text{left copy}}, \underbrace{\text{Employee.Fname, Employee.Ssn, ...}}_{\text{right copy}}$

But both copies still carry the **same qualified name** Employee.Fname — still ambiguous!

The correct approach: use ρ to **rename** one (or both) copies before forming the product:

$$\rho_{E1}(\text{Employee}) \times \rho_{E2}(\text{Employee})$$

Now the columns are E1.Fname, E1.Ssn, ..., E2.Fname, E2.Ssn, ... — all distinct.

RELAX syntax:

$\rho_{E1}(\text{Employee}) \times \rho_{E2}(\text{Employee})$

Solution — Click to expand

Run the expression below. Notice the column headers are prefixed E1. and E2., making all 20 columns unambiguous. Row count = $30 \times 30 = 900$ combinations.

Practical use: this rename-then-cross-product pattern is the foundation of **self-joins** — for example, finding every pair of employees who share the same supervisor:

$$\pi_{E1.Fname, E2.Fname}(\sigma_{E1.Super_ssn = E2.Super_ssn \wedge E1.Ssn \neq E2.Ssn}(\rho_{E1}(\text{Employee}) \times \rho_{E2}(\text{Employee})))$$

i Column Naming Rules for ×

Situation	Column names in result
$R \times S$ — no shared names	col (unqualified)
$R \times S$ — some names shared	shared columns become R.col and S.col
$R \times R$ — same table twice	ambiguous — must rename first with ρ

The general rule: **every column in a relation must be uniquely identifiable**. Whenever a Cartesian product would produce duplicate column names, use ρ to rename one operand before the product.

6. Rename — ρ (Rename Relation or Attributes)

$$\rho_{S(B_1, B_2, \dots, B_n)}(R)$$

Renames relation R to S and optionally its attributes to $B_1, \dots, B_n$. If you only need to rename the relation (not the columns), use $\rho_S(R)$.

Use cases:

When to use	Example
Self-join — need two copies of same table	$\rho_{E1}(\text{Employee}) \times \rho_{E2}(\text{Employee})$
Enable natural join when FK name $\neq$ PK name	Rename FK column to match PK column name
Shorten a long subexpression for readability	$\rho_{\text{Dept}}(\sigma_{\text{Dname} = \text{'Research'}}(\text{Department}))$

Syntax:

Format	Notation
Rename relation only	$\rho_S(R)$
Rename relation + all columns	$\rho_{S(B_1, \dots, B_n)}(R)$
RELAX — relation only	$\rho_S(R)$
RELAX — relation + columns	$\rho_S(B_1, B_2, \dots)(R)$

Exercise 6.7-ρ.1 — Rename for Readability

Create a named subexpression *Supervisor* containing SSN, first name, and last name of all employees, using shorter column names.

LaTeX:

$$\rho_{\text{Supervisor}(\text{Ssn}, \text{FName}, \text{LName})}(\pi_{\text{Ssn}, \text{Fname}, \text{Lname}}(\text{Employee}))$$

RELAX syntax:

$\rho_{\text{Supervisor}(\text{Ssn}, \text{FName}, \text{LName})}(\pi_{\text{Ssn}, \text{Fname}, \text{Lname}}(\text{Employee}))$

Solution — Click to expand

Exercise 6.7-ρ.2 — Self-Join with Rename: Find Employee–Supervisor Pairs

List the first name of every employee alongside the first name of their direct supervisor.

Both the employee and their supervisor are rows in the same `Employee` table. To join the table with itself: 1. Rename one copy to `E` (the employee) 2. Rename the other to `S` (the supervisor) 3. Join on `E.Super_ssn = S.Ssn` 4. Project only the two first names

LaTeX:

$$\pi_{E.Fname, S.Fname}(\rho_E(Employee) \bowtie_{E.Super_ssn=S.Ssn} \rho_S(Employee))$$

RELAX syntax:

$\pi \ E.Fname, \ S.Fname \ (\rho \ E \ (Employee) \ \Join \ E.Super_ssn = S.Ssn \ \rho \ S \ (Employee))$

Solution — Click to expand

Each row shows an employee's first name paired with their supervisor's first name. Employees with no supervisor (NULL Super_ssn) are excluded by the inner join.

This is especially useful for **self-joins** (joining a table with itself), where you need two distinct copies with different names.

Additional (Derived) Operations

7. Intersection — $\cap$

$$R \cap S = R - (R - S)$$

Returns tuples present in **both** R and S . Requires union compatibility. Intersection is a **derived** operation — it can always be rewritten using set difference.

Syntax:

Format	Notation
LaTeX	$R \cap S$
RELAX	$(expr1) \cap (expr2)$
SQL	$\dots \text{ INTERSECT } \dots$

Exercise 6.7- .1 — SSNs in Both Tables

Find the SSNs of employees who are assigned to at least one project.

An SSN appears in `Employee.Ssn` and also in `Works_On.Essn` only if that employee has a project assignment. Their intersection gives exactly those employees.

LaTeX:

$$\pi_{Ssn}(Employee) \cap \pi_{Essn}(Works_On)$$

RELAX syntax:

$(\pi_{\text{Ssn}}(\text{Employee})) \cap (\pi_{\text{Essn}}(\text{Works_On}))$

Solution — Click to expand

Compare the row count with the Union exercise (§6.5): union gives all SSNs that appear in *either* table, intersection gives only those in *both*.

Exercise 6.7- .2 — Departments With Both Projects and Location Records

Find department numbers that appear in both the Project table (as Dnum) and the Dept_Locations table (as Dnumber).

This verifies which departments have at least one project *and* at least one registered location.

LaTeX:

$$\pi_{\text{Dnum}}(\text{Project}) \cap \pi_{\text{Dnumber}}(\text{Dept_Locations})$$

RELAX syntax:

$(\pi_{\text{Dnum}}(\text{Project})) \cap (\pi_{\text{Dnumber}}(\text{Dept_Locations}))$

Solution — Click to expand

Note: `Project.Dnum` and `Dept_Locations.Dnumber` have **different column names**, so we first project each to a single column; set intersection then matches the values regardless of original name.

8. Natural Join — ⋈

$$R \bowtie S$$

A natural join **automatically** determines the join condition by looking for columns with **identical names** in both relations, then:

1. Keeps only rows where those common attributes have **equal values** (implicit equijoin)
2. **Eliminates the duplicate common columns** — they appear only once in the result

Result schema rule: If R has columns (A, B, C) and S has columns (B, C, D) , then $R \bowtie S$ produces columns (A, B, C, D) .

Syntax:

Format	Notation
LaTeX	$R \bowtie S$
RELAX	$R \sqcap S$
SQL	FROM R NATURAL JOIN S

! Natural Join is Name-Driven

Natural join joins on **every** column that shares the same name in both tables.

- If PK and FK have **different names** → no common columns → result is a **Cartesian product** (all combinations, probably wrong)
- If you accidentally share **too many** column names → the join condition becomes overly restrictive
- Always verify which columns will be matched before using natural join

Exercise 6.8 — Natural Join: When It Works Directly

List each department with all its office locations.

The Department table has Dnumber as its primary key.

The Dept_Locations table has Dnumber as a foreign key.

Both columns share the exact same name → natural join will automatically join on Dnumber.

Schema recap:

Table	Columns
Department	Dnumber (PK), Dname, Mgr_ssn, Mgr_start_date
Dept_Locations	Dnumber (FK), Dlocation

LaTeX:

Department $\bowtie$ Dept_Locations

RELAX syntax:

Department $\sqcap$ Dept_Locations

Solution — Click to expand

The common column Dnumber is found automatically. Result columns: Dnumber, Dname, Mgr_ssn, Mgr_start_date, Dlocation.

Exercise 6.9 — Natural Join: When Column Names Differ (Danger!)

Try to list each employee with the name of their department using natural join.

`Employee.Dno` is the FK referencing `Department.Dnumber` — but the column names are **different**.

Table	Relevant column
Employee	Dno (FK → Department)
Department	Dnumber (PK)

Since there are **no columns with identical names** between `Employee` and `Department`, a natural join produces a **Cartesian product** — every employee paired with every department, regardless of which department they belong to.

LaTeX:

$\text{Employee} \bowtie \text{Department} \rightarrow \text{Employee} \times \text{Department}$ (Cartesian! Wrong!)

RELAX syntax:

`Employee` $\bowtie$ `Department`

Solution — Click to expand and see the problem

Run this and observe the row count: with 30 employees and 8 departments you get **240 rows** — every combination, not just the correct one!

Why is this wrong? The result should have 30 rows (one per employee, matched to their department), not 240. Because `Dno` $\neq$ `Dnumber` in name, the natural join found no common column and fell back to a Cartesian product.

Exercise 6.10 — Fix with Rename ρ : Force Natural Join

Correctly list each employee with their department name by forcing the natural join using rename.

Strategy: Use ρ to rename `Dno` to `Dnumber` in `Employee`, making the column name match. Now the natural join will correctly find `Dnumber` as the common column.

Step 1 — Rename Employee's Dno column to Dnumber:

$\rho_{\text{EmpR}(\text{Fname}, \text{Minit}, \text{Lname}, \text{Ssn}, \text{Bdate}, \text{Address}, \text{Gender}, \text{Salary}, \text{Super_ssn}, \text{Dnumber})}(\text{Employee})$

Step 2 — Natural join the renamed relation with Department:

$\rho_{\text{EmpR}(\dots, \text{Dnumber})}(\text{Employee}) \bowtie \text{Department}$

RELAX syntax:

$\rho \text{ EmpR}(\text{Fname}, \text{Minit}, \text{Lname}, \text{Ssn}, \text{Bdate}, \text{Address}, \text{Gender}, \text{Salary}, \text{Super_ssn}, \text{Dnumber}) (\text{Emp})$

Solution — Click to expand

Now the join produces exactly 30 rows — one per employee matched to their correct department. The common column Dnumber is found and the join is correct.

Exercise 6.11 — Project After Join: Keep Only Wanted Columns

After the rename + join above, the result has 13 columns (all from both tables). Reduce it to just: employee first name, last name, and their department name.

Compose π outside the join to select only the useful columns. This is the standard pattern for extracting meaningful information from a join.

LaTeX:

$$\pi_{\text{Fname}, \text{Lname}, \text{Dname}}(\rho_{\text{EmpR}(\dots, \text{Dnumber})}(\text{Employee}) \bowtie \text{Department})$$

RELAX syntax:

$\pi \text{ Fname}, \text{Lname}, \text{Dname} (\rho \text{ EmpR}(\text{Fname}, \text{Minit}, \text{Lname}, \text{Ssn}, \text{Bdate}, \text{Address}, \text{Gender}, \text{Salary}, \text{Super_ssn}, \text{Dnumber}) (\text{Emp}) \bowtie \text{Department})$

Solution — Click to expand

The result is a clean 3-column table: Fname, Lname, Dname.

 Danger: Too Many Matching Column Names

If you rename **more columns than the intended join key**, natural join silently adds extra conditions.

Example: Suppose you rename both $\text{Dno} \rightarrow \text{Dnumber}$ **and** $\text{Super_ssn} \rightarrow \text{Mgr_ssn}$ in Employee before joining with Department.

Now the natural join matches on **both** Dnumber AND Mgr_ssn.

This returns only employees whose supervisor is also the manager of their department — a tiny, unintended subset!

Rule: When using ρ to enable natural join, rename **only** the FK column(s) you intend to join on. Check that no other renamed columns accidentally match columns in the other table.

9. Theta Join — $\bowtie_\theta$ (Join on Any Condition)

$$R \bowtie_\theta S = \sigma_\theta(R \times S)$$

The theta join is the most **explicit** join: you write the exact condition yourself. No column-name matching happens automatically — you control everything.

When θ uses only equality (=) it is called an **equijoin**. The difference from natural join:

	Natural Join $\bowtie$	Equijoin / Theta Join $\bowtie_{\theta}$
Join condition	Auto-detected from column names	You write it explicitly
Duplicate columns	Removed (common cols appear once)	Both columns kept in result
Safe when FK $\neq$ PK name?	No	Yes

Syntax:

Format	Notation
LaTeX	$R \bowtie_{\theta} S$
RELAX	$R \bowtie \text{condition } S$
SQL	<code>FROM R JOIN S ON condition</code>

Exercise 6.12 — Equijoin on Different-Named FK/PK

List each employee with their project number and hours, joining Employee and Works_On on $Ssn = Essn$.

Employee.Ssn is the PK; Works_On.Essn is the FK — they have different names so natural join fails.

Use a theta (equi) join with the explicit condition $Ssn = Essn$.

LaTeX:

$\text{Employee} \bowtie_{Ssn=Essn} \text{Works_On}$

RELAX syntax:

$\text{Employee} \bowtie Ssn = Essn \text{ Works_On}$

Solution — Click to expand

Both Ssn (from Employee) and Essn (from Works_On) appear in the result — theta join keeps both join columns.

10. Outer Joins

Standard (inner) joins **discard unmatched rows**. Outer joins **preserve** them by filling missing values with NULL.

Operator	Symbol	Keeps unmatched from
Left outer join	$R \bowtie S$	R (left operand)
Right outer join	$R \bowtie S$	S (right operand)
Full outer join	$R \bowtie S$	Both R and S

Syntax:

Format	Notation
LaTeX	$R \bowtie S$
RELAX	$R \bowtie \text{condition } S$
SQL	FROM R LEFT OUTER JOIN S ON condition

Exercise 6.13 — Left Outer Join: All Employees, Even Those Without Projects

List every employee with their project number and hours. Employees with no project assignment should still appear (with NULL for project columns).

An inner equijoin ($\text{Employee} \bowtie \text{Ssn} = \text{Essn Works_On}$) drops employees with no assignment. A **left outer join** keeps all employees from the left side.

LaTeX:

$\text{Employee} \bowtie_{\text{Ssn}=\text{Essn}} \text{Works_On}$

RELAX syntax:

$\text{Employee} \bowtie \text{Ssn} = \text{Essn Works_On}$

Solution — Click to expand

All 30 employees appear. Employees who have no row in `Works_On` show NULL for `Essn`, `Pno`, and `Hours`.

Note

Left vs. Inner join row count: Inner join returns only matched rows (employees with ≥ 1 project). Left outer join always returns at least $|\text{Employee}|$ rows — unmatched rows get NULLs on the right side.

Exercise 6.13b — Full Outer Join: All Employees and All Project Records

Show all employees and all Works_On records, including employees with no project and project records with no matching employee.

LaTeX:

$$\text{Employee} \bowtie_{\text{Ssn}=\text{Essn}} \text{Works_On}$$

RELAX syntax:

Employee $\bowtie$ Ssn = Essn Works_On

Solution — Click to expand

Unmatched employees get NULLs from Works_On columns; orphan Works_On rows (if any) get NULLs from Employee columns.

11. Division — $\div$ (Universal Quantification)

$$R(A, B) \div S(B)$$

Returns the values of A in R that are associated with **every** value of B in S . This answers “for all” (universal quantification) queries.

$$R \div S \equiv \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$$

Use case: “Find employee SSNs who work on **all** projects in a given set.”

Syntax:

Format	Notation
LaTeX	$R \div S$
RELAX	(expr1) $\div$ (expr2)
SQL	No direct equivalent — use NOT EXISTS double negation

Schema requirements for division:

- Dividend $R(A, B)$ — has the “key” column(s) A plus the “requirement” column(s) B
- Divisor $S(B)$ — contains only the required B values
- Result has only column(s) A

Exercise 6.14 — Division: Employees Working on All Dept-1 Projects

Find the SSNs of employees who work on every project controlled by department 1.

Works_On records (Essn, Pno) pairs. The divisor is the set of Pnumbers of dept-1 projects.

Division returns Essns that appear with **every** Pnumber in the divisor.

LaTeX:

$$\pi_{\text{Essn, Pno}}(\text{Works_On}) \div \pi_{\text{Pnumber}}(\sigma_{\text{Dnum}=1}(\text{Project}))$$

RELAX syntax:

$(\pi \text{ Essn, Pno } (\text{Works_On})) \div (\pi \text{ Pnumber } (\sigma \text{ Dnum} = 1 (\text{Project})))$

Solution — Click to expand

The divisor projects Pnumber of dept-1 projects (Pnumbers 1, 2, 3). Only Essns that appear in Works_On with **all** of those project numbers are returned.

Exercise 6.14b — Division: Employees Working on All Projects in a Location

Find the SSNs of employees who work on every project located in ‘Bellaire’.

LaTeX:

$$\pi_{\text{Essn, Pno}}(\text{Works_On}) \div \pi_{\text{Pnumber}}(\sigma_{\text{Plocation}='Bellaire'}(\text{Project}))$$

RELAX syntax:

$(\pi \text{ Essn, Pno } (\text{Works_On})) \div (\pi \text{ Pnumber } (\sigma \text{ Plocation} = 'Bellaire' (\text{Project})))$

Solution — Click to expand

Change 'Bellaire' to 'Houston' or 'Stafford' to see how the divisor set changes and which Essns satisfy the universal condition for each city.

Composed Queries — Company Database Examples

Composed Query Q1 — Employees in the ‘Research’ Department

List the first and last name of every employee who works in the Research department.

Step-by-step breakdown:

Step	Operation	Purpose
1	$\sigma_{Dname='Research'}(Department)$	Filter to Research dept
2	$Employee \bowtie_{Dno=Dnumber} (\text{result of step 1})$	Match employees to dept
3	$\pi_{Fname, Lname}(\dots)$	Keep only names

LaTeX (composed):

$$\pi_{Fname, Lname} \left(Employee \bowtie_{Dno=Dnumber} \sigma_{Dname='Research'}(Department) \right)$$

RELAX syntax:

$\pi \text{ Fname, Lname } (Employee \bowtie Dno = Dnumber (\sigma \text{ Dname} = 'Research' (Department)))$

SQL equivalent:

```
SELECT Fname, Lname FROM Employee
JOIN Department ON Dno = Dnumber
WHERE Dname = 'Research';
```

Solution — Click to expand

Composed Query Q2 — Employees Working More Than 30 Hours on Any Project

List the first name, last name, and hours of employees who have more than 30 hours on at least one project.

LaTeX:

$$\pi_{Fname, Lname, Hours} \left(Employee \bowtie_{Ssn=Essn} \sigma_{Hours>30} (Works_On) \right)$$

RELAX syntax:

$\pi \text{ Fname, Lname, Hours } (Employee \bowtie Ssn = Essn (\sigma \text{ Hours} > 30 (Works_On)))$

SQL equivalent:

```
SELECT Fname, Lname, Hours FROM Employee
JOIN Works_On ON Ssn = Essn
WHERE Hours > 30;
```

Solution — Click to expand

Composed Query Q3 — Employees With No Project Assignment

List the first and last name of employees who are NOT assigned to any project.

Strategy: Get all SSNs from Employee, subtract those that appear in Works_On, then join back to get names.

LaTeX:

$$\pi_{\text{Fname, Lname}} \left(\text{Employee} \bowtie_{\text{Ssn}=\text{Ssn}} \left(\pi_{\text{Ssn}}(\text{Employee}) - \pi_{\text{Essn}}(\text{Works_On}) \right) \right)$$

RELAX syntax:

$\pi \text{ Fname, Lname } (\text{Employee} \sqcap \text{Ssn} = \text{Ssn} ((\pi \text{ Ssn } (\text{Employee})) - (\pi \text{ Essn } (\text{Works_On}))))$

SQL equivalent:

```
SELECT Fname, Lname FROM Employee
WHERE Ssn NOT IN (SELECT Essn FROM Works_On);
```

Solution — Click to expand

12. Extended Relational Algebra — Aggregation ()

Standard relational algebra operates on **sets** of tuples with no grouping or counting. Real queries often need summaries: “count employees per department”, “average salary by gender”, “maximum project hours”. The **extended relational algebra** adds the **aggregate operator** γ (gamma).

$$\gamma_{G; F_1(A_1) \rightarrow n_1, \dots, F_k(A_k) \rightarrow n_k} (R)$$

Where: - G — a list of **grouping attributes** (may be empty for a total aggregate) - $F_i(A_i)$ — an aggregate function applied to attribute A_i - n_i — the output column name for the aggregate result

Aggregate functions:

Function	Meaning
COUNT ()	Count rows in each group
SUM(col)	Sum of values
AVG(col)	Average of values
MIN(col)	Minimum value
MAX(col)	Maximum value

RELAX syntax:

γ GroupCol1, GroupCol2 ; AGG(Col) $\rightarrow$ ResultName (Relation)

For multiple aggregates, separate them with commas after the semicolon:

γ Dno ; COUNT() $\rightarrow$ NumEmp, AVG(Salary) $\rightarrow$ AvgSal (Employee)

For a **total aggregate** (no grouping), omit the group columns:

γ ; COUNT() $\rightarrow$ Total (Employee)

Exercise 6.15 — Count Employees per Department

How many employees work in each department?

Group Employee by Dno, count the rows in each group.

LaTeX:

$\gamma_{\text{Dno}; \text{COUNT}() \rightarrow \text{NumEmp}}(\text{Employee})$

RELAX syntax:

γ Dno ; COUNT() $\rightarrow$ NumEmp (Employee)

SQL equivalent:

SELECT Dno, **COUNT**(*) **AS** NumEmp **FROM** Employee **GROUP BY** Dno;

Solution — Click to expand

Each row shows a department number and the count of employees in it. Compare the totals — they should sum to 30.

Exercise 6.16 — Average Salary per Department

What is the average salary in each department?

LaTeX:

$\gamma_{\text{Dno}; \text{AVG}(\text{Salary}) \rightarrow \text{AvgSalary}}(\text{Employee})$

RELAX syntax:

γ Dno ; AVG(Salary) $\rightarrow$ AvgSalary (Employee)

SQL equivalent:

```
SELECT Dno, AVG(Salary) AS AvgSalary FROM Employee GROUP BY Dno;
```

Solution — Click to expand

Exercise 6.17 — Multiple Aggregates: Max and Min Salary per Department

In each department, what is the highest and lowest salary?

You can compute **multiple aggregates** in a single γ by listing them after the semicolon, separated by commas.

LaTeX:

$$\gamma_{Dno; \text{MAX}(\text{Salary}) \rightarrow \text{MaxSal}, \text{MIN}(\text{Salary}) \rightarrow \text{MinSal}}(\text{Employee})$$

RELAX syntax:

```
 $\gamma$  Dno ; MAX(Salary)  $\rightarrow$  MaxSal, MIN(Salary)  $\rightarrow$  MinSal (Employee)
```

SQL equivalent:

```
SELECT Dno, MAX(Salary) AS MaxSal, MIN(Salary) AS MinSal
FROM Employee GROUP BY Dno;
```

Solution — Click to expand

Exercise 6.18 — Aggregation with Filter: Departments Averaging Over \$33,000

List department numbers where the average salary exceeds \$33,000.

Relational algebra has no HAVING clause. The pattern is: 1. Compute the aggregation with γ (producing Dno, AvgSalary) 2. Apply σ to filter the aggregated result

LaTeX:

$$\sigma_{\text{AvgSalary} > 33000} \left(\gamma_{Dno; \text{AVG}(\text{Salary}) \rightarrow \text{AvgSalary}}(\text{Employee}) \right)$$

RELAX syntax:

```
 $\sigma$  AvgSalary > 33000 ( $\gamma$  Dno ; AVG(Salary)  $\rightarrow$  AvgSalary (Employee))
```

SQL equivalent:

```
SELECT Dno, AVG(Salary) AS AvgSalary FROM Employee
GROUP BY Dno
HAVING AVG(Salary) > 33000;
```

Solution — Click to expand

Note that σ is applied **after** γ — it filters groups, not individual tuples. This is exactly what SQL's HAVING clause does.

Exercise 6.19 — Total Hours per Employee Across All Projects

For each employee, calculate the total hours worked across all their project assignments.

LaTeX:

$$\gamma_{\text{Essn}; \text{SUM}(\text{Hours}) \rightarrow \text{TotalHours}}(\text{Works_On})$$

RELAX syntax:

γ Essn ; SUM(Hours) $\rightarrow$ TotalHours (Works_On)

SQL equivalent:

SELECT Essn, **SUM**(Hours) **AS** TotalHours **FROM** Works_On **GROUP BY** Essn;

Solution — Click to expand

Employees who work on multiple projects get their hours summed. Compare this with a simple π projection of Works_On — you'd see all individual rows, whereas γ collapses them into one summary row per employee.

i γ vs. SQL GROUP BY — The Mapping

Relational Algebra	SQL
$\gamma_{G; F(A) \rightarrow n}(R)$	SELECT G, F(A) AS n FROM R GROUP BY G
σ applied after γ	HAVING clause
γ with empty G (no grouping)	SELECT F(A) FROM R (total aggregate)
Compose γ with π , σ ,	Subqueries or CTEs

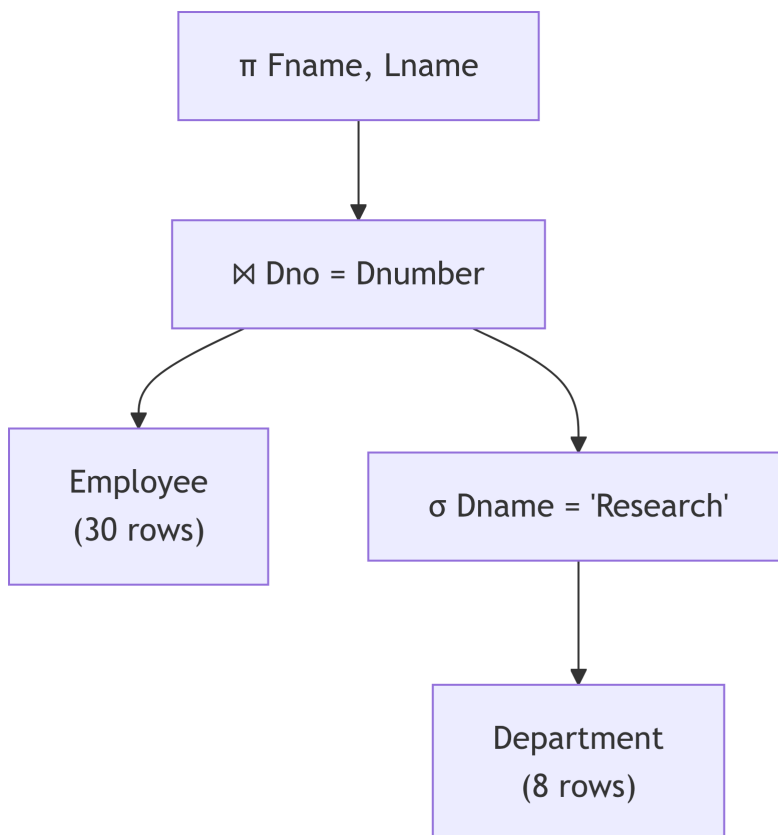
Algebra → Execution Plan and Query Optimization

When the DBMS receives a SQL query, the **DML compiler** translates it into a relational algebra expression tree. The **query optimizer** then applies **equivalence rules** to rewrite the tree into a cheaper form.

Expression Trees

An **expression tree** (also called a *query tree*) is a binary tree where: - **Leaf nodes** are operand relations (base tables) - **Internal nodes** are operators (σ , π , $\bowtie$, ...) - Data flows **bottom-up**: leaves feed into parent operators

Example tree for Q1:



Notice that σ is applied **before** the join — filtering Department from 8 rows down to 1 row before the expensive join operation.

Equivalence Rules

Rule	Statement	Optimization impact
R1 Cascade select	$\sigma_{c_1 \wedge c_2}(R) \equiv \sigma_{c_1}(\sigma_{c_2}(R))$	Apply each condition separately

Rule	Statement	Optimization impact
R2 Commutativity of select	$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$	Apply cheapest condition first
R3 Cascade project	$\pi_L(\pi_{L'}(R)) \equiv \pi_L(R)$ if $L \subseteq L'$	Eliminate intermediate projections
R4 Push select past join	$\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S$ if c involves only R	Reduce join inputs
R5 Commutativity of join	$R \bowtie S \equiv S \bowtie R$	Put smaller relation on the right (build side)
R6 Associativity of join	$(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$	Reorder joins for minimal intermediate size

Cost Estimation

The optimizer assigns a **cost estimate** to each plan. The dominant cost factor is the number of **disk I/O operations** (page reads). A simple model:

$$\text{cost}(\sigma_c(R)) = b_R \text{ page reads}$$

$$\text{cost}(R \bowtie_{\theta} S) \approx b_R + |R| \cdot b_S \text{ (nested-loop join, worst case)}$$

where b_R = number of disk pages in R and $|R|$ = cardinality of R .

Pushing a σ before a join reduces $|R|$ (and hence b_R), making the join substantially cheaper.

Before-and-After Optimization Example

Naive plan (SQL compiler's initial translation):

```
 $\pi$  Fname, Lname ( $\sigma$  Dname='Research' (Employee  $\times$  Department))
```

Cost: $30 \times 8 = 240$ Cartesian product rows, then filter.

Optimized plan (after applying R4 and R1):

```
 $\pi$  Fname, Lname (Employee  $\bowtie_{\{Dno=Dnumber\}}$  ( $\sigma$  Dname='Research' (Department)))
```

Cost: Filter 8 department rows $\rightarrow$ 1 row $\rightarrow$ join 30 employees against 1 row = 30 comparisons.

The optimization reduces tuples processed from **240 to 30** — and that ratio only grows with larger tables.

Relational Algebra vs. SQL — Correspondence

Relational Algebra	SQL Equivalent
$\sigma_c(R)$	SELECT * FROM R WHERE c
$\pi_{a,b}(R)$	SELECT DISTINCT a, b FROM R
$R \cup S$	... UNION ...
$R - S$	... EXCEPT ... (or NOT EXISTS)
$R \times S$	FROM R CROSS JOIN S
$R \bowtie S$	FROM R NATURAL JOIN S
$R \bowtie_{\theta} S$	FROM R JOIN S ON θ
$R \Join_{\theta} S$	FROM R LEFT OUTER JOIN S ON θ
$R \div S$	No direct SQL; simulated with NOT EXISTS double negation
$\gamma_{G;F(A) \rightarrow n}(R)$	SELECT G, F(A) AS n FROM R GROUP BY G
$\rho_S(R)$	... AS alias ...

Summary

! Chapter 6 — Key Takeaways

- **Relational algebra** is procedural, closed, and composable. Every SQL query translates to a RA expression internally.
- **6 fundamental operations:** σ (filter rows), π (filter columns), $\cup$ (union), $-$ (difference), $\times$ (Cartesian product), ρ (rename).
- **Set compatibility** for $\cup$, $\cap$, $-$: same degree + compatible domains; result takes *left* operand's column names.
- **Natural join** auto-matches on identically-named columns and **collapses duplicates** to one column. Always verify shared column names match your intent.
- **Theta join** $= \sigma_{\theta}(R \times S)$; equijoin uses only $=$; natural join is a special equijoin that removes duplicate columns.
- **Outer joins** preserve unmatched tuples by padding with NULLs.

- **Division ($\div$)** answers universal queries: “find tuples associated with *all* tuples in S ”.
- **Extended RA — Aggregation (γ)**: $\gamma_{G;F(A) \rightarrow n}(R)$ groups by G and computes aggregate F over each group. Combining σ after γ is the equivalent of SQL’s HAVING clause.
- **Result schema tracking**: every operation has a known degree, known column names (qualified if needed), and an estimated cardinality.
- **Query optimization**: expression trees + 6 equivalence rules. Key rule: always push σ as far down the tree as possible to minimize join input sizes.

Review Questions

Conceptual

1. What is the **closure property** of relational algebra? Why is it essential for composing operations?
2. State the three **set-compatibility requirements** for $\cup$, $-$, and $\cap$. What happens to column names in the result?
3. Explain the difference between a **natural join** and a **theta join** in terms of (a) how matching columns are determined, and (b) how duplicate columns are handled in the result.

Short-Answer / Compare

4. Why is the **Cartesian product** almost never used alone? What operation is formed when a σ is combined with $\times$?
5. Explain the **division operator** $\div$. Write a query in relational algebra that finds employees working on **all** projects located in ‘Ramallah’. Use the Company Database schema.
6. Describe the difference between pushing a σ operator **below a join** versus **below a projection**. Which equivalence rules (R1–R6) apply in each case?

True / False (with explanation)

7. “A natural join always produces a result with fewer columns than the sum of columns in both operands.” — True or False? Explain the column-collapse rule.
8. “The result of $R \cup S$ takes attribute names from S (the right operand).” — True or False?
9. “SQL’s UNION ALL corresponds to the relational algebra union operator $\cup$.” — True or False? Explain the bag vs. set distinction.
10. “A full outer join can be expressed as $(R \bowtie S) \cup (R \bowtie S)$.” — True or False? Explain.

Multiple Choice

11. Which operation answers the question “Find employees who work on *every* project in department 5”?
- (a) Natural join (b) Set difference (c) **Division** (d) Left outer join
12. After applying $\text{Employee} \bowtie_{\text{Ssn}=\text{Essn}} \text{Works_On}$, what is the degree of the result?
- (a) 10 (b) **13** (c) 12 (d) 20
- (Employee has 10 columns, Works_On has 3; equijoin keeps both Ssn and Essn, so $10+3=13$)

Design / Think-Through

13. The optimizer is given the following naïve plan:

$$\pi_{\text{Fname, Pname}} \left(\sigma_{\text{Dno}=8 \wedge \text{Dnum}=8 \wedge \text{Ssn}=\text{Essn} \wedge \text{Pno}=\text{Pnumber}} (\text{Employee} \times \text{Works_On} \times \text{Project}) \right)$$

- (a) Draw the initial expression tree.
- (b) Apply equivalence rules R4, R1, and R5 to produce an optimized tree.
- (c) Estimate the cardinality at each node before and after optimization using the Company Database row counts (Employee: 30, Works_On: 20, Project: 12).

SQL / RELAX Exercise

14. Write a RELAX expression to find the **names** of all employees who have dependents with gender 'F'. Load the Company Database into RELAX and verify your result. Then write the equivalent SQL query.
15. Write a RELAX expression for: “List the first names and project names for all employees working more than 35 hours on a project controlled by department 1.” Break it into named intermediate steps (using $\leftarrow$ assignment notation) and verify in RELAX.

Part IV: Structured Query Language (SQL)

SQL Basics

Chapter 7

From Algebra to Executable Queries

In Chapter 6 you wrote relational algebra expressions — precise mathematical statements like $\pi_{\text{Fname,Lname}}(\sigma_{\text{Salary} > 60000}(\text{Employee}))$. Relational algebra is *procedural*: it specifies an exact sequence of operators.

SQL is the executable, declarative counterpart. You tell the database *what* you want; the query optimizer translates your SQL into an algebra expression tree (Chapter 11), picks the cheapest execution plan, and returns the result.

Algebra concept	SQL keyword
σ (selection)	WHERE
π (projection)	SELECT col1, col2
$\times$ (Cartesian product)	CROSS JOIN
$\bowtie_{\theta}$ (theta join)	JOIN ... ON ...
$\bowtie$ (natural join)	NATURAL JOIN (<i>use with caution</i>)
ρ (rename)	AS alias
$\cup / \cap / -$	UNION / INTERSECT / EXCEPT

SQL Standards and Dialects

The ANSI/ISO SQL standard evolves continuously (SQL-92, SQL:1999, SQL:2003, SQL:2016). All major systems conform to the core but differ in extensions. This chapter shows examples in three dialects:

- **MySQL 8+** — widely used in web development and coursework
- **PostgreSQL 16** — closest to the standard; used in data engineering
- **Oracle 21c** — common in enterprise environments

Tabs labelled **MySQL** | **PostgreSQL** | **Oracle** appear wherever syntax diverges significantly.

SQL Overview

SQL (Structured Query Language) was originally developed at IBM (SEQUEL, 1974) and standardized by ANSI/ISO. It is:

- **Declarative:** You specify *what* you want, not *how* to get it
- **Set-based:** Operates on whole relations, not row-by-row
- **Nearly universal:** All major RDBMS support SQL-92 core and SQL:2016+

SQL Sub-languages

Sub-language	Abbreviation	Statements	Role
Data Definition	DDL	CREATE, ALTER, DROP, TRUNCATE	Schema structure
Data Manipulation	DML	INSERT, UPDATE, DELETE	Modify data
Data Query	DQL	SELECT	Retrieve data
Data Control	DCL	GRANT, REVOKE	Permissions
Transaction Control	TCL	COMMIT, ROLLBACK, SAVEPOINT	Atomicity

Data Types

Choosing the right type affects storage, performance, and data integrity. The table below compares the three target dialects.

Category	MySQL 8	PostgreSQL 16	Oracle 21c
Integer	INT, BIGINT, TINYINT	INTEGER, BIGINT, SMALLINT	NUMBER(10), NUMBER(19)
Exact decimal	DECIMAL(p, s)	NUMERIC(p, s)	NUMBER(p, s)
Floating point	FLOAT, DOUBLE	REAL, DOUBLE PRECISION	BINARY_FLOAT, BINARY_DOUBLE
Auto-increment	INT AUTO_INCREMENT	SERIAL or GENERATED ALWAYS AS IDENTITY	NUMBER + SEQUENCE
Fixed string	CHAR(n)	CHAR(n)	CHAR(n)
Variable string	VARCHAR(n)	VARCHAR(n)	VARCHAR2(n)
Unlimited text	TEXT	TEXT	CLOB
Date	DATE	DATE	DATE (<i>includes time</i>)

Category	MySQL 8	PostgreSQL 16	Oracle 21c
Date + time	DATETIME, TIMESTAMP	TIMESTAMP	TIMESTAMP
Boolean	TINYINT(1) (<i>no native bool</i>)	BOOLEAN	No native bool — use NUMBER(1)
Binary	BLOB	BYTEA	BLOB
JSON	JSON	JSON, JSONB	JSON
UUID	CHAR(36) / UUID (8+)	UUID	VARCHAR2(36)

💡 Prefer VARCHAR2 in Oracle, TEXT in PostgreSQL

Oracle's VARCHAR is technically reserved; always use VARCHAR2. PostgreSQL's TEXT is unlimited and identical in performance to VARCHAR(n) without a length limit.

DDL — Data Definition Language

DDL commands define or modify the *structure* of the database (schema objects). They implicitly commit any open transaction.

CREATE TABLE

The Company DB will be our running example. Below is the Employee table written in all three dialects, illustrating the most important syntactic differences.

MySQL

```
CREATE TABLE Department (
    Dnumber      INT          PRIMARY KEY,
    Dname        VARCHAR(15)  NOT NULL UNIQUE,
    Mgr_ssn      CHAR(9),
    Mgr_start_date DATE
);
```

```
CREATE TABLE Employee (
    Ssn          CHAR(9)      PRIMARY KEY,
    Fname        VARCHAR(15)  NOT NULL,
    Minit        CHAR(1),
    Lname        VARCHAR(15)  NOT NULL,
```

```
Bdate          DATE,
Address        VARCHAR(50),
Gender         CHAR(1)          CHECK (Gender IN ('M','F')),
Salary         DECIMAL(10,2)    CHECK (Salary >= 0),
Super_ssn      CHAR(9),
Dno            INT              NOT NULL DEFAULT 1,
CONSTRAINT fk_emp_dept FOREIGN KEY (Dno) REFERENCES Department(Dnumber)
               ON DELETE SET NULL ON UPDATE CASCADE,
CONSTRAINT fk_emp_super FOREIGN KEY (Super_ssn) REFERENCES Employee(Ssn)
               ON DELETE SET NULL
);
```

PostgreSQL

```
CREATE TABLE Department (
    Dnumber      INTEGER          PRIMARY KEY,
    Dname        VARCHAR(15)      NOT NULL UNIQUE,
    Mgr_ssn      CHAR(9),
    Mgr_start_date DATE
);
```

```
CREATE TABLE Employee (
    Ssn          CHAR(9)          PRIMARY KEY,
    Fname        VARCHAR(15)      NOT NULL,
    Minit        CHAR(1),
    Lname        VARCHAR(15)      NOT NULL,
    Bdate        DATE,
    Address      VARCHAR(50),
    Gender       CHAR(1)          CHECK (Gender IN ('M','F')),
    Salary       NUMERIC(10,2)    CHECK (Salary >= 0),
    Super_ssn    CHAR(9),
    Dno          INTEGER          NOT NULL DEFAULT 1,
    CONSTRAINT fk_emp_dept FOREIGN KEY (Dno) REFERENCES Department(Dnumber)
               ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT fk_emp_super FOREIGN KEY (Super_ssn) REFERENCES Employee(Ssn)
               ON DELETE SET NULL
);
```

Oracle

```
CREATE TABLE Department (
    Dnumber      NUMBER(4)      PRIMARY KEY,
    Dname        VARCHAR2(15)   NOT NULL UNIQUE,
    Mgr_ssn      CHAR(9),
    Mgr_start_date DATE
);

CREATE TABLE Employee (
    Ssn          CHAR(9)        PRIMARY KEY,
    Fname        VARCHAR2(15)   NOT NULL,
    Minit        CHAR(1),
    Lname        VARCHAR2(15)   NOT NULL,
    Bdate        DATE,
    Address      VARCHAR2(50),
    Gender        CHAR(1)        CONSTRAINT chk_sex CHECK (Gender IN ('M', 'F')),
    Salary        NUMBER(10,2)   CONSTRAINT chk_sal CHECK (Salary >= 0),
    Super_ssn     CHAR(9),
    Dno          NUMBER(4)        DEFAULT 1 NOT NULL,
    CONSTRAINT fk_emp_dept FOREIGN KEY (Dno) REFERENCES Department(Dnumber)
                                ON DELETE SET NULL,
    CONSTRAINT fk_emp_super FOREIGN KEY (Super_ssn) REFERENCES Employee(Ssn)
                                ON DELETE SET NULL
);

-- Oracle does not support ON UPDATE CASCADE for FK constraints
```

Constraint Summary

Constraint	Column-level example	Table-level example
NOT NULL	Fname VARCHAR(15) NOT NULL	<i>(inline only)</i>
UNIQUE	Email VARCHAR(100) UNIQUE	UNIQUE (LastName, FirstName)
PRIMARY KEY	Id INT PRIMARY KEY	PRIMARY KEY (OrderId, ItemNo)
FOREIGN KEY	DeptId INT REFERENCES Dept(Id)	FOREIGN KEY (DeptId) REFERENCES Dept(Id)
CHECK	Salary DECIMAL CHECK (Salary >= 0)	CHECK (StartDate < EndDate)
DEFAULT	Status CHAR(1) DEFAULT 'A'	<i>(inline only)</i>

! Foreign Key Referential Actions

When a referenced row is deleted or updated, three actions are available:

Action	ON DELETE	ON UPDATE
CASCADE	Delete child rows too	Update FK values to match
SET NULL	Set FK columns to NULL	Set FK columns to NULL
RESTRICT	Block the parent delete/update	Block the parent update
NO ACTION	Like RESTRICT but deferred	Like RESTRICT but deferred

Oracle only supports ON DELETE CASCADE and ON DELETE SET NULL — no ON UPDATE referential actions.

Auto-Increment / Identity Columns

Auto-generated surrogate keys are handled differently across dialects.

MySQL

```
CREATE TABLE Orders (
    OrderId INT AUTO_INCREMENT PRIMARY KEY,
    OrderDate DATE NOT NULL,
    CustomerId INT NOT NULL
);

INSERT INTO Orders (OrderDate, CustomerId) VALUES ('2026-04-06', 42);
-- OrderId is assigned automatically: 1, 2, 3, ...
```

PostgreSQL

```
-- Option 1: SERIAL shorthand (implicit sequence)
CREATE TABLE Orders (
    OrderId SERIAL PRIMARY KEY,
    OrderDate DATE NOT NULL,
    CustomerId INT NOT NULL
);

-- Option 2: SQL:2003 IDENTITY (preferred in PG 10+)
CREATE TABLE Orders (
```

```
    OrderId    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    OrderDate  DATE NOT NULL,  
    CustomerId INT NOT NULL  
);
```

Oracle

-- Option 1: Sequence + trigger (classic Oracle)

```
CREATE SEQUENCE orders_seq START WITH 1 INCREMENT BY 1;
```

```
CREATE TABLE Orders (  
    OrderId    NUMBER          PRIMARY KEY,  
    OrderDate  DATE            NOT NULL,  
    CustomerId NUMBER          NOT NULL  
);
```

```
CREATE OR REPLACE TRIGGER orders_bir  
    BEFORE INSERT ON Orders FOR EACH ROW  
BEGIN  
    :NEW.OrderId := orders_seq.NEXTVAL;  
END;
```

-- Option 2: IDENTITY column (Oracle 12c+, simpler)

```
CREATE TABLE Orders (  
    OrderId    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    OrderDate  DATE            NOT NULL,  
    CustomerId NUMBER          NOT NULL  
);
```

ALTER TABLE

-- Add a column

```
ALTER TABLE Employee ADD COLUMN Email VARCHAR(100);      -- MySQL/PG  
ALTER TABLE Employee ADD Email VARCHAR2(100);           -- Oracle
```

-- Drop a column

```
ALTER TABLE Employee DROP COLUMN Email;                  -- MySQL/PG  
ALTER TABLE Employee DROP COLUMN Email;                  -- Oracle
```

-- Add a constraint

```
ALTER TABLE Employee ADD CONSTRAINT chk_salary CHECK (Salary > 0);

-- Drop a constraint
ALTER TABLE Employee DROP CONSTRAINT chk_salary;

-- Rename a column (MySQL 8+ / PG / Oracle)
ALTER TABLE Employee RENAME COLUMN Fname TO FirstName;      -- PG / Oracle
ALTER TABLE Employee CHANGE Fname FirstName VARCHAR(15);    -- MySQL
```

DROP and TRUNCATE

MySQL

```
-- Remove table completely (structure + data)
DROP TABLE IF EXISTS Employee;

-- Remove all rows; reset AUTO_INCREMENT; keep structure
TRUNCATE TABLE Employee;

-- Drop multiple tables (respect FK order or disable checks temporarily)
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE Works_On, Dependent, Employee, Department, Project, Dept_Locations;
SET FOREIGN_KEY_CHECKS = 1;
```

PostgreSQL

```
DROP TABLE IF EXISTS Employee;

-- CASCADE also drops dependent views and FK constraints
DROP TABLE Employee CASCADE;

TRUNCATE TABLE Employee RESTART IDENTITY CASCADE;
```

Oracle

```
DROP TABLE Employee;

-- PURGE skips the recycle bin (immediate permanent deletion)
DROP TABLE Employee PURGE;
```



```
TRUNCATE TABLE Employee;
```

Feature	DELETE	TRUNCATE
Removes rows		
WHERE clause		
Fires row-level triggers		(varies)
Can be rolled back		Depends on DBMS
Resets identity/auto-incr		(with option)
Performance	Slower — logged per row	Faster — deallocates pages

DML — Manipulating Data

INSERT

```
-- Single row
```

```
INSERT INTO Department (Dnumber, Dname, Mgr_ssn)
VALUES (5, 'Research', '333445555');
```

```
-- Multiple rows (MySQL / PostgreSQL)
```

```
INSERT INTO Employee (Ssn, Fname, Lname, Salary, Dno) VALUES
('123456789', 'John', 'Smith', 30000, 5),
('333445555', 'Frank', 'Wong', 40000, 5),
('999887777', 'Alicia', 'Zelaya', 25000, 4);
```

```
-- Insert from SELECT
```

```
INSERT INTO Department_Backup
SELECT * FROM Department WHERE Dnumber IN (1, 4, 5);
```

```
-- Oracle — multi-row INSERT syntax differs
```

```
INSERT ALL
```

```
    INTO Employee (Ssn, Fname, Lname, Salary, Dno) VALUES ('123456789', 'John', 'Smith', 30000)
    INTO Employee (Ssn, Fname, Lname, Salary, Dno) VALUES ('333445555', 'Frank', 'Wong', 40000)
SELECT 1 FROM DUAL;
```

UPDATE

```
-- 10 % raise for all Research department employees
UPDATE Employee
SET    Salary = Salary * 1.10
WHERE  Dno = (SELECT Dnumber FROM Department WHERE Dname = 'Research');

-- Update multiple columns at once
UPDATE Employee
SET    Address = '731 Fondren, Houston TX',
       Salary  = 43000
WHERE  Ssn = '123456789';
```

DELETE

```
-- Remove a specific employee
DELETE FROM Employee
WHERE  Ssn = '123456789';

-- Remove all employees in a department (check FK constraints first)
DELETE FROM Employee
WHERE  Dno = (SELECT Dnumber FROM Department WHERE Dname = 'Headquarters');
```

SELECT — Querying Data

Logical Evaluation Order

SQL is *written* in SELECT–FROM–WHERE order but *evaluated* in a different order:

- FROM / JOIN – identify source rows (and join)
- WHERE – filter rows before grouping
- GROUP BY – group remaining rows
- HAVING – filter groups
- SELECT – compute output columns and aliases
- DISTINCT – eliminate duplicates
- ORDER BY – sort (can reference SELECT aliases)
- LIMIT / FETCH – restrict output count

! Why You Cannot Use a SELECT Alias in WHERE

WHERE is evaluated before SELECT, so any column alias defined in SELECT does not yet exist when WHERE runs. This is a frequent beginner mistake:

```
-- WRONG – alias 'annual' does not exist in WHERE
SELECT Salary * 12 AS annual FROM Employee WHERE annual > 600000;

-- CORRECT – repeat the expression
SELECT Salary * 12 AS annual FROM Employee WHERE Salary * 12 > 600000;

-- OR use a subquery / CTE (Chapter 8)
```

Basic SELECT

```
-- All columns
SELECT * FROM Employee;

-- Specific columns
SELECT Fname, Lname, Salary FROM Employee;

-- Column alias + expression
SELECT Fname AS "First", Lname AS "Last",
       Salary * 12 AS AnnualSalary
FROM Employee;

-- Eliminate duplicates
SELECT DISTINCT Dno FROM Employee;

-- Literal expression (no table needed in MySQL/PG)
SELECT 1 + 1; -- MySQL / PostgreSQL
SELECT 1 + 1 FROM DUAL; -- Oracle requires FROM DUAL for scalar queries
```

WHERE — Filtering Rows

Operator	Example	Notes
Comparison	Salary > 50000	=, <>, !=, <, <=, >, >=
BETWEEN ... AND	Salary BETWEEN 40000 AND 60000	Inclusive on both ends
IN (...)	Dno IN (1, 4, 5)	Membership test
LIKE	Lname LIKE 'S%'	% = any chars; _ = one char

Operator	Example	Notes
IS NULL	Super_ssn IS NULL	Never use = NULL
AND, OR, NOT	Dno = 5 AND Salary > 40000	AND binds tighter than OR

```
-- Find all female employees with salary above 30 000 in dept 4 or 5
```

```
SELECT Fname, Lname, Salary
FROM Employee
WHERE Gender = 'F'
      AND Salary > 30000
      AND Dno IN (4, 5);
```

```
-- Find employees whose last name contains 'a' as second character
```

```
SELECT Fname, Lname FROM Employee WHERE Lname LIKE '_a%';
```

```
-- Find employees with no supervisor (top-level)
```

```
SELECT Fname, Lname FROM Employee WHERE Super_ssn IS NULL;
```

ORDER BY

```
-- Sort by salary descending, then last name ascending (tie-breaker)
```

```
SELECT Fname, Lname, Salary
FROM Employee
ORDER BY Salary DESC, Lname ASC;
```

Limiting Rows / Pagination

MySQL

```
-- Top 5 highest-paid employees
```

```
SELECT Fname, Lname, Salary
FROM Employee
ORDER BY Salary DESC
LIMIT 5;
```

```
-- Pagination: skip first 10 rows, return next 5 (0-based offset)
```

```
SELECT Fname, Lname, Salary
FROM Employee
ORDER BY Salary DESC
LIMIT 5 OFFSET 10;
```

PostgreSQL

-- Top 5 highest-paid employees

```
SELECT Fname, Lname, Salary
FROM   Employee
ORDER  BY Salary DESC
LIMIT  5;
```

-- Pagination

```
SELECT Fname, Lname, Salary
FROM   Employee
ORDER  BY Salary DESC
LIMIT  5 OFFSET 10;
```

-- SQL standard syntax also works in PG

```
SELECT Fname, Lname, Salary
FROM   Employee
ORDER  BY Salary DESC
FETCH  FIRST 5 ROWS ONLY;
```

Oracle

-- Oracle 12c+ – SQL standard FETCH FIRST

```
SELECT Fname, Lname, Salary
FROM   Employee
ORDER  BY Salary DESC
FETCH  FIRST 5 ROWS ONLY;
```

-- Pagination with OFFSET

```
SELECT Fname, Lname, Salary
FROM   Employee
ORDER  BY Salary DESC
OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY;
```

-- Classic Oracle (pre-12c) – ROWNUM subquery pattern

```
SELECT * FROM (
    SELECT Fname, Lname, Salary,
           ROWNUM rn
    FROM   (SELECT Fname, Lname, Salary FROM Employee ORDER BY Salary DESC)
    WHERE  ROWNUM <= 15
)
```

```
)
WHERE rn > 10;
```

Aggregate Functions

Aggregate functions collapse a set of rows into a single value.

Function	Description	Ignores NULLs?
COUNT(*)	Total number of rows	No
COUNT(col)	Non-NULL values in a column	Yes
COUNT(DISTINCT col)	Distinct non-NULL values	Yes
SUM(col)	Sum of values	Yes
AVG(col)	Mean of values	Yes
MIN(col)	Minimum value	Yes
MAX(col)	Maximum value	Yes

Tip

AVG(Salary) divides the **sum of non-NULL salaries** by the **count of non-NULL rows**, not by total rows. If some employees have a NULL salary, AVG quietly excludes them — use COUNT(*) vs COUNT(Salary) to detect this.

GROUP BY and HAVING

```
-- Number of employees and average salary per department
SELECT  Dno,
        COUNT( *)           AS Headcount,
        ROUND(AVG(Salary), 2) AS AvgSalary,
        MAX(Salary)         AS MaxSalary
FROM    Employee
GROUP BY Dno
ORDER BY AvgSalary DESC;
```

Result (using Company DB values):

Dno	Headcount	AvgSalary	MaxSalary
5	4	33 250	40 000
4	3	31 000	43 000

Dno	Headcount	AvgSalary	MaxSalary
1	1	55 000	55 000

```
-- HAVING – filter on groups, not individual rows
-- Find departments where average salary exceeds 32 000
SELECT Dno, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Dno
HAVING AVG(Salary) > 32000;
```

! WHERE vs HAVING

	WHERE	HAVING
Filters	Individual rows	Entire groups
Evaluated	Before GROUP BY	After GROUP BY
Can reference	Column values	Aggregate results
Example	WHERE Salary > 30000	HAVING COUNT(*) > 2

You can use both in the same query:

```
SELECT Dno, COUNT(*) AS n
FROM Employee
WHERE Gender = 'F'          -- first filter rows (only female)
GROUP BY Dno
HAVING COUNT(*) >= 2;      -- then filter groups (≥ 2 female employees)
```

Joins

A join combines rows from two or more tables based on a related column. All examples use the Company DB.

CROSS JOIN (Cartesian Product)

```
SELECT E.Fname, D.Dname
FROM Employee E CROSS JOIN Department D;
-- Produces |Employee| × |Department| rows – every employee ↔ every dept
-- Useful only for generating test data or combinatorial problems
```

INNER JOIN

Returns only rows where the join condition is satisfied in **both** tables.

```
-- Algebra: Employee ⋈_{Dno=Dnumber} Department

-- Implicit syntax (old-style – avoid in new code)
SELECT E.Fname, E.Lname, D.Dname
FROM   Employee E, Department D
WHERE  E.Dno = D.Dnumber;

-- Explicit syntax (preferred)
SELECT E.Fname, E.Lname, D.Dname
FROM   Employee E
       INNER JOIN Department D ON E.Dno = D.Dnumber;
```

Multi-table join — all three levels of the Company DB:

```
SELECT E.Fname, E.Lname, P.Pname, W0.Hours
FROM   Employee   E
       INNER JOIN Works_On   W0 ON E.Ssn      = W0.Essn
       INNER JOIN Project    P  ON W0.Pno     = P.Pnumber
       INNER JOIN Department D  ON P.Dnum     = D.Dnumber
WHERE  D.Dname = 'Research'
ORDER BY E.Lname, P.Pname;
```

OUTER JOINs

An outer join preserves rows from one (or both) sides that have **no match**.

```
LEFT OUTER JOIN      – keep all rows from LEFT table
RIGHT OUTER JOIN     – keep all rows from RIGHT table
FULL OUTER JOIN      – keep unmatched rows from BOTH tables
```

```
-- LEFT JOIN: list every employee and their supervisor's name (NULL if no supervisor)
SELECT E.Fname AS Emp,
       S.Fname AS Supervisor
FROM   Employee E
       LEFT JOIN Employee S ON E.Super_ssn = S.Ssn;

-- RIGHT JOIN: list every department and its manager (NULL if no manager yet)
SELECT D.Dname, E.Fname, E.Lname
```



```
FROM Employee E
      RIGHT JOIN Department D ON E.Ssn = D.Mgr_ssn;
```

FULL OUTER JOIN — employees without a department AND departments without any employee:

MySQL

```
-- MySQL has NO native FULL OUTER JOIN — emulate with UNION:
SELECT E.Fname, E.Lname, D.Dname
FROM Employee E LEFT JOIN Department D ON E.Dno = D.Dnumber
UNION
SELECT E.Fname, E.Lname, D.Dname
FROM Employee E RIGHT JOIN Department D ON E.Dno = D.Dnumber;
-- UNION removes duplicates; use UNION ALL if you want to keep them
```

PostgreSQL

```
SELECT E.Fname, E.Lname, D.Dname
FROM Employee E
      FULL OUTER JOIN Department D ON E.Dno = D.Dnumber;
```

Oracle

```
SELECT E.Fname, E.Lname, D.Dname
FROM Employee E
      FULL OUTER JOIN Department D ON E.Dno = D.Dnumber;
-- Oracle supports FULL OUTER JOIN natively since version 9i
```

SELF JOIN

A table joined to itself using different aliases — useful for hierarchical data.

```
-- Find each employee and their direct supervisor
SELECT E.Fname || ' ' || E.Lname AS Employee,
       S.Fname || ' ' || S.Lname AS Supervisor
FROM Employee E
      LEFT JOIN Employee S ON E.Super_ssn = S.Ssn
ORDER BY Supervisor, Employee;
```

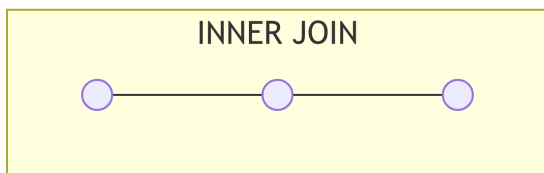
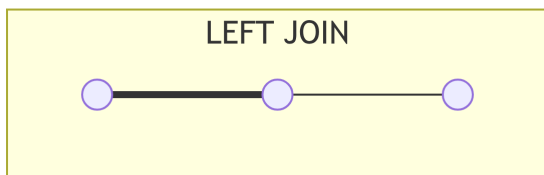
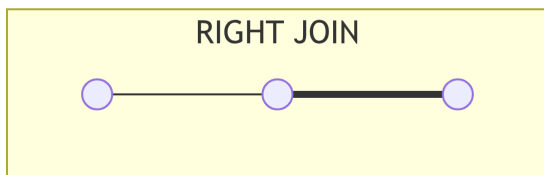
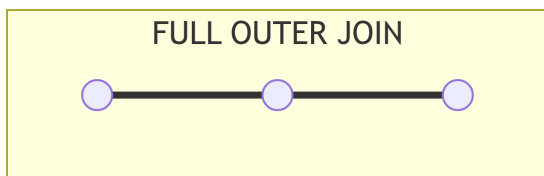
 Tip

String concatenation syntax varies:

Dialect	Operator	Example
MySQL	CONCAT()	CONCAT(Fname, ' ', Lname)
PostgreSQL		Fname ' ' Lname
Oracle		Fname ' ' Lname

MySQL also supports || when PIPES_AS_CONCAT SQL mode is enabled.

Join Type Visual Summary



Bold (===) edges indicate rows that appear in the result even when the other side has no match.

Subqueries

A **subquery** (nested query) is a SELECT statement inside another SQL statement.

Scalar Subquery

Returns exactly one row and one column; can be used wherever a single value is expected.

```
-- Employees earning more than the company average salary
SELECT Fname, Lname, Salary
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

Row / Table Subquery in WHERE

```
-- Employees who work in the same department as 'John Smith'
SELECT Fname, Lname
FROM Employee
WHERE Dno = (SELECT Dno FROM Employee WHERE Fname = 'John' AND Lname = 'Smith');

-- Employees in departments located in 'Houston'
SELECT Fname, Lname
FROM Employee
WHERE Dno IN (SELECT Dnumber FROM Department
              WHERE Dnumber IN (SELECT Dnumber FROM Dept_Locations
                                WHERE Dlocation = 'Houston'));
```

Correlated Subquery

A subquery that references a column from the outer query — re-evaluated once per outer row.

```
-- For each employee, show their salary relative to their own department's average
SELECT E.Fname, E.Lname, E.Salary,
       (SELECT AVG(E2.Salary)
        FROM Employee E2
        WHERE E2.Dno = E.Dno) AS DeptAvg,
       E.Salary - (SELECT AVG(E2.Salary)
                   FROM Employee E2
                   WHERE E2.Dno = E.Dno) AS VsAvg
FROM Employee E
ORDER BY E.Dno, VsAvg DESC;
```

i Note

Correlated subqueries are conceptually clear but can be slow on large tables because the inner query runs once per outer row. Chapter 8 introduces **Common Table Expressions (CTEs)** and **window functions** — more efficient alternatives for the pattern above.

EXISTS and NOT EXISTS

EXISTS(subquery) is TRUE if the subquery returns **at least one row**. It is typically faster than IN when the subquery result set is large, because it short-circuits at the first match.

```
-- Employees who are supervising at least one other employee
SELECT DISTINCT S.Fname, S.Lname
FROM   Employee S
WHERE  EXISTS (SELECT 1 FROM Employee E WHERE E.Super_ssn = S.Ssn);
```

```
-- Employees who have NO dependents registered
SELECT E.Fname, E.Lname
FROM   Employee E
WHERE  NOT EXISTS (SELECT 1 FROM Dependent D WHERE D.Essn = E.Ssn);
```

```
-- Projects that have at least one worker logging > 20 hours
SELECT P.Pname
FROM   Project P
WHERE  EXISTS (SELECT 1 FROM Works_On W
               WHERE W.Pno = P.Pnumber AND W.Hours > 20);
```

Set Operators: UNION, INTERSECT, EXCEPT**MySQL**

```
-- UNION: employees in dept 5 OR who supervise someone
SELECT Ssn, Fname, Lname FROM Employee WHERE Dno = 5
UNION
SELECT DISTINCT S.Ssn, S.Fname, S.Lname
FROM   Employee S INNER JOIN Employee E ON E.Super_ssn = S.Ssn;
```

```
-- MySQL 8.0.31+ supports INTERSECT and EXCEPT:
SELECT Ssn FROM Employee WHERE Dno = 5
INTERSECT
SELECT Essn FROM Works_On WHERE Pno = 1;
```

PostgreSQL

```
-- UNION ALL (keep duplicates), UNION (remove duplicates)
SELECT Ssn FROM Employee WHERE Dno = 5
UNION ALL
SELECT Essn FROM Works_On WHERE Pno = 1;

-- INTERSECT: SSNs of dept 5 employees also working on Project 1
SELECT Ssn FROM Employee WHERE Dno = 5
INTERSECT
SELECT Essn FROM Works_On WHERE Pno = 1;

-- EXCEPT: dept 5 employees NOT working on Project 1
SELECT Ssn FROM Employee WHERE Dno = 5
EXCEPT
SELECT Essn FROM Works_On WHERE Pno = 1;
```

Oracle

```
-- Oracle uses MINUS instead of EXCEPT
SELECT Ssn FROM Employee WHERE Dno = 5
MINUS
SELECT Essn FROM Works_On WHERE Pno = 1;

-- INTERSECT works the same
SELECT Ssn FROM Employee WHERE Dno = 5
INTERSECT
SELECT Essn FROM Works_On WHERE Pno = 1;
```

! Set Operator Compatibility Rules

1. Both queries must return the **same number of columns**.
2. Corresponding columns must have **compatible data types**.
3. The result column **names and order come from the first query**.
4. **UNION** (without **ALL**) eliminates duplicate rows — equivalent to $R \cup S$ in algebra.
5. **UNION ALL** keeps all duplicates (faster — no dedup step).

Interactive: Join Type Explorer

Chapter Summary

! Key Takeaways — Chapter 7

1. **SQL is declarative**; the optimizer maps it to a relational algebra expression tree. The logical evaluation order (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT) explains why SELECT aliases cannot appear in WHERE.
2. **DDL**: CREATE TABLE with six constraint types (PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT); ALTER TABLE; DROP TABLE; TRUNCATE. Auto-increment syntax differs: AUTO_INCREMENT (MySQL), SERIAL/IDENTITY (PostgreSQL), SEQUENCE (Oracle).
3. **DML**: INSERT (single/multi-row/from-SELECT), UPDATE (with WHERE and subquery), DELETE. Prefer DELETE when you need filtering or rollback; prefer TRUNCATE for wiping entire tables.
4. **SELECT pipeline**: filter with WHERE, group with GROUP BY, filter groups with HAVING, sort with ORDER BY, paginate with LIMIT/FETCH FIRST/ROWNUM.
5. **Aggregate functions** (COUNT, SUM, AVG, MIN, MAX) ignore NULLs except COUNT (*).
6. **Joins**: INNER removes non-matching rows; LEFT/RIGHT/FULL OUTER preserve unmatched rows; SELF JOIN handles hierarchies; CROSS JOIN produces the Cartesian product. MySQL lacks native FULL OUTER JOIN — emulate with LEFT JOIN UNION RIGHT JOIN.
7. **Subqueries**: scalar (one value), correlated (per outer row), EXISTS/NOT EXISTS (short-circuit, often faster than IN for large sets).
8. **Set operators**: UNION/INTERSECT/EXCEPT (Oracle: MINUS) — require column-count and type compatibility.

Review Questions

Conceptual

1. Why does SQL evaluate FROM before SELECT? Give one practical consequence of this evaluation order.

2. Explain the difference between DELETE, TRUNCATE, and DROP. Which ones can be rolled back? Which ones fire triggers?
3. What is referential integrity, and how do ON DELETE CASCADE and ON DELETE SET NULL differ? Give a concrete example for each using the Employee–Department scenario.

Applied

4. Write SQL to create the Works_On(Essn CHAR(9), Pno INT, Hours DECIMAL(5,2)) table with:
 - A composite primary key on (Essn, Pno)
 - Essn as a foreign key referencing Employee(Ssn)
 - Pno as a foreign key referencing Project(Pnumber)
 - Hours as NOT NULL with a CHECK that Hours >= 0
5. Using the Company DB, write a single SQL query to list the **full name** and **total hours worked** for every employee who has ever worked on a project in the Research department. Include only employees whose total hours exceed 30.
6. Write a query that returns each project name along with the number of distinct employees working on it. Include projects with **zero workers** (use an appropriate outer join).

Analysis

7. Predict the output of the following query:

```
SELECT COUNT(*), COUNT(Super_ssn), COUNT(DISTINCT Dno)
FROM Employee;
```

Given that the Company DB has 8 employees, 1 has no supervisor, and they span 3 departments.

8. A trainee writes:

```
SELECT Dno, AVG(Salary) AS AvgSal
FROM Employee
WHERE AvgSal > 35000
GROUP BY Dno;
```

Identify the error and write the corrected query.

Dialect Awareness

9. A query written for PostgreSQL uses `EXCEPT` to find SSNs in one table but not another. Rewrite it for Oracle, and note any limitations of MySQL 8.0.30 with `EXCEPT`.
10. Your production database is Oracle 11g (pre-12c). You need a surrogate key on a `Customer` table. Write the complete DDL including the sequence, the table, and the trigger.

Design

11. The `Employee` table has `Salary DECIMAL(10,2)`. A business rule states that no employee may earn less than 20 000 or more than 500 000. Where should this constraint be enforced — as a `CHECK` constraint, in the application layer, or both? Justify your answer.

Synthesis

12. Using `INNER JOIN`, `LEFT JOIN`, and `NOT EXISTS`, write **three different** SQL queries that all return: *employees who have no dependents registered in the `Dependent` table*. Which approach do you consider most readable? Which is likely most efficient, and why?

Advanced SQL

Chapter 8

Building on the Basics

Chapter 7 covered the foundational SELECT–FROM–WHERE pipeline, joins, and aggregates. This chapter introduces the features that make SQL genuinely powerful for real-world applications:

Feature	Purpose
CTEs (WITH)	Name and reuse intermediate results; enable recursion
Window functions (OVER)	Analytics without collapsing rows
Views	Virtual named queries; security and abstraction
Stored procedures & functions	Encapsulate reusable logic inside the database
Triggers	Automatic event-driven actions
JSON functions	Work with semi-structured data inline with relational data

Common Table Expressions (CTEs)

A **Common Table Expression** names a temporary result set scoped to a single query. It replaces deeply nested subqueries with readable, layered logic.

```
-- Basic CTE syntax
WITH cte_name AS (
    SELECT ...
)
SELECT ... FROM cte_name ...;
```

Single CTE — Readable Subquery

```
-- Employees earning more than the overall average salary
WITH AvgSalary AS (
    SELECT AVG(Salary) AS Avg FROM Employee
)
SELECT E.Fname, E.Lname, E.Salary, A.Avg
FROM   Employee E, AvgSalary A
WHERE  E.Salary > A.Avg
ORDER BY E.Salary DESC;
```

Multiple CTEs — Chained Steps

```
-- Step 1: department headcount
-- Step 2: department total salary
-- Step 3: combine and compute cost-per-head
WITH Headcount AS (
    SELECT Dno, COUNT(*) AS n
    FROM   Employee
    GROUP BY Dno
),
TotalSal AS (
    SELECT Dno, SUM(Salary) AS Total
    FROM   Employee
    GROUP BY Dno
)
SELECT D.Dname,
       H.n      AS Employees,
       T.Total   AS PayrollTotal,
       T.Total / H.n AS CostPerHead
FROM   Department D
       JOIN Headcount H ON D.Dnumber = H.Dno
       JOIN TotalSal T ON D.Dnumber = T.Dno
ORDER BY CostPerHead DESC;
```

Recursive CTE

A recursive CTE has two parts joined by UNION ALL:

Part	Role
Anchor member	Base case — the starting row(s)

Part	Role
Recursive member	References the CTE itself — adds one level per iteration

```
-- Build the complete management chain above employee '123456789'
```

MySQL

```
WITH RECURSIVE OrgChain AS (
    -- Anchor: start from the target employee
    SELECT Ssn, Fname, Lname, Super_ssn, 0 AS Depth
    FROM Employee
    WHERE Ssn = '123456789'

    UNION ALL

    -- Recursive: walk up to each supervisor
    SELECT E.Ssn, E.Fname, E.Lname, E.Super_ssn, OC.Depth + 1
    FROM Employee E
         INNER JOIN OrgChain OC ON E.Ssn = OC.Super_ssn
)
SELECT Depth, Fname, Lname, Ssn
FROM OrgChain
ORDER BY Depth;
```

PostgreSQL

```
WITH RECURSIVE OrgChain AS (
    SELECT Ssn, Fname, Lname, Super_ssn, 0 AS Depth
    FROM Employee
    WHERE Ssn = '123456789'

    UNION ALL

    SELECT E.Ssn, E.Fname, E.Lname, E.Super_ssn, OC.Depth + 1
    FROM Employee E
         INNER JOIN OrgChain OC ON E.Ssn = OC.Super_ssn
)
SELECT Depth, Fname, Lname, Ssn
```

```
FROM OrgChain
ORDER BY Depth;
```

-- Identical syntax to MySQL 8; PostgreSQL has supported RECURSIVE CTEs since version 8.4

Oracle

-- Oracle uses CONNECT BY for hierarchical queries (classic syntax)

```
SELECT LEVEL - 1 AS Depth, Fname, Lname, Ssn
FROM Employee
START WITH Ssn = '123456789'
CONNECT BY PRIOR Super_ssn = Ssn
ORDER BY Depth;
```

-- Oracle 11g R2+ also supports the SQL:1999 recursive CTE:

```
WITH OrgChain (Ssn, Fname, Lname, Super_ssn, Depth) AS (
    SELECT Ssn, Fname, Lname, Super_ssn, 0
    FROM Employee
    WHERE Ssn = '123456789'
    UNION ALL
    SELECT E.Ssn, E.Fname, E.Lname, E.Super_ssn, OC.Depth + 1
    FROM Employee E
        INNER JOIN OrgChain OC ON E.Ssn = OC.Super_ssn
)
SELECT Depth, Fname, Lname, Ssn FROM OrgChain ORDER BY Depth;
-- Note: Oracle does NOT use the RECURSIVE keyword
```

! Recursive CTE Safety

Always add a **termination condition** to prevent infinite recursion. The DBMS uses an implicit cycle-detection threshold (e.g., PostgreSQL's `cte_stack_limit`). Oracle's `CONNECT BY` automatically detects cycles when `NOCYCLE` is specified.

Window Functions

Window functions compute values *across a set of rows related to the current row* **without collapsing** those rows into a single output row — unlike `GROUP BY`.

```
FUNCTION_NAME() OVER (
```

```

[PARTITION BY col, ...] -- divide rows into groups (windows)
[ORDER BY col, ...]      -- order within each window
[ROWS/RANGE BETWEEN ...] -- optional frame specification
)

```

Ranking Functions

```

SELECT Fname, Lname, Dno, Salary,
       ROW_NUMBER() OVER (PARTITION BY Dno ORDER BY Salary DESC) AS RowNum,
       RANK()        OVER (PARTITION BY Dno ORDER BY Salary DESC) AS Rnk,
       DENSE_RANK()  OVER (PARTITION BY Dno ORDER BY Salary DESC) AS DenseRnk,
       NTILE(3)       OVER (PARTITION BY Dno ORDER BY Salary DESC) AS Tercile
FROM   Employee;

```

Difference between RANK and DENSE_RANK — when two employees share a salary:

Salary	ROW_NUMBER	RANK	DENSE_RANK
40 000	1	1	1
38 000	2	2	2
38 000	3	2	2
35 000	4	4 (<i>gap!</i>)	3 (<i>no gap</i>)

Analytic / Offset Functions

```

-- For each employee: show salary, the previous hire's salary, and next hire's salary
-- (employees ordered by Bdate as a proxy for hire date)
SELECT Fname, Lname, Bdate, Salary,
       LAG (Salary, 1) OVER (ORDER BY Bdate) AS PrevSalary,
       LEAD(Salary, 1) OVER (ORDER BY Bdate) AS NextSalary,
       FIRST_VALUE(Fname) OVER (PARTITION BY Dno ORDER BY Salary DESC) AS TopEarnerInDept
FROM   Employee;

```

Running Aggregates

```

-- Cumulative total salary spent, ordered by employee bdate
SELECT Fname, Lname, Bdate, Salary,
       SUM(Salary) OVER (ORDER BY Bdate
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
                        ) AS RunningTotal,
       AVG(Salary) OVER (PARTITION BY Dno) AS DeptAvg,
       Salary - AVG(Salary) OVER (PARTITION BY Dno) AS VsDeptAvg

```

```
FROM Employee
ORDER BY Bdate;
```

Frame Specification (ROWS vs RANGE)

MySQL

```
-- MySQL 8.0 supports frames; default is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
SELECT Fname, Salary,
       SUM(Salary) OVER (
         ORDER BY Salary
         ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING -- 3-row sliding window
       ) AS SlidingSum
FROM Employee;
```

PostgreSQL

```
SELECT Fname, Salary,
       SUM(Salary) OVER (
         ORDER BY Salary
         ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
       ) AS SlidingSum,
       -- RANGE uses value comparison; useful for date windows
       SUM(Salary) OVER (
         ORDER BY Salary
         RANGE BETWEEN 5000 PRECEDING AND CURRENT ROW
       ) AS SalaryRangeSum
FROM Employee;
```

Oracle

```
-- Oracle supports full frame specification since 8i
SELECT Fname, Salary,
       SUM(Salary) OVER (
         ORDER BY Salary
         ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
       ) AS SlidingSum
FROM Employee;
```

Window Functions vs GROUP BY

Aspect	GROUP BY	Window Function
Output rows	One per group	Same count as input
Access to non-group cols	No	Yes
Access to aggregate result	Via HAVING	Via the window expression
Typical use	Summarize	Compare each row to its group

Views

A **view** is a stored SELECT statement with a name — a virtual table. When you query a view, the DBMS substitutes its definition (called *view expansion*).

Creating and Querying Views

MySQL

```
-- Create a view of Research department employees
CREATE VIEW ResearchEmployees AS
SELECT E.Ssn, E.Fname, E.Lname, E.Salary, E.Bdate
FROM Employee E
      JOIN Department D ON E.Dno = D.Dnumber
WHERE D.Dname = 'Research';

-- Query the view like a table
SELECT Fname, Lname, Salary
FROM ResearchEmployees
WHERE Salary > 35000;

-- Replace / modify the view
CREATE OR REPLACE VIEW ResearchEmployees AS
SELECT E.Ssn, E.Fname, E.Lname, E.Salary, E.Bdate, E.Gender
FROM Employee E
      JOIN Department D ON E.Dno = D.Dnumber
WHERE D.Dname = 'Research';

DROP VIEW IF EXISTS ResearchEmployees;
```

PostgreSQL

```
CREATE VIEW ResearchEmployees AS
SELECT E.Ssn, E.Fname, E.Lname, E.Salary, E.Bdate
FROM   Employee E
       JOIN Department D ON E.Dno = D.Dnumber
WHERE  D.Dname = 'Research';

-- PG supports CREATE OR REPLACE VIEW with the same column list
CREATE OR REPLACE VIEW ResearchEmployees AS
SELECT E.Ssn, E.Fname, E.Lname, E.Salary, E.Bdate, E.Gender
FROM   Employee E
       JOIN Department D ON E.Dno = D.Dnumber
WHERE  D.Dname = 'Research';

-- MATERIALIZED VIEW (stores results physically – refreshed manually)
CREATE MATERIALIZED VIEW ResearchSummary AS
SELECT Dno, COUNT(*) AS n, AVG(Salary) AS avg_sal FROM Employee GROUP BY Dno;

REFRESH MATERIALIZED VIEW ResearchSummary;
```

Oracle

```
CREATE OR REPLACE VIEW ResearchEmployees AS
SELECT E.Ssn, E.Fname, E.Lname, E.Salary, E.Bdate
FROM   Employee E
       JOIN Department D ON E.Dno = D.Dnumber
WHERE  D.Dname = 'Research';

-- Oracle Materialized Views (MVIEW)
CREATE MATERIALIZED VIEW ResearchSummary
BUILD IMMEDIATE
REFRESH FAST ON COMMIT AS
SELECT Dno, COUNT(*) AS n, AVG(Salary) AS avg_sal FROM Employee GROUP BY Dno;
```

Updatable vs Non-Updatable Views

A view supports DML (INSERT/UPDATE/DELETE) only under strict conditions:

Condition	Updatable?
Single base table	
No DISTINCT	
No aggregate functions	
No GROUP BY / HAVING	
No set operators	
No subqueries in SELECT	
All NOT NULL columns included (for INSERT)	
JOIN (multi-table)	in most DBMSs

-- This simple view IS updatable

```
CREATE VIEW EmpSalaries AS SELECT Ssn, Fname, Lname, Salary FROM Employee;
```

```
UPDATE EmpSalaries SET Salary = 45000 WHERE Ssn = '123456789';
```

-- DBMS translates this to: UPDATE Employee SET Salary = 45000 WHERE Ssn = '123456789'

Tip

Use WITH CHECK OPTION to prevent a view update from making a row *invisible* from the view:

```
CREATE VIEW HighEarners AS
SELECT * FROM Employee WHERE Salary > 50000
WITH CHECK OPTION;
```

-- This would reject a salary reduction below 50000 via the view:

```
UPDATE HighEarners SET Salary = 30000 WHERE Ssn = '123456789'; -- ERROR
```

Stored Procedures and Functions

Stored programs run inside the database engine — they avoid round-trips and centralize business logic.

Stored Procedures

MySQL

```
DELIMITER $$
```

```
CREATE PROCEDURE GiveRaise(
```

```
    IN p_dept    INT,
    IN p_pct     DECIMAL(5,2),
    OUT p_count  INT
)
BEGIN
    UPDATE Employee
    SET    Salary = Salary * (1 + p_pct / 100)
    WHERE  Dno = p_dept;

    SET p_count = ROW_COUNT();
    SELECT CONCAT('Raise applied to ', p_count, ' employees.') AS msg;
END$$

DELIMITER ;

-- Call
CALL GiveRaise(5, 10.0, @n);
SELECT @n AS RowsUpdated;
```

PostgreSQL

```
CREATE OR REPLACE PROCEDURE GiveRaise(
    p_dept    INTEGER,
    p_pct     NUMERIC
)
LANGUAGE plpgsql AS $$
DECLARE
    v_count INTEGER;
BEGIN
    UPDATE Employee
    SET    Salary = Salary * (1 + p_pct / 100)
    WHERE  Dno = p_dept;

    GET DIAGNOSTICS v_count = ROW_COUNT;
    RAISE NOTICE 'Raise applied to % employees.', v_count;
END;
$$;

-- Call (PG 11+)
CALL GiveRaise(5, 10.0);
```

Oracle

```

CREATE OR REPLACE PROCEDURE GiveRaise(
    p_dept    IN  NUMBER,
    p_pct     IN  NUMBER,
    p_count   OUT NUMBER
) AS
BEGIN
    UPDATE Employee
    SET     Salary = Salary * (1 + p_pct / 100)
    WHERE  Dno = p_dept;

    p_count := SQL%ROWCOUNT;
    DBMS_OUTPUT.PUT_LINE('Raise applied to ' || p_count || ' employees.');
```

END;

/

-- Call via anonymous block

```

DECLARE
    v_count NUMBER;
BEGIN
    GiveRaise(5, 10.0, v_count);
    DBMS_OUTPUT.PUT_LINE('Updated: ' || v_count);
END;
/
```

Stored Functions

Functions return a value and can be used inside SQL expressions.

MySQL

```

DELIMITER $$
CREATE FUNCTION DeptHeadcount(p_dept INT)
RETURNS INT
DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE n INT;
    SELECT COUNT(*) INTO n FROM Employee WHERE Dno = p_dept;
    RETURN n;

```

```
END$$
DELIMITER ;

-- Use in a query
SELECT Dname, DeptHeadcount(Dnumber) AS Headcount FROM Department;
```

PostgreSQL

```
CREATE OR REPLACE FUNCTION DeptHeadcount(p_dept INTEGER)
RETURNS INTEGER
LANGUAGE sql
STABLE AS $$
    SELECT COUNT(*)::INTEGER FROM Employee WHERE Dno = p_dept;
$$;

SELECT Dname, DeptHeadcount(Dnumber) AS Headcount FROM Department;
```

Oracle

```
CREATE OR REPLACE FUNCTION DeptHeadcount(p_dept NUMBER)
RETURN NUMBER
AS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM Employee WHERE Dno = p_dept;
    RETURN v_count;
END;
/

SELECT Dname, DeptHeadcount(Dnumber) AS Headcount FROM Department;
```

Procedures vs Functions vs Triggers

	Stored Procedure	Stored Function	Trigger
In- voked by	CALL	SQL expression	DBMS automatically
Re- turns	Nothing (or OUT params)	Exactly one value	N/A

	Stored Procedure	Stored Function	Trigger
Commit/rollback	Yes (varies)	No (typically)	In same transaction
Inside			
Side effects	Yes	Discouraged	Yes (but use carefully)
OK?			

Triggers

A **trigger** is a named database object that fires automatically when a specified DML event occurs on a table.

Trigger Anatomy

```
CREATE [OR REPLACE] TRIGGER trigger_name
  { BEFORE | AFTER | INSTEAD OF }
  { INSERT | UPDATE [OF col] | DELETE }
  ON table_name
  [FOR EACH ROW]
  [WHEN condition]
trigger_body
```

	Available in	Available in	Available in
Pseudo-record	INSERT	UPDATE	DELETE
OLD / :OLD	— (NULL)	Row before change	Row being deleted
NEW / :NEW	Inserted row	Row after change	— (NULL)

Audit Trigger Example

MySQL

```
-- First create an audit table
CREATE TABLE SalaryAudit (
```

```
AuditId    INT AUTO_INCREMENT PRIMARY KEY,  
Ssn        CHAR(9),  
OldSalary  DECIMAL(10,2),  
NewSalary  DECIMAL(10,2),  
ChangedAt  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
ChangedBy  VARCHAR(50) DEFAULT CURRENT_USER()  
);  
  
DELIMITER $$  
CREATE TRIGGER trg_salary_audit  
AFTER UPDATE ON Employee  
FOR EACH ROW  
BEGIN  
    IF OLD.Salary <> NEW.Salary THEN  
        INSERT INTO SalaryAudit (Ssn, OldSalary, NewSalary)  
        VALUES (OLD.Ssn, OLD.Salary, NEW.Salary);  
    END IF;  
END$$  
DELIMITER ;
```

PostgreSQL

```
CREATE TABLE SalaryAudit (  
    AuditId  SERIAL PRIMARY KEY,  
    Ssn      CHAR(9),  
    OldSalary NUMERIC(10,2),  
    NewSalary NUMERIC(10,2),  
    ChangedAt TIMESTAMP DEFAULT NOW(),  
    ChangedBy TEXT        DEFAULT CURRENT_USER  
);  
  
-- Step 1: trigger function  
CREATE OR REPLACE FUNCTION fn_salary_audit()  
RETURNS TRIGGER LANGUAGE plpgsql AS $$  
BEGIN  
    IF OLD.Salary IS DISTINCT FROM NEW.Salary THEN  
        INSERT INTO SalaryAudit (Ssn, OldSalary, NewSalary)  
        VALUES (OLD.Ssn, OLD.Salary, NEW.Salary);  
    END IF;  
    RETURN NEW;  
END;
```

```

END;
$$;

-- Step 2: bind trigger to table
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON Employee
FOR EACH ROW EXECUTE FUNCTION fn_salary_audit();

```

Oracle

```

CREATE TABLE SalaryAudit (
    AuditId    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Ssn        CHAR(9),
    OldSalary  NUMBER(10,2),
    NewSalary  NUMBER(10,2),
    ChangedAt  TIMESTAMP DEFAULT SYSTIMESTAMP,
    ChangedBy  VARCHAR2(50) DEFAULT SYS_CONTEXT('USERENV','SESSION_USER')
);

CREATE OR REPLACE TRIGGER trg_salary_audit
AFTER UPDATE OF Salary ON Employee
FOR EACH ROW
WHEN (OLD.Salary <> NEW.Salary)
BEGIN
    INSERT INTO SalaryAudit (Ssn, OldSalary, NewSalary)
    VALUES (:OLD.Ssn, :OLD.Salary, :NEW.Salary);
END;
/

```

Validation Trigger — Rejecting Bad Data

```

-- PostgreSQL: prevent salary reduction via trigger
CREATE OR REPLACE FUNCTION fn_no_salary_cut()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.Salary < OLD.Salary THEN
        RAISE EXCEPTION 'Salary reduction not permitted: % → %',
            OLD.Salary, NEW.Salary;
    END IF;
    RETURN NEW;

```

```
END;  
$$;
```

```
CREATE TRIGGER trg_no_salary_cut  
BEFORE UPDATE ON Employee  
FOR EACH ROW EXECUTE FUNCTION fn_no_salary_cut();
```

! Trigger Design Principles

1. **Keep triggers short** — complex logic belongs in stored procedures.
2. **Avoid cascading triggers** — trigger A fires trigger B, which fires trigger A → infinite loop.
3. **Use INSTEAD OF triggers on views** to implement DML on non-updatable views.
4. **Document every trigger** — they are invisible to external callers and hard to debug.

JSON Functions

Modern RDBMS supports querying and manipulating JSON data directly in SQL.

JSON Storage

MySQL

```
CREATE TABLE Products (  
    Id          INT AUTO_INCREMENT PRIMARY KEY,  
    Name        VARCHAR(100),  
    Attributes  JSON                -- MySQL validates JSON on INSERT  
);  
  
INSERT INTO Products (Name, Attributes)  
VALUES ('Laptop', '{"brand":"Dell","ram_gb":16,"tags":["work","portable"]}');
```

PostgreSQL

```
CREATE TABLE Products (  
    Id          SERIAL PRIMARY KEY,  
    Name        VARCHAR(100),  
    Attributes  JSONB          -- JSONB: binary, indexed, preferred over JSON  
);
```



```
INSERT INTO Products (Name, Attributes)
VALUES ('Laptop', '{"brand":"Dell","ram_gb":16,"tags":["work","portable"]}');
```

Oracle

```
CREATE TABLE Products (
    Id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Name        VARCHAR2(100),
    Attributes  CLOB          -- store as CLOB with IS JSON constraint
                CONSTRAINT chk_json CHECK (Attributes IS JSON)
);
```

```
INSERT INTO Products (Name, Attributes)
VALUES ('Laptop', '{"brand":"Dell","ram_gb":16,"tags":["work","portable"]}');
```

Reading JSON Values

MySQL

```
-- JSON_EXTRACT – returns JSON value
SELECT Name,
       JSON_EXTRACT(Attributes, '$.brand') AS Brand,
       JSON_EXTRACT(Attributes, '$.ram_gb') AS RAM
FROM   Products;
```

```
-- Shorthand operator ->> (returns unquoted string)
SELECT Name,
       Attributes ->> '$.brand' AS Brand,
       Attributes ->> '$.ram_gb' AS RAM
FROM   Products;
```

```
-- Filter on a JSON field
SELECT Name FROM Products
WHERE Attributes ->> '$.ram_gb' > 8;
```

PostgreSQL

```
-- -> returns JSON; ->> returns text
SELECT Name,
       Attributes -> 'brand' AS Brand_JSON,
       Attributes ->> 'brand' AS Brand_text,
       Attributes ->> 'ram_gb' AS RAM
FROM   Products;

-- Nested path with #>>
SELECT Attributes #>> '{tags,0}' AS FirstTag FROM Products;

-- Filter
SELECT Name FROM Products WHERE (Attributes ->> 'ram_gb')::INT > 8;

-- JSONB index (GIN for arbitrary key search)
CREATE INDEX idx_products_attrs ON Products USING GIN (Attributes);
```

Oracle

```
-- JSON_VALUE – extract a scalar
SELECT Name,
       JSON_VALUE(Attributes, '$.brand') AS Brand,
       JSON_VALUE(Attributes, '$.ram_gb') AS RAM
FROM   Products;

-- JSON_QUERY – extract an object or array
SELECT JSON_QUERY(Attributes, '$.tags') AS Tags FROM Products;

-- JSON_TABLE – shred JSON into relational rows
SELECT P.Name, JT.Tag
FROM   Products P,
       JSON_TABLE(P.Attributes, '$.tags[*]'
                  COLUMNS (Tag VARCHAR2(50) PATH '$')
                  ) JT;
```

Modifying JSON

MySQL

```
-- Set a field
UPDATE Products
SET   Attributes = JSON_SET(Attributes, '$.ram_gb', 32)
WHERE Id = 1;

-- Add a field
UPDATE Products
SET   Attributes = JSON_INSERT(Attributes, '$.price_usd', 1299.99)
WHERE Id = 1;

-- Remove a field
UPDATE Products
SET   Attributes = JSON_REMOVE(Attributes, '$.tags')
WHERE Id = 1;
```

PostgreSQL

```
-- Replace a key (creates new JSONB)
UPDATE Products
SET   Attributes = Attributes || '{"ram_gb": 32}'
WHERE Id = 1;

-- Remove a key with the - operator
UPDATE Products
SET   Attributes = Attributes - 'tags'
WHERE Id = 1;

-- jsonb_set for nested paths
UPDATE Products
SET   Attributes = jsonb_set(Attributes, '{ram_gb}', '32')
WHERE Id = 1;
```

Oracle

```
-- Oracle 21c JSON_MERGEPATCH – merge/update JSON document
UPDATE Products
SET   Attributes = JSON_MERGEPATCH(Attributes, '{"ram_gb": 32}')
```

```
WHERE Id = 1;
```

Chapter Summary

! Key Takeaways — Chapter 8

1. **CTEs** (`WITH cte AS (...)`) name intermediate result sets, dramatically improving readability. **Recursive CTEs** (`UNION ALL`) traverse hierarchies. Oracle omits the `RECURSIVE` keyword and also offers `CONNECT BY`.
 2. **Window functions** (`OVER (PARTITION BY ... ORDER BY ...)`) perform analytics without collapsing rows — unlike `GROUP BY`. Key functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG`, `LEAD`, `SUM OVER`.
 3. **Views** are stored named queries — virtual tables. They can be updatable (single table, no aggregates) or materialized (physical snapshot, refreshed). Use `WITH CHECK OPTION` to prevent rows from escaping the view filter.
 4. **Stored procedures** (`CALL`) encapsulate multi-statement operations. **Stored functions** return a value usable in SQL expressions. Syntax differs significantly: MySQL uses `DELIMITER $$`, PG uses `$$ LANGUAGE plpgsql`, Oracle uses `AS ... END;`.
 5. **Triggers** fire automatically `BEFORE`, `AFTER`, or `INSTEAD OF DML` events. `OLD/NEW` (`:OLD/:NEW` in Oracle) access row values before/after the change. Keep trigger bodies minimal; avoid cascading trigger chains.
 6. **JSON functions** vary widely by dialect: MySQL uses `JSON_EXTRACT/->>`, PostgreSQL uses `->>/#>>/GIN` indexes on `JSONB`, Oracle uses `JSON_VALUE/JSON_QUERY/JSON_TABLE` on `CLOBs`.
-

Review Questions

Conceptual

1. Explain the difference between a CTE and a subquery. When would you choose one over the other? Can every CTE be rewritten as a subquery? Can every subquery be rewritten as a CTE?
2. What is a **materialized view**? How does it differ from a regular view in terms of data freshness and query performance? Which of the three dialects supports materialized views? Which does not natively?
3. Describe one scenario each where BEFORE, AFTER, and INSTEAD OF triggers are the appropriate choice.

Applied

4. Using a recursive CTE, write a query that lists **all employees who report (directly or indirectly) to the employee named 'Franklin Wong'** (Ssn 333445555), showing each employee's depth in the hierarchy and the full chain of supervisors.
5. For each department, use window functions to assign a **rank** and a **dense rank** to employees by salary (highest first). Also compute each employee's salary as a percentage of their department's maximum salary. Show all output columns.
6. Create a view ProjectSummary(Pname, DeptName, WorkerCount, TotalHours) that joins Project, Department, and Works_On. Then write a query against the view that shows only projects with more than 3 workers.

Analysis

7. A trainee writes the following trigger:

```
CREATE TRIGGER trg_prevent_delete
BEFORE DELETE ON Department
FOR EACH ROW
BEGIN
    DELETE FROM Employee WHERE Dno = OLD.Dnumber; -- remove employees first
END;
```

Identify two potential problems with this trigger design, and suggest a better architectural approach.

8. Compare LAG(Salary, 1) OVER (ORDER BY Bdate) with a self-join that achieves the same result. What are the correctness risks and performance implications of the self-join approach?

Dialect Awareness

9. Re-write the following PostgreSQL query for MySQL and Oracle:

```
SELECT Name, Attributes ->> 'brand' AS Brand
FROM   Products
WHERE  (Attributes ->> 'ram_gb')::INT >= 16;
```

10. A stored function written in MySQL uses DELIMITER \$\$ and BEGIN...END\$\$\$. Explain why DELIMITER is needed in MySQL but not in PostgreSQL or Oracle.

Design

11. An e-commerce application stores product attributes (color, size, weight, material) that differ by product category. Should you use a JSON column, a separate AttributeValue table, or database inheritance (via EER/table-per-type)? Justify your recommendation based on query patterns, indexing, and schema evolution.

Synthesis

12. Design a complete audit solution for the Salary column of the Employee table that:

- Logs every change with old value, new value, who made the change, and when
- Prevents any salary reduction greater than 20%
- Works in both MySQL and PostgreSQL

Write the DDL for the audit table, the trigger function (or procedure), and the trigger binding for **both** dialects.

LEFT OUTER JOIN

Returns **all rows from the left table** plus matched rows from the right. Unmatched right rows produce NULLs.

```
-- Find all employees and their project (if any)
SELECT e.fname, e.lname, p.project_name
FROM   employee e
LEFT JOIN works_on wo  ON e.ssn = wo.essn
LEFT JOIN project  p   ON wo.pno = p.pno;
-- Employees with no project still appear (with NULLs for project columns)
```

RIGHT OUTER JOIN

```
-- All departments, even those with no employees
SELECT d.dept_name, e.fname, e.lname
FROM employee e
RIGHT JOIN department d ON e.dept_no = d.dept_no;
```

FULL OUTER JOIN

```
-- All employees and all departments, matched where possible
SELECT e.fname, d.dept_name
FROM employee e
FULL OUTER JOIN department d ON e.dept_no = d.dept_no;
```

Subqueries (Nested Queries)

A **subquery** is a SELECT embedded within another SQL statement.

Scalar Subquery

Returns exactly one value:

```
SELECT fname, lname, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee);
```

Subquery with IN / NOT IN

```
-- Employees who work on at least one project in Houston
SELECT fname, lname
FROM employee
WHERE ssn IN (
    SELECT essn
    FROM works_on
    WHERE pno IN (
        SELECT pno FROM project WHERE plocation = 'Houston'
    )
);
```

Subquery with EXISTS / NOT EXISTS

EXISTS returns TRUE if the subquery produces *any* rows.

```
-- Employees who manage at least one other employee
SELECT fname, lname
FROM employee e
WHERE EXISTS (
    SELECT 1 FROM employee sub
    WHERE sub.super_ssn = e.ssn
);

-- Employees who work on ALL projects (using double negation)
SELECT fname, lname
FROM employee e
WHERE NOT EXISTS (
    SELECT * FROM project p
    WHERE NOT EXISTS (
        SELECT * FROM works_on wo
        WHERE wo.essn = e.ssn AND wo.pno = p.pno
    )
);
```

Tip

Rule of thumb: Use IN for small, uncorrelated subqueries; use EXISTS for large, correlated subqueries — EXISTS short-circuits on the first match.

Correlated Subquery

A subquery that references columns from the outer query (re-evaluated per row):

```
-- Find employees earning more than their department's average
SELECT fname, lname, salary, dept_no
FROM employee e
WHERE salary > (
    SELECT AVG(salary)
    FROM employee
    WHERE dept_no = e.dept_no    -- ← correlated: references outer e.dept_no
);
```


ANY / ALL

```
-- Salary > at least one employee in dept 5
SELECT fname, salary FROM employee
WHERE salary > ANY (SELECT salary FROM employee WHERE dept_no = 5);

-- Salary > all employees in dept 5
SELECT fname, salary FROM employee
WHERE salary > ALL (SELECT salary FROM employee WHERE dept_no = 5);
```

Common Table Expressions (CTE)

CTEs make complex queries readable by naming intermediate results:

```
-- WITH clause (CTE)
WITH high_earners AS (
    SELECT ssn, fname, lname, salary, dept_no
    FROM employee
    WHERE salary > 60000
),
dept_stats AS (
    SELECT dept_no, COUNT(*) AS cnt, AVG(salary) AS avg_sal
    FROM employee
    GROUP BY dept_no
)
SELECT he.fname, he.lname, he.salary, ds.avg_sal
FROM high_earners he
JOIN dept_stats ds ON he.dept_no = ds.dept_no;
```

Recursive CTE

Find all direct and indirect supervisors of an employee:

```
WITH RECURSIVE org_chart AS (
    -- Base: the employee himself
    SELECT ssn, fname, lname, super_ssn, 0 AS level
    FROM employee
    WHERE ssn = '123456789'

    UNION ALL
```

```
-- Recursive: each supervisor
SELECT e.ssn, e.fname, e.lname, e.super_ssn, oc.level + 1
FROM employee e
JOIN org_chart oc ON e.ssn = oc.super_ssn
)
SELECT * FROM org_chart;
```

Views

A **view** is a **named, saved SELECT statement** — a virtual table.

Creating a View

```
CREATE VIEW research_employees AS
SELECT e.ssn, e.fname, e.lname, e.salary
FROM employee e
JOIN department d ON e.dept_no = d.dept_no
WHERE d.dept_name = 'Research';
```

Querying a View

Views are queried exactly like tables:

```
SELECT * FROM research_employees WHERE salary > 55000;
```

Modifying a View

```
CREATE OR REPLACE VIEW research_employees AS
SELECT e.ssn, e.fname, e.lname, e.salary, e.bdate
FROM employee e
JOIN department d ON e.dept_no = d.dept_no
WHERE d.dept_name = 'Research';
```

Dropping a View

```
DROP VIEW research_employees;
DROP VIEW IF EXISTS research_employees CASCADE;
```

Updateable Views

A view is **updateable** (allows INSERT/UPDATE/DELETE) if it:

- References exactly one base table
- Contains no DISTINCT, GROUP BY, HAVING, aggregates, or set operations
- Contains all NOT NULL columns without defaults

```
UPDATE research_employees SET salary = 70000 WHERE ssn = '123456789';
-- Allowed if the view is simple and updateable
```

Triggers

A **trigger** is a stored procedure that automatically executes in response to a table event (INSERT, UPDATE, or DELETE).

Trigger Syntax (PostgreSQL)

```
-- Step 1: Create the function that the trigger will call
CREATE OR REPLACE FUNCTION log_salary_change()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO salary_audit (ssn, old_salary, new_salary, changed_at)
        VALUES (OLD.ssn, OLD.salary, NEW.salary, NOW());
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Step 2: Bind the trigger to the table and event
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON employee
FOR EACH ROW
EXECUTE FUNCTION log_salary_change();
```

Trigger Timing and Events

Timing	Event	Description
BEFORE	INSERT / UPDATE / DELETE	Fire before the row operation; can modify NEW
AFTER	INSERT / UPDATE / DELETE	Fire after the row operation; can read OLD and NEW
INSTEAD OF	INSERT / UPDATE / DELETE	On views; replace the default operation

OLD and NEW pseudo-records

Context	OLD	NEW
INSERT	NULL	Inserted row
UPDATE	Row before update	Row after update
DELETE	Deleted row	NULL

Stored Procedures and Functions

Stored Procedure (PostgreSQL)

```
-- Procedure: give a raise to all employees in a department
CREATE OR REPLACE PROCEDURE give_raise(
    p_dept_no  INTEGER,
    p_percent  DECIMAL(5,2)
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE employee
    SET salary = salary * (1 + p_percent / 100)
    WHERE dept_no = p_dept_no;

    RAISE NOTICE 'Raise applied to dept %', p_dept_no;
END;
$$;

-- Call a procedure
CALL give_raise(5, 10.0);
```

Function (returns a value)

```
CREATE OR REPLACE FUNCTION dept_headcount(p_dept_no INTEGER)
RETURNS INTEGER
LANGUAGE sql AS $$
    SELECT COUNT(*)::INTEGER FROM employee WHERE dept_no = p_dept_no;
$$;

-- Use in a query
SELECT dept_name, dept_headcount(dept_no) AS employees
FROM department;
```

Stored Procedures vs. Triggers vs. Functions

Feature	Stored Procedure	Function	Trigger
Called by	CALL statement	Inside SQL expression	DBMS automatically
Returns value	No (output params)	Yes	Via RETURN NEW/OLD
Can commit/rollback	Yes	No (usually)	Subject to transaction
Use case	Business operations	Computed values	Automatic audit/validation

Window Functions

Window functions compute values across *related* rows without collapsing them into groups:

```
-- Rank employees by salary within each department
SELECT fname, lname, dept_no, salary,
       RANK() OVER (PARTITION BY dept_no ORDER BY salary DESC) AS dept_rank,
       ROW_NUMBER() OVER (PARTITION BY dept_no ORDER BY salary DESC) AS row_num,
       AVG(salary) OVER (PARTITION BY dept_no) AS dept_avg
FROM employee;
```

Common Window Functions

Function	Description
ROW_NUMBER()	Sequential row number within partition

Function	Description
RANK()	Rank; gaps for ties
DENSE_RANK()	Rank; no gaps for ties
NTILE(n)	Divide into n buckets
LAG(col, n)	Value from n rows before
LEAD(col, n)	Value from n rows ahead
SUM/AVG/MIN/MAX OVER (. . .)	Running aggregate

Summary

! Chapter 8 — Key Takeaways

- **Outer joins** (LEFT/RIGHT/FULL) preserve unmatched rows padded with NULLs.
- **Subqueries** can return a scalar, a column (IN), or a boolean (EXISTS); correlated subqueries re-evaluate per row.
- **CTEs** (WITH) improve readability; recursive CTEs handle hierarchical data.
- **Views** are virtual tables; they can be updateable under strict conditions.
- **Triggers** fire automatically on INSERT/UPDATE/DELETE — use for auditing and business rule enforcement.
- **Stored procedures** encapsulate multi-statement operations; functions return values usable in queries.
- **Window functions** (OVER) perform analytics on row groups without collapsing them.

Review Questions

1. Write a query that finds all employees whose salary is **above the average salary of their own department** (use a correlated subquery).
2. What is the difference between EXISTS and IN? When is EXISTS preferred?
3. Create a view vw_manager_summary showing each manager's name and the number of employees they supervise.
4. Write a trigger that prevents salary from being reduced (an UPDATE where NEW.salary < OLD.salary is rejected).
5. Rank all employees by salary within their department using a window function. Show rank 1 as the highest earner.
6. Explain the difference between RANK() and DENSE_RANK() with an example.

Part V: Database Design Theory

Normalization & Functional Dependencies

Chapter 9

The Problem: Everything in One Table

Before diving into theory, start with a concrete problem.

Imagine a small e-commerce database where someone created a single table to “keep things simple”:

```
OrderLine(  
    OrderId,    CustomerId, CustomerName, CustomerCity,  
    ProductId,  ProductName, Category,    Price,  
    Qty,        Discount,    TotalPrice  
)
```

Sample data:

OrderId	CustomerId	CustomerName	CustomerCity	ProductId	ProductName	Category	Price	Qty	Discount	TotalPrice
1001	C01	Alice	Cairo	P10	Laptop	Electronics	1200	2	0.05	2280
1001	C01	Alice	Cairo	P20	Mouse	Peripherals	25	3	0.00	75
1002	C02	Bob	Alex	P10	Laptop	Electronics	1200	1	0.10	1080
1003	C01	Alice	Cairo	P30	Keyboard	Peripherals	80	1	0.00	80

This table has every fact about an order, customer, and product crammed into one row. Let us identify the **update anomalies** it creates and then systematically fix them.

Update Anomalies

Anomaly	What it means	Example in OrderLine
Insertion	Cannot record a fact without other unrelated facts	Cannot insert a new customer C03 (Hany, Giza) until they place an order
Deletion	Deleting a fact accidentally destroys other facts	Deleting Order 1002 (last order with P10) loses the fact that P10 costs 1200
Modification	A single real-world fact must be updated in many rows	Renaming category “Electronics” to “Computers” requires updating every row containing that category

Root cause: a single row stores facts about *three separate entities* — orders, customers, and products — all mixed together.

Functional Dependencies (Recap)

This chapter depends heavily on the **FD theory** introduced in Chapter 5 §FD-Based Key Identification. If any of the following terms feel unfamiliar, review that section before continuing:

- **Functional dependency** $X \rightarrow Y$ — whenever two rows agree on X , they must agree on Y .
- **Types of FDs** — *full, partial, transitive, trivial*. Partial and transitive FDs are what normalization eliminates.
- **Armstrong’s axioms** — *reflexivity, augmentation, transitivity* — sound and complete inference rules for deriving implied FDs.
- **Attribute closure** X^+ — the set of attributes X determines under an FD set F ; computed by the closure algorithm.
- **Superkey / candidate key / prime attribute** — defined via closures.

FDs in the OrderLine Table

Applying this machinery to our running example:

```

ProductId → ProductName, Category, Price      -- product facts depend only on product
CustomerId → CustomerName, CustomerCity       -- customer facts depend only on customer
OrderId → CustomerId, Discount                -- order header depends on order
{OrderId, ProductId} → Qty, Discount, TotalPrice -- line item facts need both

```

Closure check:

- $\{OrderId, ProductId\}^+ = \text{all columns} \rightarrow$ **candidate key** (chosen as PK).
- $\{OrderId\}^+ = \{OrderId, CustomerId, CustomerName, CustomerCity, Discount\} \rightarrow$ *not* a superkey (missing ProductId, ProductName, Category, Price, Qty, TotalPrice).
- ProductName is determined by ProductId alone (a proper subset of the key) $\rightarrow$ **partial dependency**, the red flag for 2NF.
- CustomerName is determined by CustomerId, which is determined by OrderId $\rightarrow$ **transitive dependency**, the red flag for 3NF.

The rest of this chapter uses these two FD “smells” — partial and transitive — as the decomposition triggers.

Step-by-Step Normalization of OrderLine

Step 1 — Is OrderLine in 1NF?

1NF: Every attribute value is **atomic** (single-valued, indivisible).

OrderLine is in 1NF — each cell holds one value.

Note

If Category had been stored as a comma-separated list ("Electronics, Computers"), the table would violate 1NF. The fix: extract a separate ProductCategory(ProductId, Category) table or hold one category per row.

Step 2 — Check for 2NF (Eliminate Partial Dependencies)

2NF: In 1NF, AND every non-prime attribute is **fully** functionally dependent on every candidate key. (Relevant only when the key is composite.)

Violations found:

FD	Problem
ProductId → ProductName, Category, Price	Partial — ProductId alone determines these; OrderId is irrelevant
CustomerId → CustomerName, CustomerCity	Partial — CustomerId alone determines these; ProductId is irrelevant
OrderId → CustomerId, Discount	Partial — OrderId alone determines discount and customer; ProductId is irrelevant

Anomalies caused by ProductId → ProductName, Category, Price:

- *Insert:* cannot record a product's price without first having an order for it
- *Delete:* deleting the last order for a product destroys its price
- *Update:* changing Price for P10 requires updating every order line that contains P10

Fix — decompose into:

```

Product(ProductId PK, ProductName, Category, Price)
Customer(CustomerId PK, CustomerName, CustomerCity)
Order(OrderId PK, CustomerId FK, Discount)
OrderLine(OrderId FK, ProductId FK, Qty, TotalPrice)  -- PK = {OrderId, ProductId}

```

All four relations are now in **2NF** — no partial dependencies remain.

Step 3 — Check for 3NF (Eliminate Transitive Dependencies)

3NF: In 2NF, AND for every non-trivial FD $X \rightarrow A$: - X is a superkey, **OR** - A is a prime attribute

Violation found in Order:

OrderId → CustomerId → CustomerName, CustomerCity

$\text{CustomerId} \rightarrow \text{CustomerName}, \text{CustomerCity}$ — CustomerId is NOT a superkey of Order .

This **transitive dependency** means:

- *Update*: changing Alice’s city in two separate orders requires two UPDATE statements

Fix: CustomerName and CustomerCity already moved to Customer in Step 2. $\text{Order}(\text{OrderId}, \text{CustomerId}, \text{Discount})$ — now in 3NF because CustomerId is an FK whose non-key attributes live in Customer .

Let us check $\text{Product}(\text{ProductId}, \text{ProductName}, \text{Category}, \text{Price})$:

$\text{ProductId} \rightarrow \text{Category} \rightarrow \text{Price}$? -- is there a transitive FD here?

Answer: Not inherently — a single Category does not determine Price (different laptop brands have different prices). No transitive FD. Product is in 3NF.

Step 4 — Check for BCNF

BCNF: For *every* non-trivial FD $X \rightarrow Y$: **X must be a superkey**.

BCNF is stricter than 3NF — it removes the “prime attribute” exception.

Current schema after 2NF/3NF normalization:

Table	FDs	BCNF?
$\text{Product}(\text{ProductId}, \text{ProductName}, \text{Category}, \text{Price})$	$\text{ProductId} \rightarrow \text{all}$	
$\text{Customer}(\text{CustomerId}, \text{CustomerName}, \text{CustomerCity})$	$\text{CustomerId} \rightarrow \text{all}$	
$\text{Order}(\text{OrderId}, \text{CustomerId}, \text{Discount})$	$\text{OrderId} \rightarrow \text{all}$	
$\text{OrderLine}(\text{OrderId}, \text{ProductId}, \text{Qty}, \text{TotalPrice})$	$\{\text{OrderId}, \text{ProductId}\} \rightarrow \text{all}$	

All four tables are in **BCNF**.

Step 5 — Verify Lossless Decomposition

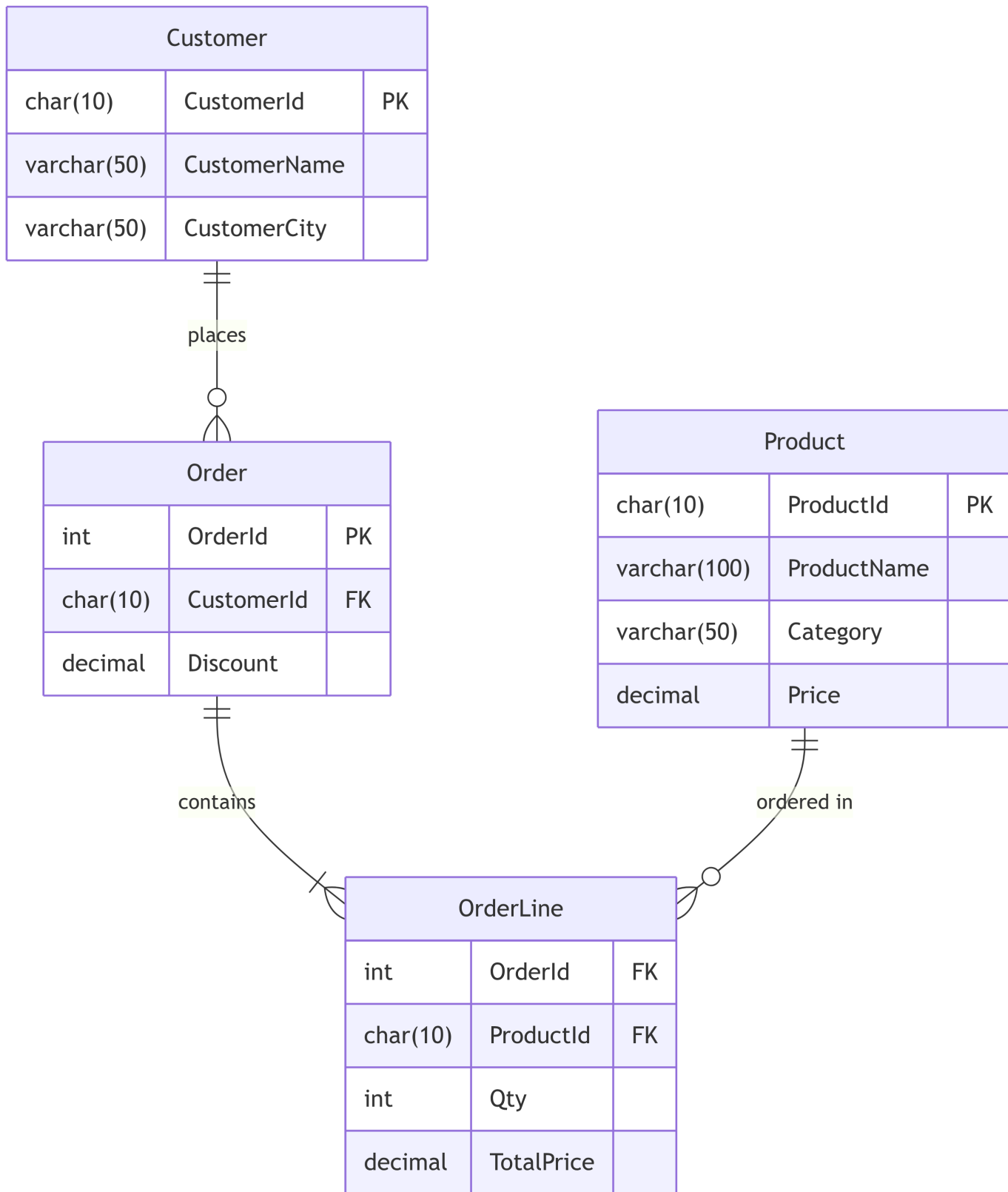
A decomposition of R into (R_1, R_2) is **lossless** iff:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2$$

Check OrderLine $\cap$ Order: shared attribute is OrderId. $\{OrderId\}^+$ in Order = all attributes of Order $\rightarrow$ lossless.

Check OrderLine $\cap$ Product: shared attribute is ProductId.
 $\{ProductId\}^+$ in Product = all attributes of Product $\rightarrow$ lossless.

Final Normalized Schema



Normal Form Summary

Normal Form	Condition	What it eliminates
1NF	All attributes atomic	Multi-valued / repeating groups
2NF	1NF + no partial FDs	Redundancy from composite-key partial deps
3NF	2NF + no transitive FDs (with prime-attr exception)	Transitive deps via non-key attributes
BCNF	Every FD determinant is a superkey	All non-trivial FD redundancy
4NF	BCNF + no non-trivial MVDs	Multi-valued dependency redundancy
5NF	4NF + no non-trivial join dependencies	Decomposition artifacts

! 3NF vs BCNF Trade-off

Property	3NF	BCNF
Lossless decomposition		
Dependency preservation	(always)	(may lose some FDs)
Eliminates all FD-based redundancy	Partial	Yes

Use BCNF when possible; fall back to 3NF when you must preserve a specific dependency.

BCNF Decomposition — A Case Where 3NF ≠ BCNF

The BCNF and 3NF distinction matters when there are **multiple overlapping candidate keys**.

Classic example:

Enrolment(Student, Course, Teacher)

FDs: - {Student, Course} → Teacher (a course has exactly one teacher per student) - Teacher → Course (a teacher teaches exactly one course)

Candidate keys: - {Student, Course} — covers all columns - {Student, Teacher} — covers all columns (because Teacher → Course)

Check for BCNF violations: - Teacher $\rightarrow$ Course: determinant Teacher is NOT a superkey $\rightarrow$ **BCNF violated**

Check for 3NF violations: - Teacher $\rightarrow$ Course: Course is a prime attribute (member of candidate key {Student, Course}) $\rightarrow$ **3NF satisfied** (prime-attr exception)

So Enrolment is in 3NF but **not** in BCNF.

BCNF decomposition:

TeacherCourse(Teacher PK, Course) -- FD: Teacher $\rightarrow$ Course
 StudentTeacher(Student, Teacher) -- PK: {Student, Teacher}

Trade-off: the FD {Student, Course} $\rightarrow$ Teacher can no longer be checked in a single table — it requires a join. That is the dependency-preservation loss inherent to BCNF decomposition.

Multivalued Dependencies and 4NF

A **multivalued dependency (MVD)** $X \twoheadrightarrow Y$ holds when, for each value of X , the set of Y values is independent of the remaining attributes Z .

Example:

EmployeeSkillHobby(Ssn, Skill, Hobby)

An employee has a set of skills and a set of hobbies, and these are independent of each other. MVDs: Ssn $\twoheadrightarrow$ Skill and Ssn $\twoheadrightarrow$ Hobby.

This causes redundancy: each (Ssn, Skill) pair must be repeated for each Hobby.

Ssn	Skill	Hobby
111	Java	Chess
111	Java	Soccer
111	SQL	Chess
111	SQL	Soccer

4NF requires that every non-trivial MVD $X \twoheadrightarrow Y$ has X as a superkey.

Fix:

EmployeeSkill(Ssn, Skill)
 EmployeeHobby(Ssn, Hobby)

Canonical Cover (Minimal Cover)

The **canonical cover** F_c of an FD set F is an equivalent, minimal FD set:

1. Every FD has a **single attribute on the right side** (decompose all compound RHS)
2. No FD has a **redundant attribute on the left side** (check if dropping a LHS attribute leaves the FD derivable)
3. No FD is **redundant overall** (check if removing the whole FD leaves it derivable from the rest)

Why compute F_c ? The 3NF synthesis algorithm uses F_c to generate the minimal, dependency-preserving, lossless schema.

Dependency Preservation

A decomposition $\{R_1, R_2, \dots, R_k\}$ of R is **dependency-preserving** iff every FD in F^+ can be verified in at least one R_i without performing a join:

$$\left(\bigcup_{i=1}^k F_i \right)^+ = F^+$$

where F_i = projection of F onto R_i .

BCNF decompositions may not be dependency-preserving (as shown in §9.7). The **3NF synthesis algorithm** always produces a lossless, dependency-preserving decomposition.

Chapter Summary

! Key Takeaways — Chapter 9

1. **Update anomalies** (insertion, deletion, modification) arise whenever a table stores facts about multiple unrelated entities. Normalization eliminates the root cause.
2. A **functional dependency** $X \rightarrow Y$ means knowing X always determines Y . Key types: full (depends on whole key), partial (depends on subset), transitive ($X \rightarrow Y \rightarrow Z$).
3. **Armstrong's Axioms** (Reflexivity, Augmentation, Transitivity) are complete — all valid FDs can be derived. Use **closure** X^+ to find keys and test derivability.
4. **Normal forms** are a ladder: 1NF (atomic) $\rightarrow$ 2NF (no partial deps) $\rightarrow$ 3NF (no transitive

deps) $\rightarrow$ BCNF (every determinant is a superkey) $\rightarrow$ 4NF (no non-trivial MVDs).

5. The **running OrderLine example** showed: partial deps caused product/customer data duplication in each order row $\rightarrow$ fixed by decomposing into four tables (2NF), then confirmed 3NF/BCNF.
6. **BCNF vs 3NF trade-off**: BCNF eliminates all FD-based redundancy but may not preserve dependencies; 3NF always allows a lossless, dependency-preserving decomposition (use 3NF synthesis algorithm).
7. A decomposition is **lossless** iff shared attributes form a key in at least one sub-relation.

Review Questions

Conceptual

1. Define **functional dependency** and explain the difference between a full and a partial FD. At what normal form does eliminating partial FDs become relevant?
2. Why does Armstrong's **Augmentation** axiom hold? Give a counterexample showing that dropping the augmented attributes from only one side would be wrong.
3. What is the fundamental difference between **BCNF** and **3NF**? Give an example of a relation that is in 3NF but not in BCNF.

Applied

4. Given $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow B\}$:
 - Compute $\{A, B\}^+$ and $\{A, C\}^+$.
 - Identify all candidate keys of $R(A, B, C, D)$.
 - Identify which attributes are prime and which are non-prime.

Run the closure algorithm from Chapter 5 §FD-Based Key Identification on each starting set. Remember: a candidate key K must satisfy $K^+ = \{A, B, C, D\}$ **and** no proper subset of K can satisfy that condition.

5. Normalize the following relation to BCNF, showing each decomposition step and the FDs that triggered it:

Invoice(InvoiceId, VendorId, VendorName, VendorCity, ItemId, ItemName, Qty, UnitPrice)

Assumed FDs:

InvoiceId $\rightarrow$ VendorId

VendorId $\rightarrow$ VendorName, VendorCity

ItemId $\rightarrow$ ItemName, UnitPrice

{InvoiceId, ItemId} $\rightarrow$ Qty

- ItemId $\rightarrow$ ItemName, UnitPrice is a **partial** dependency on the composite key {InvoiceId, ItemId} $\rightarrow$ triggers the 1NF $\rightarrow$ 2NF split.
 - InvoiceId $\rightarrow$ VendorId $\rightarrow$ VendorName, VendorCity is a **transitive** chain $\rightarrow$ triggers the 2NF $\rightarrow$ 3NF split.
 - In this example 3NF is already BCNF (every determinant is a candidate key of its own sub-relation). Check the LHS of every remaining FD explicitly.
6. For each step in Question 5, describe the specific **update anomaly** that would occur if you left the table at that lower normal form.

Analysis

7. A classmate claims: “Every table with a single-column primary key is automatically in 3NF.” Is this claim true? If not, produce a counterexample.
8. Prove or disprove: if a decomposition is lossless and dependency-preserving, then the union of all sub-relations’ FDs is equivalent to the original F^+ .

Diagram / Visualization

9. Draw the FD diagram (show attributes as nodes and dependencies as labelled arrows) for OrderLine before normalization. Circle the primary key attributes. Highlight the partial and transitive FDs.
10. Given the normalized schema from §9.5, write the SQL CREATE TABLE statements for all four tables (Product, Customer, Order, OrderLine) with all appropriate constraints (PK, FK, NOT NULL, CHECK). Include the referential actions ON DELETE and ON UPDATE.

Design

11. Consider a university database where a **student** can enrol in multiple **courses**, each course is taught by multiple **instructors**, but each student in a given course section has exactly one instructor. Model this with:
- An ER diagram (or erDiagram in Mermaid)
 - The relational schema
 - Identify all FDs and determine the highest normal form the schema achieves

Synthesis

12. The company DBA proposes storing the full employee hierarchy (employee $\rightarrow$ supervisor $\rightarrow$ their supervisor $\rightarrow$...) as a JSON column in PostgreSQL instead of the normalized Employee.Super_ssn foreign key pattern:

```
ALTER TABLE Employee ADD COLUMN HierarchyChain JSONB;
```

```
-- e.g. [{"Ssn":"333445555","Name":"Franklin Wong"}, {"Ssn":"888665555","Name":"James I
```

Evaluate this design from a normalization perspective. What normal form guarantees does it violate?

What are the practical trade-offs? Under what query patterns might the denormalized JSON approach be justified?

Part VI: Storage, Indexing & Query Processing

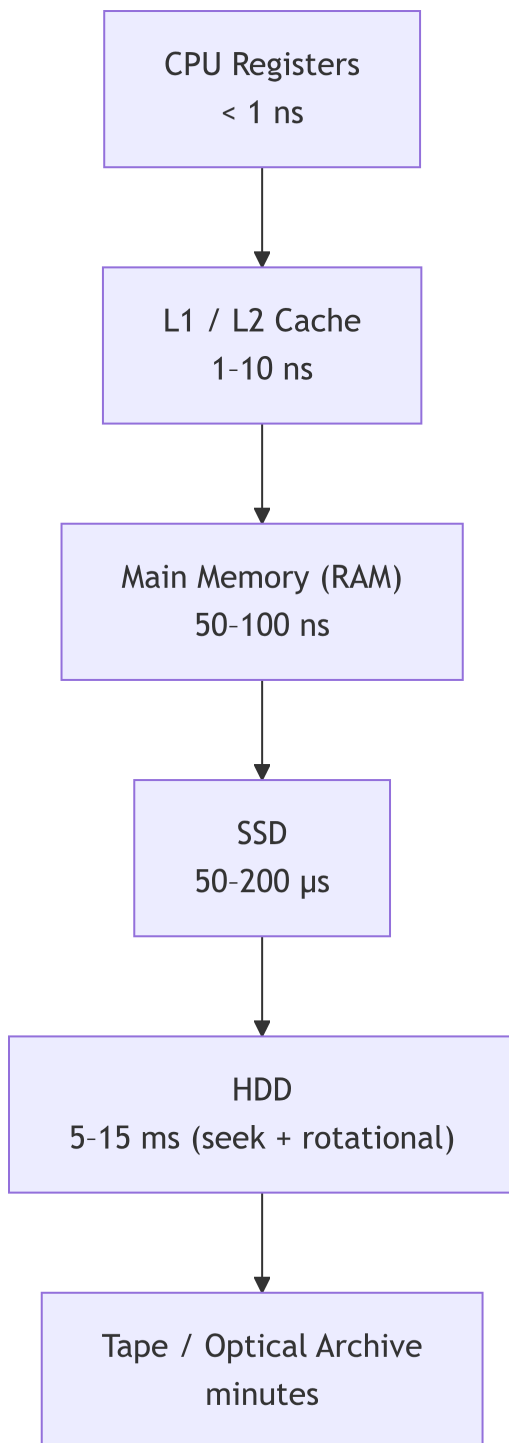
Storage & Indexing

Chapter 10

Why Storage Matters

Chapter 11 (Query Processing) will show you *how* a query executes. But all execution happens ultimately by reading **pages** from disk into memory. The number of page reads is the primary cost metric for query optimization — and indexes exist to reduce that count dramatically.

Storage Hierarchy



The DBMS **buffer manager** sits between RAM and disk. It maintains a pool of pages in RAM and applies a replacement policy (LRU, Clock) when the pool is full.

! The Page is the Unit of I/O

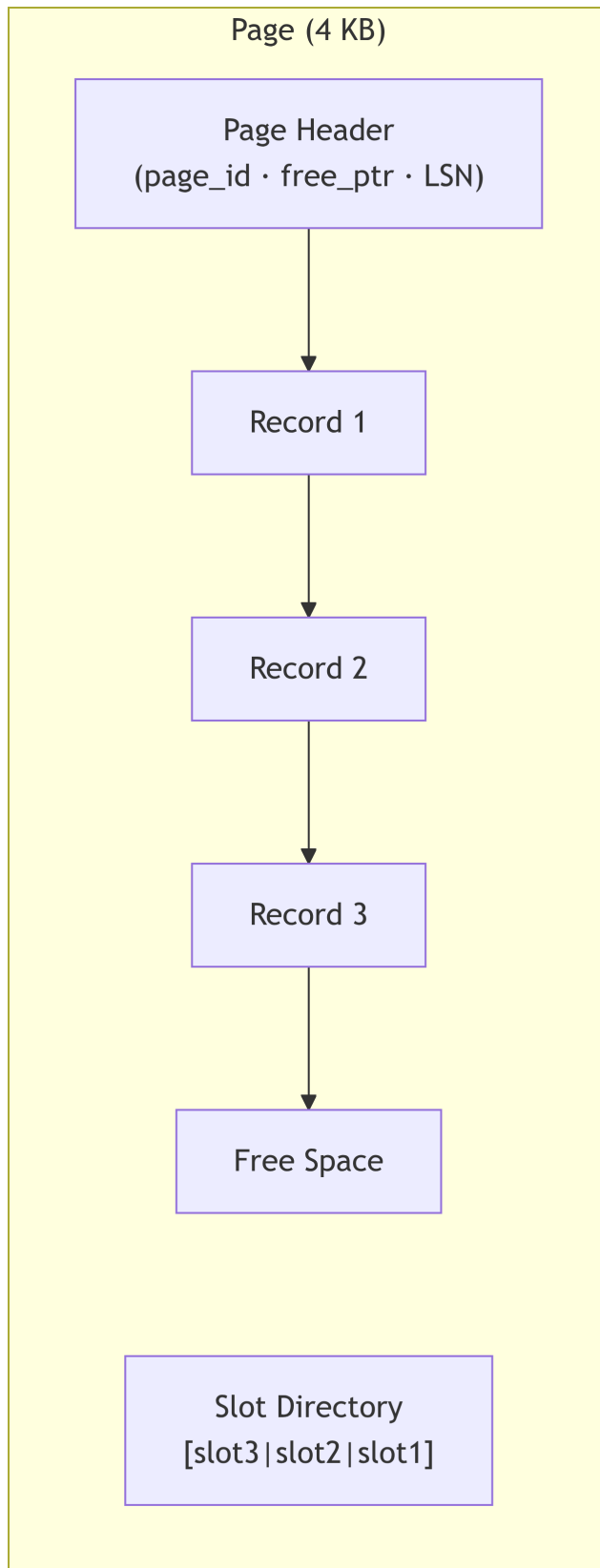
All disk reads and writes transfer exactly **one page** (typically 4–16 KB, configurable). Reading a single 10-byte row forces a full page to come into RAM. Query cost is measured in **number of page I/Os**, not rows accessed.

Records and Page Layout

Record Formats

Type	Description	Used for
Fixed-length	Each field has a fixed byte size	INT, CHAR(n), DATE
Variable-length	Fields have variable sizes; stored with offsets	VARCHAR, TEXT, JSON

Slotted Page



The **slot directory** grows from the end toward the middle. Each slot holds the (offset, length) of its record. This allows records to be moved within the page during compaction without changing their

external address (page_id, slot_id).

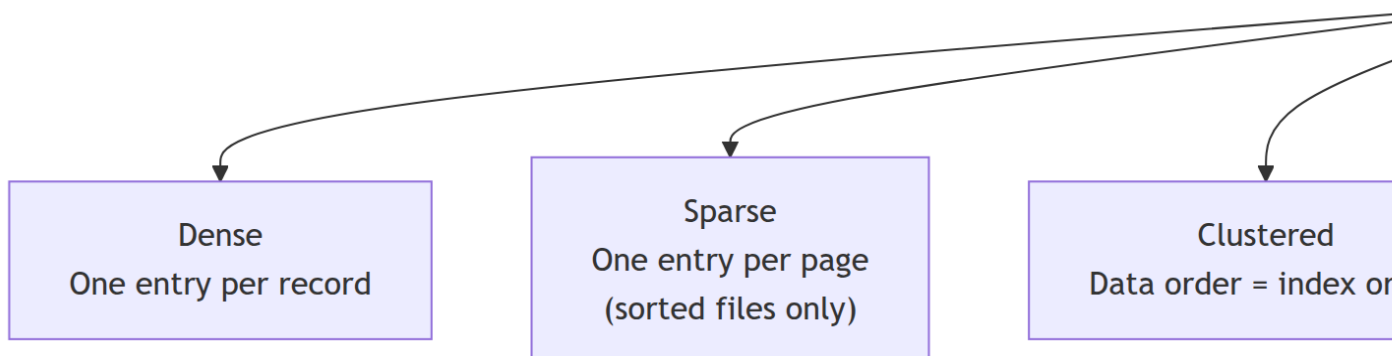
File Organizations

Organization	Layout	Best For
Heap	Records in insertion order	Bulk load; full table scans
Sequential	Records sorted by a key	Binary search; range queries on sort key
Hash	Records in buckets by hash(key)	Exact-match equality
Clustered	Records sorted to match a B+ tree index	Range queries combined with index access

Index Concepts

An **index** is an auxiliary data structure that maps **search key** → **record location (RID)**. It is separate from the data file and maintained automatically on INSERT/UPDATE/DELETE.

Taxonomy



Clustered vs Unclustered: I/O Impact

	Clustered	Unclustered
Range query (k rows)	$\sim k / \text{records_per_page}$ pages	Up to k separate pages
Equality query	$\sim \text{height}$ pages	$\sim \text{height}$ pages
Build cost	High (file must be sorted)	Lower
One per table?	Only one clustered index	Many allowed

💡 Tip

A table can have **at most one** clustered index, because the data file can have only one physical sort order. In MySQL InnoDB, the **primary key is always the clustered index**. Every secondary index stores the primary key as the RID — requiring one extra lookup (“index dive”).

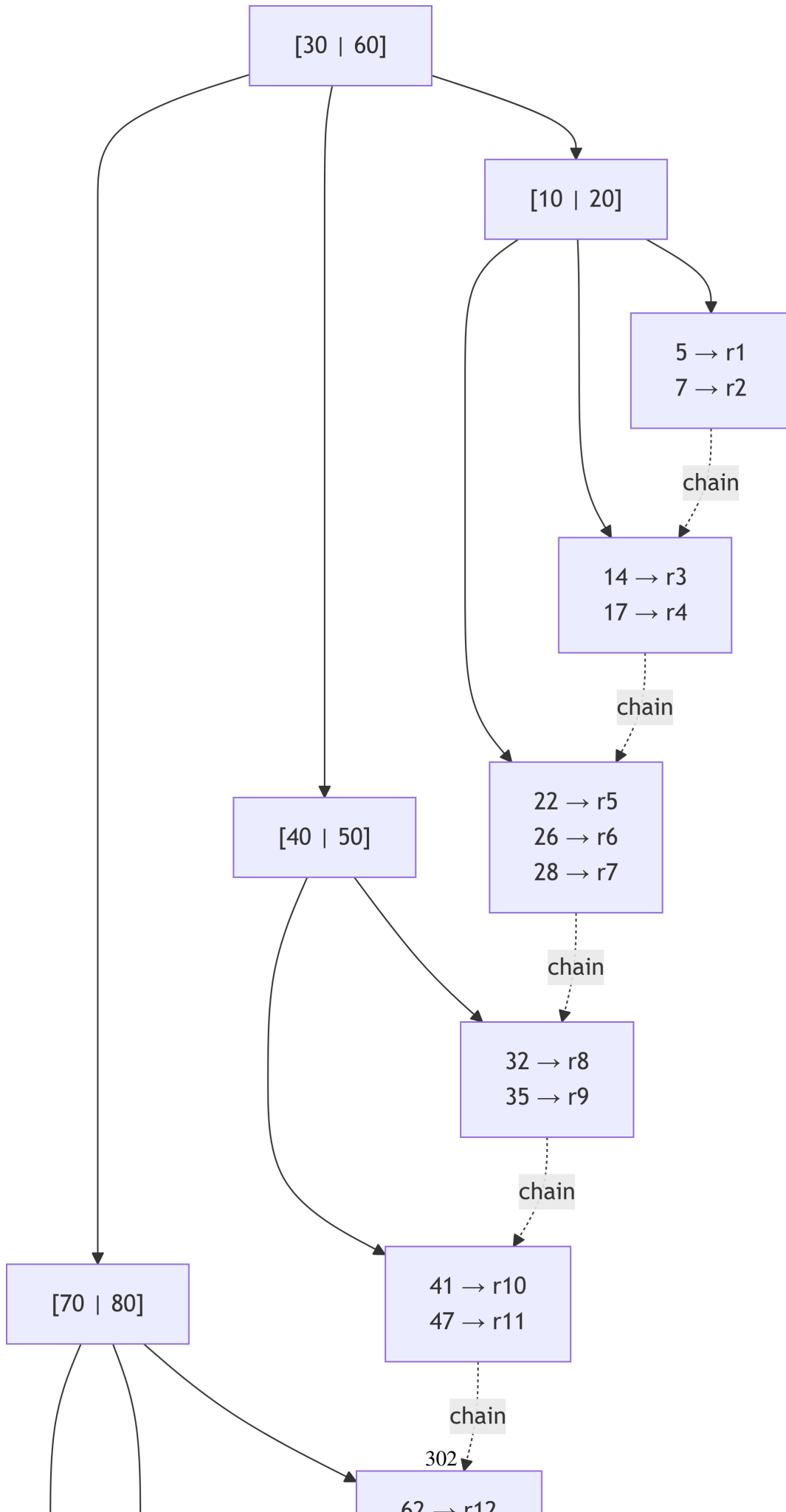
B+ Tree Index

The **B+ Tree** is the dominant index structure across all major RDBMS (MySQL, PostgreSQL, Oracle, SQL Server).

Structure

- **Balanced:** all leaf nodes at the same depth h ; guarantees $O(\log n)$ operations
- **Order d :** every internal node except the root holds between d and $2d$ search keys
- **All data pointers in leaves:** internal nodes hold only keys for routing
- **Leaf chain:** leaf nodes are linked in a doubly-linked list for sequential range scans

Storage & Indexing



Cost Analysis

Operation	I/O Cost	Formula
Equality search	h page reads	$h = \lceil \log_d n \rceil$
Range scan (k results)	$h + \lceil k/f \rceil$	$f = \text{records per leaf page}$
Insert	$\$h + \$ \text{splits}$	Amortised $O(1)$ splits
Delete	$\$h + \$ \text{merges}$	Rare in practice

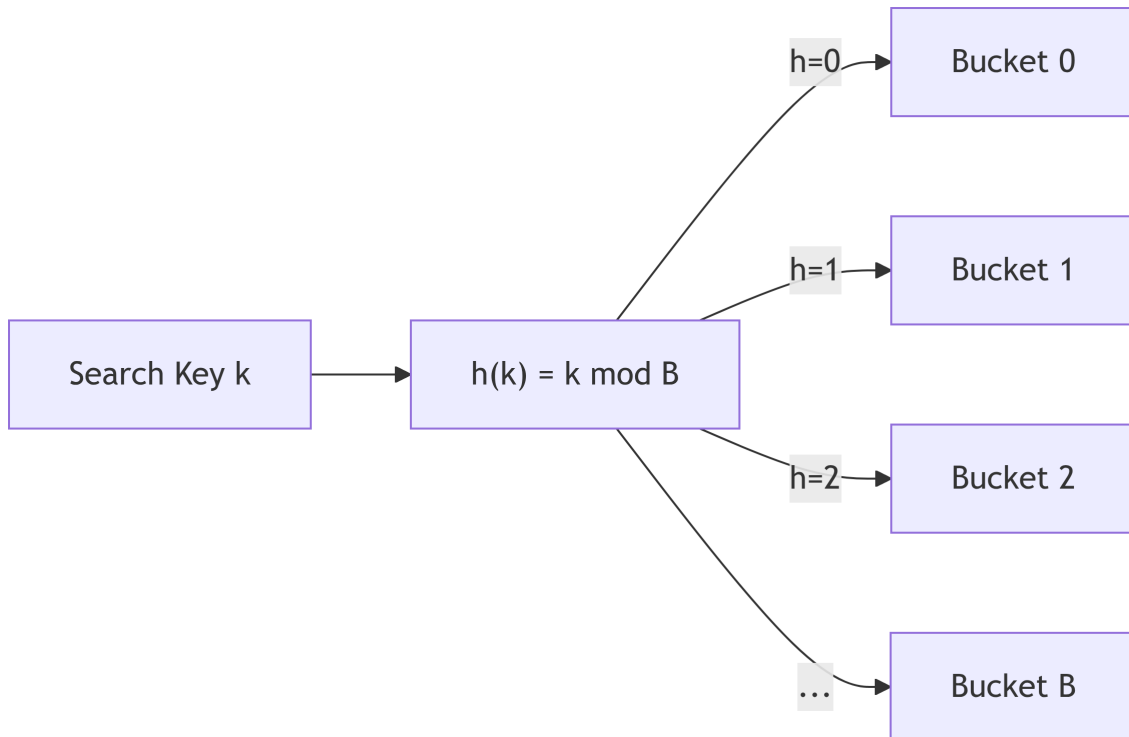
Example: order $d = 100$, $n = 10^8$ records $\rightarrow h = \lceil \log_{100} 10^8 \rceil = 4$ levels. Four page reads to find any record in a 100-million-row table.

Interactive: File Scan vs. Index Lookup

Interactive: B+ Tree Order Explorer

Hash Indexes

A **hash index** applies a hash function $h(k)$ to map a key to a **bucket** where matching records are stored.



Characteristic	Value
Equality search	O(1) theoretical
Range query	Impossible — hashing destroys order
Overflow	Buckets chain when full

Extendible Hashing

Uses a **global depth** and a **directory**. When a bucket overflows: 1. Split only the full bucket (not all buckets) 2. Double the directory only when all buckets at that depth are full

Hash indexes are available as `USING HASH` in PostgreSQL and as `MEMORY` storage engine indexes in MySQL.

Bitmap Indexes

A **bitmap index** stores, for each distinct value v of the indexed column, a **bit vector** of length N (one bit per row), where bit $i = 1$ iff row i has value v .

Example

Table Employee with 8 rows and column Gender:

Row	Ssn	Gender
1	123	M
2	333	M
3	999	F
4	987	M
5	666	F
6	453	F
7	888	F
8	963	M

Bitmap index on Gender:

Value	Bit vector
M	1 1 0 1 0 0 0 1
F	0 0 1 0 1 1 1 0

Bitwise Operations for Multi-Predicate Queries

Query: SELECT * FROM Employee WHERE Gender = 'F' AND Dno = 5

Bitmap	Vector
Gender = 'F'	0 0 1 0 1 1 1 0
Dno = 5	1 0 1 0 1 0 1 0
AND result	0 0 1 0 1 0 1 0

Rows 3, 5, and 7 satisfy both conditions — determined by **bitwise AND** in a single CPU operation across the entire table.

When to Use Bitmap Indexes

Tip

Bitmap indexes are ideal for:

- **Low-cardinality columns:** Gender, Status, Region (few distinct values → compact bitmaps)
- **Analytical / OLAP queries** with many AND/OR column predicates
- **Data warehouse star-schema** dimension columns (InnoDB does not support bitmap indexes natively — they are a first-class citizen of Oracle and PostgreSQL with specialized extensions)

Avoid bitmap indexes on: - **High-cardinality columns** (every value different → one bit vector per row = huge overhead) - **OLTP tables with frequent writes** — bit vectors must be updated on every INSERT/UPDATE/DELETE

Oracle Bitmap Index

```
-- Oracle natively supports bitmap indexes
```

```
CREATE BITMAP INDEX idx_emp_sex ON Employee(Gender);
```

```
CREATE BITMAP INDEX idx_emp_dno ON Employee(Dno);
```

```
-- Query: optimizer combines bitmaps with bitwise AND/OR automatically
```

```
SELECT * FROM Employee WHERE Gender = 'F' AND Dno = 5;
```

PostgreSQL does not have a native CREATE BITMAP INDEX, but the query planner performs **bitmap index scans** by combining multiple B+ tree indexes with BitmapAnd/BitmapOr operations internally.

Index Selection Guidelines

When to Create an Index

Create index when...	Avoid when...
Column frequent in WHERE predicates	Table has fewer than a few thousand rows
Column used in JOIN ... ON condition	Column has very low cardinality (boolean, status with 2 values) — consider bitmap

Create index when...	Avoid when...
Column in ORDER BY or GROUP BY	Table is write-heavy (INSERT/UPDATE/DELETE) — index maintenance overhead
Foreign key column (speeds up deletes of parent)	Column almost never queried alone
Covering index candidate (all SELECT cols in index)	Index is redundant with another existing index

Composite (Multi-Column) Index

MySQL

```
-- Index on (Dno, Salary) — useful for filtering by dept and sorting by salary
CREATE INDEX idx_emp_dno_sal ON Employee(Dno, Salary);

-- A covering index: includes all columns needed by the query (avoids table lookup)
CREATE INDEX idx_emp_cover ON Employee(Dno, Salary, Fname, Lname);

-- Verify index usage
EXPLAIN SELECT Fname, Lname, Salary FROM Employee WHERE Dno = 5 ORDER BY Salary DESC;
```

PostgreSQL

```
CREATE INDEX idx_emp_dno_sal ON Employee(Dno, Salary);

-- Partial index: only index rows matching a condition (smaller, faster for sparse predicate)
CREATE INDEX idx_emp_mgr ON Employee(Super_ssn) WHERE Super_ssn IS NOT NULL;

-- Functional index (index on expression)
CREATE INDEX idx_emp_lower_lname ON Employee(LOWER(Lname));

-- Verify
EXPLAIN ANALYZE SELECT Fname, Lname, Salary FROM Employee WHERE Dno = 5 ORDER BY Salary DESC;
```

Oracle

```
CREATE INDEX idx_emp_dno_sal ON Employee(Dno, Salary);
```

```
-- Function-based index
```

```
CREATE INDEX idx_emp_upper_lname ON Employee(UPPER(Lname));
```

```
-- Bitmap index for low-cardinality column
```

```
CREATE BITMAP INDEX idx_emp_sex ON Employee(Gender);
```

```
-- Verify (execution plan)
```

```
EXPLAIN PLAN FOR SELECT Fname, Lname, Salary FROM Employee WHERE Dno = 5 ORDER BY Salary DE
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

! Leftmost Prefix Rule for Composite Indexes

A composite index on (A, B, C) is usable for:

Query predicate	Index used?
WHERE A = ?	
WHERE A = ? AND B = ?	
WHERE A = ? AND B = ? AND C = ?	
WHERE B = ?	(leftmost column skipped)
WHERE C = ?	
WHERE A = ? AND C = ?	Partial for A only
ORDER BY A, B	(avoids sort step)

Chapter Summary

! Key Takeaways — Chapter 10

1. **All disk I/O transfers pages** (4–16 KB). The buffer manager caches pages; the cost metric is page reads.
2. **File organizations:** heap (insertion order), sequential (sorted), hash, clustered. MySQL InnoDB stores data in the clustered B+ tree (primary key order).
3. **Dense vs sparse:** dense index has one entry per record; sparse index has one entry per page (requires sorted file).
4. **B+ tree:** balanced, fan-out $2d$, height $h \approx \log_d n$. All leaves linked for range scans. Supports equality, range, ORDER BY, and GROUP BY efficiently. With $d = 100$ and 10^8 records: only **4 page reads** to find any record.
5. **Hash index:** $O(1)$ equality; completely useless for range queries.

6. **Bitmap index:** per-distinct-value bit vectors; bitwise AND/OR answering multi-predicate analytical queries at CPU speed. Best for low-cardinality columns in OLAP.
7. **Composite indexes:** leftmost prefix rule determines which predicates the index covers. A covering index includes all columns a query needs — eliminates all table lookups.
8. Use EXPLAIN / EXPLAIN ANALYZE to verify that the optimizer selects the intended index.

Review Questions

Conceptual

1. Why is the **page** the unit of I/O instead of the row? How does this influence the cost of a full table scan versus an index scan?
2. Explain the **clustered vs unclustered** distinction. Given a table of 1 000 000 employee rows where 100 000 are in department 5, compare the page reads for:
 - A clustered B+ tree index on Dno
 - An unclustered B+ tree index on Dno
3. Why can a table have **at most one clustered index**? What is the consequence in MySQL's InnoDB of having a large primary key (e.g., UUID)?

Applied

4. A B+ tree of order $d = 200$ stores 50 million records. Each leaf page holds 200 records.
 - What is the tree height h ?
 - What is the estimated I/O cost for a range query returning 10 000 rows?
5. Write the SQL (in MySQL and PostgreSQL) to:
 - Create a composite index on (Dno, Salary) covering the query `SELECT Fname, Lname, Salary FROM Employee WHERE Dno = 5 ORDER BY Salary DESC`
 - Create a partial index (PostgreSQL) on Super_ssn that excludes NULL supervisors
 - Use EXPLAIN to confirm the index is selected
6. Design bitmap indexes for the following query on a 10-million-row sales table:

```
SELECT SUM(Amount) FROM Sales
WHERE Region = 'North' AND Year = 2025 AND Quarter = 'Q1';
```

Show the bit vectors (conceptually, for 8 sample rows) and the bitwise AND operation.

Analysis

7. A DBA notices that queries on `Employee.Lname` are slow but an index exists on `(Dno, Lname)`. Explain why the index is not being used and what index change would fix it.
8. Compare the I/O cost of these two equivalent queries on a 2-million-row table with no indexes, then with the appropriate index:
 - Query A: `WHERE Salary = 55000`
 - Query B: `WHERE Salary BETWEEN 40000 AND 60000` Which benefits more from a B+ tree index vs a hash index? Justify.

Diagram

9. Draw a B+ tree of order $d = 2$ after inserting keys in order: 10, 20, 5, 6, 12, 30, 7, 17. Show the state of the tree after each split. Which node is the root at the end?

Design

10. An analytics team runs nightly reports on a 500M-row `Transactions` table with columns: `(TxnId, CustomerId, StoreId, ProductId, Category, Amount, TxnDate)`. The reports filter on `Category` (20 distinct values), `StoreId` (500 stores), and `TxnDate`. Design an indexing strategy (B+ tree, bitmap, composite, partial) for this table, justifying each choice with the expected query patterns and write characteristics.

Synthesis

11. MySQL InnoDB does not support native bitmap indexes. However, a DBA wants bitmap-like multi-predicate query performance on a column `Status CHAR(1)` with values 'A', 'I', 'S'. Propose two alternative approaches (using InnoDB's existing index types) that approximate bitmap behavior. What are the trade-offs compared to a native bitmap index?

Storage Hierarchy

Register (< 1 ns)

- └─ L1/L2/L3 Cache (1–10 ns)
 - └─ Main Memory / RAM (50–100 ns)
 - └─ SSD (50–200 μ s)
 - └─ HDD (5–15 ms)

└─ Tape / Archive

The DBMS **buffer manager** bridges RAM and disk, caching frequently used pages.

Units of Disk Transfer

Unit	Typical Size
Block / Page	4–16 KB (configurable)
Track	Contiguous blocks on one disk ring
Cylinder	Same track across all disk platters

All disk I/O is in **pages**. The cost metric in query optimization is the number of page reads/writes.

Records and Pages

Record Types

Type	Description
Fixed-length	All records same size; easy to locate by offset
Variable-length	Attributes of varying size (VARCHAR, TEXT); stored with length indicators or pointers

Page / Block Layout

Page Header	← page ID, free space pointer, LSN
Record 1 Record 2 ... Rn	
(free space)	
Slot n+1 ... Slot 2 Slot 1	← Slot directory (grows from end)

Slotted page design keeps a **slot array** at the end. This allows records to move within a page without changing their external address (page_id, slot_id).

File Organizations

Organization	Description	Best For
Heap (unordered)	Records in insertion order	Bulk load, full scans
Sequential (sorted)	Records sorted by a key	Range queries on sort key
Hash	Records in buckets by hash(key)	Exact-match equality
Clustered	Records sorted to match a B+ tree index	Range queries + index access

Index Concepts

An **index** is an auxiliary data structure that speeds up retrieval by mapping search key values to record locations.

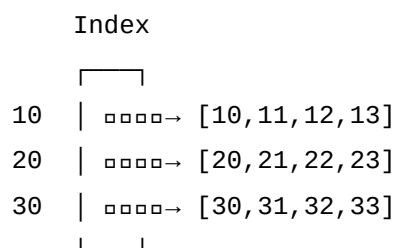
Index Terminology

Term	Definition
Search key	Attribute(s) the index is built on
Dense index	One index entry per data record
Sparse index	One index entry per page/block (only for sorted files)
Clustered index	Data file order matches index order
Unclustered index	Data file order differs from index order
Primary index	Built on ordering key of a sequential file
Secondary index	Built on any non-ordering attribute

Clustered vs. Unclustered

Clustered index:

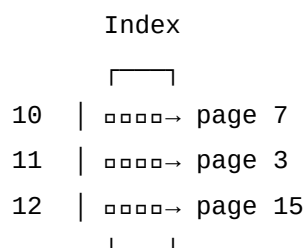
Index leaf → page range



(sequential I/O – fast for ranges)

Unclustered index:

Index leaf → scattered pages



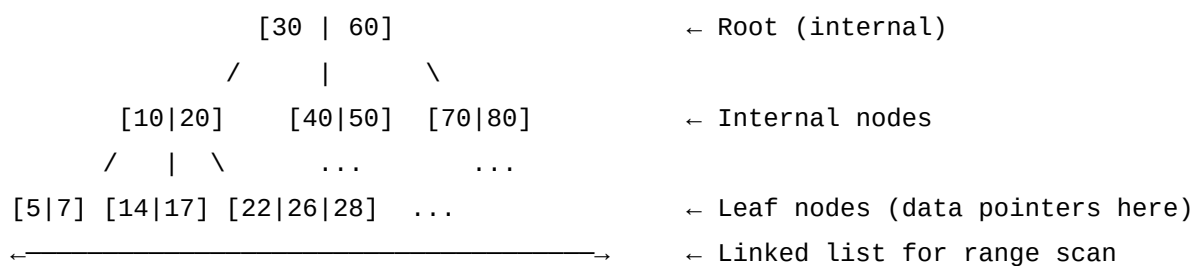
(random I/O – slow for ranges)

B+ Tree Index

The **B+ Tree** is the dominant index structure in all major RDBMS.

Properties

- Balanced tree: all leaf nodes at the same depth
- Order d : each internal node has between d and $2d$ keys
- **All data pointers are in leaf nodes** (unlike B-Tree)
- Leaf nodes are **linked** in a doubly linked list for range scans



B+ Tree Operations

Operation	Cost (height h)
Search (equality)	$O(h) = O(\log_d n)$
Range scan	$O(h + k/d)$ where k = matching records
Insert	$O(h)$ + possible node splits
Delete	$O(h)$ + possible merges/redistribution

For $d = 100$ and $n = 10^8$ records: $h \approx \lceil \log_{100} 10^8 \rceil = 4$ levels.

When to Use a B+ Tree Index

💡 Tip

Use a B+ Tree index for:

- **Equality queries:** WHERE key = value — 3–5 page reads regardless of table size
- **Range queries:** WHERE key BETWEEN a AND b — traverse linked leaf chain
- **ORDER BY** on the indexed column — already sorted, no sort needed
- **GROUP BY** on the indexed column — efficient grouping

Hash Index

A **hash index** uses a hash function $h(k)$ to map search key k to a **bucket**.

Static Hashing

Bucket 0: [records where $h(\text{key}) = 0$]

Bucket 1: [records where $h(\text{key}) = 1$]

...

Bucket B: [records where $h(\text{key}) = B$]

- **Equality search:** compute $h(k)$, go to bucket — single I/O (ideal)
- **Range search:** impossible — hashing destroys order
- **Overflow chains** needed when buckets fill up

Extendible Hashing (Dynamic)

Uses a **global depth** and **directory** to grow buckets without reorganizing the whole file:

- Split only the full bucket
- Global depth grows only when needed (doubles directory)

Index Selection Guidelines

When to Create an Index

Create index when	Avoid index when
Column frequently in WHERE	Table is tiny (< thousands of rows)
Column used in JOIN conditions	Column has low cardinality (e.g., boolean)
Column used in ORDER BY / GROUP BY	Table has heavy INSERT/UPDATE/DELETE
Foreign key columns	Column rarely queried

Multi-Column (Composite) Indexes

```
CREATE INDEX idx_emp_dept_salary ON employee(dept_no, salary);
```

The index on (dept_no, salary) benefits: - WHERE dept_no = 5 (uses leftmost prefix) - WHERE dept_no = 5 AND salary > 50000 - WHERE salary > 50000 (leftmost column missing — full scan)

! Index Rule: Leftmost Prefix

A composite index on (A, B, C) is usable for queries on: - A alone - A, B - A, B, C
But **not** for queries on B alone, C alone, or B, C.

Creating / Dropping Indexes in SQL

```
-- Create index (default: B+ tree in most RDBMS)
CREATE INDEX idx_emp_lname ON employee(lname);

-- Create unique index
CREATE UNIQUE INDEX idx_emp_ssn ON employee(ssn);

-- Create composite index
CREATE INDEX idx_emp_dept_sal ON employee(dept_no, salary);

-- Hash index (PostgreSQL)
CREATE INDEX idx_emp_ssn_hash ON employee USING HASH (ssn);

-- Drop index
DROP INDEX idx_emp_lname;
```

EXPLAIN — Verifying Index Usage

```
EXPLAIN SELECT * FROM employee WHERE lname = 'Smith';
-- Output shows: Index Scan using idx_emp_lname      (good)
-- vs.           Seq Scan on employee                (no index used)
```

Summary

! Chapter 10 — Key Takeaways

- Data is stored in **pages/blocks**; the buffer manager caches pages in RAM to reduce disk I/O.
- File organizations: heap (unordered), sequential (sorted), hash, clustered.
- **Dense index**: one entry per record; **sparse index**: one entry per page (sorted files only).
- **Clustered index**: data file order matches index — sequential I/O, ideal for range queries.
- **Unclustered index**: causes random I/O — expensive for large range scans.
- **B+ Tree**: balanced, all pointers in linked leaves; supports $O(\log n)$ equality and efficient range scans.
- **Hash index**: $O(1)$ equality; useless for range queries.

- **Composite index:** leftmost prefix rule — column order matters.
- Use EXPLAIN to verify if a query uses an index.

Review Questions

1. What is the difference between a **dense** and a **sparse** index? When can a sparse index be used?
2. Explain the **clustered vs. unclustered** index distinction. Which is better for a range query on salary?
3. Why are **all data pointers in the leaf nodes** of a B+ Tree, unlike a B-Tree?
4. A B+ Tree of order $d = 50$ has 1 million records. How many page reads are needed to find a specific record?
5. Why is a **hash index useless for range queries**?
6. Given a composite index on (city, age), which of these queries benefit from the index? Explain.
 - WHERE city = 'Cairo'
 - WHERE age > 25
 - WHERE city = 'Cairo' AND age > 25
 - WHERE age > 25 AND city = 'Cairo'

Query Processing & Optimization

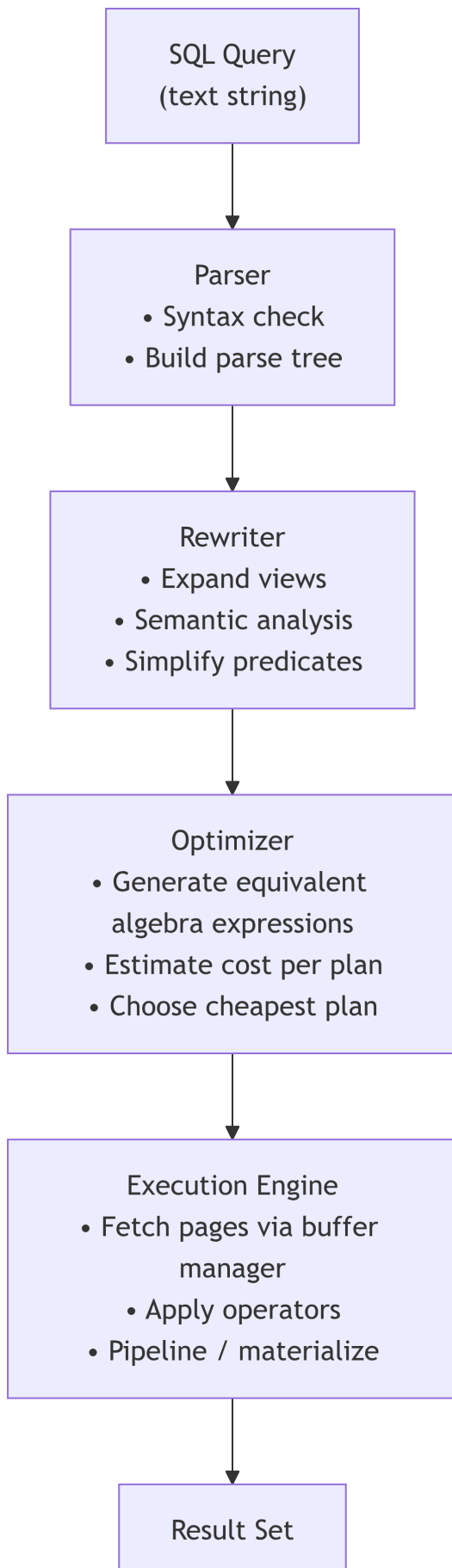
Chapter 11

The Journey of a SELECT

In Chapter 10 you learned *where* data lives (pages, B+ trees, bitmap bitmaps). In Chapter 6 you learned *what* operations transform data (relational algebra). This chapter connects them: how does a DBMS take your SELECT and produce a result *efficiently*?

The answer is a four-stage pipeline that every major RDBMS follows.

Query Processing Pipeline



Stage 1 — Parser

Verifies syntax; produces a **parse tree**.

```
SELECT E.Fname FROM Employee E WHERE E.Salary > 50000
```

→ parse tree: `Project(Filter(Scan(Employee), Salary>50000), [Fname])`

Stage 2 — Rewriter

- Replaces **view references** with the view's stored SELECT
- Applies **semantic simplifications** (e.g., `NOT (x > 5) → x <= 5`)
- Produces the initial **relational algebra expression**

Stage 3 — Optimizer

The core of this chapter. Given the algebra expression, the optimizer:

1. Applies **equivalence rules** to generate alternative expressions (logical optimization)
2. Selects physical algorithms for each operator (physical optimization)
3. Estimates the **cost** of each candidate plan using statistics from the system catalog
4. Returns the plan with the lowest estimated total cost

Stage 4 — Execution Engine

Executes the physical plan: - Fetches pages via the **buffer manager** - Applies operators using **pipelining** or **materialization** - Returns rows to the client

Translation to Relational Algebra

```
SELECT E.Fname, D.Dname
FROM   Employee E
       JOIN Department D ON E.Dno = D.Dnumber
WHERE  E.Salary > 50000;
```

Initial (naive) algebra expression:

$$\pi_{\text{Fname, Dname}}(\sigma_{\text{Salary} > 50000}(\text{Employee} \times \text{Department}))$$

After **pushing the selection down** and converting to a join:

$$\pi_{\text{Fname, Dname}}(\sigma_{\text{Salary} > 50000}(\text{Employee}) \bowtie_{\text{Dno=Dnumber}} \text{Department})$$

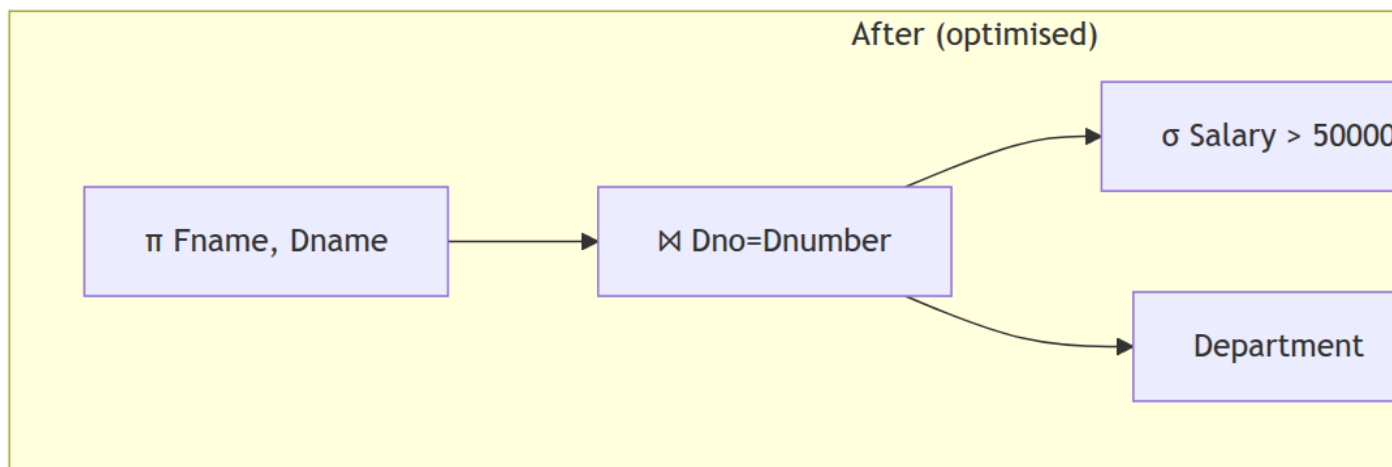
This is better — the filter reduces Employee rows *before* the join (fewer rows to join).

Logical Optimization: Equivalence Rules

The optimizer systematically applies these rules to push work closer to the leaves:

Rule	Description	Benefit
R1 — Selection commutativity	$\sigma_c(R) = \sigma_c(R)$	Trivially true
R2 — Selection cascade	$\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$	Split complex condition
R3 — Selection push-down (join)	$\sigma_c(R \bowtie S) = \sigma_c(R) \bowtie S$ when c only involves R	Reduce R early
R4 — Projection push-down	Keep only needed columns early	Reduce row width
R5 — Join commutativity	$R \bowtie S = S \bowtie R$	Choose smaller outer
R6 — Join associativity	$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$	Reorder joins freely

Before vs After optimization (expression trees):



Cost Model

The optimizer estimates every operator's I/O cost using **catalog statistics**.

Key Statistics

Statistic	Symbol	Meaning
Pages	$B(R)$	Number of disk pages in relation R
Tuples	$T(R)$	Number of rows
Distinct values	$V(A, R)$	Number of distinct values of attribute A
Selectivity	$\text{sel}(A = v)$	Fraction of rows matching $A = v$ — assumed $1/V(A, R)$

Operator Cost Estimates

Operator	I/O cost estimate
Sequential scan	$B(R)$
Equality with clustered B+ tree	$\lceil B(R)/V(A, R) \rceil$
Equality with unclustered index	up to $T(R)/V(A, R)$
Sort (M buffer pages)	$2B(R) \cdot (1 + \lceil \log_{M-1}(\lceil B(R)/M \rceil) \rceil)$
Hash join	$3(B(R) + B(S))$
Block nested-loop join (M buffers)	$\lceil B(R)/(M - 2) \rceil \cdot B(S) + B(R)$

Tip

Total cost = I/O cost + CPU cost. In most systems with spinning disks, I/O dominates. On NVMe SSDs, CPU cost becomes more relevant — some modern optimizers model both.

Selection Algorithms

Sequential Scan

Read every page of R : cost = $B(R)$.

Use when: no index, or the filter is not selective enough to justify random I/O.

Turning point: an unclustered index becomes *worse* than a full scan when the selectivity is above ~5–10% (because random I/Os exceed the savings).

Binary Search (Sorted File)

$\lceil \log_2 B(R) \rceil$ page reads. Rarely used standalone — only when file is sorted on the search key.

B+ Tree Index Scan

$\sigma_{\{\text{Salary} = 55000\}}(\text{Employee})$

→ Clustered B+ tree on Salary

→ h page reads (path) + $\lceil B(R)/V(\text{Salary}, \text{Employee}) \rceil$ pages at leaf level

Range query $\sigma_{\{\text{Salary} > 50000\}}(\text{Employee})$: - Clustered: traverse leaf chain — sequential I/O, very efficient - Unclustered: each record may require a separate random I/O — potentially worse than a full scan for high selectivity

Join Algorithms

Joins are the most expensive operation. Three fundamental algorithms:

Nested-Loop Join

```
for each tuple r in R (outer):
    for each tuple s in S (inner):
        if r.key = s.key: output (r, s)
```

Variant	Cost	Notes
Naïve NLJ	$T(R) \cdot B(S) + B(R)$	Inner scanned once per outer <i>tuple</i>
Block NLJ (M buffers)	$\lceil B(R)/(M-2) \rceil \cdot B(S) + B(R)$	Inner scanned once per outer <i>block</i>
Index NLJ	$B(R) + T(R) \cdot C_{idx}$	Uses index on inner; $C_{idx} \approx 3\text{--}5$ for B+ tree

Sort-Merge Join

Phase 1: External sort R on join key $\rightarrow R'$ (cost: $2 B(R)$ per pass)

Phase 2: External sort S on join key $\rightarrow S'$ (cost: $2 B(S)$ per pass)

Phase 3: Merge R' and S' once (cost: $B(R) + B(S)$)

Total cost $\approx 3(B(R) + B(S))$ for the common two-pass case.

Benefits: excellent when data is already sorted, or when the output needs to be sorted (avoids a second sort).

Hash Join

Build phase: partition R into h buckets via $h_1(\text{key})$

Partition: partition S into same h buckets via $h_1(\text{key})$

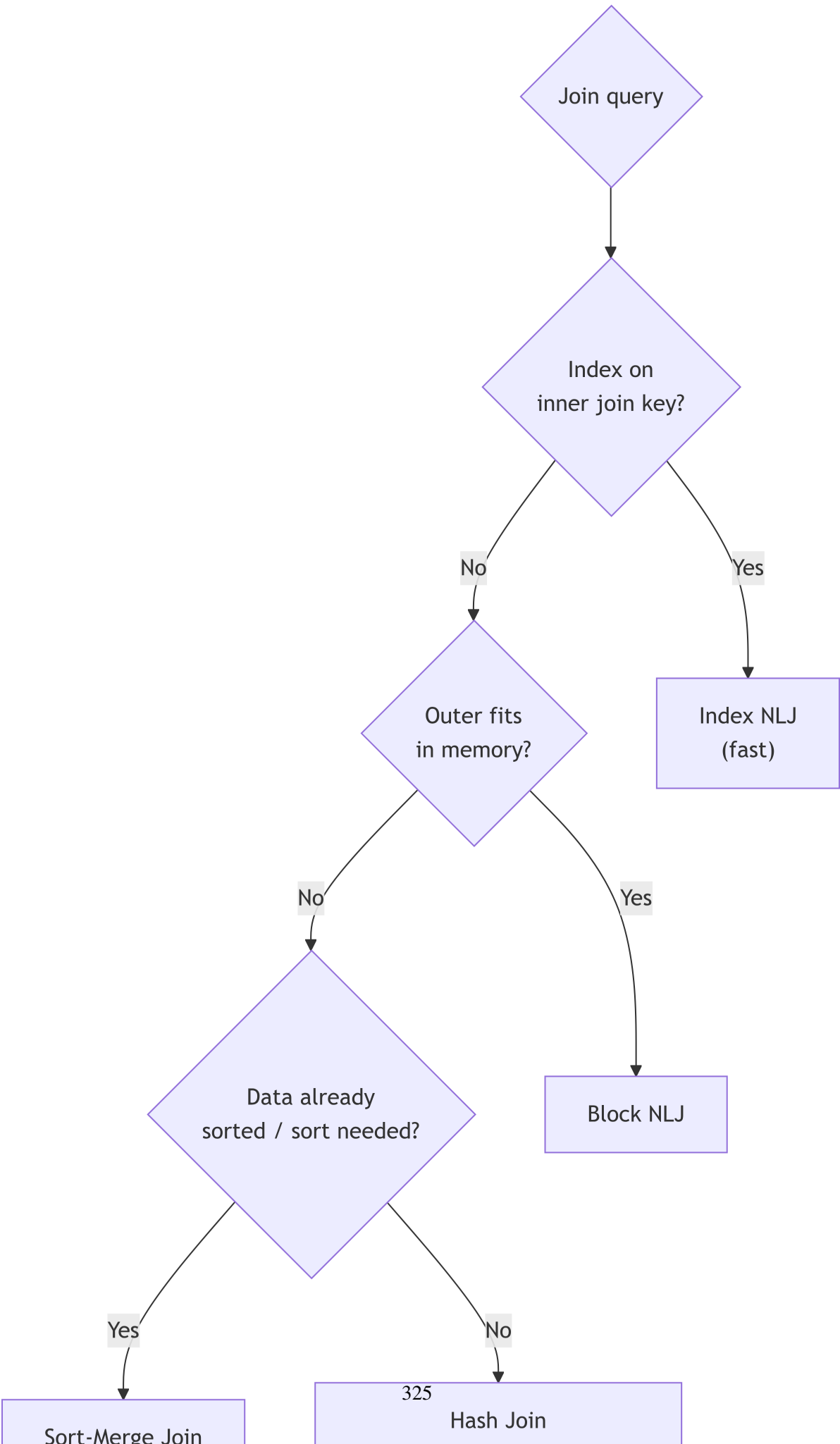
Probe phase: for each bucket i :
 load R_i into memory hash table
 probe with every s in S_i

Cost = $3(B(R) + B(S))$ (two passes over each relation: read/write partitions, then probe).

SQL: two records r, s can only match if $h_1(r.k) = h_1(s.k)$ — only matching buckets need to be compared.

Algorithm Selection Guide

Algorithm	Best When
Index NLJ	One relation small; other has index on join key
Block NLJ	Small outer relation; inner has no index
Sort-Merge	Data already sorted; output needs to be sorted; both large
Hash Join	Large unsorted equijoins; no useful sort or index

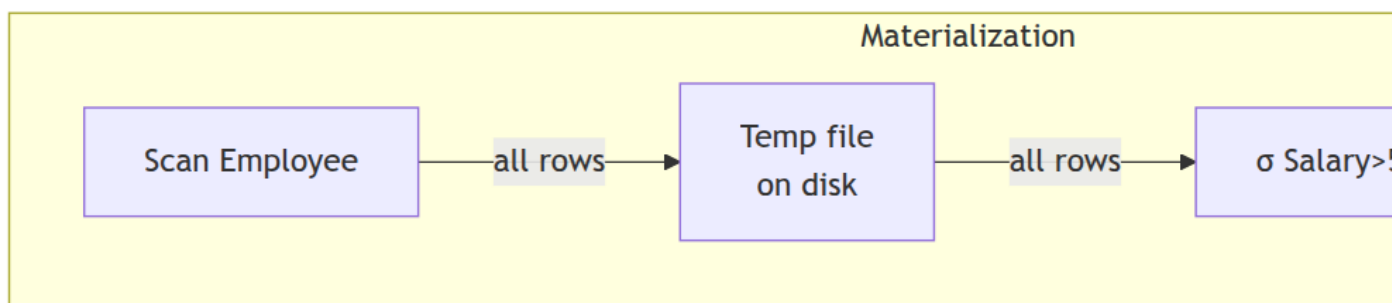


Pipeline vs Materialization

When executing a multi-operator plan, the engine can transmit rows between operators in two ways:

Strategy	Description	Memory	Speed
Pipelining	Operator passes a row immediately to the parent as soon as it produces it	Low — one row at a time	Faster — no intermediate writes
Materialization	Operator writes its entire result to a temp table; parent then reads it	High — full intermediate	Slower — extra I/O, but sometimes required

Materialisation is required when: - The operator needs the full input before producing output (e.g., external sort, hash-build phase, DISTINCT, GROUP BY) - The result is used multiple times (e.g., an inner relation in a nested-loop join)



EXPLAIN and EXPLAIN ANALYZE

EXPLAIN shows the optimizer's chosen plan without executing the query. EXPLAIN ANALYZE executes the query and reports both estimated and actual statistics.

MySQL

EXPLAIN FORMAT=TREE

SELECT E.Fname, D.Dname

```
FROM Employee E
      JOIN Department D ON E.Dno = D.Dnumber
WHERE E.Salary > 50000;
```

Sample output:

```
-> Nested loop inner join
    -> Filter: (e.Salary > 50000) (cost=2.75 rows=4)
        -> Index range scan on e using idx_emp_salary (cost=2.75 rows=4)
            -> Single-row index lookup on d using PRIMARY (Dnumber=e.Dno) (cost=0.25 rows=1)
```

-- EXPLAIN ANALYZE – adds actual rows and execution time

```
EXPLAIN ANALYZE
SELECT E.Fname, D.Dname
FROM Employee E JOIN Department D ON E.Dno = D.Dnumber
WHERE E.Salary > 50000;
```

PostgreSQL

```
EXPLAIN (FORMAT TEXT, ANALYZE, BUFFERS)
SELECT E.Fname, D.Dname
FROM Employee E
      JOIN Department D ON E.Dno = D.Dnumber
WHERE E.Salary > 50000;
```

Sample output:

```
Hash Join (cost=12.50..98.75 rows=150 width=64)
    (actual time=0.542..1.203 rows=4 loops=1)
    Hash Cond: (e.dno = d.dnumber)
    Buffers: shared hit=8
    -> Index Scan using idx_emp_salary on employee e
        (cost=0.43..85.00 rows=200 width=40)
        (actual time=0.124..0.311 rows=4 loops=1)
        Index Cond: (salary > 50000)
    -> Hash (cost=8.00..8.00 rows=360 width=24)
        (actual time=0.287..0.287 rows=3 loops=1)
        -> Seq Scan on department d (cost=0.00..8.00 rows=360 width=24)
            (actual time=0.019..0.047 rows=3 loops=1)
```

Planning Time: 0.312 ms

Execution Time: 1.421 ms

Reading an EXPLAIN Output

1. **Bottom-up:** leaf nodes (scans) execute first; their output feeds parent joins
2. `cost=start..total`: estimated startup and total cost in arbitrary optimizer units
3. `rows=n`: estimated output rows (compare with `actual rows=` to spot bad estimates)
4. `actual time=x.y ms`: wall-clock time for first/last row (`loops=n` means the node repeated n times)
5. **Buffers:** `shared hit=n` (PostgreSQL): pages served from buffer cache (no disk I/O)

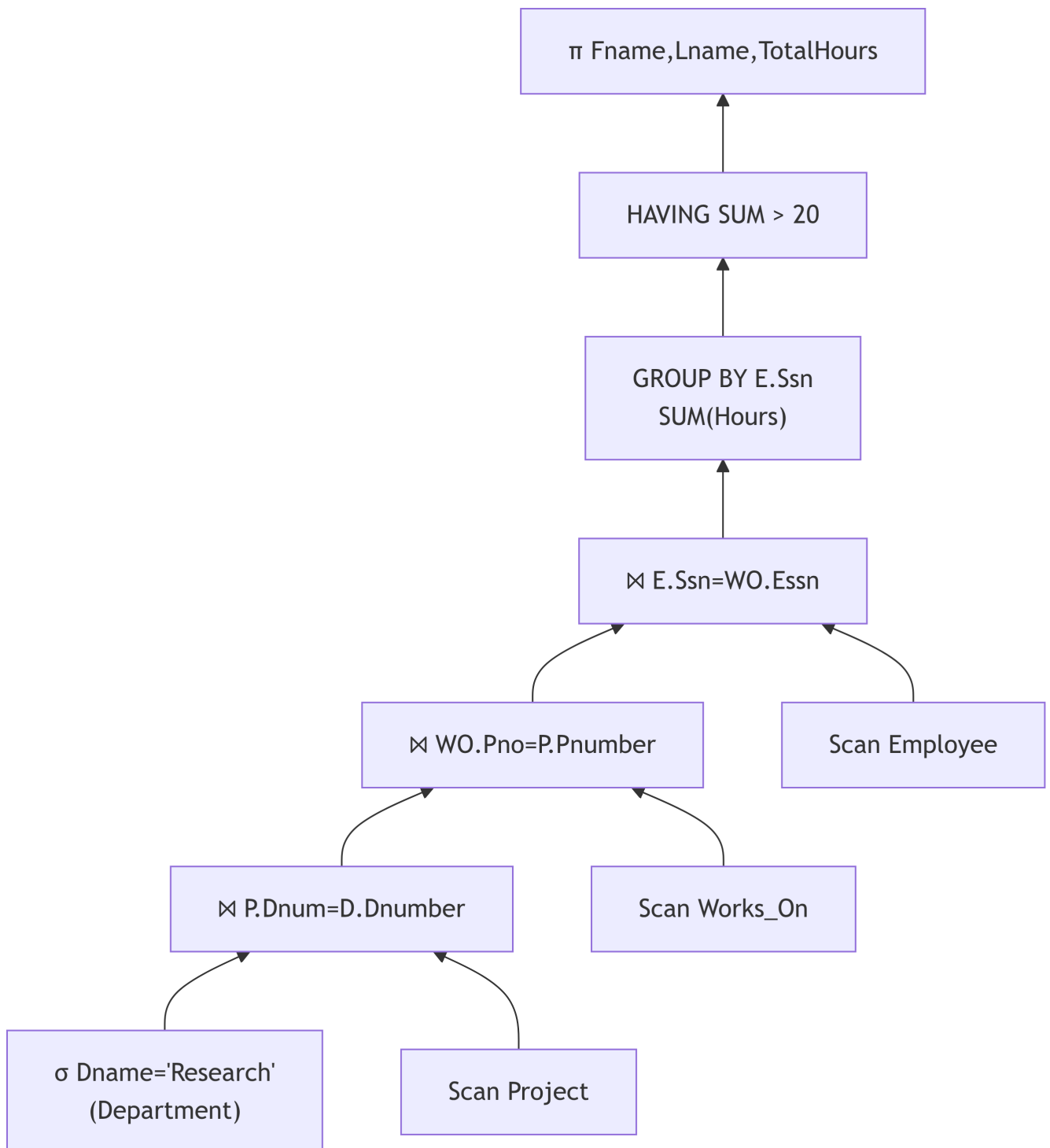
! When Estimates $\neq$ Actuals

If `rows=200` but `actual rows=4`, the optimizer had stale or missing statistics. Fix: run `ANALYZE Employee`; (PostgreSQL) or `ANALYZE TABLE Employee`; (MySQL) to refresh statistics.

Putting It All Together: A Worked Plan

```
-- Q: Names + total hours for Research dept employees working > 20 h
SELECT  E.Fname, E.Lname, SUM(W0.Hours) AS TotalHours
FROM    Employee E
        JOIN Works_On  W0 ON E.Ssn      = W0.Essn
        JOIN Project    P  ON W0.Pno    = P.Pnumber
        JOIN Department D  ON P.Dnum    = D.Dnumber
WHERE   D.Dname = 'Research'
GROUP BY E.Ssn, E.Fname, E.Lname
HAVING  SUM(W0.Hours) > 20;
```

Optimized plan (conceptual):



- Key optimization choices:
1. **Selection on Department pushed down** (only 1 row matches Dname='Research') — eliminates 3 of 4 departments before any join
 2. **Join order**: small filter result joined first; large Employee table joined last
 3. **GROUP BY after all joins** — only grouped once, not per-join

Chapter Summary

! Key Takeaways — Chapter 11

1. **Pipeline:** SQL → Parser (parse tree) → Rewriter (algebra) → Optimizer (physical plan) → Execution Engine (result).
2. **Logical optimization** applies equivalence rules to push selections/projections toward leaves, reorder joins, and reduce intermediate result sizes before choosing algorithms.
3. **Cost model** uses catalog statistics ($B(R)$, $T(R)$, $V(A, R)$) to estimate I/O. Refresh with ANALYZE when estimates diverge from actuals.
4. **Three join algorithms:**
 - *Nested-loop*: simple; best with a small outer + index on inner
 - *Sort-merge*: best when data is sorted or output needs to be sorted
 - *Hash join*: default for large unsorted equijoins; cost $\approx 3(B(R) + B(S))$
5. **Pipelining** passes rows between operators on-the-fly (low memory, faster); **materialization** writes full intermediate results to disk (required for sort, GROUP BY, DISTINCT).
6. EXPLAIN ANALYZE shows the actual plan, estimated vs actual rows, and execution time. A large discrepancy between estimated and actual rows signals stale statistics.

Review Questions

Conceptual

1. Walk through all four stages of the query processing pipeline for the query:

```
SELECT Pname FROM Project WHERE Plocation = 'Houston';
```

Describe what each stage produces.

2. Explain why **pushing selections down** the expression tree is almost always beneficial. Is there ever a case where it is NOT beneficial?
3. Compare **pipelining** and **materialization**. For each of the following operators, state which strategy must be used and why: (a) hash join build phase, (b) sort for ORDER BY, (c) simple selection filter $\sigma_C(R)$, (d) DISTINCT.

Applied

4. Given: $B(\text{Employee}) = 500$, $B(\text{Works_On}) = 200$, $T(\text{Employee}) = 40000$, $T(\text{Works_On}) = 80000$, $M = 50$ buffer pages. Compute the cost of:

- Block nested-loop join (Employee outer, Works_On inner)
- Hash join
- Sort-merge join (assume neither relation is pre-sorted; 2-pass sort)

5. Write the optimized relational algebra expression for:

```
SELECT E.Fname, P.Pname
FROM Employee E, Works_On W, Project P
WHERE E.Ssn = W.Essn AND W.Pno = P.Pnumber
AND P.Plocation = 'Stafford' AND E.Salary < 30000;
```

Show the **unoptimized** expression first, then apply pushdown rules step by step.

6. Run EXPLAIN ANALYZE on the above query in MySQL or PostgreSQL with the Company DB data. Report the chosen join algorithm, the estimated vs actual row counts at each node, and propose one index change that would improve the plan.

Analysis

7. An optimizer estimates rows=5000 for a filter, but EXPLAIN ANALYZE shows actual rows=1.

- What caused this discrepancy?
- What command fixes it?
- What does a bad row estimate do to subsequent join cost estimates?

8. A query joins four tables: Employee (500 pages), Works_On (200 pages), Project (50 pages), Department (10 pages). Without considering indexes, which join order minimizes the total data read in a left-deep plan using hash join at each step? Show your reasoning.

Diagram

9. Draw the **optimized expression tree** for:

```
SELECT D.Dname, COUNT(*) AS n
FROM Employee E JOIN Department D ON E.Dno = D.Dnumber
WHERE E.Salary > 40000
GROUP BY D.Dname
HAVING COUNT(*) >= 2;
```

Label each node with the relational algebra operator and its estimated output cardinality.

Design / Synthesis

10. The DBA notices that the following query runs in 8 seconds on a 2-million-row Sales table:

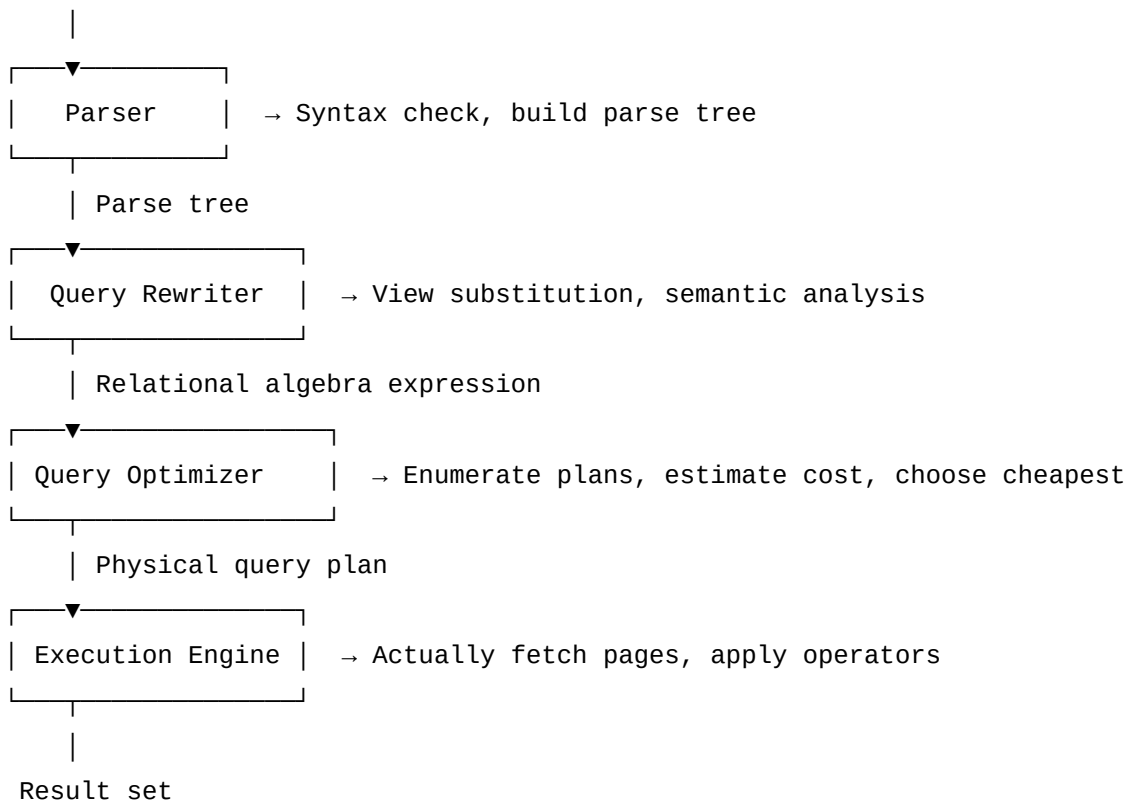
```
SELECT ProductId, SUM(Amount) FROM Sales
WHERE Region = 'North' AND SaleDate BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY ProductId;
```

EXPLAIN ANALYZE shows: Seq Scan on Sales (actual rows=2000000) at the base.

- Propose two indexes (with justification) that would improve this query.
- After adding the indexes, predict the new plan structure (index scan, filter, group).
- What cardinality statistic would the optimizer need to correctly estimate the benefit?

Query Processing Pipeline

SQL Query (text)



Parsing and Translation

The parser checks SQL syntax and builds a **parse tree**. The translator converts it to an initial **relational algebra (RA) expression**:

```
SELECT e.fname, d.dept_name
FROM employee e JOIN department d ON e.dept_no = d.dept_no
WHERE e.salary > 50000;
```

Translates to:

$$\pi_{\text{fname, dept_name}}(\sigma_{\text{salary} > 50000}(\text{employee} \bowtie_{\text{dept_no}} \text{department}))$$

Query Optimization Overview

The optimizer's job: find the **cheapest execution plan** from the many equivalent RA expressions.

Why Optimization Is Hard

- A join of n tables has $O(n!)$ possible orderings (actually more, counting bushy trees)
- For 5 tables: $5! = 120$ plans; for 10 tables: $10! = 3,628,800$ plans
- Cost depends on: table sizes, selectivities, index availability, memory

Two Phases of Optimization

1. **Logical optimization:** Apply algebraic equivalences to improve the RA tree (push selections down, push projections)
2. **Physical optimization:** Choose algorithms for each logical operator (which join algorithm? which access path?)

Cost Model

The optimizer uses a **cost model** to estimate the cost of each plan:

$$\text{Total Cost} = \text{I/O cost} + \text{CPU cost} + \text{Communication cost}$$

In most systems: I/O cost dominates (disk access » CPU).

Cost factors per operator:

Operator	Key cost factor
Sequential scan	$B(R)$ pages
Index scan (clustered)	$B(R)/V(A, R)$ pages
Index scan (unclustered)	Up to $T(R)/V(A, R)$ page I/Os

Operator	Key cost factor
Sort	$2B(R) \cdot (1 + \lceil \log_M B(R) \rceil)$
Hash join	$3(B(R) + B(S))$
Nested loop join	$B(R) + T(R) \cdot B(S)$ (worst case)

Where: - $B(R)$ = number of pages in relation R - $T(R)$ = number of tuples in R - $V(A, R)$ = number of distinct values of attribute A in R - M = number of buffer pages available

Statistics in the System Catalog

The optimizer reads statistics to estimate costs: - $B(R)$: page count - $T(R)$: tuple count - $V(A, R)$: domain cardinality (distinct values) - Histogram: distribution of attribute values

Selection Algorithms

Linear Scan (Sequential Scan)

Read every page of the table: cost = $B(R)$.

Use when: no usable index exists.

Binary Search

On sorted file: cost = $\lceil \log_2 B(R) \rceil$.

Index-Based Selection

Equality: $\sigma_{\{A = v\}}(R)$

└─ Clustered B+ tree: cost $\approx$ height + $B(R)/V(A, R)$ pages

└─ Unclustered B+ tree: cost $\approx$ height + $T(R)/V(A, R)$ page I/Os (potentially many)

Range: $\sigma_{\{A \geq v\}}(R)$

└─ Clustered B+ tree: height + fraction of pages (efficient)

└─ Unclustered B+ tree: may be slower than full scan if many matches

Join Algorithms

Joins are the most expensive operations. Three fundamental algorithms:

1. Nested-Loop Join (NLJ)

```

for each tuple r in R:           ← outer relation
    for each tuple s in S:       ← inner relation
        if r[join_attr] = s[join_attr]: output (r, s)

```

- **Cost:** $T(R) \cdot B(S) + B(R)$ page I/Os (inner scanned once per outer tuple)
- Choose the **smaller relation as outer** to minimize cost
- Improved by **block nested loop**: read a block of outer relation at a time

2. Sort-Merge Join (SMJ)

Phase 1: Sort R on join attribute $\rightarrow R'$
 Phase 2: Sort S on join attribute $\rightarrow S'$
 Phase 3: Merge R' and S' (like merge sort)

- **Cost:** $2B(R) + 2B(S) + B(R) + B(S)$ = sort costs + merge cost
- Excellent when both relations are already sorted, or when the output also needs to be sorted
- Only scans each relation **once** during merge

3. Hash Join

Phase 1 (Build): Partition R into h buckets using hash function h1
 Phase 2 (Partition): Partition S into same h buckets using h1
 Phase 3 (Probe): For each bucket i, build in-memory hash table on R_i ,
 then probe with every tuple in S_i

- **Cost:** $3(B(R) + B(S))$ for the standard case
- Best when at least one relation fits in memory after partitioning
- Most efficient for large unsorted equijoins

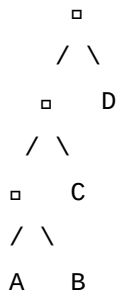
Algorithm Comparison

Algorithm	Best Case	Worst Case	Good When
Nested Loop	One relation fits in mem: $B(R) + B(S)$	$T(R) \cdot B(S)$	Small tables, inner has index
Sort-Merge	$B(R) + B(S)$ (pre-sorted)	Sort cost + merge	Data pre-sorted, output sorted
Hash Join	$B(R) + B(S)$ (build fits in mem)	$3(B(R) + B(S))$	Large unsorted equijoins

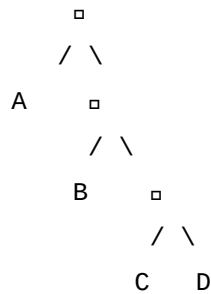
Join Ordering and Plan Enumeration

Left-Deep vs. Right-Deep vs. Bushy Trees

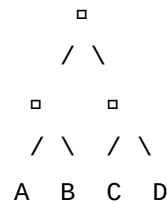
Left-deep:



Right-deep:



Bushy:



Most optimizers enumerate only **left-deep** trees ($n-1$ levels deep) — this reduces the search space from $(2n-2)!/(n-1)!$ to $n!$.

Dynamic Programming Optimization

For each subset of relations S : 1. Find the optimal plan for accessing each single relation 2. For each pair, find optimal join order 3. Build up bottom-up

This finds the global optimum in $O(3^n)$ time instead of $O(n!)$.

Query Plan Reading (EXPLAIN)

```
EXPLAIN ANALYZE
```

```
SELECT e.fname, d.dept_name
```

```
FROM employee e
```

```
JOIN department d ON e.dept_no = d.dept_no
```

```
WHERE e.salary > 50000;
```

Sample output:

```
Hash Join (cost=12.50..98.75 rows=150 width=64)
```

```
Hash Cond: (e.dept_no = d.dept_no)
```

```
-> Index Scan using idx_emp_salary on employee e
```

```
(cost=0.43..85.00 rows=200 width=40)
```

```
Index Cond: (salary > 50000)
```

```
-> Hash (cost=8.00..8.00 rows=360 width=24)
```

```
-> Seq Scan on department d (cost=0.00..8.00 rows=360 width=24)
```

Reading the plan: - The tree is read **bottom-up** — innermost operations happen first - cost=start...total: estimated startup and total cost (in page I/Os) - rows=n: estimated number of output rows - -> denotes a child operator feeding into its parent

 Tip

Always run EXPLAIN ANALYZE (not just EXPLAIN) to see *actual* rows and execution times as well as estimates.

Summary

! Chapter 11 — Key Takeaways

- **Query pipeline:** Parse → Rewrite → Optimize → Execute.
- The **optimizer** applies algebraic equivalences (push σ down, reorder joins) and selects physical algorithms.
- **Cost model** relies on catalog statistics ($B(R)$, $T(R)$, $V(A, R)$) to estimate I/O.
- **Three join algorithms:** Nested-Loop (simple, good with index), Sort-Merge (sorted data/output), Hash Join (large unsorted equijoins).
- **Left-deep trees** reduce the plan search space; dynamic programming finds the optimal join order.
- EXPLAIN ANALYZE shows the actual execution plan and can guide index creation decisions.

Review Questions

1. Sketch the **query processing pipeline** and describe what happens at each stage.
2. Compare **Nested-Loop Join**, **Sort-Merge Join**, and **Hash Join** in terms of cost and best-use scenarios.
3. Given $B(R) = 1000$ pages, $B(S) = 500$ pages, $T(R) = 80000$, $T(S) = 50000$, compute the cost of:
 - Block Nested-Loop Join (100 buffer pages)
 - Hash Join
4. Why does the optimizer prefer **left-deep join trees** over bushy trees?
5. What information does the DBMS store in the **system catalog** that the optimizer uses?
6. How does EXPLAIN ANALYZE help a DBA optimize a slow query?

Part VII: Transaction Management

Transaction Processing

Chapter 12

Why Transactions?

Chapters 7–8 showed you *how* to read and write data with SQL. But two problems arise in any real system:

1. **Multi-step operations** must either complete fully or not at all — a bank transfer that deducts money but never credits the recipient would be catastrophic.
2. **Concurrent access** by many users can corrupt data if writes interfere with reads — without coordination, two users booking the last airline seat could both succeed.

Transactions solve both problems: they provide an atomic, isolated unit of work with guaranteed durability after commit and recovery after failure.

What Is a Transaction?

A **transaction** is a sequence of one or more SQL operations that the DBMS treats as a single, indivisible unit of work.

Example — Bank Transfer of \$500 from Account A to Account B:

```
BEGIN;                                     -- start transaction
    UPDATE Accounts SET Balance = Balance - 500 WHERE Id = 'A';
    UPDATE Accounts SET Balance = Balance + 500 WHERE Id = 'B';
COMMIT;                                    -- make permanent
```

If the system crashes after the first `UPDATE` but before the second, the DBMS **rolls back** the first update automatically — money never disappears.

ACID Properties

Property	One-line definition
Atomicity	All-or-nothing: every operation in a transaction commits together or none do
Consistency	A transaction moves the database from one <i>consistent</i> (valid) state to another
Isolation	Concurrent transactions are invisible to each other; execute as if serial
Durability	Once committed, changes survive crashes, power loss, and restarts

Worked Examples — What Goes Wrong Without Each Property

Without Atomicity

```
-- T1: Transfer $500 A → B
UPDATE Accounts SET Balance = Balance - 500 WHERE Id = 'A';    -- success
-- ** crash here ** — second update never runs
UPDATE Accounts SET Balance = Balance + 500 WHERE Id = 'B';

-- Result: $500 vanishes; A debited, B never credited
-- Fix: atomicity guarantees the first update is undone too
```

Without Consistency

```
-- Business rule: total balance across all accounts must remain constant
-- T1 withdraws $500 from A and deposits $600 to B (bug in code)
-- After commit: total balance increased by $100 — DB is inconsistent
-- Fix: consistency ensures constraints and business rules hold post-commit
```

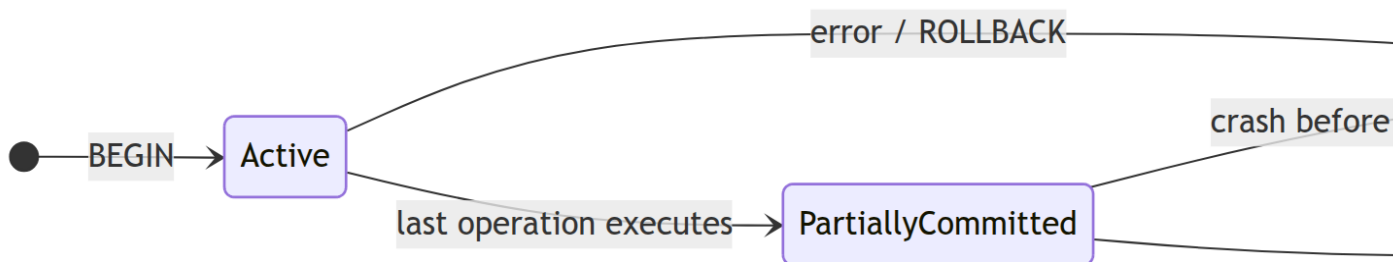
Without Isolation

```
-- T1: Read A = 500, prepare to add 100
-- T2: Read A = 500, prepare to add 200, commits: A = 700
-- T1: commits: A = 600 ← T2's update is overwritten — lost update!
-- Fix: isolation serialises T1 and T2 so one runs fully before the other reads
```

Without Durability

```
-- T1 commits: "Salary updated"
-- ** power failure 2 ms after response sent to client **
-- System restarts: salary reverted to old value
-- Fix: durability requires the committed write to survive any failure
```

Transaction States



State	Description
Active	Transaction is executing; reads/writes in progress
Partially Committed	COMMIT issued; log record written but not yet flushed to stable storage
Committed	Log flushed; changes durable; normal end
Failed	Cannot proceed — error or explicit ROLLBACK BEGIN
Aborted	All changes undone; database restored to pre-transaction state

Concurrency Anomalies

When multiple transactions run without coordination, four classes of anomaly can occur.

Lost Update

Two transactions both read a value, modify it independently, and write back — the second write overwrites the first.

Time	T1	T2	X value
t1	Read X		100
t2		Read X	100
t3	$X \leftarrow X - 50$; Write X=50		50
t4		$X \leftarrow X + 30$; Write X=130	130

Correct result should be 80. T1's update is *lost*.

Dirty Read (Reading Uncommitted Data)

```
T1: Write(X = 200)    -- not yet committed
T2:  Read(X) = 200    -- reads T1's uncommitted value
T1: ROLLBACK          -- X reverts to 100
T2: uses X = 200      -- now based on data that never officially existed
```

Non-Repeatable Read

```
T1: Read(X) = 100
      T2: Write(X = 200), COMMIT
T1: Read(X) = 200    -- same query, different result within same transaction
```

Phantom Read

```
T1: SELECT COUNT(*) WHERE Salary > 50000  → 10
      T2: INSERT employee with Salary = 80000, COMMIT
T1: SELECT COUNT(*) WHERE Salary > 50000  → 11  -- new row "appeared"
```

Isolation Levels

SQL:1992 defines four standard isolation levels, each preventing progressively more anomalies.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible

(MySQL:)

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
SERIALIZABLE	Prevented	Prevented	Prevented

MySQL

```
-- View current level
SELECT @@transaction_isolation;

-- Change for the session
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Change for next transaction only
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
    -- ...
COMMIT;
```

MySQL InnoDB default: **REPEATABLE READ** (with MVCC preventing phantoms too)

PostgreSQL

```
SHOW transaction_isolation;

-- Per-transaction
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
    -- ...
COMMIT;

-- Or: SET at start of transaction
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
    -- ...
COMMIT;
```

PostgreSQL default: **READ COMMITTED**

Oracle

```
-- Per-session
ALTER SESSION SET ISOLATION_LEVEL = SERIALIZABLE;

-- Per-transaction
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

-- ...
COMMIT;
```

Oracle supports only **READ COMMITTED** and **SERIALIZABLE** (no **READ UNCOMMITTED** or **REPEATABLE READ**).

Write-Ahead Logging (WAL)

WAL is the mechanism that implements both **Atomicity** (rollback) and **Durability** (recovery after crash).

Core WAL rule:

> Before a modified data page is written to disk, its log record must reach stable storage first.

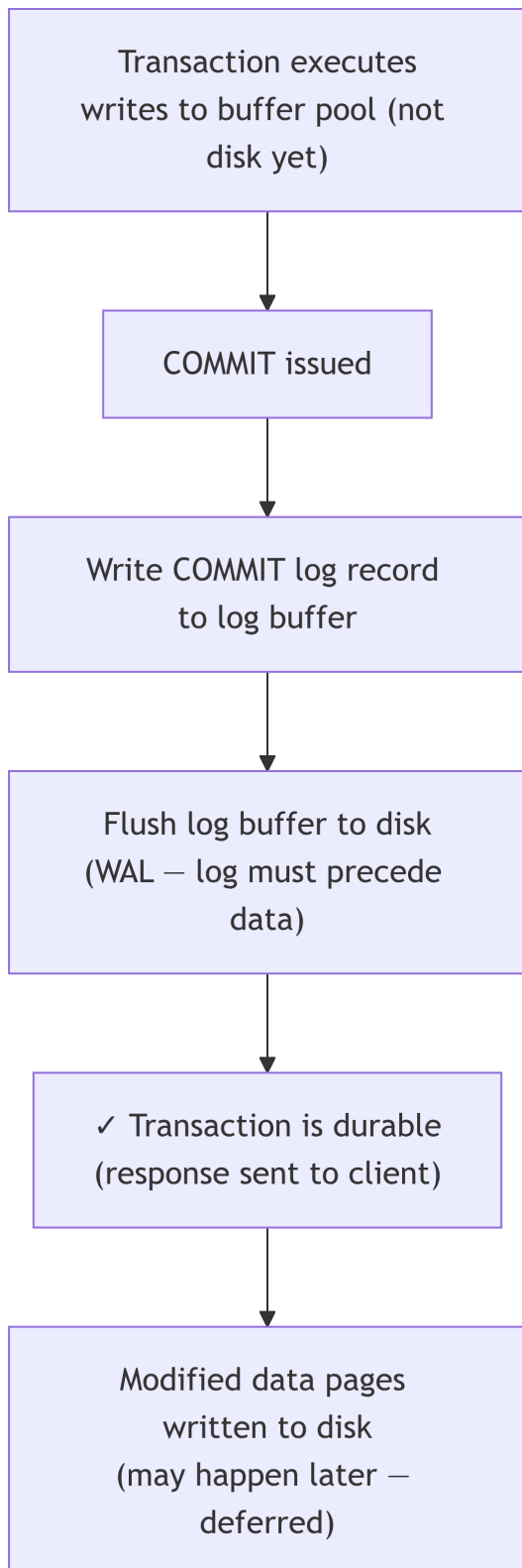
Log Record Format

Each write operation appends a record to the sequential log:

LSN | TxnId | PageId | Before-Image | After-Image | PrevLSN

Field	Meaning
LSN	Log Sequence Number — monotonically increasing ID
TxnId	Which transaction performed this write
PageId	Which data page was modified
Before-Image	Value <i>before</i> the change (needed for UNDO)
After-Image	Value <i>after</i> the change (needed for REDO)
PrevLSN	Pointer back to previous log record of this transaction (forms a chain)

Commit Flow with WAL



! Steal / No-Force Policy

Most modern DBMS use **Steal + No-Force**:

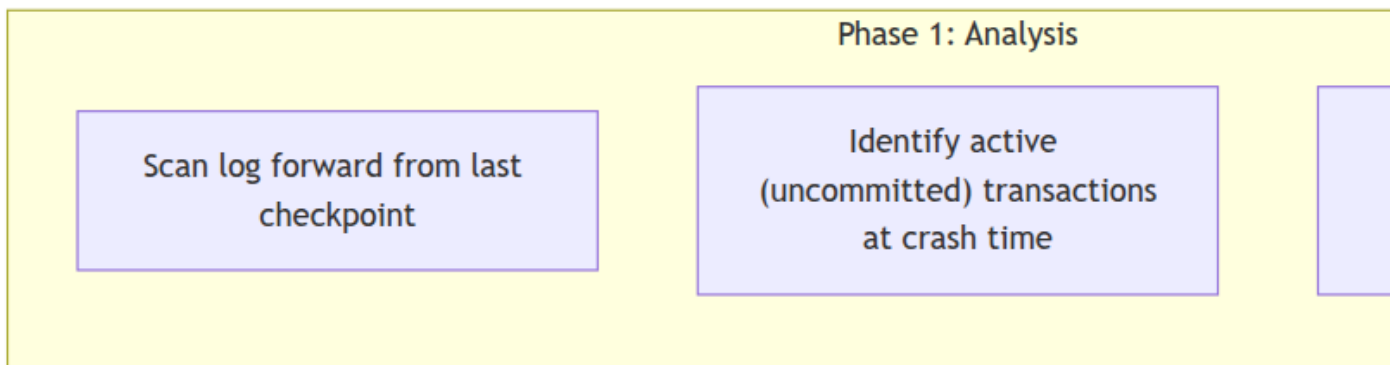
Policy	Meaning
Steal	Dirty (uncommitted) pages <i>can</i> be written to disk before commit — needed to free buffer space
No-Force	Dirty pages are <i>not</i> forced to disk at commit time — better write performance

This maximises performance but requires recovery to handle both: - **UNDO**: steal means uncommitted data may be on disk → must undo after crash - **REDO**: no-force means committed data may still be in buffer → must redo after crash

Recovery: Undo and Redo

After a crash, the recovery manager reads the log and restores a consistent state.

ARIES Recovery Algorithm (3 Phases)



Phase	Direction	Purpose
Analysis	Forward from checkpoint	Determine what needs REDO / UNDO
Redo	Forward from earliest dirty page	Re-apply all operations (even of crashed txns)
Undo	Backward for each loser transaction	Roll back uncommitted changes

Checkpointing

Checkpoints reduce recovery time by: 1. Flushing all dirty pages to disk (fuzzy checkpoint: writes a checkpoint record without full flush) 2. Writing a checkpoint log record noting which transactions were active

After a crash, recovery only needs to examine log records from the last checkpoint — not the entire log.

Savepoints

Savepoints allow partial rollback within a transaction without aborting the entire unit:

MySQL

```
BEGIN;
    UPDATE Employee SET Salary = 40000 WHERE Ssn = '123456789';
    SAVEPOINT sp1;
    UPDATE Employee SET Salary = 50000 WHERE Ssn = '333445555';
    -- oops, second update was a mistake
    ROLLBACK TO SAVEPOINT sp1;    -- undo only the second update
    -- first update still in effect
COMMIT;
```

PostgreSQL

```
BEGIN;
    UPDATE Employee SET Salary = 40000 WHERE Ssn = '123456789';
    SAVEPOINT sp1;
    UPDATE Employee SET Salary = 50000 WHERE Ssn = '333445555';
    ROLLBACK TO SAVEPOINT sp1;
    RELEASE SAVEPOINT sp1;        -- optional: free the savepoint
COMMIT;
```

Oracle

BEGIN

```
UPDATE Employee SET Salary = 40000 WHERE Ssn = '123456789';
```

```
SAVEPOINT sp1;
```

```
UPDATE Employee SET Salary = 50000 WHERE Ssn = '333445555';
```

```
ROLLBACK TO SAVEPOINT sp1;
```

```
COMMIT;
```

```
END;
```

Chapter Summary

! Key Takeaways — Chapter 12

1. A **transaction** is an atomic unit of work: all operations succeed together or all are undone.
SQL syntax: BEGIN → (operations) → COMMIT or ROLLBACK.
 2. **ACID** = Atomicity (all-or-nothing), Consistency (rules preserved), Isolation (concurrent txns appear serial), Durability (committed = permanent).
 3. **Four concurrency anomalies** — Lost Update, Dirty Read, Non-Repeatable Read, Phantom Read — caused by unsynchronised concurrent transactions.
 4. **Isolation levels** (weakest to strongest): READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE. Default: MySQL=REPEATABLE READ, PostgreSQL/Oracle=READ COMMITTED.
 5. **WAL (Write-Ahead Logging)**: log record flushed to stable storage before data page modified on disk. Implements both undo (rollback/crash) and redo (ensure committed writes survive).
 6. **ARIES recovery**: three phases — Analysis (identify active txns) → Redo (replay log forward) → Undo (rollback losers backward).
 7. **Savepoints** allow partial rollback without aborting the full transaction.
-

Review Questions

Conceptual

1. For each ACID property, describe a concrete scenario — using either the bank-transfer or the Company DB context — where violating that property would cause incorrect results.
2. Explain the difference between a **non-repeatable read** and a **phantom read**. Both involve a transaction reading different data within the same transaction — why are they classified as separate anomalies? At which isolation levels is each prevented?
3. Explain the **Steal/No-Force** buffer policy. Why is it preferred despite requiring full ARIES recovery?

Applied

4. Draw a timeline (like §12.4) showing a **lost update anomaly** involving two employees simultaneously trying to use `UPDATE Employee SET Salary = Salary + 5000 WHERE Ssn = '123456789'`. Annotate each step with the in-memory value each transaction sees.
5. For each isolation level in MySQL, write the SQL to set it and then write a script with two concurrent sessions (T1 and T2) that demonstrates the specific anomaly that level still allows.
6. Using the WAL log record format from §12.6, write out the log records produced by:

```
BEGIN;                                -- TxnId = T42
UPDATE Employee SET Salary = 55000 WHERE Ssn = '123456789';  -- page P7, was 30000
UPDATE Employee SET Salary = 45000 WHERE Ssn = '333445555';  -- page P7, was 40000
COMMIT;
```

Show the LSN, TxnId, PageId, before-image, after-image, and PrevLSN for each record.

Analysis

7. A system crashes immediately after flushing the COMMIT log record to disk for transaction T1, but before writing T1's modified page to disk. After restart:
 - Does ARIES classify T1 as a winner or a loser?
 - Which phase handles T1, and what does it do?
 - What would happen if WAL were violated (data page written before log)?
8. Transaction T2 is in the Partially Committed state when the system crashes. After recovery, is T2 committed or aborted? Justify your answer using the transaction state diagram.

Diagram

9. Draw the complete **wait-for graph** for the following scenario and identify whether a deadlock exists. If it does, name one victim that can be aborted to break it:

- T1 holds lock on Employee row E1, waits for lock on Department row D1
- T2 holds lock on Department row D1, waits for lock on Works_On row W3
- T3 holds lock on Works_On row W3, waits for lock on Employee row E1

Design

10. An e-commerce application processes 10 000 orders per second. Each order involves:

- INSERT INTO Orders ...
- UPDATE Inventory SET Qty = Qty - n WHERE ProductId = ?
- INSERT INTO Payments ...

The DBA suggests using READ UNCOMMITTED to maximize throughput. Identify the specific risks this creates for each of the three statements. Propose the minimum isolation level that would be safe, and justify the trade-off.

Synthesis

11. Compare how **WAL + ARIES** and **shadow paging** (an alternative recovery mechanism where committed changes are written to a new page, then the page table atomically updated on commit) handle each of the following:

- Crash before commit
- Crash after commit but before data page flush
- Sequential write performance
- Space overhead

Which is better for a high-throughput OLTP system, and why?

What Is a Transaction?

A **transaction** is a logical unit of work that consists of one or more database operations that must be executed as an **indivisible whole** — either all succeed or none do.

Example: Bank transfer of \$500 from Account A to Account B:

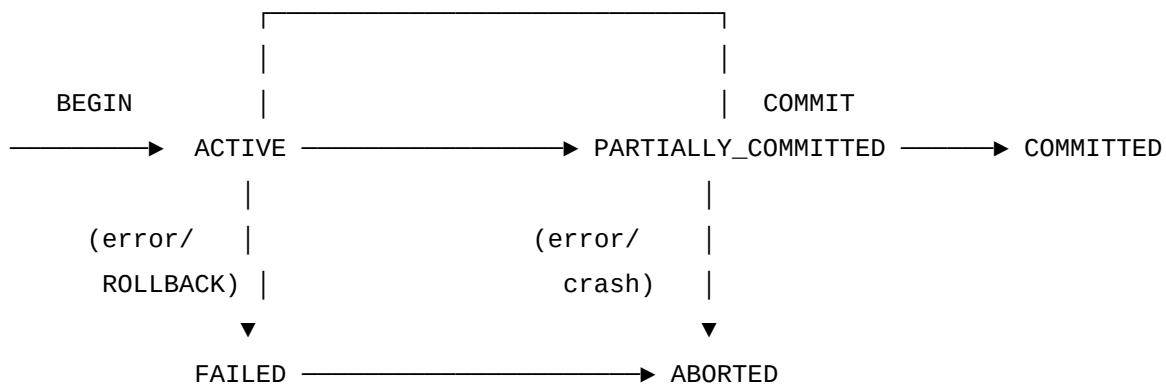
```
BEGIN;  
    UPDATE accounts SET balance = balance - 500 WHERE account_id = 'A';  
    UPDATE accounts SET balance = balance + 500 WHERE account_id = 'B';  
COMMIT;
```

If the system crashes after the first UPDATE but before the second, the transfer **must be rolled back** — we cannot allow money to vanish.

ACID Properties

Property	Definition
Atomicity	All operations in a transaction succeed, or none do. No partial execution.
Consistency	A transaction takes the database from one <i>consistent state</i> to another. Business rules preserved.
Isolation	Concurrent transactions execute as if they were serial. Intermediate states not visible to others.
Durability	Once a transaction is committed, its effects survive system failures (written to stable storage).

Transaction States



State	Description
Active	Transaction is executing
Partially committed	Final statement executed; awaiting commit
Committed	Transaction successfully completed; changes made permanent
Failed	Normal execution can no longer proceed
Aborted	Transaction rolled back; database restored to pre-transaction state
Terminated	Either committed or aborted; leaves the system

Concurrency Issues

When multiple transactions run concurrently without control, several **anomalies** can arise:

1. Lost Update

```

T1: Read(X=100)
T1: X = X - 50 = 50
T1: Write(X=50)

T2: Read(X=100)
T2: X = X + 30 = 130
T2: Write(X=130) ← T1's update is lost!

Result: X=130 (should be X=80)

```

2. Dirty Read (Temporary Update)

```

T1: Write(X=200)
T1: ROLLBACK → X restored to 100

T2: Read(X=200) ← reads uncommitted data
T2: uses X=200 ← wrong! dirty read

```

3. Unrepeatable Read

```

T1: Read(X=100)
T1: Read(X=200)

T2: Write(X=200), COMMIT
← T1 reads different value same transaction!

```

4. Phantom Read

```

T1: SELECT COUNT(*) WHERE salary>50000 → 10
T2: INSERT new employee with salary=80000, COMMIT
T1: SELECT COUNT(*) WHERE salary>50000 → 11 ← phantom row appeared!

```

Transaction Isolation Levels

SQL defines four standard isolation levels:

Level	Dirty Read	Unrepeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible

Level	Dirty Read	Unrepeatable Read	Phantom Read
SERIALIZABLE	Prevented	Prevented	Prevented

```
-- Set isolation level (PostgreSQL)
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN;
    -- your queries here
COMMIT;
```

💡 Tip

Default isolation levels: - PostgreSQL: READ COMMITTED

- MySQL/InnoDB: REPEATABLE READ

- SQL Server: READ COMMITTED

- Oracle: READ COMMITTED

Concurrency Control: Two-Phase Locking (2PL)

Two-Phase Locking is the most widely used protocol for guaranteeing serializability.

Lock Types

Lock	Symbol	Compatibility
Shared (read) lock	S	Multiple transactions can hold S simultaneously
Exclusive (write) lock	X	Only one transaction; incompatible with all others

Lock Compatibility Matrix

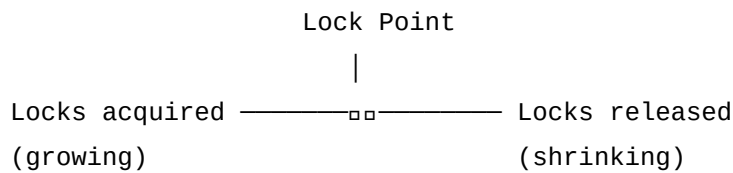
	S	X
S	(compatible)	
X		

2PL Protocol

A transaction follows **two strict phases**:

Phase 1 – Growing Phase: locks can be ACQUIRED; none released

Phase 2 – Shrinking Phase: locks can be RELEASED; none acquired



Theorem: Any schedule produced by 2PL transactions is **conflict-serializable** (guaranteed correct).

Strict 2PL

A common variant: **all locks are held until COMMIT or ROLLBACK.**

- Prevents cascading rollbacks (no dirty reads)
- Used by most real DBMS implementations

```
-- Locks in practice (explicit locking in PostgreSQL)
BEGIN;
SELECT * FROM employee WHERE dept_no = 5 FOR UPDATE; -- Acquires X lock
-- ... process rows ...
COMMIT; -- Releases all locks
```

Deadlocks

A **deadlock** occurs when two or more transactions wait for each other's locks indefinitely:

```
T1 holds X(A), waits for X(B)
T2 holds X(B), waits for X(A)
↑ circular wait → deadlock
```

Deadlock Detection

Build a **wait-for graph**: - Node per transaction - Edge $T_i \rightarrow T_j$ if T_i is waiting for a lock held by T_j - A **cycle** in the graph means deadlock

```
T1 → T2 (T1 waits for T2)
T2 → T3 (T2 waits for T3)
T3 → T1 (T3 waits for T1) ← cycle! deadlock
```

Resolution: Abort one transaction in the cycle (the “victim”) — choose by: - Youngest transaction (minimum work lost) - Transaction with fewest locks - Transaction that has not been a victim recently

Deadlock Prevention

- **Wait-Die:** older transactions wait; younger ones die (rollback and restart)
- **Wound-Wait:** older transactions wound (abort) younger ones; younger ones wait

Recovery: Logging and ARIES

Why Recovery Is Needed

- **System crash:** server power loss, OS failure
- **Transaction abort:** application error, deadlock victim
- **Media failure:** disk crash

Write-Ahead Logging (WAL)

WAL rule: Before any data page is written to disk, the corresponding **log record must be written to stable storage first.**

Every operation writes a log record:

(LSN, TransactionID, PageID, Before-Image, After-Image, PrevLSN)

Commit Protocol

Transaction decides to COMMIT

↓

Write COMMIT log record to disk (WAL)

↓

Transaction is durable (even if crash occurs now)

↓

Write modified data pages to disk (may be deferred – "steal/no-force")

Summary

! Chapter 12 — Key Takeaways

- A **transaction** is an atomic unit of work; it's the fundamental unit of DB reliability.
- **ACID properties:** Atomicity, Consistency, Isolation, Durability — enforced by the DBMS.
- **Concurrency anomalies:** Lost Update, Dirty Read, Unrepeatable Read, Phantom Read.
- **Isolation levels** (weakest to strongest): READ UNCOMMITTED → READ COMMITTED

→ REPEATABLE READ → SERIALIZABLE.

- **Two-Phase Locking (2PL)**: ensures conflict-serializability; Strict 2PL holds all locks until commit.
- **Deadlocks** arise from circular lock waits; detected via wait-for graphs; resolved by aborting a victim.
- **WAL (Write-Ahead Logging)**: log record written before data page — ensures durability.

Review Questions

1. Explain each of the four **ACID properties** and give an example of what could go wrong if each property were violated.
2. Draw a timeline showing a **dirty read anomaly** between two transactions T1 and T2.
3. Explain the **Two-Phase Locking** protocol. Why does it guarantee serializability?
4. What is a **deadlock**? Draw a wait-for graph showing a deadlock among three transactions.
5. Compare **isolation levels**: at which level are each of the four anomalies prevented?
6. Explain the **Write-Ahead Logging** rule and why it is needed for recovery.

Concurrency Control & Scheduling

Chapter 13

From Problem to Protocol

Chapter 12 identified four concurrency anomalies (lost update, dirty read, non-repeatable read, phantom read) and four isolation levels that prevent them. This chapter explains *how* DBMS implementations actually achieve isolation: by controlling the schedule in which operations execute.

Schedules

A **schedule** (or *history*) S is a total ordering of all operations from a set of concurrent transactions that preserves the intra-transaction order within each transaction.

Notation

Symbol	Meaning
$r_i(X)$	Transaction T_i reads item X
$w_i(X)$	Transaction T_i writes item X
c_i	Transaction T_i commits
a_i	Transaction T_i aborts

Serial vs Concurrent

$$S_{\text{serial}} : r_1(A) w_1(A) c_1 \mid r_2(A) w_2(A) c_2$$

$$S_{\text{concurrent}} : r_1(A) r_2(A) w_1(A) w_2(A) c_1 c_2$$

A serial schedule is always **correct** by definition (no interference) but gives zero concurrency. The goal of concurrency control is to allow non-serial schedules that are still *equivalent* to some serial schedule.

Conflict Serializability

Conflicting Operations

Two operations **conflict** iff: 1. They belong to **different** transactions 2. They access the **same data item** X 3. At least **one is a write**

Pair	Type	Conflict?
$r_i(X), r_j(X)$	Read-Read	No — both only read
$r_i(X), w_j(X)$	Read-Write	Yes
$w_i(X), r_j(X)$	Write-Read	Yes
$w_i(X), w_j(X)$	Write-Write	Yes

Conflict-Equivalent Schedules

Schedules S and S' are conflict-equivalent iff one can be transformed into the other by swapping adjacent **non-conflicting** operations.

A schedule is **conflict-serializable** iff it is conflict-equivalent to some serial schedule.

Precedence (Serialization) Graph

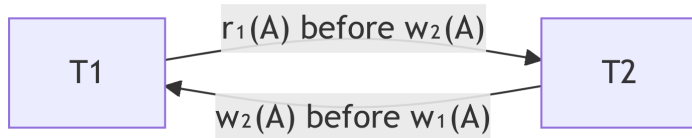
Build a directed graph from schedule S : - One **node** per transaction - Add **edge** $T_i \rightarrow T_j$ if T_i has a conflicting operation that *precedes* an operation of T_j

Theorem: S is conflict-serializable $\iff$ its precedence graph is **acyclic** (contains no cycle).

Worked Example

$S : r_1(A) \ w_2(A) \ w_1(A) \ r_2(B) \ w_2(B) \ c_1 \ c_2$

Identify conflicts: - $r_1(A)$ before $w_2(A)$: Read-Write $\rightarrow$ edge $T_1 \rightarrow T_2$ - $w_2(A)$ before $w_1(A)$: Write-Write $\rightarrow$ edge $T_2 \rightarrow T_1$



The graph has a **cycle** $(T_1 \rightarrow T_2 \rightarrow T_1) \rightarrow S$ is **NOT conflict-serializable**.

💡 Tip

If the graph were acyclic, the topological sort of the graph gives the equivalent serial order.

View Serializability

View serializability is a weaker (more permissive) criterion than conflict serializability. Two schedules are **view-equivalent** iff:

1. If T_i reads the **initial value** of X in S , it also does so in S'
2. If T_i reads a value of X **written by** T_j in S , same in S'
3. If T_i performs the **final write** of X in S , same in S'

Every conflict-serializable schedule is view-serializable.

Not every view-serializable schedule is conflict-serializable.

View serializability is NP-complete to test $\rightarrow$ not used in practice.

Recoverability

Recoverable Schedules

A schedule is **recoverable** iff: if T_j reads data written by T_i , then T_i must commit before T_j commits.

Non-recoverable example:

Time	T1	T2
t1	$w_1(X)=200$	
t2		$r_2(X)=200$ (dirty read)
t3		$c_2 \leftarrow **$ T2 commits based on T1's data**
t4	$a_1 \leftarrow **$ T1 aborts**	

T2 committed based on data that T1 never committed. **Irrecoverable** — we cannot undo T2.

Cascading Rollback

If T1 aborts and T2 has read T1's data, T2 must also abort — this cascades to T3 if T3 read T2's data:

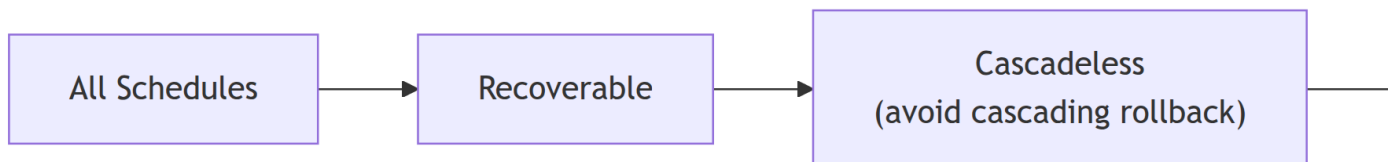
T1 aborts $\rightarrow$ T2 must abort $\rightarrow$ T3 must abort $\rightarrow \dots$

Cascadeless Schedules

In a **cascadeless** schedule, no transaction reads uncommitted data:

If T_j reads X written by T_i , then T_i commits before that read

Cascadeless $\supset$ strict (no reading OR writing uncommitted data).



Two-Phase Locking (2PL) — Full Protocol

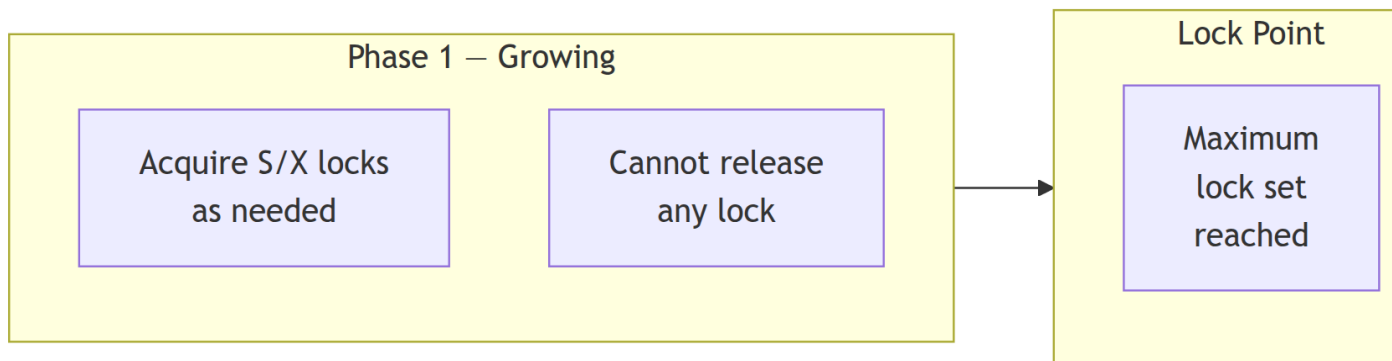
Lock Types

Lock	Notation	Compatibility
Shared (read)	S	Multiple transactions can hold S simultaneously
Exclusive (write)	X	Only one transaction; blocks all other S and X

Lock Compatibility Matrix:

Request	Held	S	X
S			
X			

The Two Phases



Theorem: Any schedule produced by 2PL transactions is **conflict-serializable**.

2PL Variants

Variant	Rule	Prevents
Basic 2PL	Two phases as above	Non-serializable schedules
Strict 2PL	Hold all X locks until commit/abort	Dirty reads + cascading rollback
Rigorous 2PL	Hold ALL locks (S and X) until commit/abort	Strictest; simplest to reason about
Conservative 2PL	Acquire ALL needed locks before any operation	Deadlock-free (pre-declaration required)

Most databases implement **Strict 2PL** — release all locks only at commit or rollback.

```
-- Explicit locking in PostgreSQL (Strict 2PL semantics by default)
BEGIN;
SELECT * FROM Employee WHERE Ssn = '123456789' FOR UPDATE;  -- X lock
-- ... process row ...
COMMIT;  -- releases all locks
```

-- Shared lock (no wait; fail if cannot acquire immediately)

SELECT * FROM Employee FOR SHARE NOWAIT;

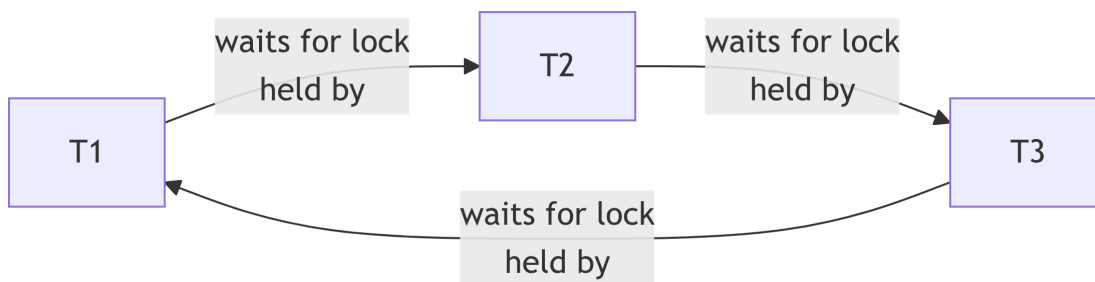
Deadlocks

A **deadlock** is a circular wait where each transaction holds a lock that another transaction is waiting for.

Wait-For Graph

Build a directed graph where edge $T_i \rightarrow T_j$ means “ T_i is waiting for T_j to release a lock.”

Deadlock iff the wait-for graph contains a cycle.



This three-way cycle is a deadlock. The DBMS detects it by running cycle detection on the wait-for graph (periodically or immediately on lock request).

Deadlock Resolution

Once detected, abort one transaction (the **victim**). Selection criteria:

Criterion	Strategy
Minimum work done	Abort the youngest transaction
Minimum cost to restart	Abort the one with fewest locks
Starvation prevention	Never choose the same victim twice

! Important

After aborting a victim, the DBMS **restarts** the transaction automatically (usually). The application must be prepared to retry its transaction on deadlock.

Deadlock Prevention Strategies

Instead of detection + abort, prevent cycles from forming:

Strategy	Rule	Characteristic
Wait-Die	Old waits for young; young dies (aborts, restarts)	Non-preemptive victim choice
Wound-Wait	Old wounds (preempts) young; young waits	Fewer aborts for old transactions
No-Wait	Never wait; abort immediately if lock unavailable	Low deadlock overhead; high abort rate

Timestamp-Based Concurrency Control

Every transaction receives a timestamp $TS(T)$ at start. Two additional timestamps are maintained per data item X :

- $R_TS(X)$: timestamp of the last transaction that successfully **read** X
- $W_TS(X)$: timestamp of the last transaction that successfully **wrote** X

Rules

Operation	Condition	Action
$r_i(X)$	$TS(T_i) < W_TS(X)$	Abort T_i — it would read a future-written value
$r_i(X)$	$TS(T_i) \geq W_TS(X)$	OK; set $R_TS(X) = \max(R_TS(X), TS(T_i))$
$w_i(X)$	$TS(T_i) < R_TS(X)$	Abort T_i — its write would overwrite data already read by a newer transaction
$w_i(X)$	$TS(T_i) < W_TS(X)$	Thomas Write Rule: skip this write (it's already obsolete, no abort needed)
$w_i(X)$	otherwise	OK; set $W_TS(X) = TS(T_i)$

Advantage: No deadlocks (no waiting for locks).

Disadvantage: Long-running transactions may repeatedly abort and restart (starvation).

Multiversion Concurrency Control (MVCC)

MVCC is used by the majority of modern DBMS (PostgreSQL, MySQL InnoDB, Oracle, CockroachDB).

Core Idea

Each **write** creates a new **version** of the item; old versions are retained for a period. Each **read** sees the snapshot of the database as of its transaction's start time — a read never blocks a write and a write never blocks a read.

Item X versions:

v1: value=100, created by T1 (TS=1), expired by T5 (TS=5)

v2: value=150, created by T5 (TS=5), active (never expired)

- Transaction T_3 (TS=3): reads $v1$ (the version valid at TS=3)
- Transaction T_7 (TS=7): reads $v2$ (the version valid at TS=7)

MVCC and Isolation Levels

Level	MVCC snapshot point	Blocks on writes?
READ COMMITTED	Beginning of each SQL statement	No
REPEATABLE READ	Beginning of the transaction	No
SERIALIZABLE (PG SSI)	Beginning + conflict detection	Only on detected write-write conflict

PostgreSQL's **Serializable Snapshot Isolation (SSI)** extends MVCC with anti-dependency detection, achieving true serializable isolation with minimal blocking overhead.

Optimistic Concurrency Control (OCC)

OCC assumes conflicts are **rare** — so validate only at commit time rather than locking proactively.

Three Phases

Phase	Activity
Read	Execute transaction in a local workspace; read from DB, write to workspace; no locks
Validate	Check if any conflict exists between this transaction's reads/writes and concurrently committed transactions
Write	If valid: apply workspace changes to DB; if invalid: abort and restart

OCC is ideal for **read-heavy, low-conflict** workloads. It can cause high abort rates under write-heavy contention.

Interactive: Isolation Level Anomaly Explorer

Chapter Summary

! Key Takeaways — Chapter 13

1. A **schedule** is a total ordering of operations from concurrent transactions. **Serial schedules** are always correct; **concurrent schedules** need validation.
2. **Conflict serializability** (the practical standard): a schedule is correct iff its **precedence graph** is **acyclic**. An acyclic graph's topological sort gives the equivalent serial order.
3. **Recoverable schedules**: T_j commits only after all T_i whose data it read. **Cascadeless** schedules: no dirty reads at all — prevents cascading rollback.
4. **Two-Phase Locking (2PL)**: acquire all locks in the growing phase; release in the shrinking phase. **Strict 2PL** (hold X locks until commit) is what real DBMS implement. Produces conflict-serializable, cascadeless schedules.
5. **Deadlocks**: circular lock-wait chains; detected via **wait-for graph** cycles. Resolved by

aborting a victim; prevented by Wait-Die or Wound-Wait timestamp ordering.

6. **MVCC**: readers never block writers, writers never block readers. Each transaction reads a snapshot valid at its start time. Used by PostgreSQL (with SSI for `SERIALIZABLE`), MySQL InnoDB, Oracle.
7. **OCC**: validate at commit, not during execution — best for low-conflict, read-heavy workloads.

Review Questions

Conceptual

1. Define a **conflicting operation pair**. Using the operations $r_1(X)$, $w_1(X)$, $r_2(X)$, $w_2(X)$, list all four pairs and classify each as conflicting or non-conflicting.
2. Explain the difference between **conflict-serializability** and **view-serializability**. Why does practice rely on conflict-serializability even though view-serializability is more permissive?
3. Explain how **MVCC** allows readers and writers to coexist without blocking each other. What happens to old versions over time, and who is responsible for cleaning them up?

Applied

4. For the following schedule, identify all conflicting operations, draw the precedence graph, and determine whether the schedule is conflict-serializable. If yes, state the equivalent serial order.

$$S : w_1(A) \ r_2(A) \ r_1(B) \ w_2(B) \ c_1 \ c_2$$

5. Explain why the following schedule is **not recoverable**, and modify it minimally to make it recoverable:

$$S : w_1(X) \ r_2(X) \ c_2 \ a_1$$

6. A database uses **Strict 2PL**. Transaction T1 holds an X lock on row R7. Transaction T2 requests an S lock on R7, then T3 requests an X lock on R7. Draw the lock request queue and show what happens when T1 commits. Will T2 and T3 both proceed? In what order?

Analysis

7. Show the wait-for graph for the following deadlock scenario and identify the minimal set of victims that, when aborted, breaks all cycles:
 - T1 holds X(A), waiting for X(B)
 - T2 holds X(B), waiting for X(C)
 - T3 holds X(C), waiting for X(A)
 - T4 holds X(D), waiting for X(E)
 - T5 holds X(E), waiting for X(D)
8. In the **Timestamp Protocol**, transaction T1 (TS=5) tries to write X where $R_TS(X) = 8$. What happens? Why is this the correct behavior?

Diagram

9. Draw a timeline diagram (with two rows: T1 and T2) showing:
 - A **non-repeatable read** anomaly at READ COMMITTED level
 - How REPEATABLE READ prevents the same scenario (using MVCC or locking)

Design

10. An online banking system runs thousands of concurrent transactions per second. Most transactions are balance reads; 10% involve writes (transfers, deposits). A DBA debates between SERIALIZABLE isolation (maximum safety) and READ COMMITTED (maximum throughput).
 - Describe a specific sequence of events in which READ COMMITTED would produce an incorrect bank balance.
 - Describe whether PostgreSQL's SERIALIZABLE (SSI) or REPEATABLE READ (MVCC snapshot) would catch this.
 - What is the performance cost of each approach?

Synthesis

11. Compare **Two-Phase Locking** and **MVCC** across the following dimensions: | Dimension | 2PL | MVCC | |———|———|———| | Readers block writers? | | | Writers block readers? | | | Deadlock possible? | | | Stale read possible? | | | Write skew anomaly possible? | | | Storage overhead? | | |
 Complete the table and then justify which mechanism you would choose for: (a) a high-concurrency web app (mostly reads), (b) a financial ledger system (strict correctness required).

Notation:

- $r_i(X)$: transaction T_i reads item X

- $w_i(X)$: transaction T_i writes item X
- c_i : transaction T_i commits
- a_i : transaction T_i aborts

Serial schedule: All operations of one transaction complete before the next starts — always correct, but low concurrency:

$$S_1 : r_1(X) \ w_1(X) \ c_1 \ r_2(X) \ w_2(X) \ c_2$$

Concurrent (non-serial) schedule: Operations interleaved:

$$S_2 : r_1(X) \ r_2(X) \ w_1(X) \ w_2(X) \ c_1 \ c_2$$

Serializability

Conflict Serializability

Two operations **conflict** if they: 1. Belong to **different** transactions 2. Access the **same data item** X 3. At least **one is a write**

Conflicting operation pairs: - $r_i(X)$ and $w_j(X)$ — read-write conflict - $w_i(X)$ and $r_j(X)$ — write-read conflict - $w_i(X)$ and $w_j(X)$ — write-write conflict

A schedule S is **conflict-serializable** iff it is conflict-equivalent to some serial schedule — i.e., we can transform S to a serial schedule by swapping **non-conflicting** adjacent operations.

Precedence Graph (Serialization Graph)

Given a schedule S over transactions $T_1, \dots, T_n$:

- One node per transaction
- Add edge $T_i \rightarrow T_j$ if T_i has an operation that **conflicts with and precedes** an operation of T_j

Theorem: S is conflict-serializable iff its precedence graph is **acyclic** (no cycles).

Example:

$$S : r_1(A) \ w_2(A) \ r_1(B) \ w_2(B) \ c_1 \ c_2$$

Conflicts: - $r_1(A)$ before $w_2(A) \rightarrow$ edge $T_1 \rightarrow T_2$ - $r_1(B)$ before $w_2(B) \rightarrow$ edge $T_1 \rightarrow T_2$ (redundant)

Graph: $T_1 \rightarrow T_2$ — no cycle $\rightarrow$ **conflict-serializable** (equivalent to serial order T_1, T_2).

View Serializability

View serializability is a weaker (more permissive) notion:

Two schedules S and S' are **view-equivalent** iff for each data item X :

1. **Initial reads:** if T_i reads the initial value of X in S , it does so in S'
2. **Updated reads:** if T_i reads X written by T_j in S , it does so in S'
3. **Final writes:** if T_i performs the last write of X in S , it does so in S'

Relationship: Every conflict-serializable schedule is view-serializable. The converse is **not** always true.

Note

View serializability is NP-complete to check, so most DBMS use conflict-serializability (which is polynomial via precedence graphs).

Recoverability

A schedule is **recoverable** iff no transaction T_i commits before all transactions T_j that wrote data read by T_i have committed.

Non-recoverable example:

```
T1: w(X)
T2: r(X) ← reads T1's uncommitted write (dirty read)
T2: c     ← T2 commits
T1: a     ← T1 aborts! But T2 already committed based on T1's data → non-
recoverable
```

Cascading Rollback

If T_1 aborts and other transactions read data written by T_1 , they must also abort — cascading effect:

```
T1: w(X)           T2: r(X)           T3: r(X)
                        T1 aborts → T2 must abort → T3 must abort
```

Cascadeless Schedules

A **cascadeless schedule** ensures no transaction reads uncommitted data:

If T_j reads X written by T_i , then T_i must commit before that read

Hierarchy:

Serial $\subset$ Cascadeless $\subset$ Recoverable $\subset$ All schedules

All practical DBMS implement (at minimum) recoverable, cascadeless schedules.

Concurrency Control Protocols

2PL Variants (Review and Extension)

Variant	Rule	Guarantees
Basic 2PL	Two-phase; locks released in shrinking phase	Conflict-serializability
Strict 2PL	Hold all locks until commit/abort	Conflict-serializable + no dirty reads
Rigorous 2PL	Even shared locks held until commit	Equivalent to basic serial ordering
Conservative 2PL	Acquire all locks before starting	No deadlocks, but low throughput

Timestamp-Based Concurrency Control

Each transaction receives a **timestamp** $TS(T)$ at start. Operations are validated against timestamps:

Rules:

- $r_i(X)$: if $TS(T_i) < W_TS(X) \rightarrow T_i$ tries to read a value overwritten by a newer transaction $\rightarrow$ **abort** T_i and restart with a new timestamp
- $w_i(X)$: if $TS(T_i) < R_TS(X) \rightarrow$ abort T_i ; if $TS(T_i) < W_TS(X) \rightarrow$ skip write (Thomas Write Rule)

Advantage: No deadlocks; lock-free

Disadvantage: Starvation if a transaction is always restarted; long-running transactions may repeatedly abort

Multiversion Concurrency Control (MVCC)

Used by PostgreSQL, MySQL InnoDB, Oracle:

- Each write creates a **new version** of the data item
- Readers never block writers; writers never block readers
- Each transaction sees a **snapshot** of the database at a fixed point in time

Item X:

Version 1: value=100, created_by=T1, expires_at=T5

Version 2: value=150, created_by=T5, expires_at= ∞

- Transaction T_3 (timestamp between T1 and T5) reads Version 1
- Transaction T_7 (after T5) reads Version 2

Optimistic Concurrency Control (OCC)

Assumes conflicts are rare. Three phases per transaction:

Phase	Activity
Read	Read from DB; store in local workspace; no locks
Validate	Check if transaction's reads/writes conflict with committed transactions
Write	If valid: write changes; if invalid: abort and restart

Good for read-heavy workloads with few conflicts.

Summary: Isolation Implementations

Approach	Mechanism	Used By
Locking (2PL)	Shared/exclusive locks	SQL Server, older MySQL
MVCC	Snapshots and versions	PostgreSQL, MySQL InnoDB, Oracle
Optimistic	Validate before commit	In-memory DBs, some NoSQL
Serializable Snapshot Isolation (SSI)	MVCC + conflict detection	PostgreSQL ≥9.1, serializable level

Timestamp Protocol Example

Transactions: T1 (TS=1), T2 (TS=2)

Initial state: R_TS(X)=0, W_TS(X)=0

T1: w(X) → TS(T1)=1 ≥ R_TS(X)=0 and W_TS(X)=0 → OK; W_TS(X)=1

T2: r(X) → TS(T2)=2 ≥ W_TS(X)=1 → OK; R_TS(X)=2

T2: w(X) → TS(T2)=2 ≥ R_TS(X)=2 and W_TS(X)=1 → OK; W_TS(X)=2

T1: r(X) → TS(T1)=1 < W_TS(X)=2 → ABORT T1, restart with new TS

Summary

! Chapter 13 — Key Takeaways

- A **schedule** interleaves operations from multiple transactions; it must ensure correct results.
- **Conflict-serializability**: a schedule is correct if it's equivalent to a serial one — checked via the **precedence graph** (acyclic = serializable).
- **View serializability** is less restrictive but NP-complete to check.
- A **recoverable** schedule never commits before its dirty-read sources commit; **cascadeless** schedules prohibit dirty reads entirely.
- **Strict 2PL** (lock everything until commit) is the dominant practical protocol.
- **Timestamp ordering** avoids deadlocks but may cause starvation.
- **MVCC** allows readers and writers to not block each other — used in PostgreSQL, InnoDB.
- **OCC** (optimistic) validates at commit time — good for low-conflict workloads.

Review Questions

1. Define a **schedule**. What is the difference between a serial and a concurrent schedule?
2. For the schedule $S : r_1(A) w_2(A) w_1(A) r_2(B) w_2(B) c_1 c_2$:
 - Identify all conflicting operations
 - Draw the precedence graph
 - Is S conflict-serializable? If so, what is the equivalent serial order?
3. Explain the difference between **conflict-serializable** and **view-serializable** schedules.
4. What is a **cascading rollback**? How does Strict 2PL prevent it?
5. Explain how **MVCC** allows readers and writers to not block each other.
6. When would you choose **optimistic concurrency control** over locking?

Part VIII: NoSQL & Modern Databases

Introduction to NoSQL Databases

Chapter 14

Beyond the Relational Model

Chapters 5–13 showed how relational databases guarantee ACID transactions, enforce declarative constraints, and answer ad-hoc queries through SQL. These properties assume a clean, stable schema that fits well on one server. From roughly 2005 onward, companies like Google, Amazon, and Facebook discovered that these assumptions break down at internet scale: billions of users, petabytes of data, and millisecond latency requirements across globally distributed data centres.

This chapter introduces the *NoSQL* family of databases that emerged to address those scale constraints.

Motivation: Challenges at Internet Scale

Challenge	Relational Model's Weakness
Horizontal scale-out	Adding servers is hard — a single RDBMS enforces ACID across all rows, requiring distributed coordination
Schema evolution	ALTER TABLE on billions of rows is expensive; zero-downtime schema changes require elaborate migrations
Impedance mismatch	Object-oriented and document-centric data don't map cleanly to flat rows and columns
Write throughput	Millions of inserts/sec saturate locks and WAL on a single RDBMS server
Diverse data shapes	Social graphs, JSON events, time-series sensor readings fit poorly in normalized tables
Cost	Enterprise RDBMS licences (Oracle, SQL Server) are prohibitive at hyperscale

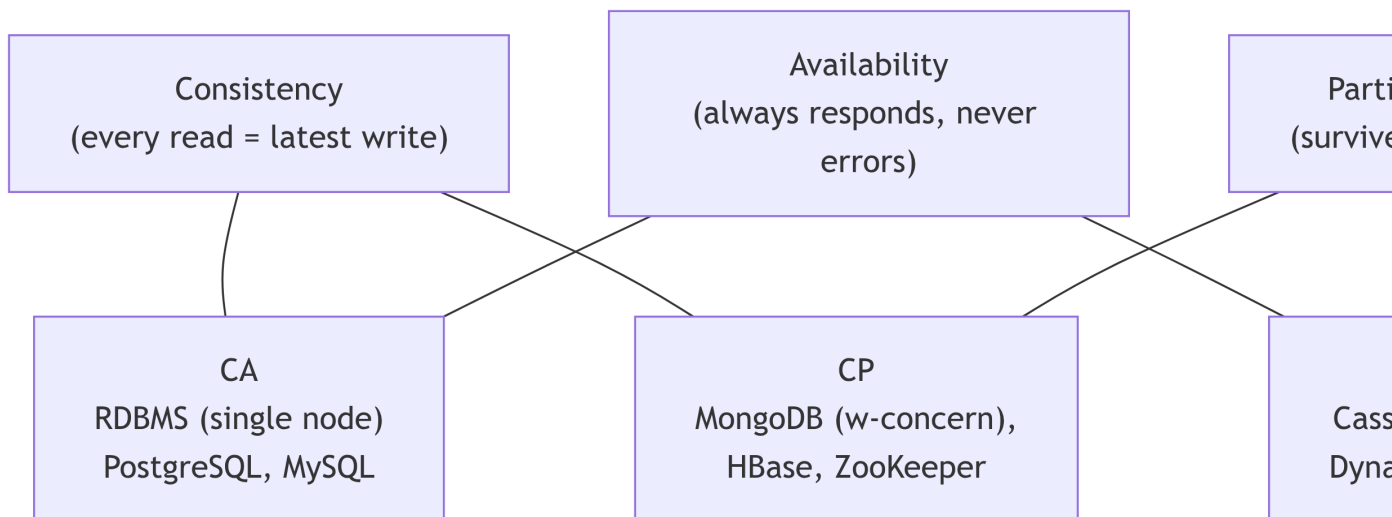
NoSQL (Not Only SQL) systems trade some ACID guarantees for **horizontal scalability**, **schema flexibility**, and **high availability under network partitions**.

The CAP Theorem

Proposed by Eric Brewer (PODC 2000) and formally proved by Gilbert & Lynch (2002):

A distributed data store can simultaneously provide at most **two** of the following three guarantees.

Guarantee	Symbol	Definition
Consistency	C	Every read receives the most recent write, or an error
Availability	A	Every request receives a non-error response (may not be the latest data)
Partition Tolerance	P	The system continues operating even when nodes cannot communicate



! Important

In real distributed systems, network partitions are unavoidable. Therefore, the practical design choice is always between **CP** and **AP** — not whether to tolerate partitions.

Mode	Behaviour under partition	Examples
CP	Returns an error rather than stale data	MongoDB (majority write concern), HBase
AP	Returns potentially stale data, never an error	Cassandra, DynamoDB (default), CouchDB

BASE vs. ACID

NoSQL systems replace ACID with **BASE**:

Property	ACID	BASE
Atomicity / Basically Available	Transaction succeeds or fails atomically	System stays available even if some replicas are stale
Consistency / Soft State	DB moves between only valid states	State may change over time even without new input (as replicas sync)
Isolation / Eventually Consistent	Concurrent transactions see consistent snapshots	Replicas converge to agreement eventually — reads may be stale in the interim
Durability	Committed data survives failure	Usually provided, but may have weaker guarantees

Eventual consistency means: given no new updates, all replicas will eventually return the same value. The window of inconsistency is typically milliseconds to seconds.

The Four NoSQL Data Models

Key-Value Stores

The simplest model: a distributed hash map. Each key maps to an opaque **value** (blob, string, JSON, binary).

key: "user:123" → {"name": "Ali", "score": 9820}
key: "session:tok_abc" → <binary token>
key: "counter:page_views" → 42571

Redis (Core Commands)

```
-- String
SET user:123 '{"name":"Ali","score":9820}'
GET user:123
EXPIRE user:123 3600          -- expire in 1 hour

-- Hash (object fields)
HSET user:123 name "Ali" score 9820
HGET user:123 name
HGETALL user:123

-- List (push/pop queue)
LPUSH notifications:user:123 "You have a new message"
LRANGE notifications:user:123 0 9    -- last 10 notifications

-- Sorted set (leaderboard)
ZADD leaderboard 9820 "Ali"
ZADD leaderboard 8550 "Sara"
ZREVRANGE leaderboard 0 9 WITHSCORES -- top 10
```

Use Cases

Pattern	Redis Command
Session cache	SET / GET / EXPIRE
Rate limiting	INCR / EXPIRE per user per window
Leaderboard	ZADD / ZREVRANGE sorted set
Pub/Sub messaging	PUBLISH / SUBSCRIBE
Distributed lock	SET key value NX EX 30 (SETNX + expire)

Document Stores

Each record is a **self-describing document** (JSON/BSON). Documents in the same collection may have different fields.

```
{
  "_id": "emp001",
  "name": { "first": "Sara", "last": "Hassan" },
  "dept": "Engineering",
  "skills": ["Python", "MongoDB", "Docker"],
  "salary": 85000,
  "address": { "city": "Cairo", "country": "Egypt" }
}
```

Feature	Detail
Schema	Flexible — each document defines its own structure
Queries	Rich filter on any field, embedded documents, arrays
Relationships	Either embed sub-documents, or reference by <code>_id</code>
Examples	MongoDB, CouchDB, Firestore, Couchbase
Best for	Catalogs, user profiles, content management, event logging

Column-Family Stores (Wide-Column)

Data is stored in rows where each row can have a different set of columns within a **column family**.

Row Key	info CF	metrics CF
"user1"	name:"Ali" city:"Cairo"	views:1200 shares:45
"user2"	name:"Sara"	views:800
"user3"	name:"Omar" dept:"Sales"	(no metrics yet)

- A **column family** groups columns stored together on disk — enables fast column-subset reads
- Designed for **massive write throughput** and **time-series** data
- Schema is defined per column family but columns within a family are flexible

Cassandra CQL

```
-- Create keyspace (defines replication strategy)
CREATE KEYSPACE company
  WITH replication = {'class':'SimpleStrategy', 'replication_factor':3};
```



```
-- Table design: optimized for a specific query (denormalize)
-- Query: "find all employees in a department ordered by hire_date DESC"
CREATE TABLE employees_by_dept (
    dept_id    UUID,
    hire_date  TIMESTAMP,
    emp_id     UUID,
    name       TEXT,
    salary     DECIMAL,
    PRIMARY KEY ((dept_id), hire_date, emp_id)
) WITH CLUSTERING ORDER BY (hire_date DESC);

INSERT INTO employees_by_dept (dept_id, hire_date, emp_id, name, salary)
VALUES (uuid(), '2022-03-15', uuid(), 'Ali Hassan', 85000.00);

SELECT * FROM employees_by_dept
WHERE dept_id = ? AND hire_date > '2022-01-01';
```

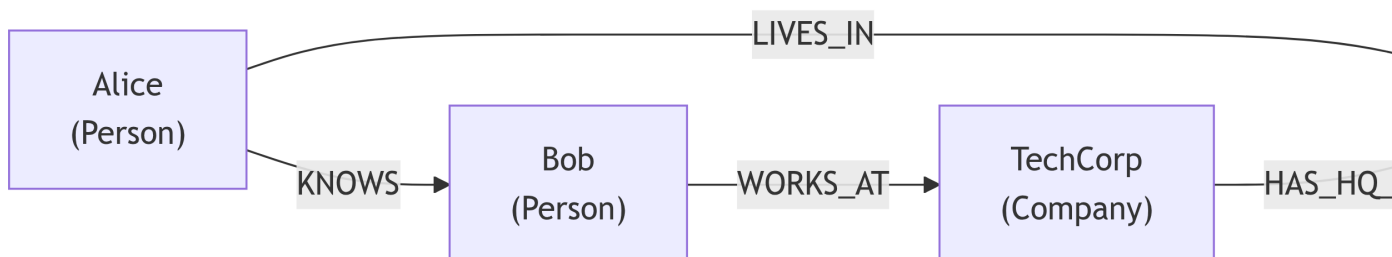
Design Principles

Cassandra forces you to design tables around access patterns, not data relationships.

RDBMS Principle	Cassandra Equivalent
One table per entity	Create one table per query
Normalize to avoid redundancy	Denormalize; redundancy is fine
Use JOINS	No JOINS; embed data you need together
Primary key = row identity	Partition key → which node; clustering key → sort order within partition

Graph Databases

Data is stored as **nodes** (entities) and **edges** (relationships), each with properties.



Neo4j Cypher

```
-- Create nodes
CREATE (:Person {name:'Alice', age:30})
CREATE (:Person {name:'Bob', age:25})
CREATE (:Company {name:'TechCorp'})

-- Create relationship
MATCH (a:Person {name:'Alice'}), (b:Person {name:'Bob'})
CREATE (a)-[:KNOWS {since:2020}]->(b)

-- Query: friends of Alice
MATCH (a:Person {name:'Alice'})-[:KNOWS]->(friend)
RETURN friend.name

-- Query: who works at TechCorp, and their friends?
MATCH (p:Person)-[:WORKS_AT]->(:Company {name:'TechCorp'})
OPTIONAL MATCH (p)-[:KNOWS]->(friend)
RETURN p.name, collect(friend.name) AS friends

-- Find shortest path
MATCH path = shortestPath(
  (a:Person {name:'Alice'})-[*]-(b:Person {name:'Omar'})
)
RETURN path
```

Use Cases

Use Case	Why Graph?
Social networks	Friend-of-friend traversal is O(relationships) not O(rows)
Recommendation engines	“People who liked X also liked Y” — relationship traversal
Fraud detection	Find circular transaction paths or shared identity attributes
Knowledge graphs	Multi-hop entity relationships (concept A → concept B → concept C)

NoSQL vs. RDBMS: Full Comparison

Dimension	RDBMS	Document (MongoDB)	Column-Family (Cassandra)	Graph (Neo4j)
Data model	Tables, fixed schema	JSON documents, flexible schema	Rows + column families	Nodes + edges + properties
Query language	SQL (standard)	MQL (MongoDB Query Language)	CQL (SQL-like)	Cypher
Schema	Rigid (DDL)	Flexible (schema-optional)	Semi-rigid (column families defined)	Node/edge types optional
ACID	Full (1 server or distributed)	Multi-document ACID (v4+, replica set)	Lightweight transactions (CAS only)	Full ACID (single server)
Scaling	Vertical + limited sharding	Horizontal sharding built-in	Masterless horizontal scale-out	Vertical primarily
Joins	Native	No joins (embed or \$lookup)	No joins (denormalize)	Relationship traversal
Consistency	Strong	Tunable (from eventual to majority)	Tunable (ONE, QUORUM, ALL)	Strong
Best for	Transactions, reporting, ERP	Catalogs, profiles, CMS	Time-series, IoT, analytics	Social, recommendations, fraud

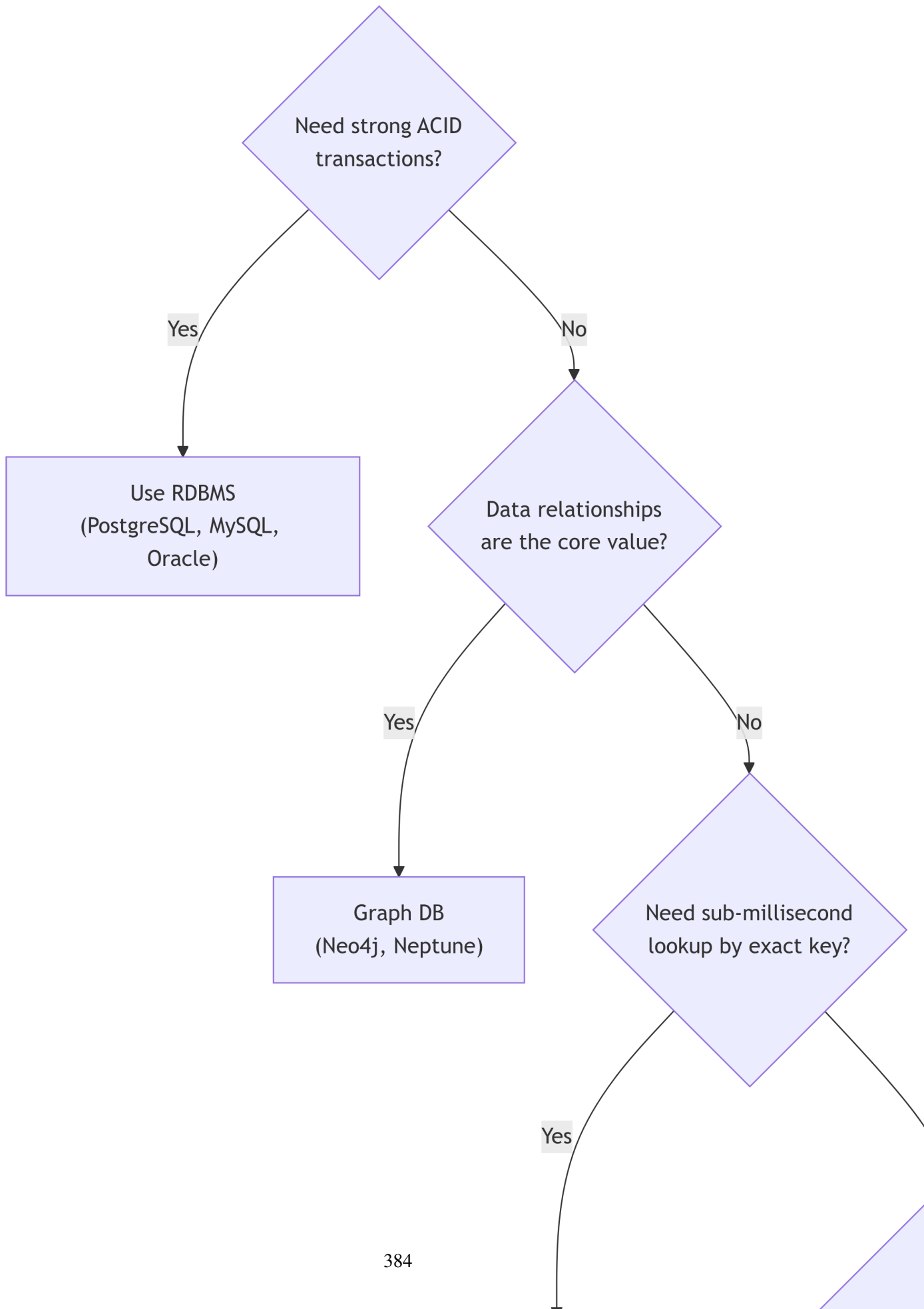
💡 Polyglot Persistence

Modern architectures combine multiple database types for different workloads in the same system:

- **PostgreSQL** → orders, payments (ACID required)
- **MongoDB** → product catalog (flexible schema)
- **Redis** → session cache, rate limiting (sub-ms reads)
- **Cassandra** → clickstream / IoT time-series (huge write volume)

- **Neo4j** → recommendation graph (relationship traversal)
-

When to Choose NoSQL



Chapter Summary

! Key Takeaways — Chapter 14

1. **RDBMS limitations at scale:** horizontal scaling is hard; schema changes are expensive; object/document data has impedance mismatch.
2. **CAP Theorem:** distributed systems can guarantee at most 2 of Consistency, Availability, Partition Tolerance. Since partitions are unavoidable, the real choice is CP vs AP.
3. **BASE** replaces ACID: Basically Available, Soft State, Eventually Consistent.
4. **Four NoSQL models:**
 - **Key-Value** (Redis): lookup by exact key; sessions, caching, counters
 - **Document** (MongoDB): flexible JSON; catalogs, profiles, CMS
 - **Column-Family** (Cassandra): denormalized by query; IoT, time-series, analytics
 - **Graph** (Neo4j): nodes + edges; social, fraud, recommendations
5. **Polyglot persistence:** use each database type where it excels.
6. The choice between RDBMS and NoSQL depends on consistency requirements, query patterns, data shape, and scale targets — not fashion.

Review Questions

Conceptual

1. In the CAP theorem, explain why **Partition Tolerance cannot be avoided** in a distributed system. Given that P must be guaranteed, what is the actual choice a distributed database designer makes?
2. Distinguish between **BASE** and **ACID**. Give a concrete example (e.g., a bank balance vs. a Twitter like-count) where BASE is acceptable and one where it is not.
3. Why do column-family stores like Cassandra require you to **design tables around query patterns** rather than around entity relationships? What is the consequence of violating this rule?

Applied

4. A social media company needs to store posts. Each post can have 0–100 tags, 0–N embedded comments, and optional media attachments (image URL, video URL). Some posts are text-only, others are media-only.
 - (a) Model this in a relational schema (normalized).
 - (b) Model this as a MongoDB document.
 - (c) Which representation makes more queries simpler? Which allows easier ad-hoc analysis?
5. A leaderboard for a mobile game needs to:
 - Add/update a user's score in real time (millions of events/minute)
 - Return the top-100 players in <5 ms
 - Expire scores older than 24 hours

Explain how **Redis Sorted Sets** (ZADD, ZREVRANGE, EXPIRE) implement this requirement. Could a relational database satisfy all three constraints? What would break first?

6. Design a Cassandra table for the following access pattern: *“Find all sensor readings for sensor X in the last 24 hours, ordered by timestamp descending.”*
 - Choose an appropriate partition key and clustering key
 - Explain how the partition key affects which Cassandra node stores the data

Analysis

7. For each system below, state whether it is **CP** or **AP** under the CAP theorem, and explain the practical consequence (what happens to the user during a network partition?):
 - Cassandra (default consistency level ONE)
 - MongoDB with `writeConcern: {w: "majority"}`
 - A single-server PostgreSQL instance
8. **Fraud detection** must follow chains of transactions: *“Find all accounts that received money from account X, directly or through at most 3 intermediary accounts.”*
 - (a) Write this query in SQL (with recursive CTE).
 - (b) Write this query in Neo4j Cypher.
 - (c) Which approach is more efficient as the graph gets deeper (e.g., 10 hops)? Explain why.

Diagram

9. Draw a **Mermaid graph** (or ASCII diagram) showing the polyglot persistence architecture for an e-commerce platform that uses: PostgreSQL for orders, MongoDB for product catalog, Redis for cart, and Cassandra for clickstream analytics. Label each database with its NoSQL type and the data it owns.

Design

10. A healthcare company needs to store patient electronic health records (EHR). Each EHR contains: a fixed core (name, dob, blood type) plus a variable set of diagnoses (ICD-10 codes + dates), medications (drug name, dosage, frequency + start/end dates), and procedure records.
 - Propose a MongoDB document schema for a single patient
 - Identify which fields belong in a relational DB instead (and why)
 - What would you do differently if EHRs must be queried with complex multi-field filters for population health analytics?

Synthesis

11. A startup is building a real-time event-sourcing system. Every user action triggers an event (JSON object with type, payload, user_id, timestamp). Requirements:
 - Ingest 500,000 events/second
 - Query: recent 100 events for a user
 - Query: aggregate counts by event type per hour
 - Replay all events from a given timestamp (for audit/replay)

Propose a combination of NoSQL systems that handles all four requirements. For each requirement, identify which system handles it and why the others fall short.

12. Compare **MongoDB** and **Cassandra** across: (a) schema flexibility, (b) consistency guarantees, (c) primary query patterns, (d) horizontal scaling mechanism, (e) multi-document transaction support. Conclude with: *given a ride-sharing app that needs to store trips, drivers, and GPS coordinates streaming in at 50,000 writes/sec, which would you choose and why?

MongoDB — Document Databases

Chapter 15

From Concepts to Queries

Chapter 14 introduced the four NoSQL data models and positioned document stores as the right choice for flexible-schema, nested-data workloads. This chapter drills into **MongoDB** — the world’s most popular document database — covering the full lifecycle: document structure, CRUD operations, the aggregation pipeline, indexing, schema design patterns, and multi-document ACID transactions.

MongoDB Architecture

Core Concepts

MongoDB Term	Description	RDBMS Equivalent
Database	Container for collections	Database
Collection	Group of documents	Table
Document	JSON-like record	Row
Field	Key-value pair in a document	Column
_id	Auto-generated unique identifier	Primary Key
Index	B-tree or other structure on a field	Index
Aggregation Pipeline	Multi-stage data transformation	JOINS + GROUP BY + HAVING

BSON — Binary JSON

MongoDB stores documents in **BSON** (Binary JSON): - Extends JSON with additional types: `ObjectId`, `Date`, `BinData`, `NumberDecimal`, `NumberLong`, `Timestamp` - Efficient for encoding/decoding; supports rich type system

```
{
  "_id":      { "$oid": "507f191e810c19729de860ea" },
  "name":     "Ahmad Hassan",
  "age":      28,
  "gpa":      3.75,
  "enrolled": { "$date": "2024-09-01T00:00:00Z" },
  "courses": ["DB Systems", "Networks", "OS"],
  "address": {
    "city":    "Alexandria",
    "country": "Egypt"
  }
}
```

Getting Started with MongoDB Shell

```
// Show all databases
show dbs

// Switch to (or create) a database
use university_db

// Show all collections
show collections

// Check current database
db.getName()
```

CRUD Operations

Create — Inserting Documents

```
// Insert one document
db.students.insertOne({
  name: "Sara Ali",
```

```

    age: 21,
    major: "Computer Engineering",
    gpa: 3.8,
    courses: ["Databases", "Algorithms"],
    address: { city: "Cairo", country: "Egypt" }
  })

// Insert many documents
db.students.insertMany([
  { name: "Omar Hassan", age: 22, major: "CS", gpa: 3.5 },
  { name: "Layla Ahmed", age: 20, major: "IT", gpa: 3.9 },
  { name: "Khaled Nour", age: 23, major: "CE", gpa: 2.7 }
])

```

Read — Querying Documents

```

// Find all documents in collection
db.students.find()

// Find with a filter (equality)
db.students.find({ major: "Computer Engineering" })

// Find with multiple conditions (implicit AND)
db.students.find({ major: "CS", gpa: { $gt: 3.5 } })

// Projection: include only name and gpa fields (1=include, 0=exclude)
db.students.find(
  { major: "CS" },
  { name: 1, gpa: 1, _id: 0 }
)

// Find one document
db.students.findOne({ name: "Sara Ali" })

// Count matching documents
db.students.countDocuments({ gpa: { $gte: 3.5 } })

```

Comparison Query Operators

Operator	Meaning	SQL Equivalent
\$eq	Equal to	= value
\$ne	Not equal	<> value
\$gt	Greater than	> value
\$gte	Greater or equal	>= value
\$lt	Less than	< value
\$lte	Less or equal	<= value
\$in	In a list	IN (a, b, c)
\$nin	Not in a list	NOT IN (a, b, c)

Logical Query Operators

```
// OR: find CS or CE students
db.students.find({
  $or: [ { major: "CS" }, { major: "CE" } ]
})

// AND (explicit – use when same field needs multiple tests)
db.students.find({
  $and: [
    { gpa: { $gte: 3.0 } },
    { gpa: { $lt: 3.5 } }
  ]
})

// NOT
db.students.find({ gpa: { $not: { $lt: 3.0 } } })
```

Querying Arrays and Nested Documents

```
// Array contains a value
db.students.find({ courses: "Databases" })

// All elements in array match (exact match with $all)
db.students.find({ courses: { $all: ["Databases", "Algorithms"] } })

// Array size
db.students.find({ courses: { $size: 2 } })
```

```
// Nested document field (dot notation)
db.students.find( { "address.city": "Cairo" } )
```

Update — Modifying Documents

```
// Update one document
db.students.updateOne(
  { name: "Sara Ali" },           // filter
  { $set: { gpa: 3.9, year: 3 } } // update operator
)

// Update many documents
db.students.updateMany(
  { major: "CS" },
  { $inc: { age: 0 } }           // $inc increments a field
)

// Common update operators
// $set    – set a field to a value
// $unset  – remove a field
// $inc    – increment a numeric field
// $push   – append to an array
// $pull   – remove from an array
// $addToSet – add to array only if not present

// Add a course to a student's courses array
db.students.updateOne(
  { name: "Sara Ali" },
  { $push: { courses: "Distributed Systems" } }
)

// Replace entire document (use with caution)
db.students.replaceOne(
  { name: "Sara Ali" },
  { name: "Sara Ali Mohamed", age: 21, major: "CE", gpa: 3.9 }
)

// Upsert: update if exists, insert if not
db.students.updateOne(
  { name: "New Student" },
  { $set: { gpa: 3.0 } },
  { upsert: true }
```

```
{ upsert: true }
)
```

Delete — Removing Documents

```
// Delete one document
db.students.deleteOne({ name: "Khaled Nour" })

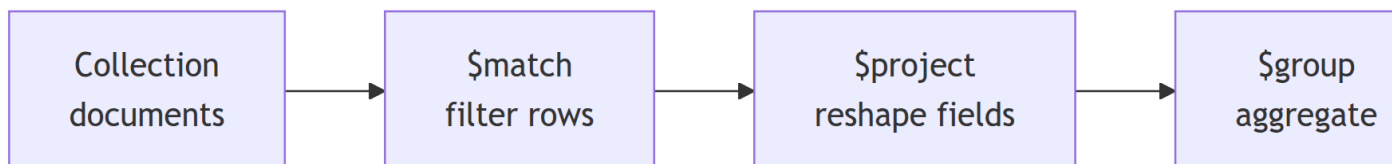
// Delete many documents
db.students.deleteMany({ gpa: { $lt: 2.0 } })

// Delete all documents in a collection (keep the collection)
db.students.deleteMany({})

// Drop the entire collection
db.students.drop()
```

Aggregation Pipeline

The **aggregation pipeline** is MongoDB's most powerful feature — a sequence of **stages** that transform documents, similar to a Unix pipe:



Each stage receives the output of the previous stage; stages can be reordered, repeated, or omitted.

Common Pipeline Stages

Stage	Purpose	SQL Equivalent
\$match	Filter documents	WHERE
\$project	Select / reshape fields	SELECT
\$group	Group + aggregate	GROUP BY + aggregates
\$sort	Sort results	ORDER BY
\$limit	Limit result count	LIMIT
\$skip	Skip N documents	OFFSET
\$lookup	Join with another collection	JOIN
\$unwind	Deconstruct an array field	Normalize 1→N

Stage	Purpose	SQL Equivalent
\$addFields	Add computed fields	Computed columns
\$count	Count documents in this stage	COUNT(*)

Aggregation Examples

// Average GPA per major (GROUP BY equivalent)

```
db.students.aggregate([
  {
    $group: {
      _id: "$major",
      avg_gpa: { $avg: "$gpa" },
      count: { $sum: 1 },
      max_gpa: { $max: "$gpa" }
    }
  },
  { $sort: { avg_gpa: -1 } }
])
```

// Full pipeline: filter → project → group → sort → limit

```
db.students.aggregate([
  { $match: { gpa: { $gte: 3.0 } } },           // Stage 1: filter
  { $addFields: {
    gpa_rounded: { $round: ["$gpa", 1] }       // Stage 2: add field
  } },
  { $group: {
    _id: "$major",
    total: { $sum: 1 },
    avg_gpa: { $avg: "$gpa_rounded" }
  } },                                           // Stage 3: group
  { $match: { total: { $gt: 5 } } },           // Stage 4: HAVING
  { $sort: { avg_gpa: -1 } },                  // Stage 5: sort
  { $limit: 5 }                                // Stage 6: top 5
])
```

\$lookup — Join Between Collections

// "Join" orders with customers (like LEFT JOIN)

```
db.orders.aggregate([
  {
```



```

    $lookup: {
      from:          "customers",      // foreign collection
      localField:    "customer_id",    // field in orders
      foreignField:  "_id",            // field in customers
      as:            "customer_info"   // output array field
    }
  },
  { $unwind: "$customer_info" },      // flatten array (INNER JOIN effect)
  { $project: {
    order_date: 1,
    total:      1,
    "customer_info.name": 1,
    "customer_info.email": 1
  }}
])

```

Warning

\$lookup is expensive — it performs a cross-collection join at query time. If you frequently join the same collections, consider **embedding** related data instead.

\$unwind — Deconstructing Arrays

```

// Count enrolments per course
db.students.aggregate([
  { $unwind: "$courses" },          // one doc per (student, course) pair
  { $group: {
    _id: "$courses",
    enrolled_count: { $sum: 1 }
  }},
  { $sort: { enrolled_count: -1 } }
])

```

Indexing in MongoDB

Creating Indexes

```

```javascript
// Single-field ascending index
db.students.createIndex({ gpa: 1 }) // 1=ascending, -1=descending

// Compound index

```

```
db.students.createIndex({ major: 1, gpa: -1 })

// Unique index (like UNIQUE in SQL)
db.students.createIndex({ email: 1 }, { unique: true })

// Text index (for full-text search)
db.students.createIndex({ name: "text", major: "text" })

// TTL index (auto-delete after N seconds – for session/log data)
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

## Viewing and Removing Indexes

```
// List all indexes on a collection
db.students.getIndexes()

// Drop a specific index
db.students.dropIndex({ gpa: 1 })

// Drop all indexes except _id
db.students.dropIndexes()
```

## EXPLAIN in MongoDB

```
db.students.find({ gpa: { $gte: 3.5 } }).explain("executionStats")
// Shows: IXSCAN (index scan) vs. COLLSCAN (full collection scan)
// winningPlan.stage == "IXSCAN" → good!
```

## Schema Design Patterns

Unlike SQL, MongoDB documents can **embed** related data. The key design decision: **embed vs. reference**.

### Embedding (Denormalization)

```
{
 "_id": "order_001",
 "customer": {
 "name": "Ahmed Ali",
 "email": "ahmed@example.com"
```

```

},
"items": [
 { "product": "Laptop", "qty": 1, "price": 1200 },
 { "product": "Mouse", "qty": 2, "price": 25 }
],
"total": 1250,
"date": "2026-04-06"
}

```

**Embed when:** - Data is always accessed together (one-to-one or one-to-few) - The embedded data doesn't change frequently - You need the best read performance (one document = one disk read)

## Referencing (Normalization)

```

// orders collection
{ "_id": "order_001", "customer_id": { "$oid": "abc123" }, "total": 1250 }

// customers collection
{ "_id": { "$oid": "abc123" }, "name": "Ahmed Ali", "email": "ahmed@example.com" }

```

**Reference when:** - Data is shared by many documents (one-to-many or many-to-many) - The referenced data changes frequently - Document size would grow unboundedly (e.g., a product with thousands of reviews)

## Common Patterns

Pattern	Description	Example
<b>Bucket</b>	Group data by time period	IoT sensor readings per hour
<b>Subset</b>	Keep only most recent/popular items	Last 10 reviews in product
	embedded	doc
<b>Computed</b>	Cache computed values to avoid repeated aggregation	total_orders field on customer
<b>Extended Reference</b>	Copy frequently-read fields from referenced doc	Embed product_name in order items

## MongoDB vs. SQL — Side-by-Side

```

// SQL: SELECT * FROM students WHERE gpa > 3.5 ORDER BY name ASC LIMIT 10
db.students.find(
 { gpa: { $gt: 3.5 } }
)

```

```

).sort({ name: 1 }).limit(10)

// SQL: SELECT major, COUNT(*), AVG(gpa) FROM students GROUP BY major HAVING COUNT(*) > 5
db.students.aggregate([
 { $group: { _id: "$major", count: { $sum: 1 }, avg_gpa: { $avg: "$gpa" } } },
 { $match: { count: { $gt: 5 } } }
])

// SQL: UPDATE students SET gpa = 4.0 WHERE name = 'Sara'
db.students.updateOne({ name: "Sara Ali" }, { $set: { gpa: 4.0 } })

// SQL: DELETE FROM students WHERE gpa < 2.0
db.students.deleteMany({ gpa: { $lt: 2.0 } })

```

## Transactions in MongoDB

Since MongoDB 4.0, **multi-document ACID transactions** are supported:

```

const session = client.startSession()
session.startTransaction()
try {
 await db.collection("accounts").updateOne(
 { _id: "A" }, { $inc: { balance: -500 } }, { session }
)
 await db.collection("accounts").updateOne(
 { _id: "B" }, { $inc: { balance: +500 } }, { session }
)
 await session.commitTransaction()
} catch (err) {
 await session.abortTransaction()
} finally {
 session.endSession()
}

```

## Chapter Summary

### ! Key Takeaways — Chapter 15

1. MongoDB stores data as **BSON documents** in **collections**; every document has a unique `_id`.
2. **CRUD commands**: `insertOne/insertMany`, `find` (with query operators: `$gt`, `$in`, `$and`, `$elemMatch`, dot-notation), `updateOne/updateMany` (`$set`, `$inc`, `$push`, `$pull`, `$addToSet`), `deleteOne/deleteMany`.
3. The **aggregation pipeline** is the core analytics tool: a sequence of stages (`$match`, `$group`, `$project`, `$sort`, `$lookup`, `$unwind`, `$addFields`) transforms documents in order.
4. **Indexing** uses B-trees. `createIndex()` creates single, compound, unique, text, or TTL indexes. Always verify with `.explain("executionStats")` and check for `IXSCAN` vs `COLLSCAN`.
5. **Schema design**: choose **embedding** when data is always read together (fast reads); choose **referencing** when data is shared or grows unboundedly (avoids large documents).
6. **Multi-document ACID transactions** (MongoDB 4.0+) require a replica set and are used for critical operations that span multiple collections.
7. `$lookup` is MongoDB's JOIN — use it when referencing, but avoid it in hot paths; embed frequently co-accessed data for best performance.

## Review Questions

### Conceptual

1. Explain the difference between a MongoDB **collection** and a **document**. How do these correspond to (and differ from) a relational table and a row? Specifically, what constraint on rows does MongoDB collections **not** enforce?
2. Why does MongoDB allow documents in the same collection to have different fields? Give one scenario where this flexibility is genuinely useful and one where it causes problems.
3. Explain when you would use a **TTL index** in MongoDB. What underlying piece of MongoDB infrastructure removes the expired documents? Could you replicate this behaviour with a relational DB? How?

## Applied

4. Write MongoDB JavaScript commands to:

- Find all employees with salary between 50,000 and 80,000 whose department is "Research", returning only name, salary, and department (omit `_id`).
- Find employees whose `skills` array contains **both** "Python" and "SQL".
- Find employees who have at least one project with status: "active" and hours: `{ $gt: 10 }`. (Hint: use `$elemMatch`.)

5. Write a MongoDB update command to:

- Add "Docker" to the `skills` array of employee "Sara Ali" **only if it isn't already there**.
- Remove "COBOL" from the `skills` array of **all** employees who have it.
- Increment the salary of all employees in department "Research" by 5,000.

6. Write an aggregation pipeline that produces a report: *“For each department, show the department name, number of employees, average salary, and the name of the highest-paid employee.”*

The pipeline should:

- Filter out employees with salary null or missing
- Group by department
- Use `$first`, `$avg`, `$sum`, `$max` accumulators
- Sort by average salary descending

## Analysis

7. Analyse the following schema decision for a blog platform:

### Option A — Embedded:

```
{ "_id": "post1", "title": "...", "comments": [
 { "author": "Ali", "text": "Great!" },
 { "author": "Sara", "text": "Nice!" }
]}
```

### Option B — Referenced:

```
// posts: { "_id": "post1", "title": "..." }
// comments: { "_id": "c1", "post_id": "post1", "author": "Ali", "text": "Great!" }
```

Under which conditions does Option A outperform Option B? Under which conditions does Option B become necessary? What happens to Option A when a post has 50,000 comments?

8. Explain MongoDB's **\$lookup** stage. Write a \$lookup aggregation that joins an orders collection with a products collection to compute the total revenue per product name (product name comes from the products collection; quantity and price come from the order line items array).

## Diagram

9. Draw a **Mermaid flowchart** (or describe) the five stages of an aggregation pipeline that answers: *"Find the top 5 cities by total order value, for orders placed in 2025, where at least 3 items were ordered."* Label each stage with its MongoDB stage name and what transformation it performs.

## Design

10. A ride-sharing app stores trips. Each trip has:
  - A fixed set of fields: `_id`, `driver_id`, `rider_id`, `status` (active/completed/cancelled), `start_time`, `end_time`, `fare`
  - A route field with an array of GPS coordinates (can have hundreds of entries)
  - A rating object: `{driver: 4.5, rider: 4.0}` (optional, added after completion)
  - (a) Write the MongoDB document structure for a completed trip.
  - (b) Should the GPS coordinates be embedded or referenced? Justify using document size limits.
  - (c) Which indexes would you create to support: (i) finding all active trips for a driver, (ii) finding all trips in the last 7 days, (iii) full-text search on a notes field?

## Synthesis

11. Translate the following normalized SQL schema and query into a MongoDB document design and equivalent aggregation pipeline:

### SQL Schema:

```
Employee(Ssn PK, Name, Salary, Dno FK→Department(Dnumber))
Department(Dnumber PK, Dname)
Works_On(Essn FK→Employee(Ssn), Pno FK→Project(Pnumber), Hours)
Project(Pnumber PK, Pname, Plocation)
```

### SQL Query:

```
SELECT E.Name, D.Dname, SUM(W.Hours) AS total_hours
FROM Employee E
JOIN Department D ON E.Dno = D.Dnumber
JOIN Works_On W ON E.Ssn = W.Essn
GROUP BY E.Name, D.Dname
```

```
HAVING total_hours > 20
ORDER BY total_hours DESC;
```

- Propose **two** possible MongoDB document designs (embedding vs referencing) for this schema.
  - For each design, write the aggregation pipeline to produce the same output.
  - Which design is more efficient for this specific query? Why?
12. A DBA argues: “*MongoDB supports ACID transactions now, so we can use it for financial systems instead of PostgreSQL.*” Write a 3-paragraph response that:
- Acknowledges what MongoDB ACID transactions do support
  - Identifies the limitations compared to a relational DBMS
  - Concludes with a clear recommendation



# Database APIs and Connectivity

Appendix A

# Database APIs and Connectivity

## §A.0 Connecting Applications to Databases

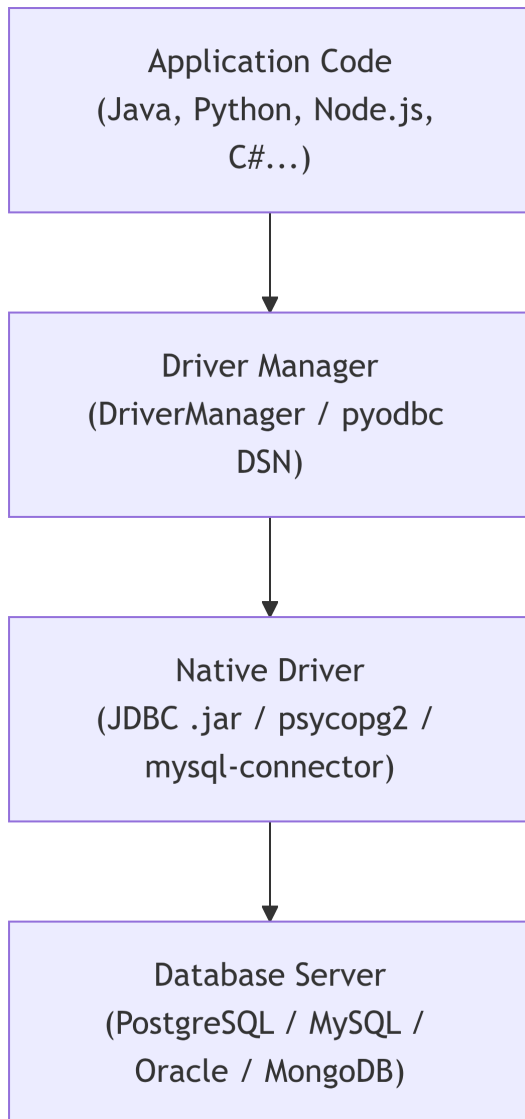
Chapters 7–8 taught SQL; Chapters 14–15 taught MongoDB query syntax. In production, however, no query runs in isolation inside a database shell. Every real application — a web server, a mobile backend, an analytics job — must **connect to a database, issue queries, handle errors, and close connections** through a programmatic API.

This appendix covers the standard layers in that connectivity stack: drivers (JDBC/ODBC), Python DB-API libraries, REST APIs, connection pooling, and ORMs.

---

## A.1 Database Connectivity Standards

### The Connectivity Stack



#### ODBC (Open Database Connectivity)

- Microsoft standard (1992); language-neutral C interface
- Available on Windows, Linux (unixODBC), macOS
- Configure a **DSN** (Data Source Name) in the OS
- Used from C/C++, Python (pyodbc), R, Excel

#### JDBC (Java Database Connectivity)

- Sun/Oracle standard for Java (1997)
- All major DBs ship JDBC drivers (.jar files)
- 4 driver types; Type 4 (pure Java TCP) is the modern standard
- URL format: `jdbc:postgresql://host:5432/dbname`

## A.2 JDBC — Java Database Connectivity

### Connection Lifecycle

```
import java.sql.*;

// 1. Load the driver (automatic in JDBC 4.0+)
// Class.forName("org.postgresql.Driver"); // no longer needed

// 2. Establish connection
String url = "jdbc:postgresql://localhost:5432/university";
String user = "db_user";
String pass = System.getenv("DB_PASS"); // NEVER hardcode passwords!

try (Connection conn = DriverManager.getConnection(url, user, pass)) {

 // 3. Create a statement
 Statement stmt = conn.createStatement();
 ResultSet rs = stmt.executeQuery("SELECT name, gpa FROM students");

 // 4. Iterate the result set
 while (rs.next()) {
 String name = rs.getString("name");
 double gpa = rs.getDouble("gpa");
 System.out.printf("%-20s %.2f\n", name, gpa);
 }

} catch (SQLException e) {
 e.printStackTrace();
}

// Connection auto-closed by try-with-resources
```

### PreparedStatement — Preventing SQL Injection

#### SQL Injection Risk

**Never** concatenate user input into SQL strings!

```
// VULNERABLE — DO NOT USE
String query = "SELECT * FROM students WHERE name = '" + input + "'";
// An attacker can input: ' OR '1'='1 — returns all rows
```

```
// SAFE: PreparedStatement uses parameterised queries
String sql = "SELECT * FROM students WHERE major = ? AND gpa > ?";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
 pstmt.setString(1, majorInput);
 pstmt.setDouble(2, minGpa);
 ResultSet rs = pstmt.executeQuery();
 // process rs...
}
```

## Transaction Control in JDBC

```
conn.setAutoCommit(false); // Begin transaction
try {
 pstmt1.executeUpdate();
 pstmt2.executeUpdate();
 conn.commit(); // Commit
} catch (SQLException e) {
 conn.rollback(); // Rollback on error
}
```

## A.3 Python Database Connectivity

### psycopg2 — PostgreSQL from Python

```
import psycopg2
import os

Connect – never hardcode credentials; use env vars
conn = psycopg2.connect(
 host= "localhost",
 port= 5432,
 dbname= "university",
 user= "db_user",
 password= os.environ["DB_PASS"]
)

Parameterized query (safe against SQL injection)
cur = conn.cursor()
cur.execute(
```

```
"SELECT name, gpa FROM students WHERE major = %s AND gpa > %s",
("CS", 3.5) # Note: tuple, not f-string
)

rows = cur.fetchall()
for name, gpa in rows:
 print(f"{name:<20} GPA: {gpa:.2f}")

conn.commit()
cur.close()
conn.close()

Insert with transaction
try:
 cur.execute(
 "INSERT INTO students (name, major, gpa) VALUES (%s, %s, %s)",
 ("Nour Khaled", "CE", 3.7)
)
 conn.commit()
except psycopg2.Error as e:
 conn.rollback()
 print(f"Error: {e}")
```

## **pymongo — MongoDB from Python**

```
from pymongo import MongoClient
import os

Connect to MongoDB
client = MongoClient(os.environ["MONGO_URI"]) # e.g. "mongodb://localhost:27017"
db = client["university_db"]
col = db["students"]

Insert
col.insert_one({ "name": "Sara Ali", "major": "CS", "gpa": 3.8 })

Query – parameterized automatically (no injection risk in pymongo)
cursor = col.find({"major": "CS", "gpa": {"$gte": 3.5}}, {"name": 1, "gpa": 1})
for doc in cursor:
 print(doc["name"], doc["gpa"])
```

```
Update
col.update_one({"name": "Sara Ali"}, {"$set": {"gpa": 3.9}})

Aggregation
pipeline = [
 {"$match": {"gpa": {"$gte": 3.0}}},
 {"$group": {"_id": "$major", "avg_gpa": {"$avg": "$gpa"}}},
 {"$sort": {"avg_gpa": -1}}
]
for result in col.aggregate(pipeline):
 print(result)

client.close()
```

## A.4 REST APIs and Databases

Modern web applications expose database operations via **REST APIs** over HTTP.

### HTTP Verbs to CRUD Mapping

HTTP Method	SQL Operation	MongoDB Operation	URL Example
GET	SELECT	find()	GET /api/students
GET (by id)	SELECT WHERE id=?	findOne()	GET /api/students/42
POST	INSERT	insertOne()	POST /api/students
PUT	UPDATE (full)	replaceOne()	PUT /api/students/42
PATCH	UPDATE (partial)	updateOne()	PATCH /api/students/42
DELETE	DELETE	deleteOne()	DELETE /api/students/42

### Minimal Flask REST API (Python)

```
from flask import Flask, request, jsonify
import psycopg2
import os
```

```
app = Flask(__name__)

def get_conn():
 return psycopg2.connect(
 host=os.environ["DB_HOST"], dbname=os.environ["DB_NAME"],
 user=os.environ["DB_USER"], password=os.environ["DB_PASS"]
)

@app.route("/api/students", methods=["GET"])
def list_students():
 conn = get_conn()
 cur = conn.cursor()
 cur.execute("SELECT id, name, major, gpa FROM students ORDER BY name")
 rows = cur.fetchall()
 cur.close(); conn.close()
 return jsonify([
 {"id": r[0], "name": r[1], "major": r[2], "gpa": r[3]} for r in rows
])

@app.route("/api/students", methods=["POST"])
def create_student():
 data = request.get_json()
 conn = get_conn()
 cur = conn.cursor()
 cur.execute(
 "INSERT INTO students (name, major, gpa) VALUES (%s, %s, %s) RETURNING id",
 (data["name"], data["major"], data["gpa"])
)
 new_id = cur.fetchone()[0]
 conn.commit(); cur.close(); conn.close()
 return jsonify({"id": new_id}), 201
```

## REST API — JSON Response Examples

```
// GET /api/students/42
{
 "id": 42,
 "name": "Sara Ali",
 "major": "Computer Engineering",
 "gpa": 3.8
}
```



```

}

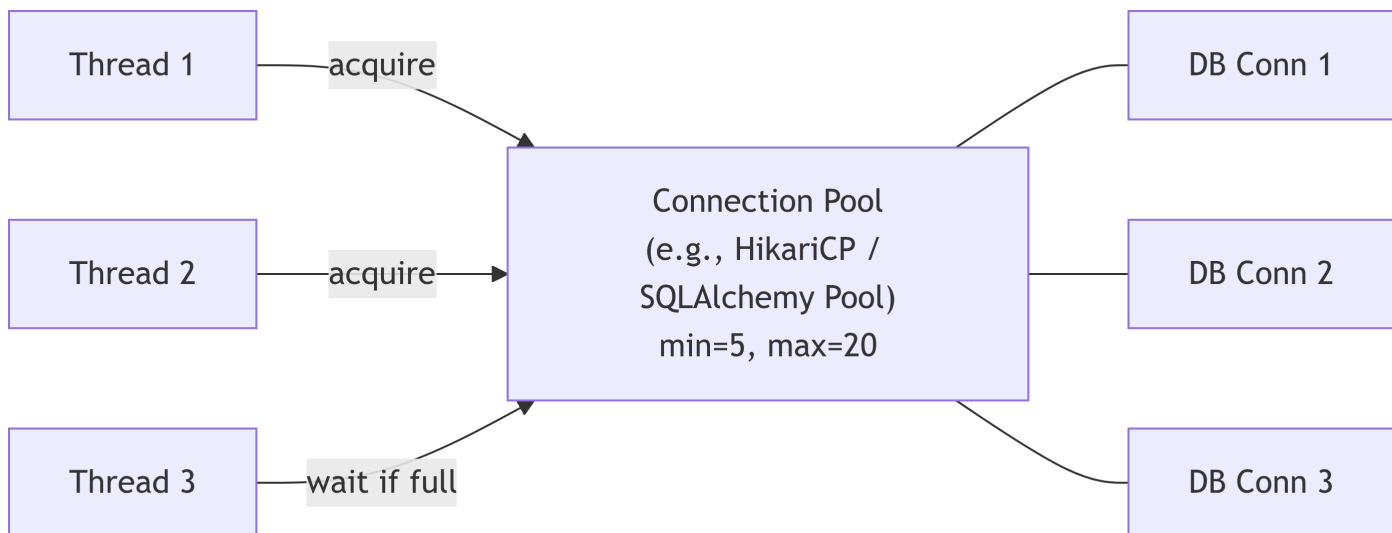
// POST /api/students (response: 201 Created)
{ "id": 43 }

// Error response (4xx)
{ "error": "Student not found", "code": 404 }

```

## A.5 Connection Pooling

Opening a new database connection for every query is expensive — each connection involves a TCP handshake, authentication, and memory allocation on the server. **Connection pooling** maintains a pool of pre-opened connections that are reused by application threads.



### Java (HikariCP)

```

HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:postgresql://localhost:5432/university");
config.setUsername(System.getenv("DB_USER"));
config.setPassword(System.getenv("DB_PASS"));
config.setMaximumPoolSize(20); // max concurrent connections
config.setMinimumIdle(5); // keep 5 warm at all times
config.setConnectionTimeout(3000); // ms to wait for a connection

HikariDataSource ds = new HikariDataSource(config);

// Use like a normal connection – returned to pool on close

```

```
try (Connection conn = ds.getConnection()) {
 // ...
}
```

## Python (SQLAlchemy)

```
from sqlalchemy import create_engine
import os
```

```
engine = create_engine(
 os.environ["DATABASE_URL"],
 pool_size=10, # persistent connections
 max_overflow=20, # extra connections under burst load
 pool_timeout=30, # seconds to wait before raising PoolTimeout
 pool_recycle=1800 # close connections older than 30 min (avoids server-side timeout)
)
Each `with engine.connect() as conn` borrows from the pool
```

## psycopg2 (ThreadedConnectionPool)

```
import psycopg2.pool, os
```

```
pool = psycopg2.pool.ThreadedConnectionPool(
 minconn=2, maxconn=10,
 host=os.environ["DB_HOST"],
 dbname=os.environ["DB_NAME"],
 user=os.environ["DB_USER"],
 password=os.environ["DB_PASS"]
)
```

```
conn = pool.getconn()
try:
 cur = conn.cursor()
 cur.execute("SELECT 1")
 conn.commit()
finally:
 pool.putconn(conn) # return to pool
```

---

## A.6 ORM — Object Relational Mapping

An **ORM** maps database tables Python classes automatically, eliminating raw SQL for common operations.

### SQLAlchemy — Python ORM

```
from sqlalchemy import create_engine, Column, Integer, String, Float
from sqlalchemy.orm import DeclarativeBase, Session
import os

engine = create_engine(os.environ["DATABASE_URL"]) # e.g. postgresql+psycopg2://...

class Base(DeclarativeBase):
 pass

Define a model (maps to a table)
class Student(Base):
 __tablename__ = "students"

 id = Column(Integer, primary_key=True)
 name = Column(String(100), nullable=False)
 major = Column(String(50))
 gpa = Column(Float)

 def __repr__(self):
 return f"<Student {self.name}, GPA={self.gpa}>"

Create tables if they don't exist
Base.metadata.create_all(engine)
```

### ORM CRUD Operations

```
with Session(engine) as session:

 # Create (INSERT)
 new_student = Student(name="Ahmad Hassan", major="CS", gpa=3.6)
 session.add(new_student)
 session.commit()

 # Read (SELECT)
```

```

students = session.query(Student).filter(Student.gpa >= 3.5).all()
top_cs = session.query(Student)\
 .filter(Student.major == "CS")\
 .order_by(Student.gpa.desc())\
 .limit(5).all()

Update
student = session.get(Student, 1)
student.gpa = 3.9
session.commit()

Delete
session.delete(student)
session.commit()

```

## ORM vs Raw SQL

	Raw SQL (psycopg2)	ORM (SQLAlchemy)
<b>Boilerplate</b>	More setup/parsing code	Minimal once model defined
<b>Type safety</b>	Manual	Automatic via model
<b>Flexibility</b>	Full SQL power	Limited for complex queries
<b>Performance</b>	Potentially faster	Slight overhead
<b>Portability</b>	DB-specific SQL	Mostly DB-agnostic
<b>Best for</b>	High-performance bulk ops	CRUD-heavy app logic

## Summary

### ! Appendix A — Key Takeaways

- **ODBC/JDBC** are standard interfaces allowing applications to connect to any database via a driver.
- **JDBC** uses `Connection` → `PreparedStatement` → `ResultSet`; always use `PreparedStatement` to prevent SQL injection.
- **psycopg2** connects Python to PostgreSQL; **pymongo** connects Python to MongoDB. Both use parameterized operations to avoid injection.
- **REST APIs** map HTTP verbs to database CRUD and return JSON — the dominant pattern for web + mobile apps.
- **ORM** (SQLAlchemy) maps DB tables to Python classes; great for app development, but raw SQL is preferred for complex analytics.

- **Never hardcode credentials** — always use environment variables or a secrets manager.

## Review Questions

### Conceptual

1. Explain the difference between **ODBC** and **JDBC**. In what situations would you use each? What is the role of the *driver manager* in the JDBC connectivity model?
2. Why does `Statement.executeQuery("SELECT ... WHERE name = '" + input + "'")` create a **SQL injection vulnerability**? Demonstrate with an example attacker input that would  
(a) return all rows and (b) delete all rows from the `students` table.
3. Explain the difference between `conn.commit()` and `conn.rollback()` in the context of JDBC. What happens if `conn.setAutoCommit(false)` is set and the program exits without committing?

### Applied

4. A Java application issues 10,000 short SQL queries per second. Each query currently opens a new JDBC Connection and closes it immediately.  
(a) What is the performance problem with this approach?  
(b) How does a **connection pool** (e.g., HikariCP) solve it?  
(c) What two parameters would you tune first and why?
5. Rewrite the following **vulnerable** Python code to be safe:

```
def get_student(name):
 conn = psycopg2.connect(...)
 cur = conn.cursor()
 cur.execute(f"SELECT * FROM students WHERE name = '{name}'")
 return cur.fetchall()
```

Your solution must:

- Use parameterized queries
  - Use `with` statements (context managers) for the connection and cursor
  - Read the database password from an environment variable
6. Write a minimal Python Flask endpoint `PATCH /api/students/<int:student_id>` that:

- Accepts a JSON body with optional fields name, major, gpa
- Updates only the provided fields (ignores absent fields)
- Returns 404 with {"error": "Not found"} if the student doesn't exist
- Uses a parameterized UPDATE statement

## Analysis

7. A web application is designed so that the backend constructs SQL queries using string formatting with user-provided data. The application passes a security scan because all inputs are checked for the characters ' , " , and ; .
  - (a) Explain why this character-blacklist approach is **insufficient** for SQL injection prevention.
  - (b) Name the one technique that completely prevents SQL injection at the query level.
  - (c) Name two additional layers of defence in depth that complement parameterized queries.
8. Explain the **ORM vs. Raw SQL** trade-off in the following scenarios:
  - (a) A CRUD-heavy REST API for a student registration system (each request touches 1–3 rows)
  - (b) A monthly batch job that must aggregate 50 million order rows with complex window functions
  - (c) A startup prototype where the schema changes weekly

For each, state whether you would prefer ORM, raw SQL, or a hybrid, and why.

## Design

9. Design the database connectivity architecture for a web application with these requirements:
  - Backend: Python (FastAPI)
  - Primary database: PostgreSQL (transactional data)
  - Cache: Redis (session tokens, rate limiting)
  - Document store: MongoDB (product catalog)
  - Peak load: 5,000 concurrent users

Describe: (a) which Python library you would use for each database, (b) how you would manage connection pools for each, (c) how credentials would be stored and injected into the application.

10. A junior developer writes the following code to handle a user login:

```
username = request.form["username"]
password = request.form["password"]
cur.execute(f"SELECT * FROM users WHERE username='{username}' AND password='{password}'")
user = cur.fetchone()
if user:
 login_user(user)
```

Identify **every** security vulnerability in this code (there are at least three beyond SQL injection). Rewrite it securely using best practices.

# A Guided Tour of Database Families

Appendix B



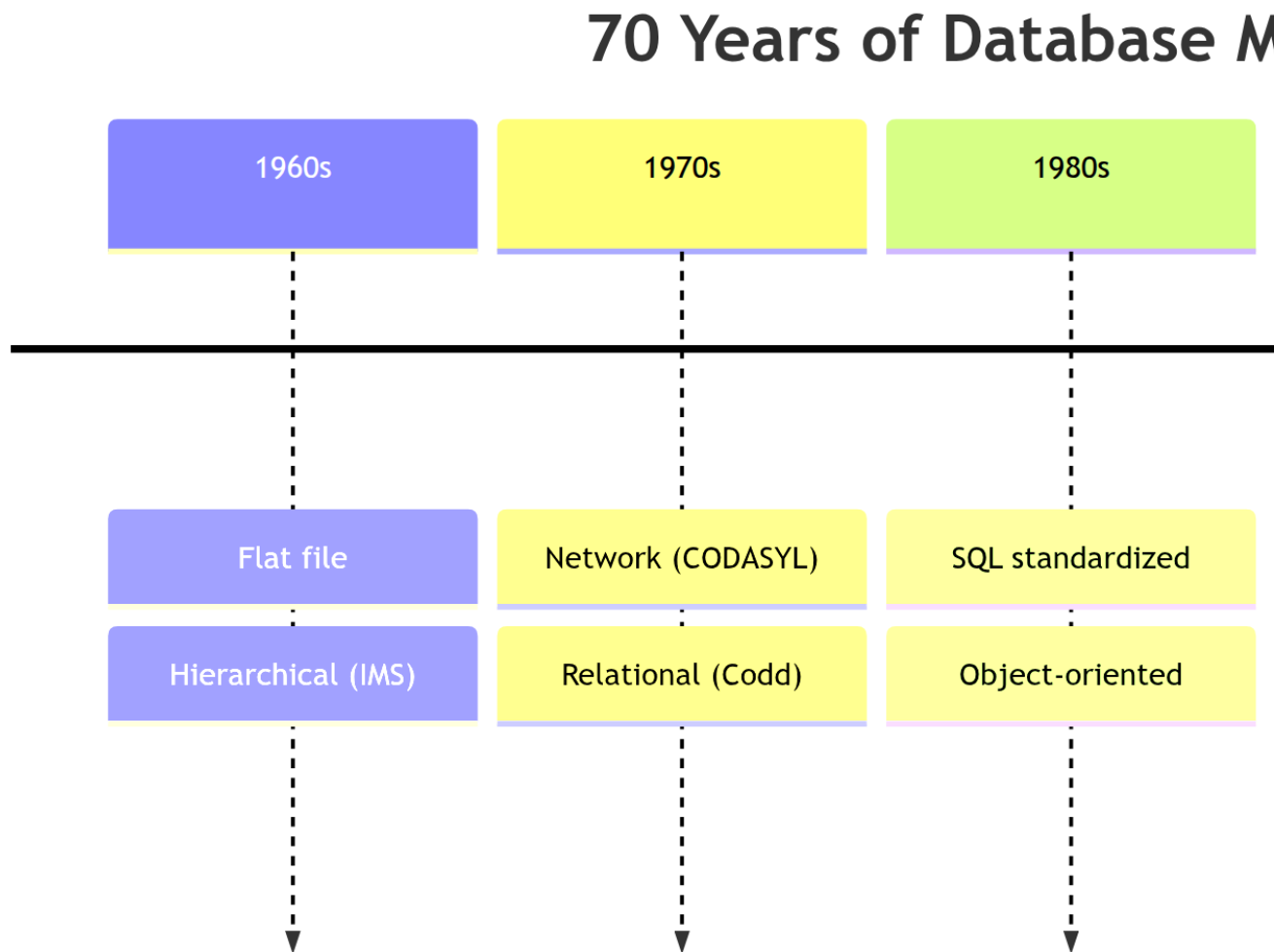
## A Guided Tour of Database Families

Chapter 1 introduced the **relational model** as today's dominant model and referenced its place in the wider family of data models. This appendix expands that reference: it surveys the main database families that exist in industry today, explains *why* each one was invented, and where each is still used.

Read it as a reference chapter. You do not need this material to follow Chapters 2–13 — the relational model carries those chapters — but when you encounter NoSQL in Part VIII, or need to justify a technology choice in a project, this tour is your map.

---

## 70 Years of Database Models



### 1. Flat File Model

One file, one record per line, delimited columns. No relationships, no constraints.

**Real-world examples:** Excel / Google Sheets, CSV exports, `/etc/passwd`, Windows `.ini` files, small `.log` files.

**Example — `/etc/passwd` style flat file** (7 colon-separated fields: *username* : *password* : *UID* : *GID* : *full name* : *home directory* : *login shell*):

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
hussein:x:1000:1000:Hussein Khalidi,,,:/home/hussein:/bin/bash
ahmed:x:1001:1001:Ahmed Mansour,,,:/home/ahmed:/bin/zsh
sara:x:1002:1002:Sara Odeh,,,:/home/sara:/bin/bash
mysql:x:119:128:MySQL Server,,,:/nonexistent:/bin/false
```

Each line is one record; fields are positional and separated by `:`. There is no schema file, no type checking, no index — every lookup is a full scan, and any program that reads it must agree on the field order by convention only.

### **Advantages**

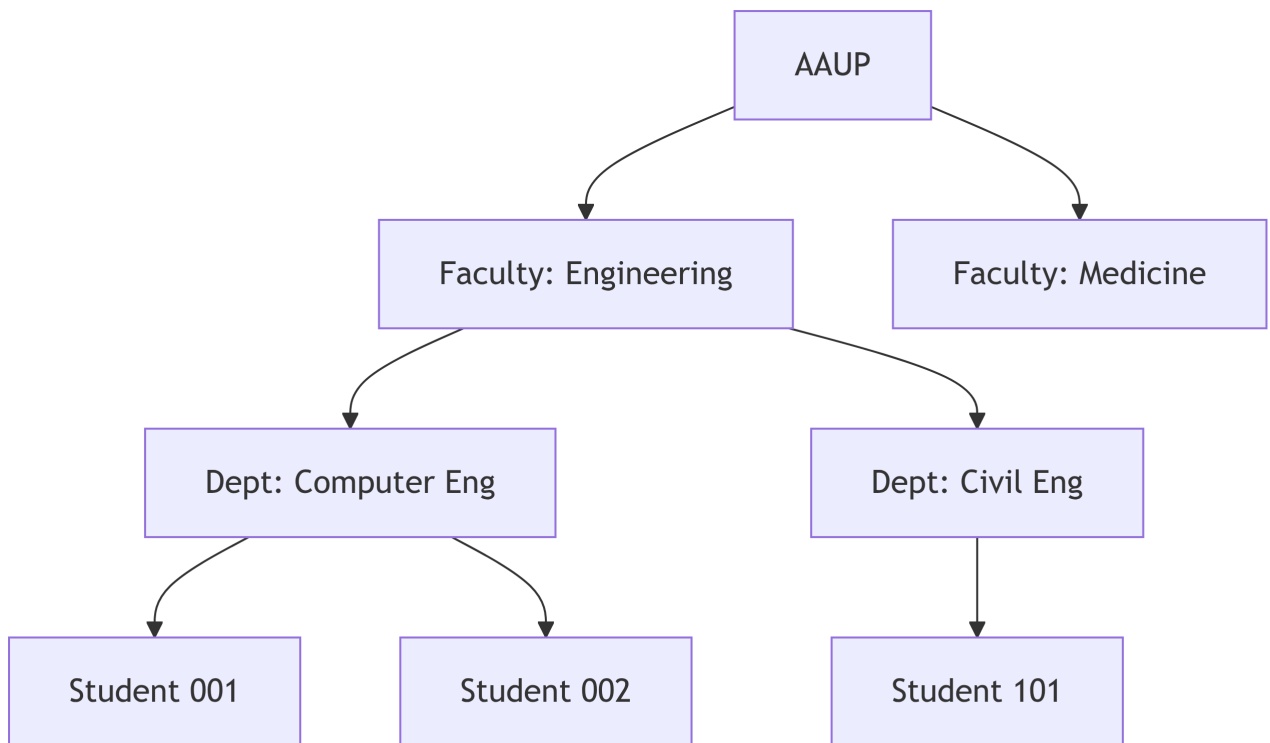
- Extremely simple and human-readable
- Works in any tool (Notepad, Excel, any language's string library)

### **Disadvantages**

- All the Notepad Scenario problems: no integrity, no concurrency, redundancy
  - Cannot scale beyond a few thousand rows
- 

## **2. Hierarchical Model**

Data is organized as a **tree** — each record has exactly **one parent**. Developed in the 1960s for mainframe systems.



**Real-world examples:** IBM IMS (still runs many banks and airlines today!), XML documents, JSON trees, LDAP directories, the Windows Registry.

#### Advantages

- Very fast for tree-shaped traversals (org charts, file systems, bill-of-materials)
- Simple, predictable navigation — follow parent/child pointers
- Enforces strict 1-to-many structure natively

#### Disadvantages

- Awkward for **many-to-many** relationships — a student enrolled in two departments would need to be duplicated
- Schema changes require rebuilding the tree
- No declarative query language — the programmer walks the hierarchy by hand

---

### 3. Network Model (CODASYL)

Like the hierarchical model, but a record can have **many parents** — so many-to-many relationships are supported directly through explicit pointers.

**Real-world examples:** IDMS (still running inside some European banks and insurance companies), Raima DB, older mainframe systems at airlines and telecoms.

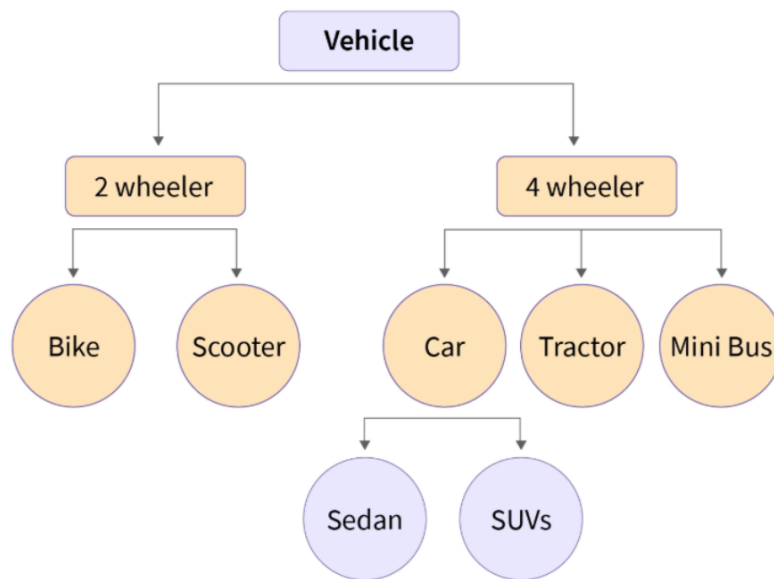


Figure 1: Network model — records (nodes) can belong to multiple parent sets through explicit pointer links, unlike a strict tree.

### Advantages

- Supports many-to-many relationships without duplication
- Very fast when the access paths are known in advance

### Disadvantages

- No declarative query language — the programmer navigates pointers by hand
- Schema changes are extremely painful
- Largely replaced by relational databases since the 1980s

#### **i** Note

Both hierarchical and network models were displaced by the relational model because Codd's math-based foundation allowed **declarative queries** (“what” instead of “how”) and automatic query optimization.

## 4. Relational Model

Data as **tables** (rows + columns); relationships via **foreign keys**; queries via **SQL**. Invented by E.F. Codd in 1970, still the #1 database model today.

**Real-world examples:** Facebook (MySQL), Instagram (PostgreSQL), Booking.com (Oracle + MySQL), Uber's core services (PostgreSQL + MySQL), most banks worldwide, AAUP's own student portal.

### Advantages

- Declarative SQL — you say *what*, the database figures out *how*
- Strong ACID guarantees (safe for money, grades, medical records)
- Huge ecosystem: tools, drivers, hosting, experts everywhere

### Disadvantages

- Rigid schema — changing columns on a live table is expensive
- Joins get slow on very large datasets
- Not ideal for deeply nested JSON or graph-shaped data

This is the model that dominates **Parts III – VII** of this book.

---

## 5. Object-Oriented (OODB)

Stores objects exactly as they exist in a Java / C++ / C# program — no mapping to tables.

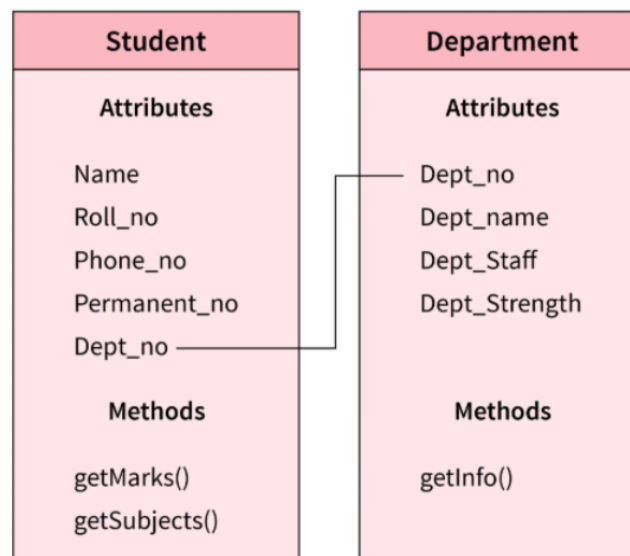


Figure 2: OODB example — each entity is a class with *attributes* and *methods*; relationships are direct object references (e.g. `Student.Dept_no` → `Department`) rather than foreign-key joins.

**Real-world examples:** ObjectDB (used inside some CAD and telecom systems), db4o (embedded in apps), Gemstone/S (financial trading). Niche today.

### Advantages

- No impedance mismatch between code and database
- Natural fit for inheritance and complex nested types

### Disadvantages

- No dominant query language (OQL never caught on)
  - Tiny ecosystem; lost the market to **relational + ORM** (Hibernate, Django ORM, Entity Framework)
- 

## 6. Object-Relational (ORDBMS)

A relational database with object-like extras: user-defined types, arrays, JSON columns, spatial types, inheritance.

**Real-world examples:** **PostgreSQL** (the gold standard — used by Apple, Instagram, Reddit), Oracle Database, modern SQL Server and MySQL (JSON columns, GEOMETRY, arrays).

### Advantages

- Keeps SQL + ACID but adds rich types (JSONB, ARRAY, GEOMETRY, UUID)
- Single database can handle both traditional and document-style data

### Disadvantages

- Extensions are vendor-specific — moving from Oracle to PostgreSQL is harder than moving pure SQL
- 

## 7. NoSQL Models

A family of **non-relational** databases born in the 2000s at web-scale companies (Google, Amazon, Facebook) to handle data volumes and write rates that single-server relational databases could not match.

Four main sub-families:

### 7a. Key-Value Stores

A giant distributed hash table: `get (key) / put (key, value)`. The simplest NoSQL model.

**Real-world examples:** **Redis** (Twitter timeline cache, Stack Overflow, GitHub, Shopify), **Memcached** (Facebook), **Amazon DynamoDB** (Amazon's own shopping cart, Lyft, Airbnb sessions), etcd (inside Kubernetes).

### Advantages

- Blazingly fast (often in-memory, microsecond latency)

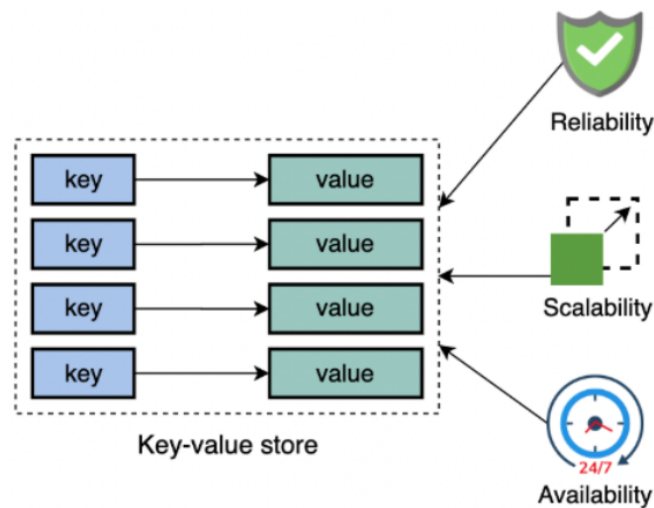


Figure 3: Key-value store — each key maps directly to an opaque value; the design prioritises reliability, horizontal scalability, and 24/7 availability over rich querying.

- Scales horizontally to millions of operations per second

### Disadvantages

- No queries across values — you can only fetch by key
- No joins, no WHERE, no rich schema

## 7b. Document Stores

Store self-contained **JSON-like documents**; each document can have a different shape. Query on any nested field.

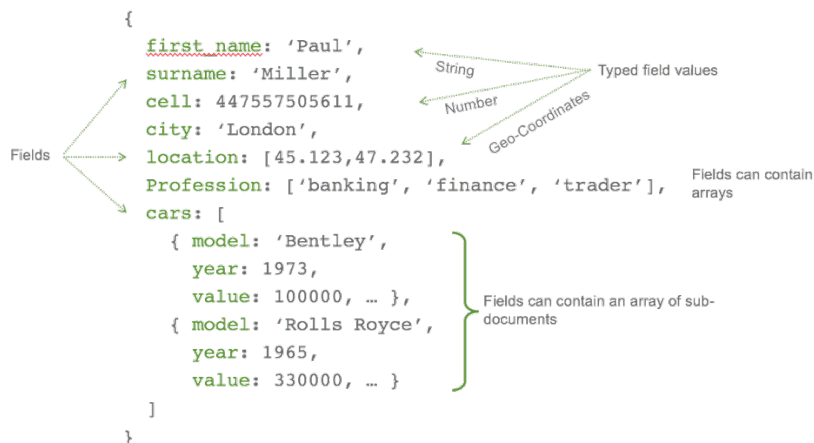


Figure 4: Document store — a single MongoDB document can embed sub-documents and arrays, keeping related data together instead of splitting it across joined tables. (Source: Studio 3T)

**Real-world examples:** **MongoDB** (The New York Times, Forbes, eBay, Adobe), **Couchbase** (LinkedIn, PayPal), **Firebase Firestore** (many mobile apps), Amazon DocumentDB.



### Advantages

- Flexible schema — add fields without migrations
- Natural fit for JSON from web and mobile APIs
- Good horizontal scalability

### Disadvantages

- Cross-document joins are awkward (or absent)
- Easy to end up with inconsistent document shapes over time

MongoDB is covered in depth in **Chapter 15**.

## 7c. Column-Family (Wide-Column) Stores

Data is stored by **column families** rather than whole rows; optimized for huge, sparse tables and write-heavy workloads. Inspired by Google's **Bigtable** paper (2006).

Row A	Column 1	Column 2	Column 3
	Value	Value	Value
Row B	Column 1	Column 2	Column 3
	Value	Value	Value

Figure 5: Wide-column store — each row is keyed by a row-key and holds a sparse set of (column, value) pairs; different rows can have different columns. (*Source: ScyllaDB*)

**Real-world examples:** **Apache Cassandra** (Netflix viewing history, Apple iCloud, Instagram messaging), **HBase** (Facebook Messenger, Pinterest), **Google Bigtable** (Google Search, Gmail, Google Analytics), **ScyllaDB** (Discord messages).

### Advantages

- Massive write throughput — millions of writes/sec across a cluster
- Linear horizontal scaling by adding nodes
- Great for time-series, IoT, logs

### Disadvantages

- Ad-hoc queries are limited — you must design around known access patterns
- No joins, weaker consistency by default

## 7d. Graph Databases

First-class support for **nodes** (entities) and **edges** (relationships). Queries are traversals like “*friends of friends within 3 hops*”.

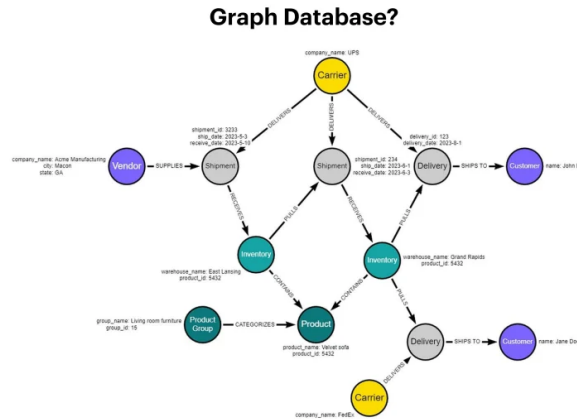


Figure 6: Graph database — data is modelled as *nodes* connected by typed *edges* (relationships), each carrying its own properties. (Source: OpenXcell)

**Real-world examples:** **Neo4j** (NASA’s lessons-learned graph, eBay delivery routing, UBS fraud detection), **Amazon Neptune** (Amazon product recommendations), **TigerGraph** (Jaguar, China Mobile), LinkedIn’s own social graph engine.

### Advantages

- Relationship queries are orders of magnitude faster than SQL joins
- Natural fit for social networks, recommendations, fraud detection, knowledge graphs

### Disadvantages

- Poor fit for plain tabular analytics
- Smaller ecosystem than relational or document stores

## 8. NewSQL & Cloud-Native Distributed SQL

A 2010s answer to: “*Can we get NoSQL’s horizontal scalability **and** relational SQL + ACID?*” Yes — at the cost of more complex infrastructure.

**Real-world examples:** **Google Spanner** (Google Ads, Gmail metadata), **CockroachDB** (DoorDash, Netflix billing), **YugabyteDB**, **TiDB** (Shopee, Bank of China), **Amazon Aurora** (most large AWS customers).

### Advantages

- Standard SQL and ACID transactions
- Scales horizontally across regions / continents
- No need to re-learn a new query language

### Disadvantages

- Operationally complex — more moving parts than a single MySQL server
- Young ecosystem; fewer DBAs with deep experience
- Cross-region writes still have latency costs

## Side-by-Side Comparison

Family	Shape of data	Query style	ACID	Horizontal scale	Example
Flat file	Rows in one file	Parse by hand			CSV
Hierarchical	Tree (one parent)	Navigate	Partial	Limited	IBM IMS, XML
Network (CODASYL)	Graph via pointers	Navigate	Partial	Limited	IDMS
<b>Relational</b>	Tables + FKs	<b>SQL (declarative)</b>		Medium	MySQL, Postgres, Oracle
Object-oriented	Objects	OQL / API	Partial	Limited	ObjectDB
Object-relational	Tables + rich types	SQL + extensions		Medium	PostgreSQL
Key-value	(key, value) blobs	get / put	Per-key		Redis, DynamoDB
Document	JSON documents	Query DSL over JSON	Per-document		<b>MongoDB</b>
Column-family	Sparse wide rows	CQL / API	Tunable		Cassandra, HBase
Graph	Nodes + edges	Cypher / Gremlin			Neo4j
NewSQL	Tables + FKs	SQL			Spanner, CockroachDB



How to choose (rule of thumb)

- **Tabular data with real relationships?** → Relational (start here by default).
- **Each record is a self-contained document with nested fields?** → Document (MongoDB).
- **Distributed cache or session store?** → Key-value (Redis).
- **Relationships between entities are the main query pattern?** → Graph (Neo4j).
- **Petabyte-scale writes, time series, telemetry?** → Column-family (Cassandra).
- **SQL + ACID and global horizontal scale?** → NewSQL (Spanner, CockroachDB).

Modern systems often combine **several** of these — this is called **polyglot persistence**. The AAUP portal might use PostgreSQL for grades, Redis for sessions, and Elasticsearch for search, all in the same application.

NoSQL families are covered in depth in **Part VIII (Chapters 14–15)**.

# Advanced Constraint Enforcement

Appendix C

# Advanced Constraint Enforcement

Some real-world business rules cannot be expressed with ordinary PRIMARY KEY, FOREIGN KEY, or simple CHECK constraints. They require **triggers**, **stored procedures**, deferrable constraints, or subquery-based CHECKs — machinery that is introduced formally only in Chapter 5 (foreign-key actions), Chapter 8 (advanced SQL), Chapter 12 (transactions), and Chapter 13 (concurrency).

To keep Chapters 3 and 4 focused on *modelling* rather than *implementation*, the deep-dive walkthroughs are collected here. Each walkthrough references the ER/EER model from which the rule originated.

The examples below come from Chapter 3, §Min-Max Notation.

---

## C.1 Parking-Permit Reassignment — ON DELETE SET NULL with a Status Trigger

*Originating rule:* HOLDS\_PERMIT is a  $(0, 1) : (0, 1)$  relationship — a permit survives when its student leaves and becomes available for the next student (Chapter 3, Example 2).

### Representing & Implementing Reassignment (“Permit becomes available when student leaves”)

The  $(0, 1):(0, 1)$  constraint captures a subtle real-world rule: a permit is **not destroyed** when a student leaves — it simply becomes unassigned and available for the next student. This must be modelled and enforced at the database level.

#### Key design decisions:

Decision	Choice	Reason
student_id nullable?	<b>Yes</b> (NULL)	A permit can exist without an owner
On student deletion	ON DELETE SET NULL	Permit survives; student_id becomes NULL
Permit lifecycle	status column (AVAILABLE / ASSIGNED / SUSPENDED)	Tracks whether permit is ready to reassign

**SQL implementation:**

```
CREATE TABLE ParkingPermit (
 permit_id VARCHAR(20) PRIMARY KEY,
 issue_date DATE NOT NULL,
 status ENUM('AVAILABLE', 'ASSIGNED', 'SUSPENDED') DEFAULT 'AVAILABLE',
 student_id INT NULL, -- NULL = unassigned
 FOREIGN KEY (student_id) REFERENCES Student(student_id)
 ON DELETE SET NULL -- Critical for reassignment!
);

-- When student leaves:
DELETE FROM Student WHERE student_id = 123;
-- → DB automatically sets permit.student_id = NULL via ON DELETE SET NULL
-- → Permit status can be updated to 'AVAILABLE' via trigger
```

**Why ON DELETE SET NULL — not ON DELETE CASCADE?**

- CASCADE would **delete** the permit when the student is removed — destroying a physical asset the university still owns.
- SET NULL **releases** the permit back into the pool, preserving it for the next student.

**Optionally, a trigger can automate the status reset:**

```
CREATE OR REPLACE FUNCTION release_permit()
RETURNS TRIGGER AS $$
BEGIN
 UPDATE ParkingPermit
 SET status = 'AVAILABLE'
 WHERE student_id = OLD.student_id;
 RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_release_permit
AFTER DELETE ON Student
FOR EACH ROW EXECUTE FUNCTION release_permit();
```

The ER diagram's (0, 1):(0, 1) pairs say nothing about *how* to implement reassignment — that detail lives entirely in the physical schema. This is a good example of why ER modelling and physical implementation are separate concerns.

## C.2 OFFERS Timing — Enforcing “Within the First Semester” with a Trigger

*Originating rule:* Every new department must create its first course within one semester of activation (Chapter 3, Example 3 — OFFERS).

### ⚠ Timing Constraint: “Within the first semester”

The  $(1, N)$  constraint on Department enforces that every department has **at least one course in the current database state** — but it says nothing about *when* that course must be created. The “within first semester” requirement is a temporal business rule that a static ER diagram cannot capture.

It must be enforced through one of the following mechanisms:

- **Application workflow** — block department activation (e.g., status remains PENDING) until at least one course is created and linked.
- **Database triggers with status flags** — a BEFORE INSERT or AFTER UPDATE trigger checks the department’s course count before allowing its status to change to ACTIVE.
- **Business process controls** — documented policy enforced by administrators; not representable in a static ER model.

-- Example: prevent activating a department with no courses

```
CREATE OR REPLACE FUNCTION check_dept_has_course()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
 IF NEW.status = 'ACTIVE' THEN
```

```
 IF (SELECT COUNT(*) FROM Course WHERE dept_id = NEW.dept_id) = 0 THEN
```

```
 RAISE EXCEPTION
```

```
 'Department % cannot be activated without at least one course.', NEW.dept.
```

```
 END IF;
```

```
 END IF;
```

```
 RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_dept_activation
```

```
BEFORE UPDATE ON Department
```

```
FOR EACH ROW EXECUTE FUNCTION check_dept_has_course();
```



## C.3 Specific Value Constraints (0, 3), (2, 5), ... — Enforcement Strategies

*Originating rule:* Some relationships require exact counts (e.g. “a project team must have 2 to 5 members”). Standard SQL foreign keys cannot count; four complementary strategies are available.

### How to Enforce Specific Value Constraints in the Database

Four complementary strategies exist, each with different trade-offs:

#### 1. CHECK Constraints with Subqueries

Some DBMSs — notably **PostgreSQL** — permit a subquery inside a CHECK constraint, letting the database count related rows at write time.

```
-- PostgreSQL: enforce that a project has at most 5 members
ALTER TABLE WORKS_ON
 ADD CONSTRAINT chk_team_size
 CHECK (
 (SELECT COUNT(*)
 FROM WORKS_ON w
 WHERE w.pno = NEW.pno) <= 5
);
```

**Portability note:** MySQL, SQL Server, and SQLite do **not** support subqueries inside CHECK constraints. Using this approach ties the schema to a specific DBMS.

---

#### 2. Triggers (BEFORE INSERT / UPDATE / DELETE)

Triggers fire automatically before (or after) a data-modification statement, giving you full procedural logic to count rows and raise an error.

```
-- PostgreSQL trigger: enforce (2,5) team size on WORKS_ON
CREATE OR REPLACE FUNCTION check_team_size()
RETURNS TRIGGER AS $$
DECLARE
 cnt INTEGER;
BEGIN
 SELECT COUNT(*) INTO cnt
 FROM WORKS_ON
 WHERE pno = NEW.pno;
```

```
IF cnt >= 5 THEN
 RAISE EXCEPTION
 'Team size limit exceeded for project %: max 5 members.', NEW.pno;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_team_size_insert
BEFORE INSERT OR UPDATE ON WORKS_ON
FOR EACH ROW EXECUTE FUNCTION check_team_size();

-- Also enforce the minimum (2) on DELETE
CREATE OR REPLACE FUNCTION check_team_min()
RETURNS TRIGGER AS $$
DECLARE
 cnt INTEGER;
BEGIN
 -- Count remaining rows after the delete
 SELECT COUNT(*) - 1 INTO cnt
 FROM WORKS_ON
 WHERE pno = OLD.pno;

 IF cnt < 2 THEN
 RAISE EXCEPTION
 'Project % must retain at least 2 members.', OLD.pno;
 END IF;
 RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_team_size_delete
BEFORE DELETE ON WORKS_ON
FOR EACH ROW EXECUTE FUNCTION check_team_min();
```

Triggers work across all major DBMSs (PostgreSQL, MySQL, SQL Server, Oracle) though the exact syntax differs.

---

### 3. Application Logic

Business rules that are hard to express in SQL can live in the application layer (Java, Python, Node.js, etc.). The application queries the current count before every insert or delete and rejects the operation when the constraint would be violated.

```
Python / SQLAlchemy example
def add_member_to_project(session, emp_ssn, proj_no):
 count = session.query(WorksOn)\
 .filter_by(pno=proj_no)\
 .count()
 if count >= 5:
 raise ValueError(
 f"Project {proj_no} already has the maximum of 5 members."
)
 session.add(WorksOn(essn=emp_ssn, pno=proj_no))
 session.commit()
```

**Caution:** Application-layer enforcement can be bypassed by direct database access tools (e.g., a DBA running a raw SQL script). Pair it with a trigger for defence-in-depth.

---

#### 4. Stored Procedures

Encapsulate all insert/update/delete operations for the affected table inside stored procedures and revoke direct-write privileges from application users. The procedure performs the count check before executing the DML.

```
-- MySQL stored procedure enforcing (0,3) on ORDERS per CUSTOMER
DELIMITER $$
CREATE PROCEDURE add_order(IN p_cust_id INT, IN p_order_data VARCHAR(255))
BEGIN
 DECLARE v_count INT;

 SELECT COUNT(*) INTO v_count
 FROM ORDERS
 WHERE cust_id = p_cust_id;

 IF v_count >= 3 THEN
 SIGNAL SQLSTATE '45000'
 SET MESSAGE_TEXT =
 'Customer has reached the maximum of 3 active orders.';
 END IF;
```

```
INSERT INTO ORDERS (cust_id, order_data)
VALUES (p_cust_id, p_order_data);
END$$
DELIMITER ;
```

**i** Note

**Summary: choosing an enforcement strategy**

Strategy	Works in	Bypassed by direct SQL?	Notes
CHECK + subquery Triggers	PostgreSQL only  All major DBMSs	No  No	Least portable Best database- level guarantee
Application logic	Any	Yes (direct access)	Flexible; pair with triggers
Stored procedures	All major DBMSs	Only if user has direct write rights	Centralises business logic

For production systems, **triggers** or **stored procedures** are the most reliable choices because they enforce the rule at the database level regardless of which client is writing data.

# RELAX — Interactive Relational Algebra Calculator

Appendix D

## How to Use

Use the **Schema Definition** textarea to define your tables using inline-relation syntax, then type a relational algebra expression in the **Query Editor** and click **Execute** (or press **Ctrl+Enter**).

## Quick Reference — Symbol Bar

Group	Symbols	Meaning
<b>RA Ops</b>	$\pi \sigma \rho \tau \gamma$	Projection, Selection, Rename, Order-by, Group-by
<b>Assign</b>	$\leftarrow \rightarrow$	Assignment arrows
<b>Logic</b>	$\square \square \neg$	And, Or, Not
<b>Compare</b>	$= \neq \geq \leq$	Equality and inequalities
<b>Set</b>	$\cap \square \div -$	Intersection, Union, Division, Difference
<b>Join</b>	$\square \square \square \square \square \square \square$	Natural/outer/semi/anti/cross joins
<b>Misc</b>	$-- / * \{ \}$	Comment, arithmetic, grouping

## Example Queries

Try these on the pre-loaded **Enhanced Company Database**:

```
-- Select high-salary employees
 σ Salary > 65000 (Employee)
```

```
-- Project only names and salary
 π Fname, Lname, Salary (Employee)

-- Natural join employee with their assignments
Employee $\bowtie$ Works_On

-- Left outer join (keep all employees, null if no assignment)
Employee $\ltimes$ Works_On

-- Employees who are also managers (intersection by SSN)
 π Ssn (Employee) $\cap$ π Mgr_ssn (Department)

-- Employees working on any project in Department 1
 π Fname, Lname (σ Dnum = 1 (Employee $\bowtie$ Works_On $\bowtie$ Project))

-- Rename relation and column
 ρ Emp $\leftarrow$ π Ssn, Fname, Salary (Employee)
```

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of database systems*, 7th ed. Hoboken, NJ: Pearson, 2015.
- [2] MongoDB, Inc., “MongoDB manual.” Accessed: Jan. 01, 2024. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [3] S. Bradshaw, E. Brazil, and K. Chodorow, *MongoDB: The definitive guide*, 3rd ed. O’Reilly Media, 2019.
- [4] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970, doi: [10.1145/362384.362685](https://doi.org/10.1145/362384.362685).